

From Trajectories to Experience: A Survey of Experience-Driven LLM Agents

Xiaohui Yan¹, Yingchen Zhang², Shiyao Liu², Xiaofei Huang², Yuxin Yue², Zaiyu Xia², Jiayu Yao², Xinjie Chen², Yun Hu¹, FuRong Li¹, Shenghua Liu², Yixing Fan², Zixuan Li², Ruqing Zhang², Jiafeng Guo²

Affiliations: ¹Huawei Technologies Co., Ltd, ²Institute of Computing Technology, Chinese Academy of Sciences

Large language model (LLM) agents can plan, use tools, and recover from errors within a task, yet many carry little from one task to the next and repeat mistakes they have already resolved. The root cause is a lack of experience accumulation: trajectories record what failed and how the agent recovered, but this information is seldom turned into guidance for later tasks. Research on this capability spans reflection, agent memory, workflow learning, and skill acquisition, but these lines describe similar content in different terms and lack a common account of experience itself, leaving basic questions open: what counts as experience, how it should be accumulated, used, and revised, and how to tell whether it improves later behavior. This survey addresses these questions with a unified framework. We define *agent experience* as action-guiding information grounded in task trajectories and retained to guide later episodes where it applies, whether as a trajectory precedent, a semantic lesson, or a reusable procedure. We focus on explicitly accessible experience, including its use as a training signal for internalization into model parameters. We describe each unit of experience by its guidance, applicability conditions, execution evidence, and reliability evaluation, so that memory entries, workflows, and skills become directly comparable, and follow experience through four lifecycle functions—formation, organization, utilization, and maintenance—that determine how it is accumulated, used, and revised. To judge reported gains, we distinguish three improvement claims—fixed experience reuse, feedback-driven sustained improvement, and improvement of the learning mechanism—by the evidence each requires. Current studies most clearly support fixed experience reuse, chiefly on tasks with tests or reference answers; evidence for the other two claims remains partial. Our analysis yields recommendations for seven application settings and a central design principle: keep guidance connected to its applicability conditions and to the evidence that supports or challenges it. Finally, we outline eight open problems, from learning with human expert records to testing whether gains accumulate, toward agents whose improvement is cumulative, transferable, and correctable.

✉ **Main Contact:** yanxiaohui2@huawei.com, guojiafeng@ict.ac.cn

Contents

1	Introduction	4
2	Foundations of Agent Experience Learning	7
2.1	Definition of Agent Experience	7
2.2	Core Terms, Forms, Dimensions, and Types of Experience	8
2.3	Formalization of Experience-Driven Agents	12
2.4	Lifecycle of Agent Experience	14
2.5	Experience-Driven Self-Improvement	15
2.6	Survey Scope and Related Directions	16
3	Experience Formation: From Trajectories to Reusable Guidance	19
3.1	Trajectory Acquisition for Experience Formation	20
3.2	Trajectory-Level Experience: Constructing Precedents for Reuse	22
3.3	Semantic-Level Experience: Abstracting Lessons from Trajectories	25
3.4	Procedural-Level Experience: Constructing Reusable Workflows and Skills	29
3.5	Comparing Forms and Formation Checks	32
4	Experience Organization: Structuring Experience for Reuse	35
4.1	Flat Experience Organization	35
4.2	Tree-Structured Experience Organization	37
4.3	Graph-Structured Experience Organization	38
4.4	Hierarchical Graph Experience Organization	40
4.5	When to Add Structure	42
5	Experience Utilization: Guiding Action with Past Experience	43
5.1	Experience Triggering: When to Seek Experience	44
5.2	Experience Selection: Which Experience to Use	46
5.3	Experience Adaptation: What to Change before Reuse	48
5.4	Experience Application: How Guidance Shapes Behavior	52
5.5	Experience Internalization: From Explicit Guidance to Model Parameters	54
5.6	Runtime Choices under One Budget	55
6	Experience Maintenance: Deciding What to Retain, Revise, or Retire	57
6.1	Experience Admission	57
6.2	Experience Consolidation	60
6.3	Experience Refinement	61
6.4	Experience Retirement	63
6.5	Coordinating Experience Maintenance	64
6.6	Safeguarding Experience Evolution	65
6.7	Unresolved Trade-offs in Experience Maintenance	66
7	Evaluation of Agent Experience	68
7.1	Scoring Reliability and Feedback Access	68
7.2	Evaluating Intrinsic Experience Quality	69
7.3	Evaluating the Experience Utilization Process	72
7.4	Evaluating Downstream Effects: Task Benefits and Transfer	75
7.5	Evaluating Downstream Effects: Sustained Improvement and Improvement of the Learning Mechanism	77
7.6	Matching Evaluations to Improvement Claims	81
8	Applications of Agent Experience	84
8.1	Question Answering	84
8.2	Software Engineering	85

8.3	Computer Use	86
8.4	Recommender Systems	87
8.5	Embodied Control	89
8.6	Scientific Research	89
8.7	Financial Trading and Forecasting	90
8.8	Comparing Application Conditions	91
8.9	Design Decisions across Applications	93
9	Challenges and Future Directions	94
9.1	Forming Experience from Human Expert Records	94
9.2	Preserving Informative Exploration during Reuse	96
9.3	Transferring Principles across Domains	97
9.4	Composing Experience for Unseen Tasks	97
9.5	Co-evolving Experience Representations and Executors	98
9.6	Tracing and Correcting Propagated Errors	99
9.7	Learning from Experience under Adversarial Interaction	100
9.8	Establishing Cumulative Capability Gains	101
9.9	How the Open Problems Depend on One Another	101
10	Conclusion	102
A	Experience-Formation Method Index	120
B	Experience-Organization Method Index	122
C	Experience-Maintenance Method Index	125
D	Evaluation Protocol Index	128

1 Introduction

Since ChatGPT’s public launch in November 2022, LLM research has moved from what models know to what agents can do, yet agents still carry little of what one task teaches into the next. In the **Era of LLMs** (OpenAI, 2022), pretraining, retrieval augmentation, and chain-of-thought prompting let models acquire, access, and reason with knowledge (Brown et al., 2020; Lewis et al., 2020; Wei et al., 2022), a *knowledge-centric paradigm* summarized as *Know*. The **Era of Agents** (Yao et al., 2023; Wang et al., 2024b) extends LLMs from dialogue to acting in external environments through planning, tool use, and runtime feedback, an *execution-centric paradigm* summarized as *Know + Act*. Such agents can recover from errors within a task, but many repeat mistakes they have already resolved once the next task begins. Much of what makes later tasks easier can be learned only through practice: which check exposes a fault quickly, which tool call tends to fail, which plan works on a particular website. Trajectories record what failed and how the agent recovered, but this information is seldom turned into guidance for later tasks.

Retaining what practice reveals and reusing it in later episodes lets an agent’s capabilities grow with use instead of staying fixed at deployment. Silver and Sutton (2025) argue that learning of this kind will define the next stage of AI, an **Era of Experience** in which agents learn continually from their own interaction with the environment, and such interaction eventually outweighs human data as the main source of improvement. LLM agents record their interaction as trajectories, and a growing body of work turns trajectories into reusable skills, insights, and strategies that guide later tasks (Wang et al., 2024a; Zhao et al., 2024a; Ouyang et al., 2026; Ni et al., 2026). We call this direction the *experience-centric paradigm*: the agent retains what its trajectories reveal, uses it to guide later episodes, and can revise it as feedback accumulates. It adds the capability to *Evolve*, shifting the emphasis from completing a task to learning across episodes; Figure 1 presents *Know*, *Know + Act*, and *Know + Act + Evolve* as cumulative capability emphases. The paradigm echoes Kolb (1984)’s *Experiential Learning Theory*, which connects practice, reflection, and further experimentation, much as agents turn trajectories into guidance and revise it through feedback. The experience-driven agents built on this paradigm are the subject of this survey.

Research on turning trajectories into reusable experience remains fragmented across reflection, agent memory, workflow learning, and skill acquisition (Shinn et al., 2023; Hu et al., 2025b; Wang et al., 2025e; Zhou et al., 2026c), lines that have pursued precedents, lessons, and procedures in parallel since 2023 (Figure 3). They often describe similar content in different terms: a distilled lesson is called a self-reflection in Reflexion (Shinn et al., 2023), an insight in ExpeL (Zhao et al., 2024a), a causal abstraction in CLIN (Majumder et al., 2024), and a memory item in ReasoningBank (Ouyang et al., 2026). Conversely, one name can cover different content: a memory entry may hold a fact, a diagnostic precedent, or a procedure, and a skill may wrap a single action or an entire workflow. Because names and content do not line up, it is hard to see which guidance a method retains and when that guidance applies, and hence to compare mechanisms and reported gains. The field lacks a common account of experience itself, which leaves three basic questions open: what counts as experience, how it should be accumulated, used, and revised, and how to tell whether it improves later behavior. Existing surveys cover much of this literature, but they organize it around memory (Hu et al., 2025b; Luo et al., 2026), self-evolving systems in which memory is one component among several (Gao et al., 2026a; Fang et al., 2025; Jiang et al., 2026a), or skills followed through closely related lifecycle stages (Zhou et al., 2026c; Wu and Zhang, 2026) (Table 6). This survey addresses the three questions with a unified framework that describes every retained unit, whether a memory entry, a workflow, or a skill, by the same analytical dimensions, so that methods storing experience differently can be compared directly by what they retain.

We define *agent experience* as action-guiding information grounded in task trajectories and retained to guide later episodes where it applies. It can take three forms: a trajectory retained as a precedent (Zheng et al., 2024), a semantic lesson (Shinn et al., 2023; Zhao et al., 2024a), or a reusable procedure such as a workflow (Wang et al., 2025e). The source trajectories may come from the agent, other agents, or human practice, including practice distilled into authored skills. The survey centers on experience kept in an explicitly accessible form, so that the guidance can be inspected and revised, and also covers how such explicit experience serves as a training signal for internalization into model parameters. Learning that updates parameters directly from

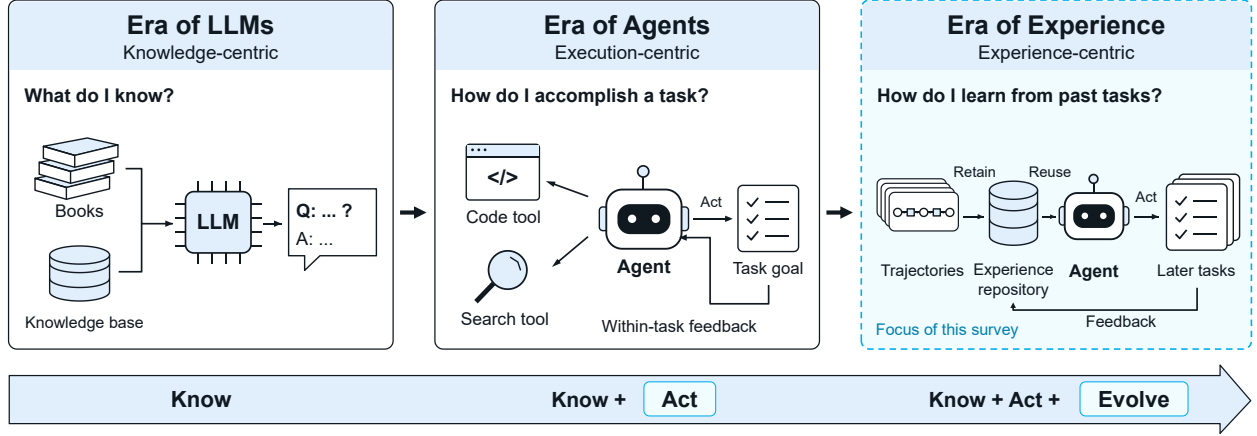


Figure 1 Three capability emphases, from LLMs to experience-driven agents. In the Era of LLMs, models answer questions by drawing on and reasoning with knowledge (*Know*). In the Era of Agents, they move beyond dialogue to complete tasks by acting in environments (*Know + Act*). In the emerging Era of Experience, agents also retain what their trajectories reveal, reuse it in later tasks, and can revise it as feedback accumulates (*Know + Act + Evolve*). Each emphasis keeps the capabilities of the previous one and adds to them; the highlighted panel marks the focus of this survey.

trajectories, without passing through explicit experience, lies outside the core scope, as do stored records that guide no action and instructions with no task practice behind them (Sections 2.1 and 2.6).

At runtime, an *experience-driven* agent consults experience retained from past trajectories to guide the current episode; an *experienceless* agent does not, and may repeat diagnostic work it has already done. Figure 2 illustrates the difference with a recurring dependency error. After tracing an import failure to a missing dependency and checking the repair, the agent stores a lesson in an *experience repository*, a persistent store of retained experience, and the lesson guides dependency checks in a later episode. Experience-driven agents also differ in whether their retained experience changes during later work. In *fixed-repository reuse*, the repository stays fixed; in *closed-loop learning*, the agent forms new experience from later trajectories or revises existing experience, and the updated repository guides subsequent episodes. We call this cycle the *experience loop*; Section 2.3 formalizes the two modes.

To make different forms comparable and to explain how experience is accumulated, used, and revised, our framework first describes what any retained experience contains and then follows it through its lifecycle. The description covers four analytical dimensions: the guidance the experience provides, the applicability conditions under which it holds, the execution evidence that supports it, and the reliability evaluation of that support. These dimensions apply to precedents, lessons, and procedures alike and keep them comparable even when a method records only some of them (Section 2). The *experience lifecycle* consists of four functions that determine how experience is produced, made accessible, used, and revised. **Formation** selects or derives candidate precedents, lessons, and procedures from trajectories. **Organization** structures and links retained experience to support access. **Utilization** determines when and which experience to use, adapts it to the current task, and applies it as guidance or as a training signal for internalization. **Maintenance** governs admission, consolidation, refinement, and retirement as evidence accumulates. Sections 3–6 review the four functions in turn, including methods that implement only some of them.

To tell whether experience improves later behavior, we match each reported gain to the improvement claim that its experimental design can support (Section 2.5). The three claims require increasingly demanding evidence. *Fixed experience reuse* attributes a gain to experience formed before testing and left unchanged during it, which a matched comparison with and without that experience can establish. *Feedback-driven sustained improvement* requires that repeated updates from new trajectories keep adding capability relative to a copy whose starting repository stays frozen. *Improvement of the learning mechanism* requires that a lasting change in how the agent forms or evaluates updates improves later learning relative to keeping the original mechanism; both of these comparisons need matched tasks and comparable budgets. Section 7 develops

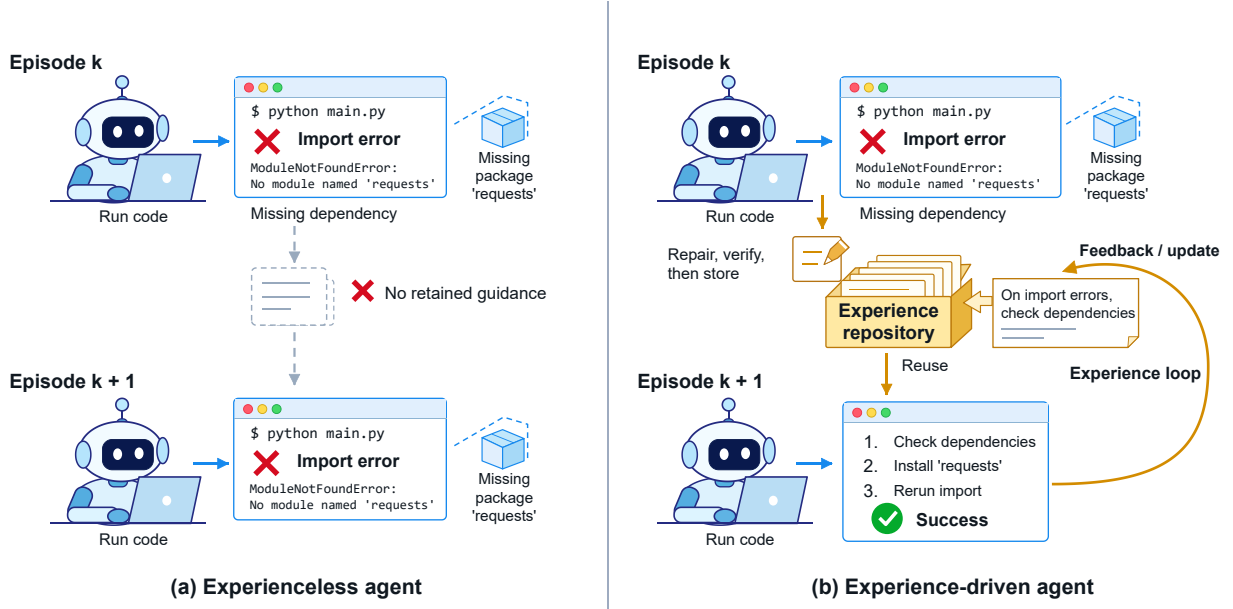


Figure 2 An experienceless agent versus an experience-driven agent. In episode k , both agents run code that fails with an import error because the `requests` package is missing. (a) The experienceless agent retains no guidance from this failure and repeats the same mistake in episode $k+1$. (b) The experience-driven agent repairs the failure, verifies the fix, and only then stores the lesson “on import errors, check dependencies” in its experience repository. In episode $k+1$ it reuses the lesson to check dependencies, install the missing package, and rerun the import, avoiding the earlier diagnostic detour. In closed-loop learning, the outcome of each new episode also feeds back through the experience loop to update the repository.

these comparisons alongside evaluation of intrinsic experience quality, the experience utilization process, and downstream effects on later tasks relative to cost. The clearest current evidence supports fixed experience reuse, mostly on tasks whose outcomes tests or reference answers can check; evidence for the other two claims remains partial.

Across the lifecycle, the analysis leads to one design principle: from formation through maintenance, keep guidance connected to its applicability conditions and to the evidence that supports or challenges it. Two findings about experience reuse motivate this principle. First, condensing experience can remove applicability conditions: shortening “if an import fails because a required package is missing, install that package” to “if an import fails, install the package” keeps the action but drops its condition, making the lesson inappropriate for an import-path error (Sections 3 and 5.3). Second, shared experience must fit the receiving agent as well as the task: LEGOMem’s results depend on which agent roles receive memory (Han et al., 2026a), and memory from a more capable teacher is not always more helpful (Kim et al., 2026b) (Sections 5.2.3 and 5.3.1). Two further findings call for checks beyond the principle. Third, adversarial tasks can induce experience entries that are judged harmless individually but elicit harmful behavior when used together, so combinations of entries need checking (Yan et al., 2026a). Fourth, rising scores over a task stream can reflect easier or repeated tasks as well as learning, so longitudinal evaluation must account for changes in the task mix (Section 7.5). These findings rest on mechanism analysis and evidence from individual systems; few studies vary one design choice while holding the others fixed.

By following experience from its source trajectories through reuse and revision, this survey makes four main contributions:

- We provide a unified framework for comparing experience across representations and storage formats, and for tracing failures to how experience was formed, accessed, applied, or revised.
- We specify the comparisons needed to support each improvement claim and locate the sources of gains, distinguishing published evidence from proposed tests and identifying what remains unestablished.

- Across seven application settings, we relate task recurrence, feedback, outcome checks, and environmental change to decisions about whether reuse justifies its cost, what to retain, how strongly it should guide action, and when to revise it (Section 8).
- We develop an agenda of eight open problems, each paired with a research direction and the evidence needed to show progress toward improvement that is *cumulative*, *transferable*, and *correctable* (Section 9).

2 Foundations of Agent Experience Learning

To understand how agents learn from past trajectories, we first ask three questions: what counts as agent experience, how it guides later behavior, and what evidence shows that it improves the agent. We use one code-repair episode to define experience and distinguish its forms and analytical dimensions, then explain how agents reuse and revise it. These operations lead to the four lifecycle functions reviewed in the following sections. Finally, we distinguish the evidence required for different improvement claims and use the definitions to delimit the survey’s scope.

2.1 Definition of Agent Experience

Consider again the simple failure in Figure 2. An agent’s tests fail at the import stage; the agent traces the failure to a declared dependency missing from the active environment, restores the dependency, and confirms that the import now succeeds. What should the agent keep from this episode? The same trajectory can serve as an audit log, as a record of what was installed, or as guidance for fixing the next failure. This subsection shows the forms such guidance can take, derives a definition from them, and explains why the other two uses fall outside it.

Table 1 shows three forms of experience that the agent can retain from this episode: a precedent to consult, a conditional lesson, and a checking procedure. Each row separates what is saved from how it guides later troubleshooting. The lesson shown in Figure 2, “on import errors, check dependencies,” is a compressed version of the conditional lesson in the table. The episode supports checking dependencies in comparable failures; it does not show that every import failure has the same cause. Section 2.2.2 develops the three representational forms, and Section 2.4 returns to the risk of over-generalization.

Table 1 Three forms of reusable experience from one code-repair episode: the agent traces an import failure to a missing dependency, restores it, and verifies that the import succeeds. Each form retains guidance for later troubleshooting.

Experience form	What is retained	How it guides later tasks
Trajectory-level experience (precedent)	A diagnostic example: the import error, relevant environment details, dependency checks, repair action, and successful import check.	Consult how a comparable failure was diagnosed and fixed to choose the next diagnostic step.
Semantic-level experience (conditional lesson)	When tests fail during import in a comparable setup, check declared dependencies and the active environment before changing application logic.	Prioritize an environment check without assuming that the new failure has the same cause.
Procedural-level experience (checking procedure)	Inspect the import error; compare declared dependencies with the active environment; resolve a confirmed mismatch within task permissions; rerun the import check.	Follow the ordered steps, using current observations to decide whether a repair is warranted.

All three forms in the table derive from a task trajectory and preserve guidance beyond the episode that produced it. We define *agent experience* as **action-guiding information that is grounded in one or more task-execution trajectories and retained to guide later episodes where it applies**. Here an *episode* is one bounded attempt at a task; Section 2.2 states its boundaries. The definition has three requirements: *execution grounding* (the information derives from task-execution trajectories), *cross-episode retention* (it is kept beyond the episode that produced it), and *behavioral relevance* (its role is to shape later behavior). Correctness and

benefit are not required; later sections treat them as properties to check. Behavior here covers interpreting a question, producing an answer, and evaluating a solution, as well as acting in an environment. The survey focuses on experience kept in an explicitly accessible form; learning solely through parameter updates is a neighboring direction (Section 2.6).

Grounding concerns where the information comes from, not who writes it down or how it is phrased: a person may record it, and a model may rewrite it. A reflection, a compressed trace, or a synthesized example is grounded when it is derived from particular trajectories, although derived content is not itself an observed result. ECHO, for example, rewrites a failed attempt in hindsight into a trajectory for a goal that the attempt could have reached and stores it without running it (Hu et al., 2025a). By contrast, BREW reruns the agent on the relabeled task, so its trajectories are observed rather than only derived (Kirtania et al., 2026). Moreover, the trajectories need not be the agent’s own. A skill or checklist that people write down from their own work on a task is experience they formed from their trajectories, even when those trajectories are unavailable to the agent. Grounding excludes only guidance with no task practice behind it, such as a generic instruction or a tool’s interface documentation. How strongly grounded guidance is supported is a separate question for reliability evaluation (Section 2.2); for authored skills, that support may rest only on the authors’ practice rather than on inspectable trajectories.

Behavioral relevance concerns the role of retained information, not its storage format. A trajectory archived only for audit, or an observation kept only to establish which dependency was previously missing, falls outside the survey’s core because it is not retained as guidance for later tasks; the same record can serve as experience if it is retained as a precedent for how to diagnose and repair a comparable failure. For a published method, this role must be read from how the study reports obtaining, keeping, retrieving, and using the information. When a paper reports only that information is stored or retrieved, that wording alone leaves open whether it is retained as guidance for later behavior.

2.2 Core Terms, Forms, Dimensions, and Types of Experience

The same repair episode can leave the agent with a diagnostic precedent, a conditional lesson, or a checking procedure (Section 2.1). To compare these forms of experience, we need to describe what guidance each retains, when it applies, and what supports it. We first define the basic terms, then distinguish how guidance is expressed (representational forms), what information describes an experience unit (analytical dimensions), and which outcomes its guidance draws on (experience types). Section 2.4 then examines how a system forms, organizes, uses, and maintains experience.

2.2.1 Episodes, Guidance, and Carriers

Because experience must outlive the episode that produced it, we need to state where an episode ends: with completion, explicit termination, or an exhausted budget. Repeated edits to a patch within one attempt belong to one episode; restarting the task begins another. Reuse across episodes therefore does not require a different task. In addition, experience may be formed before an episode ends, for example from a completed subtask, as long as it is kept for later episodes. A correction discarded together with the attempt is not experience.

The action-facing part of experience is its *guidance*: what to do, avoid, or check, or which precedent to consult. A *lesson* is guidance stated as a sentence distilled from trajectories; precedents and reusable procedures can also guide behavior without first being rewritten as lessons. A *trajectory* records the execution that supplies the evidence. A *carrier*, such as a saved trace, a text entry, a workflow, or a program, is where the experience is stored or exposed. One carrier may hold several pieces of experience, and the same guidance can appear in different carriers. In the repair episode, the trajectory records the import error, the dependency check, the installation, and the rerun import check; the guidance is to check declared dependencies and the active environment first; and a saved trace, a text entry, or a checking script can each carry that guidance.

2.2.2 Three Representational Forms

We distinguish **trajectory-level**, **semantic-level**, and **procedural-level** experience, matching the three rows of Table 1. Trajectory-level experience keeps a trajectory, or a segment of it, together with its context as

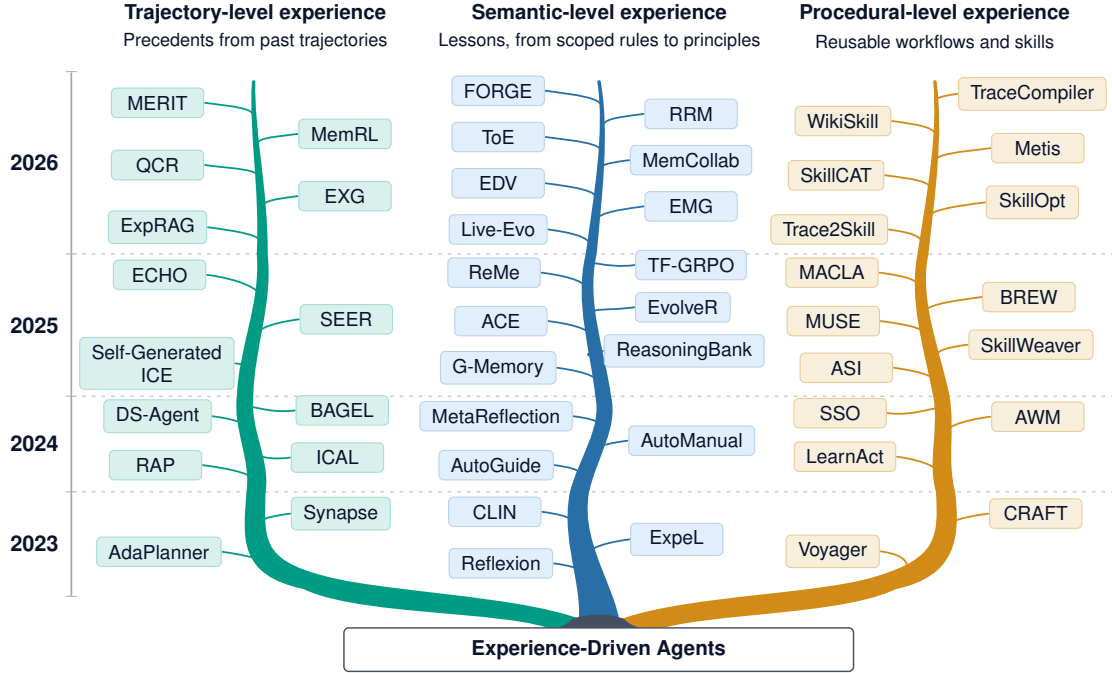


Figure 3 Representative experience-driven agents by the principal form of experience they retain and their year of first release. All three forms have developed side by side since 2023: agents reuse past trajectories as precedents (e.g., Synapse), distill them into semantic lessons (e.g., Reflexion, ExpeL), or package them as reusable workflows and skills (e.g., Voyager, AWM). Branches group methods by form and do not indicate lineage; several systems retain more than one form. ICE: in-context examples; SSO: Skill Set Optimization; TF-GRPO: Training-Free GRPO.

a *precedent* for comparable situations; precedents placed in the prompt as demonstrations are often called in-context examples or exemplars. Semantic-level experience states a lesson that can be understood without replaying the source episode; lessons range from scoped rules that state when they apply to broader principles that hold across tasks. Procedural-level experience organizes behavior into reusable steps, dependencies, parameters, or executable calls.

Form describes only how guidance is expressed, independent of file format. The names echo the episodic, semantic, and procedural long-term memory of CoALA (Sumers et al., 2024), which instead divides memory by module and content: its semantic memory also holds world knowledge read from external sources, and its procedural memory consists of model weights and agent code. Any form can be built directly from trajectories, and several forms can coexist, so the levels are not stages of maturity. Nor do they rank generality: a scoped lesson can apply as narrowly as the precedent it came from. A compressed episode remains trajectory-level, and a workflow written in natural language is procedural-level. A unit of any form can cover a whole task, a subtask, or a single step, so a precedent may be a full episode or one segment, and a procedure may wrap one action or an entire workflow. Granularity affects which situations a unit matches and which agent should receive it, as the LEGOMem example in the introduction shows; Section 4 examines structures that link coarse and fine units, such as a routine and its constituent steps. Section 3 organizes formation mechanisms around these forms and their connections.

Figure 3 groups representative methods by their principal experience form and year of first public release. They serve as concrete entry points into the method sections; Section 2.6 explains how studies were selected. Many systems, such as Voyager, ExpeL, ReasoningBank, ReMe, and AWM, handle several parts of the lifecycle and therefore appear in several sections; a gain measured on such a full system usually needs additional controls before it can be attributed to one part.

2.2.3 Analytical Dimensions

Whatever its form, a piece of experience, which we call an *experience unit*, can be described along four dimensions:

$$E_i = \langle g_i, c_i, e_i, r_i \rangle, \quad (1)$$

where *guidance* g_i is what the unit tells the agent to do, avoid, check, or consult; *applicability conditions* c_i state the task, environment, tool, and executor assumptions under which it applies; *execution evidence* e_i points to the trajectories, or parts of them, that support it and to any counterexamples; and *reliability evaluation* r_i summarizes how strong that support is and how uncertain it remains, not necessarily as a calibrated probability. Execution evidence is not a separate kind of record: the same trajectory serves as evidence for each unit it supports. A system need not store the four dimensions as fields; they are questions to ask about any unit. A unit without recorded evidence or reliability, such as a skill written from its authors’ practice, is harder to assess, but it still counts as experience if it is grounded in trajectories. All four dimensions can be revised as evidence accumulates.

Reliability asks whether the guidance is well supported under its stated conditions. *Utility* asks whether using it pays off for a particular executor and task, given its costs and the other experience available. Consequently, utility depends on context and is not a fifth fixed attribute of the unit. The distinction recurs in utilization, maintenance, and evaluation (Sections 5–7).

For the conditional lesson in Table 1, guidance g_i is to check declared dependencies and the active environment before changing application logic. Conditions c_i are an import-stage test failure, a comparable setup, and permission to inspect the environment. Evidence e_i consists of the error, the inspection, the repair, and the passing import check. After a single episode, reliability r_i remains provisional: the lesson supports a diagnostic check, not a general explanation of import failures. If the executor already performs this check unprompted, the lesson may be reliable yet add little utility.

The four dimensions can also be read off units that published methods report. Table 2 applies them to four such units: a trajectory precedent, a search lesson, an agent-built mining procedure, and a human-authored computer-use skill. The guidance and reported checks are taken from the papers; conditions described as implicit follow from the interfaces or code they use. The final column identifies what each study’s experiments support, without ranking scores from different benchmarks.

Three conclusions follow from this comparison. First, more explicit guidance does not ensure more explicit applicability: the precedent needs a comparable interface, the lesson assumes another search is possible, and the procedures still depend on inventory or environment conditions. Conditions must therefore be examined alongside guidance in every form.

Second, reliability signals answer different questions. A successful source trajectory, model agreement on a lesson, a construction-time success check, and a completed authored-skill example each provide support, but none alone estimates how often that unit will help in a new situation.

Third, the origin or sophistication of a unit does not determine the improvement claim. Human-authored and agent-built guidance can both support fixed experience reuse. Showing that further updates add capability requires the longitudinal comparisons in Section 7.5, which none of the four studies reports.

2.2.4 Common Types of Experience

Besides its form, experience can be classified by the execution outcome its guidance draws on, because that outcome determines what the guidance can tell the agent: what worked, what went wrong, what distinguished the two, or how an error was fixed. We call the corresponding types *success*, *failure*, *comparative*, and *corrective* experience. Type and form are independent: each type can be kept as a precedent, a lesson, or a procedure, and one episode can supply several types. The repair episode’s initial import failure, the fix that resolved it, and the passing rerun supply failure, corrective, and success experience. Comparative experience also needs a comparable attempt that went differently, such as an earlier attempt at a similar import failure that changed application code first and still failed. Table 3 shows the guidance each type can supply in this example, alongside representative methods and the main caution for each type.

Table 2 Four reported experience units through guidance and applicability (g_i, c_i), supporting evidence and reliability (e_i, r_i), and the improvement claim supported by the study. Unit-level checks and system-level results have different scopes.

Method and unit	Guidance and conditions	Evidence and reliability	Supported improvement claim
Synapse: terminal precedent (Zheng et al., 2024)	Consult a successful trajectory that lists files before deleting one with the requested extension. Implicit: a compatible terminal interface and a comparable task.	The abstracted observation–action trajectory retains the demonstrated steps and successful outcome. Selection uses task metadata; success on the source task provides no calibrated transfer estimate.	Fixed experience reuse: memory and exemplar ablations test the reusable set, without isolating this precedent’s contribution.
ExpeL: search lesson (Zhao et al., 2024a)	Vary search keywords when initial attempts are inconclusive. Stated: an unhelpful search; implicit: access to another keyword search.	Extraction reads successful groups or success–failure pairs, but the lesson is not linked to particular supporting trajectories. Its vote-and-edit count records model agreement.	Fixed experience reuse: held-out comparisons vary the frozen trajectory pool and insights; they do not test later updates.
Voyager: mining procedure (Wang et al., 2024a)	Equip an iron pickaxe, find cobbled deepslate below $y = 0$, and mine ten blocks. The code requires the pickaxe and constrains the search; reachable blocks and helpers are implicit.	Construction uses program runs and environment feedback; admission follows GPT-4’s success check. The description-indexed skill has no explicit source-trajectory link or calibrated reuse estimate.	Library ablations and new-world tests support reuse and transfer. Exploration curves leave the contribution of successive updates unisolated.
CUA-Skill: authored folder-creation skill (Chen et al., 2026f)	Navigate to a destination in Windows File Explorer and create a named folder. Arguments specify path and name; an accessible, writable destination is implicit.	Authors report encoding human computer-use knowledge. The published Downloads/Logs example completes the task, but no human source trajectories or per-skill reliability estimate are provided.	Fixed experience reuse: fixed-library comparisons and aggregate executability checks support skill use; they do not test agent-side formation.

Table 3 Four common types of experience, distinguished by the execution outcome they draw on and illustrated with the code-repair episode of Table 1; the comparative row adds an earlier attempt that changed application code first. Each type can be retained as a precedent, a lesson, or a procedure; the third column gives one example of the guidance each type can supply.

Type	Evidence drawn on	Guidance in the repair example	Main caution	Examples
Success	A successful trajectory	Restore a missing declared dependency in the active environment, then rerun the import check.	Success does not show which steps were necessary, and its label may be wrong.	AWM (Wang et al., 2025e) Voyager (Wang et al., 2024a)
Failure	A failed trajectory, with its feedback	Confirm that declared dependencies are installed before running tests.	A diagnosis that blames the wrong step yields an overly broad prohibition.	Reflexion (Shinn et al., 2023) ReasoningBank (Ouyang et al., 2026)
Comparative	Successful and failed attempts at comparable tasks	An earlier attempt that changed application code first still failed, whereas this one checked the environment first and succeeded, so check the environment before changing application logic when imports fail in a comparable setup.	Unless attempts are aligned, a difference may reflect the environment or task difficulty.	ExpeL (Zhao et al., 2024a) AutoGuide (Fu et al., 2024)
Corrective	An error linked to the change that resolved it	An import error for <code>requests</code> was fixed by installing the declared package into the active environment.	The same error may have a different cause in a new setting.	EXG (Jin et al., 2026)

How far each type can be trusted depends on the outcome evidence it draws on, so each needs a different check before its guidance is relied on (the *Main caution* column of Table 3). Success experience needs a

trustworthy success label, failure experience a correct diagnosis, comparative experience genuinely comparable attempts, and corrective experience a recorded link between an error and the change that resolved it. Many systems combine types: ExpeL, ReasoningBank, and ReMe (Cao et al., 2026) each draw on successes, failures, and comparisons. Section 3.1 discusses how such evidence is collected, and Section 3.3 how reflection and comparison turn it into lessons.

2.3 Formalization of Experience-Driven Agents

An agent can consult a retained lesson during a new task and use the resulting trajectory to revise what it keeps. Reusing experience and updating it are distinct operations: the first can occur while the repository remains fixed, whereas the second changes what is available for later tasks. We use *agent experience learning* for the recurring process of deriving, retaining, reusing, and revising experience. Building on the contrast in Figure 2, we formalize how an agent interacts with its environment, how experience guides its decisions, and how new trajectories lead to repository updates. Table 4 collects the notation.

2.3.1 Agent Interaction and Memory Roles

An LLM agent combines a language model with tools and memory to pursue a task (Yao et al., 2023; Wang et al., 2024b). The *executor* is the agent whose behavior the retained experience is meant to guide. Its model, tools, and surrounding program determine what it can already do, and therefore which guidance can help it. Let k index episodes and t steps within an episode. At step t of episode k , the agent uses the current observation $o_{k,t}$ and its memory to select an action $a_{k,t}$, and the environment responds with a new observation. The agent need not observe the full environment state, and execution may be stochastic.

Following the functional taxonomy of Hu et al. (2025b), the memory that the agent draws on plays three roles: factual memory holds declarative information, working memory holds the active context, and the *experience repository* R_k , a collection of retained experience units (Equation 1), supplies trajectory-derived guidance. These are roles, and one store can play more than one; retrieved facts and experience both enter the working context alongside observations and intermediate reasoning. Only the repository needs a symbol in what follows, and it carries no step index because the basic formulation changes it only between episodes.

For a task request q , the trajectory of an episode with n actions is

$$T = (q, \mathbf{h}, f), \quad \mathbf{h} = (o_0, a_0, o_1, a_1, \dots, o_{n-1}, a_{n-1}, o_n), \quad (2)$$

where $\mathbf{h}$ records the interaction history and f the outcomes and feedback available; feedback is distinct from the task output y , such as a proposed patch. Feedback can come from the environment, such as a reward or a changed state; from tests or other verifiers; from the agent’s own judgment or that of another model; or from people. Some of these sources check an outcome more independently than others. In particular, a self-assessment is widely available but can mislabel an attempt, and experience formed from a wrong label inherits the error. Section 3.1 examines how formation connects outcome labels to observed results.

2.3.2 Experienceless and Experience-Driven Agents

An *experienceless agent* may retain facts and correct errors within an attempt; however, it does not systematically reuse explicitly retained experience across episodes. Such an agent can still change persistently through additional training such as fine-tuning. Reflexion and ExpeL motivate retaining experience rather than fine-tuning the model, noting that fine-tuning requires extensive training samples and computation, and that the weights of many models accessible only through APIs are unavailable (Shinn et al., 2023; Zhao et al., 2024a).

An *experience-driven agent* uses retained experience to guide behavior across episodes. With a fixed repository it performs *fixed-repository reuse*; when it also forms and maintains experience from new trajectories, it performs *closed-loop learning*. A closed-loop agent need not retrieve experience or change its repository in every episode. Many studies build a repository *offline*, from training tasks or recorded demonstrations, and then freeze it for evaluation, which is fixed-repository reuse at test time. In an *online* setting, the agent keeps forming and updating experience while it works through the test tasks, which is closed-loop learning; AWM, for example, reports both settings (Wang et al., 2025e).

The distinction between experienceless and experience-driven agents concerns their behavior at run time. An executor trained through experience internalization (Section 5.5) and then run without supplied guidance is therefore experienceless at run time, even though explicit experience shaped its training. Its gain from that experience is measured by a training-time comparison rather than by run-time access (Section 2.5).

2.3.3 Situated Reuse and the Experience Loop

An experience-driven agent does two different things with retained experience: during an episode it turns retained units into guidance for the current decision, and in closed-loop learning it also updates the repository from the resulting trajectory. We formalize the two steps in turn. During episode k , retained experience is turned into guidance for the current decision:

$$A_{k,t} = \text{Utilize}(R_k, q_k, \mathbf{h}_{k,\leq t}). \quad (3)$$

Here $\mathbf{h}_{k,\leq t} = (o_{k,0}, a_{k,0}, \dots, a_{k,t-1}, o_{k,t})$ is the interaction history available at the current decision, including the current observation. Utilize selects applicable experience and adapts it to this history; other working context, including retrieved facts and intermediate reasoning, is an implicit input. It returns $\emptyset$ when nothing applies. The resulting *situated guidance* $A_{k,t}$ can help the agent interpret the task, generate an answer, make a decision, reuse a procedure, or evaluate a result. Three objects should be kept apart: $E_i \in R_k$ is a retained unit, g_i its guidance content, and $A_{k,t}$ the guidance supplied for one decision. Producing $A_{k,t}$ does not by itself retain a new unit. Fixed-repository reuse and closed-loop learning both use this step; an experienceless agent has no such path, so $A_{k,t} \equiv \emptyset$.

Closed-loop learning adds a path from execution back to the repository:

$$\begin{aligned} (y_k, T_k) &= \text{Execute}(q_k, R_k), \\ \mathcal{E}_k &= \text{Form}(T_k, R_k, \mathcal{H}_{<k}), \\ R_{k+1} &= \text{Update}(R_k, \mathcal{E}_k, T_k). \end{aligned} \quad (4)$$

Execute runs the agent in its environment, drawing on situated guidance at each step, and returns output y_k and trajectory T_k . Form uses the new trajectory, available earlier trajectories $\mathcal{H}_{<k}$, and optionally existing experience to select or derive $\mathcal{E}_k$, the possibly empty set of *candidate* units from episode k . Each candidate has the same form $\langle g, c, e, r \rangle$ as a retained unit in Equation 1; it differs only in status: it has not yet been admitted. The subscript i in E_i identifies a unit, whereas k identifies an episode. Update decides which candidates in $\mathcal{E}_k$ to admit into R_{k+1} and may also revise, merge, reweight, or retire existing experience, producing R_{k+1} .

The executor need not run Form and Update itself. An agent can reflect on its own attempts, but another model, such as a more capable teacher, can also form experience, and experience formed with one executor can be offered to another. Such transfer is not automatic; Section 5.2.3 compares systems that share or transfer experience between agents.

As experience accumulates, earlier tasks can influence how the agent handles later ones. The ordered sequence of tasks $q_1, q_2, \dots$ that an agent receives forms its *task stream*; in closed-loop learning, what the agent retains, and therefore how it performs, can depend on task order as well as content (Section 7.5). The repository may start empty, and the agent may decline to reuse anything or leave $R_{k+1} = R_k$ when no update is justified. Fixed-repository reuse keeps R_k unchanged throughout the stream.

Equation 4 is a simplification: it changes the repository only between episodes, whereas implementations may also form, admit, and reuse experience during an attempt. Feedback may be partial or delayed; a forecast, for example, is submitted long before its outcome is known. Such later evidence can trigger an update without a new task, provided it can be linked back to the trajectory it concerns. Section 8.7 develops this case.

Table 4 Core notation. k indexes episodes and t steps within an episode; indices are omitted when only one episode is discussed.

Symbol	Meaning
q, y	Task request and produced output.
o_t, a_t	Available observation and selected action.
$\mathbf{h}, f, T$	Interaction history, available feedback, and trajectory $T = (q, \mathbf{h}, f)$.
$\mathbf{h}_{k, \leq t}$	History of episode k through the current observation at step t .
$E_i = \langle g_i, c_i, e_i, r_i \rangle$	Experience unit i , candidate or retained: guidance content, applicability conditions, evidence, and reliability.
R_k, R_{k+1}	Experience repository, a set of retained units, before and after an episode-level update.
$\mathcal{H}_{<k}, \mathcal{E}_k$	Earlier trajectories available for formation; set of candidate units formed in episode k , not yet admitted.
$A_{k,t}$	Situated guidance supplied from retained experience for the current decision, possibly empty.

2.4 Lifecycle of Agent Experience

Making the experience loop work requires four kinds of decisions: what to learn from a trajectory, how to store it so that it can be found, how to use it to guide behavior or train the executor, and what to keep or change as evidence accumulates. We call the corresponding functions *formation*, *organization*, *utilization*, and *maintenance*. They echo Kolb’s learning cycle from the introduction only in part: trajectories play the role of concrete experience, formation corresponds to reflective observation and abstract conceptualization, and utilization applies the result to new situations, as active experimentation does. Organization and maintenance have no counterpart in that cycle, which describes a learner who carries experience internally; experience retained outside the learner must instead be made findable and kept correct explicitly. Figure 4 shows how the functions connect during runtime reuse, and Table 5 summarizes each function.

The functions pass a unit of experience between them: formation derives a lesson, maintenance decides whether the evidence suffices to admit it, organization makes it findable, utilization applies or skips it, and maintenance later narrows, revises, or retires it as exceptions appear or its added benefit fades. The same division helps locate a problem. For the dependency-checking lesson of Section 2.1, a version claiming that every import failure requires installing a dependency points to unsupported generalization during formation; a retained lesson missing from the lookup index points to organization; applying it despite incompatible environment assumptions points to utilization; and leaving it unchanged after new evidence contradicts it

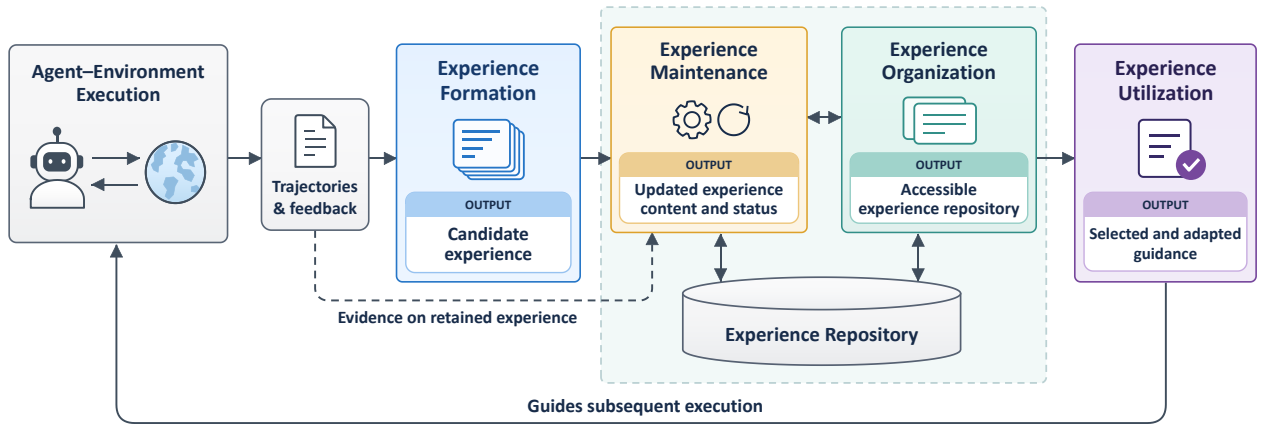


Figure 4 The agent experience lifecycle illustrated through runtime reuse. Formation derives candidates; organization structures access to the repository; utilization supplies guidance for execution; and maintenance decides which candidates to admit and which retained units to revise or retire. Arrows show information flow and feedback. Training-time internalization provides another utilization route (Section 5.5).

Table 5 The four lifecycle functions: the question each answers, its main responsibility, and its counterpart in Equations 3 and 4.

Function	Core question	Main responsibility	In the equations
Formation (§3)	What can be learned from past trajectories?	Select or derive candidate precedents, lessons, or procedures from trajectories, and check that the trajectories support them.	Form, producing $\mathcal{E}_k$
Organization (§4)	How should experience be stored so that it can be found and interpreted?	Index experience and link it to its use contexts, applicability conditions, and related units; runtime ranking belongs to utilization.	Access structure over R_k (not modeled); read by Utilize, revised in Update
Utilization (§5)	How should experience guide behavior or train the executor?	At runtime, decide when to seek experience, which units to use, and how to adapt them, then supply guidance or none; in training, use experience to shape trajectories and supervision.	Utilize yields $A_{k,t}$, applied in Execute; training route not modeled
Maintenance (§6)	What should be admitted, kept, revised, or retired?	Decide which candidates to admit, and revise, consolidate, or retire retained experience as evidence and conditions change.	Update, producing R_{k+1}

points to maintenance. These distinctions are especially useful for comparing methods that all report storing and retrieving experience: one may enhance extraction, another access, and another the correction of earlier guidance. Responsibilities can overlap; admitting the unsupported lesson also exposes a maintenance problem.

In practice, a system may combine several responsibilities in one operation and revisit them as feedback arrives. The four functions therefore need not correspond to separate modules or follow a fixed sequence. Whether this process improves later behavior is a separate question: retained experience may be irrelevant to a new task, introduce errors, or cost more to use than it saves. The next subsection distinguishes the improvement claims that evaluations of this process can support.

2.5 Experience-Driven Self-Improvement

Higher performance with experience does not by itself show that an agent keeps improving. To tell which kind of improvement a result shows, we distinguish three questions: whether fixed experience helps on new tasks, whether learning from new trajectories produces sustained gains, and whether changes to the learning mechanism improve subsequent learning. Each question requires its own experimental comparison.

Drawing on self-improvement research (Zhang et al., 2026g,h; Liu et al., 2026c), we call the corresponding conclusions *fixed experience reuse*, *feedback-driven sustained improvement*, and *improvement of the learning mechanism*. An *improvement claim* states which of these improvements a study reports, and its evaluation must provide evidence for that conclusion. Fixed experience reuse is the least demanding of the three: it credits the benefit of existing experience, not the agent’s ability to improve itself. In contrast to the two modes in Section 2.3, which describe whether a system uses fixed-repository reuse or closed-loop learning, these claims concern what its evaluation establishes. Each claim below needs more demanding evidence than the one before.

- *Fixed experience reuse*: a fixed body of experience built from earlier trajectories can improve later tasks even if its contents never change. When the experience is supplied at run time, comparing the same agent with and without access to the repository, under the same conditions, can establish this benefit, for example by testing whether a frozen repository helps on new tasks of the kind it was built from. The same claim covers experience used for training rather than supplied at run time: when a frozen collection of experience trains the executor (Section 5.5), the matched comparison trains with and without it from the same initial

policy and tests both without the guidance (Section 7.4.4). Neither comparison shows that the agent keeps learning from later trajectories.

- *Feedback-driven sustained improvement*: the retained guidance is updated from new trajectories, and the updated guidance shapes later behavior and subsequent learning. Establishing sustained improvement requires evidence that multiple updates add capability over a predeclared interval of comparable tasks, relative to a copy that keeps its starting repository fixed. The advantage need not increase at every round: plateaus and local regressions are compatible with learning, provided that the comparison identifies further contributions from later updates. Merely retaining an earlier gain does not count as acquiring further capability. When the repository starts empty, the frozen copy is the same agent without experience. Acquisition, retention, regressions, resource costs, and uncertainty are reported separately under the evaluation standards in Section 7.5. An implemented feedback loop, a growing repository, or a higher final score alone does not establish this claim.
- *Improvement of the learning mechanism*: a system may also change how it forms or evaluates future updates, for example by revising the procedure that extracts lessons from failed repairs rather than only the lessons themselves. The change must persist into later learning rounds, and its contribution must be tested against keeping the original mechanism under comparable tasks and budgets. Updating repository contents under fixed formation and update procedures does not meet this criterion. Revising one component, such as an access policy or a memory-operation skill, changes only part of the learning mechanism; when a separate optimizer or a human designer makes that revision, the agent has not modified itself. A mechanism comparison shows what a revision changed and whether it helped, not who made it; a claim that the agent improved its own learning also needs a record of how the revision was produced.

Sections 7.4 and 7.5 develop these comparisons; Table 18 lists, for each claim, the necessary evidence separately from recommended designs and diagnostics, and Table 28 in Appendix D indexes the protocols that representative studies implement and their limits.

2.6 Survey Scope and Related Directions

Many cited systems combine experience with facts, logs, or parameter updates, so readers need to know which components later sections treat as experience and which serve as neighboring evidence. We derive inclusion criteria from the definition and lifecycle, explain how studies were selected and used as evidence, and distinguish the core survey from neighboring research. The literature search was last updated on October 8, 2026.

The core survey covers methods that address any lifecycle function for explicitly accessible experience. A method can address a single function, so formation-only mechanisms and fixed-repository reuse qualify. Inclusion depends on the execution grounding, cross-episode retention, and behavioral relevance defined in Section 2.1, not on reported gains, which we assess separately. The trajectories may come from the focal agent, other agents, or human work, including offline collections and skills that people author from their own practice. Because such authored skills are formed outside the system, they enter the method sections through organization, utilization, and maintenance rather than formation. Audit-only logs, factual or preference memory that does not guide behavior, and instructions with no task practice behind them fall outside the core. We also examine how explicit experience supplies training signals and is internalized into model or adapter parameters, which connects the core scope to parameter learning (Section 5.5).

This is a selective, mechanism-oriented survey rather than a comprehensive systematic review: it uses representative studies to explain how experience changes later behavior and how those effects are evaluated. Studies were selected for their fit to the inclusion criteria and for the distinct mechanism, comparison, or application constraint they clarify. Beyond known studies and their references, targeted web searches filled gaps in particular lifecycle functions and application settings, using queries about agent experience, memory access, workflow and skill reuse, and experience internalization. Each study these searches added was checked against the original paper and official publication pages. Because the searches supplemented the corpus instead of screening the whole field uniformly, later statements that a comparison is missing or that evidence is limited refer to the studies reviewed here. Examples given without a citation, such as the code-repair episode above, are our own illustrations.

Table 6 This survey in relation to recent neighboring surveys. The main-content column follows each survey’s own framing. Two differences apply to every row and are not repeated: this survey describes each experience unit by its guidance, applicability conditions, execution evidence, and reliability evaluation, and it separates fixed experience reuse, feedback-driven sustained improvement, and improvement of the learning mechanism by the evidence each needs. The last column gives the further difference specific to each survey.

Survey	Main content	Connection to this survey	Key difference
Agent memory			
Hu et al. (2025b)	Agent memory by forms, functions, and dynamics; experiential memory includes cases, strategies, skills, and hybrids.	We adopt its three functional roles (Section 2.3); its experiential memory overlaps all three forms here.	Experiential memory is one of its three roles; we analyze the units within that role.
Luo et al. (2026)	Evolution of agent memory in three stages: storing, reflecting on, and abstracting trajectories into experience.	Also centers on experience derived from trajectories, placing it as the most recent stage.	We organize by lifecycle function, not developmental stage; a trajectory kept as a precedent already counts as experience.
Self-evolving and self-improving agents			
Gao et al. (2026a)	Self-evolving agents, organized by what (models, memory, tools, architecture), when, how, and where to evolve.	Memory, which can hold experience, is one of the components that evolve.	We examine one evolving component, explicitly accessible experience, function by function.
Fang et al. (2025)	Self-evolving agentic systems as a loop of system inputs, agent system, environment, and optimizers, with domain chapters.	Both study how environment feedback changes later agent behavior.	It centers on the optimizer that changes the system; we center on the retained guidance.
Jiang et al. (2026a)	Post-deployment improvement through an external path (skills, memory, context, tools, environments), a parameter path, and meta-evolution.	Skills and memory on its external path overlap our core; meta-evolution relates to improving the learning mechanism.	We treat its parameter path as adjacent unless explicit experience supplies the training signal.
Agent skills			
Zhou et al. (2026c)	Agent skills as reusable procedural artifacts, organized by a lifecycle of representation, acquisition, retrieval, and evolution.	Its lifecycle stages parallel our functions; skills acquired from trajectories or authored from human practice are procedural-level experience.	It takes the skill artifact as its object; we analyze the guidance a skill carries, which can also be kept as a precedent or lesson.
Wu and Zhang (2026)	Agent skills as procedural memory, organized by acquisition, representation, invocation, and refinement, with multiple skill representations.	Its four stages closely parallel our lifecycle functions and cover overlapping representations.	We treat the skill as a carrier and compare the experience units it holds by their support.

Studies of memory and skills can help explain how experience is stored or used even when the material they examine does not meet our definition of experience. We distinguish three evidence roles to make clear what each study can support. *Direct experience studies* investigate experience or lifecycle functions that meet the core inclusion criteria. *Adjacent mechanism studies*, such as work on factual and conversational memory, inform organization, maintenance, or access without necessarily showing that trajectory-derived guidance carries over to later behavior. *Complementary diagnostics*, such as memory-to-action benchmarks and retrieval tests over crawled skill corpora, evaluate an interface or outcome that matters for experience, but on material not shown to meet these criteria. Tests of skills that people author from their own practice count as direct evidence for the utilization and downstream questions they examine; because the supporting trajectories of such skills are rarely recorded, these tests cannot examine formation or audit execution evidence. One study can play several roles through different components; Sections 4–7 mark the role where it affects interpretation.

For authored skills, a paper usually reports only where the skills came from; treating them as practitioner experience is our reading. Direct[†] marks the four studies whose tests of skill use we count as direct evidence on that reading; the mark does not certify that each skill came from practice. Human authorship or public

Table 7 Agent experience learning in relation to neighboring research directions. One system can belong to several directions, since the boundaries separate mechanisms and the evidence about them.

Direction	Main content	Connection to agent experience	Key difference
Mechanisms that experience builds on			
Acting and reflection	Interleaving reasoning with actions (Yao et al., 2023), and reflecting on failed attempts in words (Shinn et al., 2023).	Acting produces the trajectories and feedback that experience comes from; reflection is one way to turn them into lessons.	Solving or correcting a task within one attempt is not experience; a reflection counts only when kept to guide later episodes.
Memory and context construction	Storing and retrieving information for agents (Chhikara et al., 2025), and supplying it through RAG (Lewis et al., 2020) or in-context examples (Brown et al., 2020).	Provides the storage, retrieval, and context interfaces through which experience is organized and used.	The content may be facts, preferences, or documents; it is experience only when drawn from trajectories and kept to guide action.
Skills and workflows	Reusable procedures, such as workflows induced from past runs (AWM; Wang et al., 2025e) or skills learned through exploration (Voyager; Wang et al., 2024a).	Procedural-level experience often takes this form.	Skills that people write from their own practice count as experience, but a procedure specified without any task practice does not; conversely, experience can be a precedent or lesson that is not executable.
Related learning paradigms			
Case-based reasoning	Solving new problems by retrieving, reusing, revising, and retaining past cases (Aamodt and Plaza, 1994).	The closest conceptual precedent: past cases guide new problems and are revised as they are reused.	The principle is shared; we extend the scope to LLM agents and add semantic lessons and procedures to cases.
Training and continued learning	Improving agents over time from execution feedback, by updating parameters (Luo et al., 2025) or by revising retained guidance, as in CLIN (Majumder et al., 2024) and ACE (Zhang et al., 2026i).	Revising retained guidance is experience learning; experience internalization (Section 5.5) connects it to training.	Learning solely through parameter updates is adjacent even when it uses trajectories; reusing a fixed repository does not show sustained improvement.
Reinforcement learning	Experience as interaction data, such as state transitions, replayed to update a policy or value function (Lin, 1992); agents learning mainly from their own interaction (Silver and Sutton, 2025).	Shares the term and the idea that agents improve from their own interaction.	Our experience stays explicitly accessible guidance that can be inspected, applied, and revised without changing parameters.

availability alone is insufficient to establish that origin. Appendix D records each reported source and the basis and limits of our reading; automatically collected corpora with unestablished authorship and practice origin remain complementary diagnostics.

Agent experience learning shares mechanisms with several neighboring directions, and the boundary between them lies mainly in what the retained information is: where it comes from, whether it persists across episodes, whether it later guides action, and whether adaptation changes accessible guidance or only model parameters. Table 7 draws this boundary for acting, reflection, memory, skills, case-based reasoning, training, and reinforcement learning, stating for each its connection to agent experience and the key difference. Reinforcement learning needs particular care, because it shares our central term but uses it for interaction data that updates parameters. Table 6 lists the recent surveys compared in the introduction.

Some methods train a model to generate guidance on demand instead of retaining it. MemGen (Zhang et al., 2026b) supplies latent token sequences to a frozen reasoning model and so provides no inspectable precedents, lessons, or procedures. SEAM (Li et al., 2026e) emits readable guidance for each new problem (Section 5.5), but that guidance is neither retained across episodes nor grounded in particular trajectories, as core experience requires. We therefore treat both as neighboring examples.

Table 8 Classifications that recur across the survey. The main axes organize the whole survey; the others serve particular sections and are restated where they are used. Each answers a different question, so one unit, method, or study is described on several of them at once. Classifications introduced within a single section, such as organization patterns or maintenance check levels, are defined where they are used.

Classification	Question it answers	Members	Defined	Mainly used
Main axes used throughout the survey				
Representational forms	How is the guidance expressed?	Trajectory-level (precedents), semantic-level (lessons), procedural-level (procedures)	§2.2.2	§3
Analytical dimensions	What should be examined in any experience unit?	Guidance g_i , applicability conditions c_i , execution evidence e_i , reliability evaluation r_i	§2.2	§3–7
Lifecycle functions	What must a system do to manage and use experience?	Formation, organization, utilization, maintenance	§2.4	§3–6
Improvement claims	What does an evaluation show about how the agent improved?	Fixed experience reuse, feedback-driven sustained improvement, improvement of the learning mechanism	§2.5	§7.4–7.6, §8.8
Evaluation targets	Which link from retained experience to outcomes is assessed?	Intrinsic experience quality, the experience utilization process, downstream effects	§7	§7.2–7.5
Classifications used in particular sections				
Experience types	Which execution outcome does the guidance draw on?	Success, failure, comparative, corrective	§2.2.4	§3
Modes of experience-driven agents	Which mechanisms does the agent have?	Fixed-repository reuse, closed-loop learning	§2.3	§7.5
Utilization questions	Which runtime decisions turn retained experience into situated guidance?	Triggering, selection, adaptation, application	§5	§5, §7.3
Experience application functions	What does supplied guidance contribute to the current operation?	Interpretation, generation, decision-making, procedure reuse, evaluation	§5.4	§7.3
Evidence roles	What can a cited study show about experience?	Direct experience studies, adjacent mechanism studies, complementary diagnostics; §7.6 adds Boundary for system-level comparisons	§2.6	§4–7

A method may select a reusable procedure at runtime, keep its repository fixed, and report a benefit on later tasks. Describing this method requires several distinctions: the procedure is procedural-level experience, its selection belongs to utilization, and the benefit may support a fixed experience reuse claim. Table 8 brings these distinctions together, with the question each answers and where it is defined and used. Five of them—representational forms, analytical dimensions, lifecycle functions, improvement claims, and evaluation targets—organize the whole survey; the others serve particular sections. Form and type vary independently, a method can address several lifecycle functions, and a study’s evidence role is evaluated separately from the improvement claim its results support. Evaluation targets specify what is examined; improvement claims specify what the downstream comparisons establish.

The following sections apply these distinctions, starting with how experience is formed from trajectories.

3 Experience Formation: From Trajectories to Reusable Guidance

Experience formation turns trajectories into candidate experience for future use: a precedent to consult, a lesson to apply, or a procedure to follow. Following Section 2.2.2, we organize formation around three representational forms: *trajectory-level* precedents, *semantic-level* lessons, and *procedural-level* workflows and skills. The central question is what each formation mechanism derives from the available evidence and

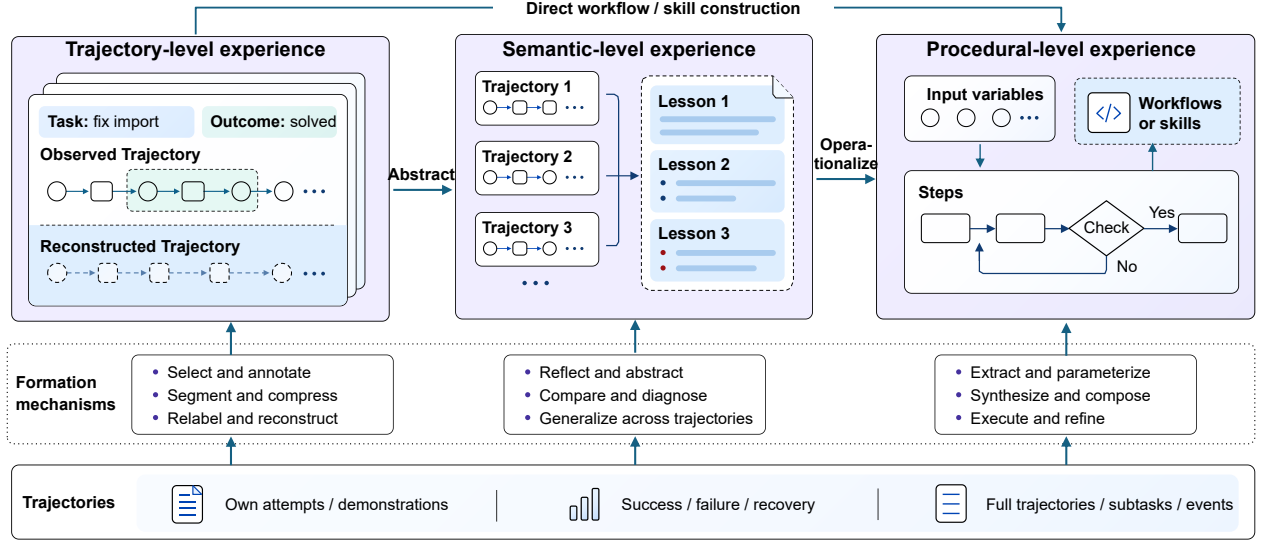


Figure 5 Trajectories support three forms of reusable experience: trajectory-level precedents constructed through selection, compression, or reconstruction; semantic-level lessons derived through reflection, comparison, or generalization; and procedural-level workflows and skills built through extraction or synthesis and refined through execution. Trajectories can be abstracted into lessons and lessons turned into procedures, while workflows and skills can also be constructed directly from trajectories.

how that evidence supports the resulting guidance. We argue that the more a mechanism adds beyond what the trajectories recorded, the more its support must come from outside them, through counterexamples, tests beyond the source tasks, or new execution. Figure 5 summarizes the three forms and their associated mechanisms.

Experience formation takes the new trajectory T_k , available earlier trajectories $\mathcal{H}_{<k}$, and optionally existing experience in R_k as inputs, and produces a possibly empty collection of candidates $\mathcal{E}_k$, as formalized in Section 2.3. Selecting a trajectory as a behavioral precedent already constitutes formation, while merely archiving a log does not. The section moves from trajectory acquisition to the mechanisms that construct each form and ends with their connections and limitations. Our focus is constructing candidates and checking their support, not deciding which of them enter the repository.

3.1 Trajectory Acquisition for Experience Formation

Experience formation needs trajectories that link task conditions, ordered actions, observations, and the feedback evaluating them. A final “success” label alone hides the checks that located a fault, and a recorded fix alone hides whether it worked. These trajectories may come from the agent’s own attempts or external collections of human or agent demonstrations. General factual documents and stored preferences alone do not record such task behavior and feedback (Section 2.3). Acquiring this evidence involves four choices, grouped into two parts. The first decides which trajectories are available: *where trajectories come from*, and, when the agent itself is the source, *which additional trajectories to collect* (Section 3.1.1). The second decides what each trajectory retains: *how outcomes are judged* and *how much context to keep* (Section 3.1.2).

3.1.1 Which Trajectories Are Available

The trajectories available for experience formation depend on their source and, when the agent collects its own trajectories, on which additional attempts it chooses to run. The source determines who performs the actions and how outcomes are evaluated, shaping the reliability of the evidence and the checks needed before reuse. Additional attempts then determine how much behavior the evidence covers and how much it reveals about any one task.

Trajectories come from four sources, which differ in who produces the actions and what judges their outcome

Table 9 Sources of trajectories for experience formation. Each source differs in who produces the actions, what judges their outcome, and what to check before reusing the resulting evidence.

Source	Actions produced by	Outcome judged by	Check before reuse	Example
Prepared demonstrations	Humans or other agents, before the receiving agent attempts the task	Curation of the collection	Whether the receiving agent has compatible tools and environment conditions	AWM, offline (Wang et al., 2025e)
Agent’s own attempts	The receiving agent	Environment signals or a model evaluator	Whether a model-evaluated label is correct	AWM, online; ExpeL (Zhao et al., 2024a)
Human-corrected demonstrations	Non-expert humans, video, or agents; then re-executed by the agent	A human observer, for failed actions	How much human effort each correction round costs	ICAL (Sarch et al., 2024)
Agent attempts on logged requests	The agent, answering recorded user requests	Logged user feedback: items that received positive feedback in the recorded conversation	Whether that feedback judges the outcome or only reflects one user’s preference	CRAVE (Zhu et al., 2025)

(Table 9). Agent Workflow Memory (AWM) ([Wang et al., 2025e](#)) uses the first two: offline, it builds workflows from annotated demonstrations; online, it runs incoming tasks itself and keeps the trajectories that a language-model evaluator judges successful. A demonstrated action is reusable only where the receiving agent has compatible tools and environment conditions; the agent’s own attempts show that its tools could perform the actions, at least in the setting where they were collected.

In the remaining two sources, human judgment, given live or taken from logs, evaluates actions that the agent itself runs. ICAL ([Sarch et al., 2024](#)) shows how a human can correct noisy demonstrations without labeling every step. A vision-language model rewrites each demonstration into an example annotated with the causal relations and state changes it exhibits. The example is executed in a similar environment; when an action fails, a human observer gives natural-language feedback on that failure, the model revises the example, and the agent retries it, until the example succeeds or a preset number of feedback rounds is reached. The authors report that fewer feedback rounds and environment interactions are needed as the example library grows. CRAVE instead relies on judgments already logged: its agents make recommendations for historical conversations, and CRAVE then reflects on each recommendation against the items that received positive user feedback in that conversation ([Zhu et al., 2025](#)).

Of these sources, only the agent’s own attempts can be expanded without new demonstrations, human corrections, or logged requests, and only when the environment permits repeated interaction. The agent can then spend extra execution budget on new tasks, to cover more behavior, or on further attempts at one task, to gather more evidence about it. For coverage, the automatic curriculum of Voyager ([Wang et al., 2024a](#)) reads the agent’s current Minecraft state, including its inventory, together with completed and failed tasks, and proposes a next task that seeks new behavior within the agent’s current resources and capabilities. Previously learned skills in turn make new states and tasks reachable. SkillWeaver ([Zheng et al., 2025](#)) applies the same idea to websites: a model proposes skills beyond those the agent already has, the agent practices them, and successful practice is distilled into callable APIs.

Further attempts at one goal yield different evidence depending on how they are run. Retrying after a failure yields paired failed and successful attempts: ExpeL ([Zhao et al., 2024a](#)) attempts each training task and, after a failure, generates a reflection to guide a retry until it succeeds or exhausts a fixed retry budget. For tasks eventually solved, keeping the failed and successful attempts associated lets later formation compare what changed; saving only the final success would lose that comparison. Running several attempts exposes alternatives or records corrections: ReasoningBank’s memory-aware test-time scaling (MaTTS) ([Ouyang et al., 2026](#)) either generates several memory-guided trajectories for the same query in parallel, exposing alternative approaches, or refines one trajectory sequentially after its first completion, retaining intermediate corrections and reasoning notes that show how it was revised.

Because each additional attempt costs a full rollout, the budget is best spent on the evidence the intended candidate needs: comparable successes and failures at one goal for diagnosis, different task instances for induction, and repeated runs of the candidate itself for executable skills, which also require an environment that tolerates repeated attempts (Table 10).

3.1.2 What Each Trajectory Retains

Once a trajectory is available, experience formation depends on what its record keeps, which again involves two choices. The first concerns evaluation: which feedback is attached to the trajectory, at what granularity, and how far that feedback can be checked. The second concerns scope: how much of the episode accompanies the behavior being learned. Both guard against records that look usable but cannot support a candidate, such as a success verdict stored without the observations behind it or an action span stored without the setup it depended on.

Feedback needs to distinguish whether an action ran, whether it produced the intended change, and whether the task was completed. On a website, for instance, submitting a form can return a confirmation page even though the record it was meant to create was never saved. Conversely, when a later step of the same task fails, the submission itself may still have worked. Attaching feedback to the action or subtask it evaluates preserves these distinctions.

Some outcome signals can be checked more directly than others. Environment feedback and interpreter errors, which Voyager (Wang et al., 2024a) collects, can be read against the environment’s own state; Voyager separately asks a model to judge completion from the task and current state. Online AWM and ReasoningBank (Ouyang et al., 2026) rely on model evaluators alone; ReasoningBank’s evaluator labels success or failure from the query and trajectory without ground-truth answers. Model evaluators extend coverage to tasks lacking a reference but leave the label unverifiable on its own, and such labels can be wrong. Auditing publicly released computer-use trajectories that benchmarks had scored as failures, Dong et al. (2026) find a non-trivial fraction of evaluator false negatives alongside invalid tasks. Because only recorded failures were sampled, the audit leaves the overall error rate open, and mistaken passes would show up only in a similar audit of recorded successes. If the observations behind an evaluation are kept along with its verdict, a mistaken label can be corrected later.

The evidence passed to formation should preserve enough context to reconstruct the decision being learned, so its boundary follows the prerequisites and consequences of the behavior rather than a fixed number of turns. Acquisition only sets this boundary; segmenting and compressing the retained trajectory into a reusable precedent belongs to trajectory-level experience formation (Section 3.2). The following subsections turn such evidence into precedents, lessons, and procedures; the *Evidence used* column of Table 22 in Appendix A summarizes what representative systems build on.

3.2 Trajectory-Level Experience: Constructing Precedents for Reuse

Trajectory-level experience formation makes a trajectory available as a behavioral example. The guidance remains embedded in what was attempted, observed, and achieved; no independently stated lesson or parameterized procedure replaces it. Formation must therefore keep enough context for a later agent to see what the trajectory did and when to imitate it. Turning trajectories into precedents raises three questions: which trajectories are worth keeping and what to record about them, how much of each to keep, and whether a failed trajectory can still yield a precedent. Selection and annotation, segmentation and compression, and reconstruction answer them in that order (Figure 6). The first two routes keep or trim recorded behavior, whereas reconstruction proposes goals or steps that the source trajectory never showed, so part of the resulting precedent was never observed.

3.2.1 Selecting and Annotating Trajectories

Proposing a trajectory as a precedent involves two decisions: whether it is a promising candidate, and what to record with it so that a later agent can tell when it applies. Selection makes the first decision by proxies, because the value of a precedent appears only after it is reused; annotation makes the second by describing

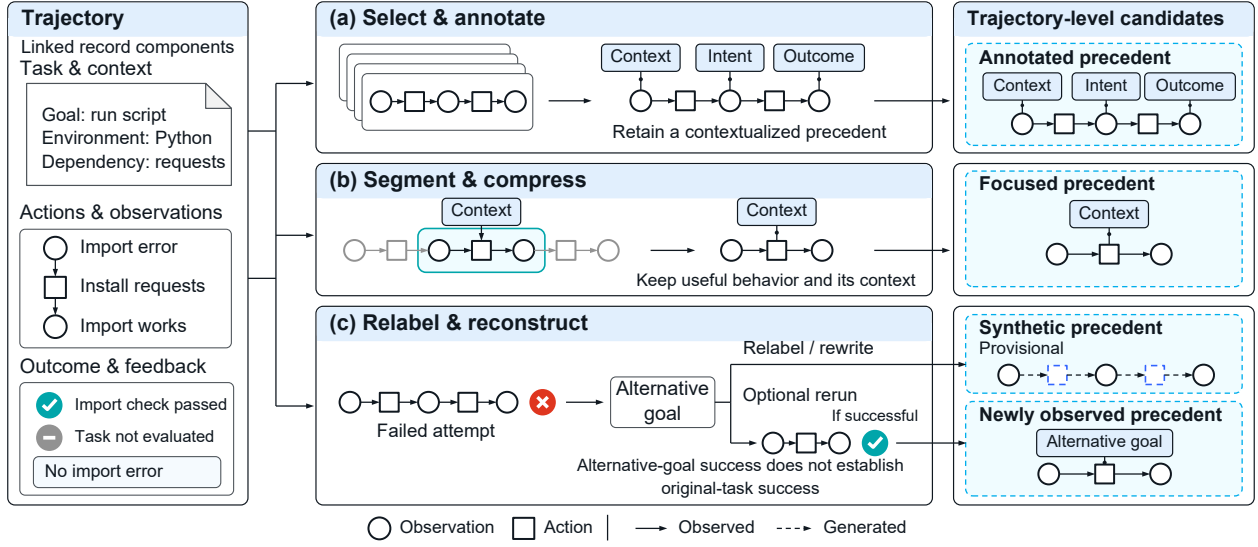


Figure 6 Trajectory-level experience formation through three complementary routes. The left column uses the dependency-repair example to show linked components of a trajectory. Selection and annotation retain trajectories as contextualized precedents; segmentation and compression expose useful behavior while preserving relevant context and action–effect relations. Reconstruction reinterprets or rewrites evidence for alternative goals. Generated changes remain provisional without confirming execution; successful reruns provide newly observed precedents.

the task, intent, or situation that the trajectory served. Both add claims that the trajectory itself does not verify, so we examine each and then ask when an annotation helps a later agent.

Selection asks which past trajectories to propose as precedents for a future agent; whether a proposed precedent is admitted is decided in maintenance (Section 6). A trajectory is a valuable precedent if later agents that retrieve it handle their tasks better with it than without it; this value is relative to those tasks, the receiving agent, and the other stored precedents (Section 6.4). The usual proxy for it is source success, used by RAP (Kagaya et al., 2024) and by the trajectory database of self-generated in-context examples (Self-Generated ICE) (Sarukkai et al., 2025). Success is not required by every design: ExpRAG (Ferraz et al., 2026) indexes full trajectories, including unsuccessful episodes, by default, although it does not test whether the unsuccessful ones help. Nor is success sufficient: a successful trajectory may contain detours or accidental shortcuts that a later agent would copy. Formation should therefore keep the outcome evidence so that the proxy can be rechecked once later attempts measure how useful a precedent actually is. Self-Generated ICE does this in its exemplar-curation step, which selects exemplars by their measured usefulness as in-context examples (Section 6).

Annotation then records what a kept trajectory was for and under what conditions it worked, making explicit the applicability conditions that a bare trace leaves implicit. SEER (Cui et al., 2025) tags each successful tool-use trajectory with a user intent, which the LLM chooses from a predefined set of categories, and with the chain of tools the trajectory called. JARVIS-1 (Wang et al., 2025b) stores each successfully executed Minecraft plan with its task and the game state observed when the plan was made, so plans for the same task can be told apart by situation. TRAD (Zhou et al., 2024) annotates single steps: for each step of an expert demonstration, an LLM writes a thought consistent with the recorded action, and a later agent retrieves the steps whose thoughts match its own. These annotations leave the guidance in the trajectory and change only what a later agent can know about when to use it. ICAL (Section 3.1.1) goes further: annotating demonstrations with the causal relations and state changes they exhibit already states lessons, which add semantic-level guidance alongside the precedent and share the risks of semantic abstraction (Section 3.3).

Annotations help only when they describe what a new task needs to match, and when the retrieving agent can produce a comparable description. Which annotation matters depends on what distinguishes the tasks: in SEER’s ToolQA ablation with Qwen2.5-72B, removing intent matching mainly lowers accuracy on easy questions, whereas removing toolchain matching mainly lowers it on hard ones, and JARVIS-1 reports higher

success on a subset of Minecraft tasks when retrieval compares the observed state as well as the task text. A predefined label set can describe only the tasks its designers anticipated. An annotation can also depend on information the retrieving agent lacks: TRAD’s annotator sees the correct action when writing each thought, whereas the retrieving agent must write its thought without it; the authors note that performance depends heavily on thought quality and retrieve neighboring steps to tolerate the mismatch. Despite this mismatch, on ALFWorld with GPT-4 and a shared expert memory, TRAD reports 96.8% success against 89.6% for Synapse’s whole-trajectory retrieval, although the comparison changes annotation and retrieval granularity together.

3.2.2 Segmenting and Compressing Trajectories

Whole trajectories preserve concrete practice but carry irrelevant observations and actions into reuse, because the behavior worth reusing often occupies only part of a trace. Segmentation isolates a coherent subtask or event sequence; compression removes irrelevant observations or repetitive material while preserving the relation between actions and their consequences. These operations change the size and presentation of a precedent, but need not change the kind of guidance it supplies.

Synapse (Zheng et al., 2024) illustrates the point: its state abstraction removes task-irrelevant interface information, while its in-context exemplars remain trajectories of abstracted states and actions. The resulting guidance is still a trajectory-level precedent. Segmentation instead cuts a trajectory at task or subtask boundaries, as Agent S (Agashe et al., 2025) does, so that a reusable span need not occupy an entire episode.

The trade-off is between keeping enough context and keeping the precedent short. A short span can omit a prerequisite, such as the environment inspection that preceded a repair, while a verbose record can hide its useful behavior. At the short extreme, one baseline in the Evo-Memory benchmark (Wei et al., 2025) keeps only each task’s input, output, and feedback, so a later agent sees what was achieved but not the steps that achieved it. A compressed precedent remains trajectory-level when it still describes a particular trajectory; stating an independent lesson or a parameterized action structure requires a further abstraction step, discussed below.

3.2.3 Reconstructing Trajectories

Selecting by source success discards failed trajectories, even though a failed attempt may still accomplish something useful along the way. Reconstruction puts such trajectories to use by forming a new precedent from their evidence. It adapts hindsight relabeling from reinforcement learning: after the fact, a trajectory that missed its intended goal is reinterpreted as a successful demonstration of a goal it did reach.

Reconstruction methods offer three levels of support, which determine how far a later agent can trust the result. Relabeling alone keeps the observed actions and changes only the task they are said to accomplish; its risk lies in the new label, since the model may wrongly judge that the trajectory reached the proposed goal. Rewriting also changes the action sequence, so the precedent contains steps that were never executed. Re-execution runs the agent on the proposed goal and keeps the resulting attempt, replacing a claimed success with an observed one.

ECHO (Hu et al., 2025a) combines relabeling with rewriting. It asks the LLM to identify alternative goals that a failed attempt could have reached and to generate an optimized trajectory or description for each. The resulting synthetic examples enter memory without being executed first. A rewritten trajectory remains a trajectory-level precedent; a description that specifies reusable steps is procedural-level experience. A post-hoc check on sampled XMiniGrid examples finds that the rewrites usually, though not always, guide an agent to the imputed goal, with some failures traced to infeasible steps.

BREW (Kirtania et al., 2026) checks the relabeled task by rerunning the agent. It summarizes the failed trajectory’s key decisions and point of deviation, asks an LLM for a task description consistent with what the agent did, and runs the agent on that task. About half of the failing trajectories pass after relabeling; the remaining failures are kept as negative evidence. The resulting trajectories join the input from which BREW distills natural-language recipes (Section 3.4). A successful rerun supports the relabeled task, but may follow different actions from the original trajectory; the original task still counts as a failure.

Both methods report downstream gains, but for different interventions. ECHO’s evaluation covers the whole framework, its memory update rule included: with GPT-4o on a stateful version of XMiniGrid, it improves average reward over ReAct by 80% and also outperforms Reflexion and AWM. Because memory is built while the evaluated queries are processed and only the final reward is reported, the result does not show how reward changes across successive updates (Section 2.5). BREW compares its pipeline with and without relabeling, building recipes on training tasks and keeping them fixed on test tasks. On OSWorld-Verified, with a GTA1-7B agent paired with o3 or GPT-5.2, and on τ^2 -Bench and SpreadsheetBench with o4-mini, recipes built with relabeling raise the success measure by 0.7 to 6.3 points under each of the three search strategies tested.

The choice between rewriting and rerunning depends on the cost and availability of further interaction. Relabeling and rewriting add no rollouts, but rely on the model’s judgment of what a trajectory achieved or could achieve. Re-execution supplies an observed result at the cost of an extra rollout per relabeled failure (Section 3.1.1) and requires an environment that can be rerun, which excludes the costly or hard-to-reset settings that motivate ECHO. Where rerunning is not possible, marking observed and proposed steps separately would at least let a later agent or maintainer distinguish their support.

The closest evidence on what rerunning adds comes from BAGEL (Murty et al., 2024), which relabels exploratory trajectories instead of failed ones. A language model labels each explored trajectory with a synthetic instruction, and the agent re-executes that instruction; the two steps alternate until a filter model accepts the pair. The accepted demonstrations are then retrieved as in-context precedents and stay fixed during testing. With PaLM-2, these demonstrations raise average reward on MiniWoB++ from 53.0 to 61.0 and F1 on ToolQA from 40.9 to 42.2 relative to labeling the explored trajectories once under the same filter. Each rerun also replaces the explored actions, so the gain covers rerunning and rewriting together.

Relabeling without rerunning can still help when the source instructions describe the recorded behavior poorly. Learn-by-Interact (Su et al., 2025) has a model attempt tasks generated from environment documentation, many of which its trajectories do not accomplish. For each sub-trajectory, an LLM then writes a new instruction that summarizes the steps or abstracts their purpose; an LLM committee discards pairs it judges incoherent or misaligned, and the remaining pairs are retrieved as in-context precedents. With Claude-3.5-Sonnet on OSWorld, retrieving these precedents raises success from 11.4% without examples to 22.5%, against 11.9% for the same pipeline with the original instructions. Relabeling every sub-trajectory also multiplies the number of stored pairs, so the comparison changes how many precedents are available as well as how well their instructions fit. Whether a relabeled trajectory accomplishes its new instruction is judged by the committee, not by execution.

Reconstruction helps most when failed attempts complete coherent subgoals, and even then it leaves the original failure unexplained. Because the new goals fit what the agent already did, the recovered precedents do not show how to solve the original task; that failure still needs to be retained or diagnosed (Section 3.3).

All three routes in Figure 6 yield contextualized behavior, recorded or proposed, and leave the receiving agent to determine which details transfer. The next subsection turns to formation that states transferable content explicitly.

3.3 Semantic-Level Experience: Abstracting Lessons from Trajectories

Semantic-level experience formation derives candidate lessons from trajectories about what to do, avoid, or check in a later situation. Stating the guidance makes it easier to reuse without replaying the source sequence, but can also hide the conditions that made it useful. We compare three ways to derive a lesson: interpreting one trajectory, contrasting attempts, and integrating patterns across trajectories (Figure 7). The closing comparison explains the evidence and characteristic risks of each.

3.3.1 Reflective Abstraction

Reflection interprets a trajectory to produce a lesson. It may summarize successful decisions, diagnose an error, explain a recovery, or identify a constraint that future attempts should respect. Its defining feature is direct interpretation of a single trajectory, even if the system repeats this operation over many trajectories.

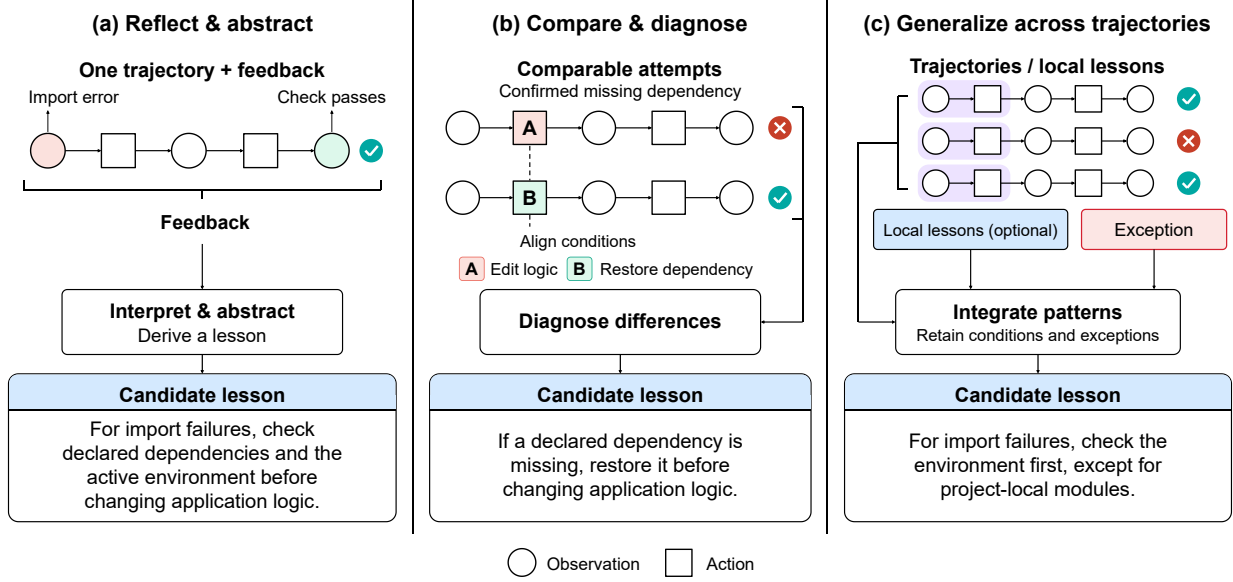


Figure 7 Semantic-level experience formation through three complementary inference mechanisms. (a) Reflection interprets a trajectory and its feedback to abstract an action-guiding lesson. (b) Comparison aligns attempts and examines differences to propose conditional guidance. (c) Generalization integrates recurring patterns from trajectories or local lessons while retaining conditions and exceptions; Figure 8 details its three routes.

Reflexion (Shinn et al., 2023) provides a concrete construction loop. After an attempt, an evaluator supplies task-dependent feedback, such as answer matching, heuristic or model-based evaluation, or results from self-generated tests. The reflection model reads that feedback together with the trajectory and retained reflections, then describes what went wrong and how a later attempt should change. This text is appended to a bounded memory that conditions subsequent attempts, and it tells the next attempt more than the outcome label alone. Reflexion reuses these reflections only in later trials of the same task, so its reported gains measure how much reflection helps an agent retry the task it failed.

Applied to the code-repair episode of Table 1, a reflector reads the import error, the missing declared dependency in the active environment, and the successful check after restoration. It can connect the failure to the environment mismatch and state: “For import-stage failures, check declared dependencies and the active environment before changing application logic.” The transformation removes instance-specific package names while retaining a diagnostic priority and its applicable situation. The source episode supports that check; it leaves the cause of a future import failure open.

Other designs vary what the reflector reads and what it writes: inferred action effects in CLIN (Majumder et al., 2024), tips written from each collected experience and stored alongside it in CFGM (Yang et al., 2025b), lessons drawn at subtask boundaries of long-horizon tasks in MUSE (Yang et al., 2025a), and both successful strategies and failure lessons from self-evaluated outcomes in ReasoningBank (Ouyang et al., 2026). ACE (Zhang et al., 2026i) makes the formation–maintenance boundary explicit: a reflector derives lessons, and a separate curator decides how they update the playbook.

Reflection can make qualitative constraints explicit with little evidence. However, its explanation may overfit incidental details or credit an unnecessary action. The inferred diagnosis therefore remains provisional, including when it is phrased causally. Comparing attempts can show which parts of that interpretation hold beyond the single trajectory.

3.3.2 Trajectory Comparison and Diagnosis

A single trajectory often cannot show which decisions mattered to the outcome and which were incidental. A successful attempt may contain unnecessary steps, while a failed attempt may contain useful decisions followed by an unrelated error. A lesson that simply repeats a success, or forbids whatever preceded a

failure, can therefore mislead. Contrastive diagnosis instead compares the decisions and conditions behind different outcomes. Synthesizing the systems discussed below, we describe it in three steps: align comparable tasks, states, or subgoals; locate differences in actions, observations, or assumptions; and state the resulting diagnosis as a conditional rule. Comparisons may pair success with failure, a failed attempt with its recovery, or alternative branches of the same task. The intended output is conditional guidance of the form “when condition c holds, prefer, avoid, or check behavior g .”

AutoGuide (Fu et al., 2024) makes these steps concrete: it pairs offline trajectories for the same task with different returns and locates the timestep at which their actions first diverge. A context-identification model summarizes the shared trajectory prefix into a description of the agent’s state. A guideline-extraction model then reads both attempts together with this context and proposes a preferred behavior for that situation. The resulting guideline couples an applicable state, summarized from the shared prefix, with a preferred action motivated by the later outcomes. Locating the first divergence narrows the comparison to one action, although a later difference may be what changed the return.

AutoGuide’s WebShop ablation supports making context explicit: with a GPT-3.5-turbo acting model, the full system reaches 46% success, compared with 37% for guideline extraction and selection without explicit contexts. Context also guides runtime selection in this ablation, so the gain belongs to context-aware formation and use together.

ExpeL (Zhao et al., 2024a) and ReMe (Cao et al., 2026) use comparison alongside other routes: ExpeL also extracts insights from groups of successful experiences, and ReMe separates success-pattern extraction, failure analysis, and comparative insights. Training-Free GRPO (TF-GRPO) (Cai et al., 2025) is one of the few systems that measures what comparison adds over reflecting on one attempt. For each training problem it samples a group of attempts, has the model contrast them against the reference answer, and edits a library of natural-language experience accordingly; the library then stays fixed during testing. With DeepSeek-V3.1-Terminus and a code interpreter, a library learned from groups of five attempts on 100 DAPO-Math problems raises Mean@32 on AIME24 and AIME25 from 80.0% and 67.9% without experience to 82.7% and 73.3%, whereas the same procedure with one attempt per problem reaches 80.4% and 69.3%, and an equal number of experiences generated without trajectories does not help. In this setting, contrasting attempts contributes beyond reflecting on each one.

The intended recipient can also shape formation. MemCollab (Chang et al., 2026) contrasts different backbone models’ trajectories on the same task to derive proposed reasoning constraints. Each constraint records which two models’ trajectories produced it, so that retrieval can keep only constraints whose source pair includes the model now receiving them. Section 5.2.3 connects this construction choice to shared access and recipient adaptation.

A diagnosis can also become an edit to existing experience. SkillCAT (Chen et al., 2026e) makes the comparison local to repeated runs of the same task: it contrasts successful and failed traces, identifies behavioral divergence, and derives a lesson that motivates a candidate skill edit. Its subsequent replay stage tests that edit, as discussed under execution-guided verification below. Because SkillCAT keeps the lesson apart from the edit, the contrast between traces supports the diagnosis, while only the replay tests whether the edit repairs the skill.

Contrastive diagnosis produces a more discriminating lesson when tasks, tools, and evaluation conditions are aligned. Otherwise, an apparent strategy difference may reflect a change in the environment or task difficulty; even matched attempts can retain uncontrolled differences. The systems above mostly align attempts at the same task: AutoGuide pairs offline trajectories of one task with different returns, ExpeL pairs failed and successful trials of one task, SkillCAT compares repeated runs, and MemCollab compares models on one task. They differ more in how they state and check the diagnosis. AutoGuide makes the applicable state an explicit part of each guideline, but like ExpeL it keeps the diagnosis without testing it at construction time, whereas SkillCAT replays the proposed edit before it can be merged. Whether a rule supported on one task holds beyond the task it came from is the subject of the next subsection.

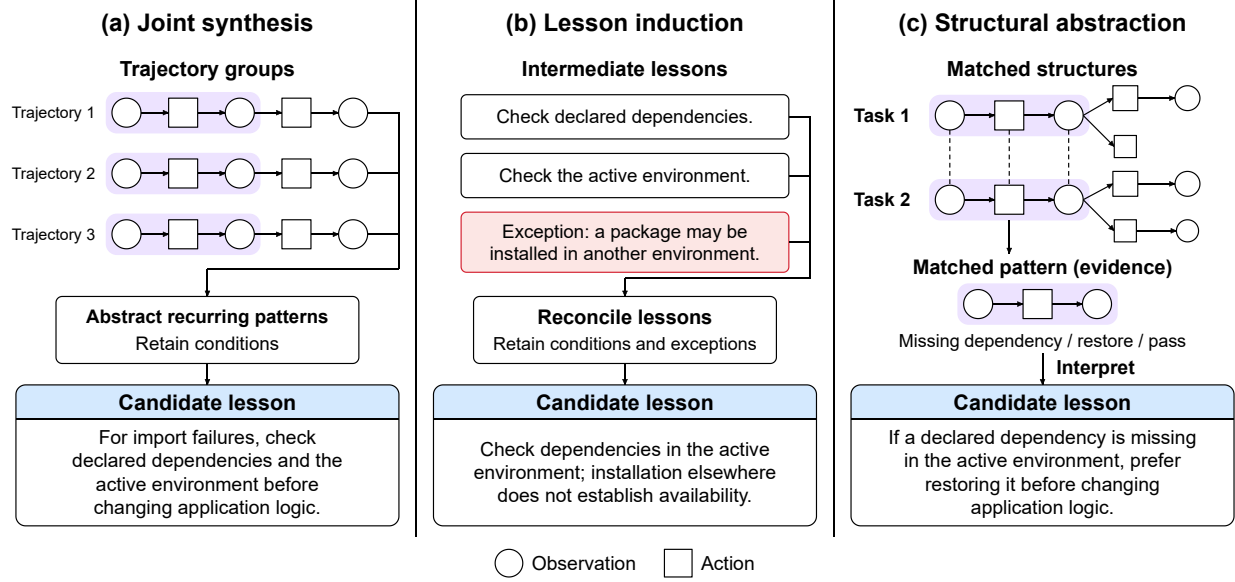


Figure 8 Three complementary routes to candidate semantic lessons from multiple trajectories. (a) Joint synthesis abstracts recurring patterns from trajectory groups while retaining conditions. (b) Lesson induction reconciles intermediate lessons while retaining conditions and exceptions. (c) Structural abstraction matches action–observation patterns, then interprets the shared structure as a conditional lesson.

3.3.3 Cross-Trajectory Generalization

Cross-trajectory generalization integrates multiple trajectories to propose shared guidance, seeking to preserve the conditions and exceptions that qualify it. Figure 8 distinguishes three routes by the material passed to abstraction: groups of trajectories, intermediate lessons, or matched trajectory structures. How evidence is grouped—by task, category, modality, or executor—is a separate choice that can apply to any route. Here we focus on forming lessons; Section 3.4 turns to procedures.

One route derives shared guidance directly from several trajectories. Besides the success–failure comparisons discussed above, ExpeL extracts insights from groups of successful experiences, whereas ReasoningBank’s parallel scaling (Section 3.1.1) looks for patterns that recur across attempts at the same query. EDV (Zhu et al., 2026c) broadens the comparison across executors: heterogeneous agents attempt the same task, and a separate distillation agent integrates their trajectories into reusable candidate experiences. What varies within the evidence group is thus task instances in ExpeL, sampled attempts in ReasoningBank, and executors in EDV. Agreement among attempts at one task, as in ReasoningBank and EDV, can make a candidate lesson more robust for that task; transfer must be tested on tasks outside the group. EDV’s subsequent executor-consensus check is also distinct from independent execution verification: it tests agreement with the candidate, not its behavior on an unseen task.

A second route integrates local abstractions instead of repeatedly processing all raw trajectories. AutoManual (Chen et al., 2024a) allows its Builder to derive more general rules from existing rules while recording their dependencies; its Consolidator can revisit supporting trajectories when combining overlapping rules, the kind of merge that Section 6.2 examines for experience already stored. Trace2Skill (Ni et al., 2026) instead identifies recurring patterns across a pool of trajectory-local patch proposals; how it turns them into a skill directory is discussed in Section 3.4. The shared operation reconciles what several trajectories have already suggested. This can reduce the amount of raw evidence considered together, but omitted conditions or mistaken interpretations in the intermediate lessons can then propagate into the shared guidance.

A third route matches the structure of trajectories, their decisions and actions, and turns the structure they share into a lesson. Experience Memory Graph (EMG) (Wang et al., 2026f) represents each successful expert trajectory as a graph of the agent’s decisions and actions, matches these graphs across similar tasks, and writes the shared subgraphs out in natural language as cross-task insights. A shared subgraph becomes a

semantic lesson only once it is interpreted as guidance on when its behavior should be preferred. In EMG’s Qwen3-4B ablation on unseen ScienceWorld tasks, adding cross-task insights to intra-task memory raises success from 9.48% to 16.11%. This supports their contribution in that configuration, while the scope of any individual insight still depends on task grouping and how actions and observations are represented. Matching structure at a finer grain, across similar high-reward subtrajectories, yields procedural-level skills instead, as in Skill Set Optimization (SSO) (Nottingham et al., 2024), discussed in Section 3.4.

In all three routes, more trajectories do not guarantee a more reliable lesson: added trajectories may repeat the same shortcut, and each merge can drop distinctions that the sources kept. Zhang et al. (2026a) have GPT-5.4 build a lesson bank from ground-truth solutions to a 19-problem ARC-AGI slice that it already solves without memory, so any loss of accuracy cannot come from incorrect inputs. In their illustrative run, after ten refresh rounds, consolidating the whole pool at once keeps accuracy at 100%, whereas updating the bank after each problem lowers it to 52.6%. The same trajectories thus yield different memories depending on how often the bank is rewritten, and in their tested settings a control that simply keeps the raw trajectories remains competitive. Section 6.2.2 returns to this result as a risk of repeatedly consolidating stored experience. These results argue for retaining source evidence and comparing induced guidance against direct reuse of precedents; Section 3.5 discusses a complementary comparison in ExpeL that builds insights and trajectory precedents from the same evidence.

Reflection can over-interpret a single trajectory, comparison can mistake an uncontrolled difference for a cause, and generalization can extend a shared pattern beyond its support. Each mechanism therefore needs a check suited to the inference it makes (Table 10). In terms of the types in Section 2.2.4, reflection turns one trajectory into success, failure, or corrective experience depending on what it records; comparison yields comparative experience, or corrective experience when it pairs a failure with its recovery; and generalization integrates any of these across trajectories. These operations can also contribute to procedural construction: when guidance is organized into reusable steps or dependencies, its output is procedural-level experience, whether or not a separate semantic lesson was retained first. The next subsection examines that construction problem.

3.4 Procedural-Level Experience: Constructing Reusable Workflows and Skills

A precedent or lesson still leaves the agent to work out the steps each time it is reused. Procedural-level experience formation reduces this repeated planning: it turns trajectories, and sometimes lessons or existing skills, into steps that an agent can follow or call directly. We use *procedure* for both workflows and skills, whether written in natural language or code. A candidate procedure specifies relevant inputs, dependencies, applicability conditions, and checks. Some decisions can remain open until execution: a procedure may branch on observations or ask the agent to resolve a step.

Building such a procedure starts with a practical question: *do the available trajectories already show the steps the intended routine needs, in a usable order?* When they do, extraction and parameterization preserve those steps while making them reusable. When the desired behavior needs additional steps, connections, or control logic, synthesis and composition propose the missing structure from trajectories, lessons, or existing skills. The routes are construction operations, and one system can use both: extracted components can later be composed. Neither route compensates for unreliable evidence: a complete-looking procedure built on such evidence needs further observation or a narrower scope, and both routes need the execution checks discussed last (Figure 9).

Whichever route is used, the procedure can be written as text or as code. Text leaves the agent to interpret observations and choose concrete actions; code fixes more of these operations through tool calls and argument bindings, so the choice depends on how much behavior can be specified before execution. Compiling generated code is only an initial check; whether the candidate accomplishes the intended task is tested by running it (Section 3.4.3). Section 3.5 reviews a direct comparison of text and code procedures.

3.4.1 Procedure Extraction and Parameterization

Extraction turns recorded trajectories into a procedure without adding steps they did not show, so its work lies in deciding what to carry over from the source instance to new ones: which span to keep, which values to

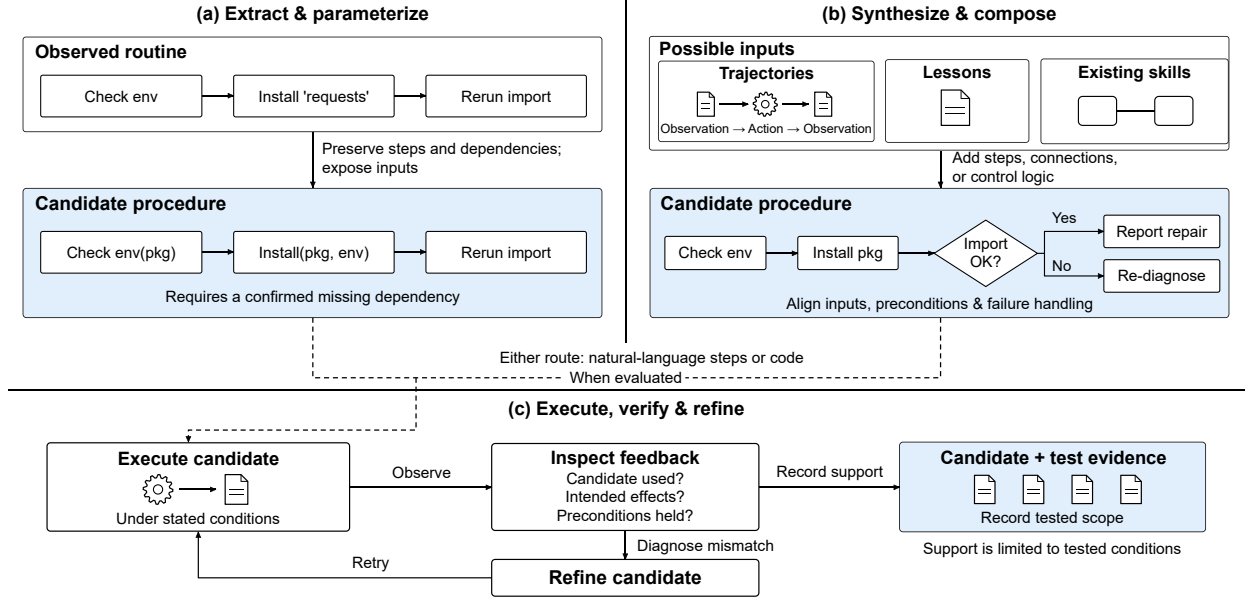


Figure 9 Procedural-level experience formation. (a) Extraction and parameterization preserve steps and dependencies from trajectories while exposing inputs for reuse. (b) Synthesis and composition add steps, connections, or control logic. Either route can produce text or code. (c) Execution checks candidate use, effects, and preconditions; feedback guides refinement and re-execution.

expose as inputs, and which step orderings to keep as dependencies. Each decision can err in two directions: tying the procedure to its source instance or discarding a constraint that made the source trajectory work.

A span worth extracting follows an identifiable subtask: the conditions needed to begin, the actions that accomplish it, and the result that marks completion. As in trajectory-level segmentation (Section 3.2), a setup step outside the selected span may still be a prerequisite. AWM (Wang et al., 2025e) prompts a language model to find common subtasks in one or more experiences, such as searching for a product within a purchase task, and to replace concrete values with descriptive variables. The workflow retains its goal and steps pairing observations with reasoning and actions. MobileGPT similarly decomposes app interactions into reusable subtasks with parameterized actions (Lee et al., 2024). SSO extracts similar high-reward subtrajectories from different trajectories and writes each set out as a subgoal with instructions; with GPT-4, skills learned on training variants of ScienceWorld tasks and then held fixed outperform CLIN on held-out variants (Nottingham et al., 2024). A successful source trajectory supplies a candidate routine, but does not show that every recorded step was necessary; its outcome label must also be trustworthy (Section 3.1.2).

Within that span, extraction then sorts out which values belong only to the source instance. Some values become caller-supplied inputs; others must be obtained from observations or preceding actions. In a product-search workflow of the kind AWM induces, the product name can become an input, whereas the item to open must be read from the current results page. Exposing that item as a caller-supplied input would skip the step that checks the results against the request, and keeping the source product name fixed would tie the workflow to its source instance.

Extraction must also decide which step orderings to keep, and steps that appear in a consistent order do not necessarily depend on one another. TraceCompiler (El Yadouni and Li, 2026) recovers data dependencies between tool calls: it orders two calls when a later argument can be traced to an earlier output and other possible sources have been ruled out, and leaves unresolved relations as suspected dependencies that impose no order. It also labels each argument as a constant, a user input, a copied or transformed earlier output, or a decision requiring an LLM at runtime. A session token, for instance, changes on every execution while its dependency on the authentication call stays fixed, a structure that replacing strings alone would miss. In the authors’ large-scale evaluation, however, the implemented rule in effect assigns each value to the most recent

candidate call instead of ruling out other sources, so those results do not test the exclusion of alternative sources described above.

3.4.2 Procedure Synthesis and Composition

Synthesis and composition are needed when the source trajectories do not show the required behavior directly, so the constructor must add steps, connections, or control logic that no single trajectory recorded. Synthesis turns a lesson into explicit steps, and composition connects existing components. In both, the added structure has no direct support, so the constructor must use execution evidence to justify it and to state when it applies.

Synthesis starts from a lesson that identifies a risk without specifying the action, trigger, or check needed to address it. Trace2Skill (Ni et al., 2026) consolidates trajectory-local proposals into a skill directory, as introduced in Section 3.3. Its spreadsheet example makes a correction explicit: after writing formulas, run a recalculation script, then reopen the workbook to inspect evaluated values. Compared with a warning about stale cells, this instruction supplies an ordered operation and an observable check. BREW (Kirtania et al., 2026) likewise distills insights from graded trajectories into natural-language recipes. Because such recipes cannot be checked by compiling them, BREW ties them to execution evidence through search: each candidate rewrite of a recipe is scored by the agent’s accuracy on held-out queries when using it, together with how readily the recipe is retrieved, and Monte Carlo tree search keeps the best-scoring versions.

Composition starts from existing skills that already provide useful behavior and specifies how they work together. Voyager (Wang et al., 2024a) generates a program using the current goal and state together with retrieved code skills. Its prompt encourages reuse of available functions and asks the model to check required items before relying on them, collecting missing items when needed. The generated code thus contributes the connections between retrieved skills and the handling of prerequisites for the new goal. At a connection, the preceding component must provide the inputs and state the next one assumes; otherwise the new procedure needs an explicit conversion, check, or recovery step.

Composition therefore has to supply decisions that neither component contains. A website search skill and a checkout skill, for instance, do not yet constitute a purchase procedure: the constructor must decide which search result to pass on, what to do when the chosen item is unavailable, and how to confirm that the order went through. Returning to the search when no result fits adds a branch that neither source trajectory contained. These decisions are part of the constructed behavior even if neither component changes internally, and since each component can succeed on its own while the connection between them fails, the combination needs execution evidence of its own.

3.4.3 Execution-Guided Verification and Refinement

A procedure can look complete and still fail when run, so its support has to come from running it. Verification runs the candidate and records where its behavior departs from what was intended. Refinement uses these failures to change the candidate and test it again.

Verification can run a candidate inside a task that needs it or on its own. Agent skill induction (ASI) (Wang et al., 2025d) rewrites each source trajectory so that it calls the induced skills, and drops the trailing actions that could reproduce the answer without using them. It executes this rewritten prefix and only then lets the agent finish the task, checking model-evaluated task success, actual calls to new skills, and environment changes caused by those calls. A pass shows that the candidate can take part in a successful run; showing that the success depended on it would take a matching run without it. SkillWeaver (Zheng et al., 2025) instead tests each synthesized website API on its own: it runs the API, using model-generated argument values as test cases when the API takes parameters, and debugs it when the run fails. These runs exercise each API apart from any task that would use it.

ASI also measures what its whole construction procedure gains. On WebArena shopping tasks with claude-3.5-sonnet, program skills that passed verification and were then converted to text and added to memory reached 39.0% task success, against 32.6% for text skills induced directly without verification. The two arms differ in how skills are induced as well as in whether they are verified.

When verification exposes a failure, refinement changes the candidate and runs it again. Voyager feeds

environment feedback, interpreter errors, and model-generated critiques into subsequent code generation (Wang et al., 2024a). For textual procedures, SkillCAT (Chen et al., 2026e) applies each proposed patch to a temporary copy of the skill and replays clones of the source tasks, recording whether the patch repairs a failure or preserves a success; because the patch revises an existing skill, the merge decision that uses this record belongs to maintenance (Section 6.3.3). SkillOpt (Yang et al., 2026e) similarly has an optimizer propose budgeted revisions to a skill from the scored trajectories of a frozen executor; Section 6.3.3 discusses how it decides which revisions replace the stored skill.

Before editing a failed candidate, the constructor should locate what failed. We find it useful to distinguish four refinement targets: steps, parameters, preconditions, and component connections. In the dependency-repair example, a missing final import check is a step defect, repaired by adding the check. Installation into the wrong environment is a parameter defect: the fix changes how the environment argument is obtained. An unsupported package manager violates a precondition, so applicability is narrowed or a supported branch added, and a checker output that the installer cannot consume is a connection defect. Matching each edit to a diagnosed target avoids an indiscriminate rewrite. If feedback cannot distinguish a bad step from an unmet prerequisite, further observation or a narrower candidate is preferable to an unsupported correction. A changed candidate still needs the affected behavior rerun and previously successful cases rechecked for regressions, and composed procedures and merged patches need checks as a whole.

Even after verification and refinement, procedural-level experience formation yields only a candidate whose evidence covers the conditions actually tested. Source-task execution supports local feasibility; changed inputs and other tasks test whether the proposed parameters and conditions support broader reuse. Unresolved failures leave the candidate unverified for the affected conditions. Sections 7.4.3 and 7.1 discuss transfer and independent evaluation in detail. Refinement here changes a candidate before it is retained. Admitting it, revising a stored procedure, replacing one, and rolling back a change belong to maintenance (Section 6); formation supplies the refined candidate and the evidence for what it has been shown to do.

3.5 Comparing Forms and Formation Checks

A designer must choose what to retain from the available trajectories and how much to check before proposing it for reuse. The three forms in Sections 3.2–3.4 make different commitments: a precedent leaves interpretation to the receiving agent, a lesson states a conclusion, and a procedure specifies steps. We first weigh these commitments against the available evidence and the few direct comparisons between forms, then turn to what formation checks establish and how to choose a mechanism, compared in Table 10; the detailed system index appears in Table 22 (Appendix A). The subsection ends with the comparisons still missing and with work that trains the formation mechanism itself from the outcomes of later reuse.

3.5.1 What Each Experience Form Decides in Advance

Reusing a past trajectory requires interpreting what it implies for a new task and planning the actions that follow; the three forms differ in how much of this work formation does in advance and how much it leaves to the agent that receives the experience. In the code-repair example, a precedent keeps the environment setup, the import error, and the restoring command, leaving the receiving agent to recognize that the environment was at fault. A lesson states in advance that declared dependencies and the active environment should be checked before application logic is changed, but leaves the receiving agent to plan the checks and the fix. A repair procedure also fixes the installation steps, so it fails on a package manager it does not support. Deciding more during formation therefore pays off only while the conditions behind those decisions stay stable across tasks.

The same trade-off appears within procedural-level experience, where code fixes more decisions than text. Metis (Dai et al., 2026b) converts repeatedly selected textual plans into parameterized code helpers, and its profiling study evaluates text and code memory on AppWorld with a GPT-4o executor. When memory was built from and tested on the same 90 tasks, text reached 78.9% task success and code 75.6%; when each task could use only memory from earlier tasks, text reached 73.3% and code 53.3%, and building code memory took about 2.5 times as many LLM calls. The larger drop for code fits the trade-off above, in the one environment the study covers. These results suggest keeping a procedure in text while its behavior still requires contextual interpretation, and considering code when operations and interfaces are explicit and stable across tasks.

Direct comparisons suggest that the best form depends on the task: when forms are built from the same trajectories, none wins on every benchmark, and combining them can help. In ExpeL (Zhao et al., 2024a), with a gpt-3.5-turbo-0613 actor and gpt-4-0613 insight extraction, insights alone outperform retrieved trajectories alone on HotpotQA (36% versus 31% success), whereas trajectories do better on ALFWorld (55% versus 50%); the authors attribute this to HotpotQA’s reliance on general reasoning guidelines and ALFWorld’s reliance on specific action sequences, and the full agent, which uses both, outperforms either. On ALFWorld with GPT-4o, Memp (Fang et al., 2026a) reports 56.4% test success for abstract procedural scripts alone, 74.3% for source trajectories alone, and 77.9% for both, with the same ordering under two other backbones. In both studies, which build experience before testing and hold it fixed, the results favor retaining complementary forms over choosing a single one.

3.5.2 Choosing an Experience-Formation Mechanism

A designer choosing among formation mechanisms needs to know what each mechanism’s checks establish about a candidate. Formation methods check a candidate in different ways before it is retained, and each check supports only the claim it actually tests. AWM’s success judgment concerns the source trajectories, not the workflow extracted from them; ECHO stores rewritten precedents without running them; BREW reruns the agent on each proposed alternative task; and ASI replays source trajectories rewritten to call the induced skills. Source success shows only that the recorded behavior reached its outcome, provided the outcome label is correct (Section 3.1.2), and a model’s judgment that a rewrite or lesson is correct is still an unverified interpretation. A successful rerun or replay shows that the candidate works under the conditions tested, and only held-out tasks test the transfer for which the candidate was formed.

Choosing a formation mechanism for a new setting involves two decisions, made in order: which form of experience later decisions need, and which mechanism can build that form from the available trajectories. Table 10 follows these two decisions: its group headers state what later decisions need from each form, such as a concrete example from a trajectory-level precedent, and its rows pair each mechanism with its main risk and check. Whatever the form, the checks become more demanding as a mechanism moves further from what the trajectories recorded: selection and segmentation need only keep the context of observed behavior; inferred explanations and shared patterns need counterexamples or tests beyond the source; and proposed goals, steps, or inputs need new execution.

Few of the recommendations above rest on controlled comparisons, because the studies discussed in this section rarely vary one formation choice while holding the others fixed. The forms have been compared only in pairs: trajectories against lessons in ExpeL, trajectories against procedures in Memp, and text against code within procedures in Metis, which covers a single environment; none sets a lesson against a procedure built from the same trajectories. Checking a candidate before retention always changes together with something else: neither ECHO nor BREW compares relabeling with and without re-execution, BAGEL’s rerun also rewrites the precedent, and ASI’s verified and unverified arms also induce skills differently (Sections 3.2 and 3.4). Systems that combine comparison with reflection or pattern extraction, such as ExpeL and ReMe, do not report what comparison contributes, which leaves Training-Free GRPO’s single-attempt arm as one of the few controlled contrasts between two formation mechanisms (Section 3.3). Future comparisons should also report construction cost and include a baseline that keeps the source trajectories under the same budget, since such a baseline remained competitive in the settings of Zhang et al. (2026a) (Section 3.3).

3.5.3 Learning to Form Experience from Later Practice

Experience formation can be trained through feedback from later task practice, using the contribution of candidate experience to task performance as the learning signal. Evolving-RL (Fan et al., 2026b) trains one model both to extract textual skills from a source trajectory and to solve related tasks with them, rewarding extraction by the average reward its skills obtain on those tasks. Part of its gain, however, sits in the model’s parameters, since the trained model also improves when no skills are injected. Its Mind2Web evaluation has a separate scope limit: the agent may collect experience from test tasks, so those results measure adaptation rather than transfer to unseen tasks.

Freezing the agent that acts separates improvements to experience formation from changes to the executor’s parameters. Retroformer (Yao et al., 2024) is an early instance: it fine-tunes a LongChat-7B reflection model

Table 10 Choosing an experience-formation mechanism. Group headers state what later decisions need from each form; each row gives the trajectories a mechanism works from, how it typically goes wrong, and the check that catches that failure.

Mechanism	Trajectories it works from	How it goes wrong	What catches it
Trajectory-level experience: later decisions need a concrete example			
Selection and annotation	Successful or otherwise informative trajectories can be kept with their task and situation	Source success may hide detours or shortcuts; predefined labels cover only anticipated tasks	Keep outcome evidence so selection can be rechecked; record when the behavior worked
Segmentation and compression	The reusable behavior occupies only part of a long trajectory	A short span drops setup that made an action valid; a long one hides the useful behavior	Keep the span’s prerequisites and the effects of its actions
Reconstruction	Failed trajectories still complete coherent subgoals	New goals and rewritten steps rest on model judgment; the original failure stays unexplained	Mark observed versus proposed steps; rerun on the new goal when the environment can be reset
Semantic-level experience: later decisions need a rule that applies across cases			
Reflection	One trajectory and its feedback are available, without comparable attempts	Over-interpreting incidental details or crediting an unnecessary action	Separate observations from inferred explanations; keep the lesson’s conditions and source trajectory
Comparison and diagnosis	Successful and failed attempts can be aligned on task, state, or subgoal	Mistaking an uncontrolled difference, such as task difficulty, for the cause	Confirm task, tool, and evaluation alignment; test the proposed correction
Cross-trajectory generalization	A pattern recurs across task instances, attempts, executors, or local lessons	Over-extending the pattern, or losing distinctions when merging	Seek counterexamples; test beyond the source instances; compare with reusing the source trajectories directly
Procedural-level experience: later decisions need a routine for a recurring subtask			
Extraction and parameterization	Trajectories already show the subtask’s steps in a usable order	Keeping unneeded steps, reading a consistent order as a dependency, or exposing the wrong inputs	Check dependencies and preconditions; execute the procedure with changed inputs
Synthesis and composition	Trajectories, lessons, or existing skills leave steps, connections, or control logic missing	Added steps lack execution support; one component may not supply what the next assumes	Test added steps, interfaces, and branches; execute the whole composition, not only its parts

with policy-gradient updates, rating each reflection by how much the episode return changes on the next trial, while a GPT-3 or GPT-4 actor stays frozen. As in Reflexion, the reflections serve later trials of the same task. After four trials on ALFWorld, its agents succeed on every evaluated task with either actor, against about 85% with Reflexion’s untrained reflector; on WebShop the margin is at most two points. [Shalev et al. \(2026\)](#) train a Qwen2.5-7B reflector with reinforcement learning to write system prompts from an actor’s trajectories, scoring each candidate prompt by replaying tasks with the frozen actor; the trained reflectors outperform the untrained one on most held-out task families in ALFWorld and MiniHack. MCMA ([Liang et al., 2026](#)) trains a Qwen3-4B memory copilot on preference pairs ranked by downstream task performance to decide how trajectories are structured and abstracted, while the Qwen3-8B or Qwen3-32B task model stays frozen, and reports gains under distribution shift and cross-task transfer. With the actor fixed, these gains can be credited to the learned formation mechanism, within the task families each study holds out. Such training bears on improvement of the learning mechanism (Section 2.5); the changes here remain limited to one component trained by a fixed algorithm. Section 5.5 further distinguishes training experience-processing components from internalizing guidance into the executor.

Whatever mechanism formed it, each candidate should carry forward what later checks will need: its source trajectories, its applicability conditions, and the uncertainty that formation’s own checks left unresolved. These records let agents recover details omitted from a lesson or procedure. When reuse exposes an exception, comparing it with the source trajectories shows where the guidance stops holding, which later experience formation can use. The next section examines how retained experience and its supporting context are organized for access.

4 Experience Organization: Structuring Experience for Reuse

A stored precedent, lesson, or procedure derived from trajectories helps a later decision only if the agent can find it and then reach both the guidance and the context needed to read it. Finding can fail quietly: a record filed under the task in which it was learned may not match a new task that has the same problem but a different description. Even a correct match leaves four questions open:

- Q1. Was the fix that worked recorded alongside the failure?
- Q2. Is the condition under which that fix worked kept with it, so that a remedy for one cause is not applied to a different cause of the same symptom?
- Q3. Is the match one step of a longer procedure, or related to other precedents?
- Q4. Can the agent open what lies behind or inside the match—the trajectory that produced the lesson, the code that carries it out, or the individual parts of a skill—to check whether it fits or to use only the part that applies?

Success in the source episode answers none of these questions for the current task. Organization decides which search keys, links, and accompanying context a repository retains. This section asks what each structure contributes to finding the right unit and to reaching what a correct match leaves out, and what it costs to keep usable. The questions above concern only what the stored structure makes reachable; deciding when to consult a match and how to use it belongs to the utilization questions of Section 5.

We distinguish four organization patterns by the relations they retain for access:

- *Flat* organization stores independently searchable entries, each with a key, guidance, and context: the conditions under which the guidance applies and, where kept, the trajectory that supports it.
- *Tree-structured* organization adds parent–child relations that group records or relate guidance through specialization, composition, or inheritance.
- *Graph-structured* organization connects a match to related precedents, observed corrections, possible continuations, or supporting skills through relations that need not follow a hierarchy.
- *Hierarchical graphs* connect units across layers: layered variants link a unit to material of a different kind, such as the trajectory it came from or the code that implements it, and nested variants expose the internal parts of a matched skill. In both, the matched unit stays connected to other units, and its source, implementation, or parts are opened only when needed.

Figure 10 contrasts these structures; Table 11 maps them to the information needed after a match (Q1–Q4). Within each pattern, we compare what is organized, what its relations mean, what a match returns, and the costs of keeping it usable. Section 4.5 brings these comparisons together as design choices and evidence requirements. General-memory studies appear where they clarify access, marked as adjacent evidence in the sense of Section 2.6.

4.1 Flat Experience Organization

Flat experience organization is the simplest way to make experience reusable: each unit is stored as an independently searchable record that associates a search key with guidance and its context, and is found by matching the key against the current task. The main design choices are which description serves as the key and which additional signals accompany each record; we cover keys based on task or skill descriptions, keys based on conditions of use, and per-entry signals, and end with where independent records fall short.

Task and skill descriptions provide search keys for locating richer experience content: Voyager (Wang et al., 2024a) indexes executable skills by their descriptions, and ReasoningBank (Ouyang et al., 2026) matches the current request against stored task queries to retrieve reasoning guidance. Synapse (Zheng et al., 2024) and ExpeL (Zhao et al., 2024a) also match the current task against descriptions of stored tasks, but Synapse returns trajectory exemplars and ExpeL returns demonstrations together with distilled insights. The information matched thus differs from the content supplied, and the key determines which later tasks can find the record: a reasoning item keyed by the query of its source task is found only by tasks that resemble that query and can be missed by a different task that it would help.

Condition-oriented indexing makes the situation in which guidance may be useful an explicit search key.

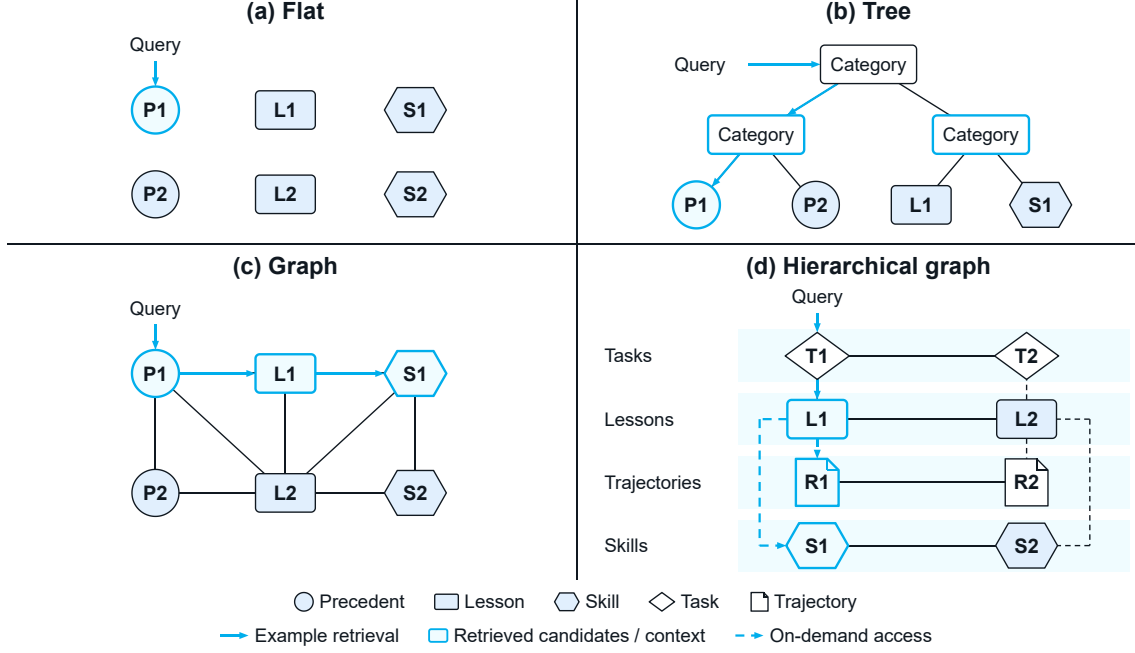


Figure 10 Example retrieval paths over four experience organization patterns. (a) Flat: match an entry’s search key. (b) Index tree: follow a category branch to a record. (c) Graph: match a seed and expand to related candidates. (d) Hierarchical graph (layered variant): enter through a task, reach a lesson, and access its source trajectory or implementation on demand. Shapes distinguish precedents (P), lessons (L), skills (S), tasks (T), and source trajectories (R). Cyan arrows show example retrieval, cyan outlines on experience and source nodes mark retrieved candidates and context, and dashed cyan arrows show on-demand access. Trees need not be traversed level by level, and retrieval does not fix the final selection.

AutoGuide (Fu et al., 2024) keys guidelines by natural-language descriptions of their applicable contexts, and ReMe (Cao et al., 2026) associates distilled experience with usage-scenario descriptions, so guidance can be found without matching the complete source task. MemGovern’s repair cards (Wang et al., 2026e) go further by indexing what is observable before the remedy is known: a normalized problem summary and diagnostic signals such as exception types, error signatures, and component tags. A condition key shifts the burden to whoever writes it, who must state the condition precisely enough that a similar task finds the guidance and a task that violates its assumptions does not.

Besides a key, guidance, and context, a record can carry signals about its own reuse, which help distinguish entries with similar descriptions without introducing relations among them. Feedback-revised weights in Live-Evo (Zhang et al., 2026m) and learned utility values in MemRL (Zhang et al., 2026j) summarize past reuse, whereas LivingRAG (Cui et al., 2026) keeps with each verified experience a sparse map of the knowledge-graph entities its retrieval activated, so a new query is matched by the entities it activates as well as by its wording. A usefulness value estimates utility, which depends on the executor and task, rather than the reliability of the guidance itself (Section 2).

Flat records suffice only when their keys anticipate later needs and their content can be interpreted independently. A source-task key can hide guidance whose condition applies elsewhere: a lesson learned while booking a flight may help with a hotel checkout that uses the same payment form, yet the task descriptions differ, and when no key matches, the agent cannot tell that a relevant record was missed. Independent storage also leaves relationships for the agent to reconstruct: near-duplicate lessons can occupy retrieval slots that complementary guidance needs, guidance to retry a timed-out request can conflict with guidance to stop after the first timeout on a payment page, and a procedure may require a helper stored elsewhere. Copying every prerequisite into each record restores local completeness, but revising a shared helper then requires updating every copy, and per-entry usefulness signals fix neither a missing key nor a missing relation.

MemGovern’s symptom keys and short self-contained lessons show where flat storage is enough: the need is observable at decision time, and each record carries the context required to act. Missing or misleading keys should be improved first; duplicate and conflicting content is a maintenance problem (Section 6), and a match that needs other records or another representation leads to the structures in the following subsections.

4.2 Tree-Structured Experience Organization

When a repository holds many similar entries, an agent needs somewhere narrower to look; when a matched routine is made of smaller steps, it needs to reach the step it can use or the routine that contains it. Tree-structured organization addresses both with parent–child links, which an agent can follow down to narrow the search or up to recover the containing context. An *experience index tree* groups records under search categories, such as a category that collects checkout precedents; an *experience content tree* relates reusable guidance through abstraction, composition, or inheritance, as a checkout routine contains its payment step. Tree-structured here refers to access through parent–child links, and the topology need not be a strict tree: shared components may have several parents, and one node can both guide search and carry guidance. Table 23 in Appendix B compares the designs by what their links mean. Index trees, whose links only organize access, come before content trees, whose links relate the guidance itself, and the subsection closes with the records and dependencies that tree access can miss.

4.2.1 Experience Index Trees

An experience index tree groups records under progressively narrower access points: a path from shopping tasks to checkout to payment-form precedents, for example, narrows where a web agent looks. In some systems the levels are fixed and search follows them: topic groups in H-MEM (Sun et al., 2026a), an adjacent dialogue-memory system, narrow the search by subject, whereas stage nodes in HMT (Tan et al., 2026c), such as searching for products or completing checkout in a shopping task, connect the current goal and page state to relevant action patterns.

Tool-based access instead lets the agent choose how to inspect the hierarchy: in the filesystem study (Zhou et al., 2026a) it opens files, reads selected lines, and searches directories, and in HORMA (Hsu et al., 2026), an adjacent working-memory system, it navigates with actions learned through reinforcement learning. The returned context then depends on the agent’s browsing decisions. At ecosystem scale, AgentSkillOS (Li et al., 2026a) builds a capability tree over pools of 200 to 200,000 publicly shared skills and reports that tree-based retrieval approaches oracle skill selection, which counts only as complementary evidence because the skills’ origin is unknown.

4.2.2 Experience Content Trees

In contrast to an index tree, which only groups records, an experience content tree uses parent–child links to relate pieces of reusable guidance to one another. Its links can mean three different things: *abstraction* relates general guidance to a more concrete realization, *composition* relates a complete routine to its constituent steps, and *inheritance* relates a shared base to a variant that modifies it, such as a checkout workflow adapted to a site with an extra confirmation step.

What the agent receives depends on which level or path is returned as guidance. MetaFlowLLM (Fan et al., 2025) returns a workflow skeleton whose open subtasks the agent completes, and Tree-of-Experience (ToE) (Deng et al., 2026) conditions the answer on selected analytical paths and their reliability. PsychAgent (Yang et al., 2026f) instead selects an atomic skill suited to the current dialogue state. Composition preserves how steps work together: AWM (Wang et al., 2025e) reuses workflow components to build larger procedures, and MACLA (Forouzandeh et al., 2026) adds coordination instructions such as when to continue, repeat, or stop, which an isolated step would omit. Inheritance shares a base while retaining only what a variant changes, as in DeltaMem (Tan et al., 2026a), so a selected variant must be reconstructed from its base and the changes on its ancestor path.

Recent studies have started to measure which level to supply, and the answer can depend on the executor. STAIR (Xu et al., 2026c) abstracts each past repair trajectory into a tree that runs from fine-grained diagnostic actions to repair strategies, and adapts nodes from several levels into a plan for a new issue. With the

abstraction built once offline and MiniMax M2.5 as the repair agent’s backbone, plans drawn from all levels resolve 80.0% of a 125-issue SWE-bench Verified subset, against 58.4–64.0% for any single level and 57.6% for raw trajectories. SkillX (Wang et al., 2026a) builds planning, functional, and atomic skills from trajectories; on AppWorld, planning skills reduce execution steps for every backbone tested, whereas for Qwen3-32B adding functional and atomic skills can lower performance because the model imitates them too closely.

4.2.3 Branch Omissions and Inherited Dependencies

Branch-restricted access can silently omit relevant records: if a precedent for an expired login session is filed under authentication failures, a search confined to checkout failures will miss it even when a checkout failed for that reason. MemTree (Rezazadeh et al., 2025) provides adjacent evidence from conversation and document memory that keeping only a few nodes per level misses information that matching across levels recovers, at the price of inspecting more candidates.

Level-removal ablations remove a level together with the content stored there, so they measure what that content contributes. With GPT-4, removing HMT’s intent and stage levels, and with them the intents and stages stored there, lowers Mind2Web cross-website step success from 39.7% to 33.2% and WebArena task success from 38.7% to 32.1% (Tan et al., 2026c). H²R (Ye et al., 2025) gives a similar ablation for a planner and an executor within one agent: it keeps planning lessons and execution lessons at separate levels, retrieved by the planner and the executor respectively. With Qwen3-235B-A22B, removing the high-level or the low-level memories lowers PDDLGame success from 80.5% to 52.8% or 61.1%, averaged over three runs on three held-out test episodes, which gauges how much each kind of lesson contributes.

Content trees introduce dependencies between the returned part and its ancestors: a variant such as “pay with a saved card instead of entering one” needs its base checkout procedure, and revising a shared base can affect every dependent variant. Moreover, storing only changes does not make access cheaper: DeltaMem’s residual nodes reduce repeated content, but matching still scans all nodes, and a matched variant must then be reconstructed along its ancestor path. Such costs are acceptable when what a match lacks lies along its path in the hierarchy. An observed correction, a possible continuation, or a dependency across branches lies off that path and needs graph relations.

4.3 Graph-Structured Experience Organization

Graph organization connects experience through explicit relations of any kind, not only parent–child links, so following a link can return material that would not have matched the request alone, such as the correction recorded for a failed attempt (Q1) or a continuation or supporting skill (Q3). We distinguish *record graphs*, which connect precedents and lessons, *transition graphs*, which connect operations through possible continuations, and *skill graphs*, which connect capabilities through dependencies and other joint-use relations. Figure 11 contrasts the three, and Table 24 in Appendix B compares their nodes, edges, and value for reuse.

4.3.1 Record, Transition, and Skill Graphs

In a *record graph*, a link helps most when it says how the connected record relates to the match. Associative links in A-MEM (Xu et al., 2025b), an adjacent general-memory system, and in ExpGraph (Feng et al., 2026a) and CoEvo-Mem (Ye et al., 2026a) expose neighboring material without explaining how to use it with the match. EXG’s directed *fixed_by* relation (Jin et al., 2026) explains more: it leads from a failure to an observed correction, answering Q1 without requiring the agent to know the remedy in advance, though the recorded fix still needs its applicability checked. A relation may also carry guidance itself: EMG (Wang et al., 2026f) places cross-task lessons on edges, so access that returns only the connected records would omit part of the experience.

Transition graphs answer what may follow the current operation; the designs below vary the context attached to each transition and how far ahead access looks. H-EPM (Li et al., 2026c) suggests next tools from observed transitions and their weights, and Procedural Graphs (Lu et al., 2026b) attaches conditions, guidance, and pitfalls to transitions and revises them from execution feedback. ProPlay (Ma et al., 2026b) looks further ahead, assembling a whole path of reusable procedures before the task begins; the path preserves more

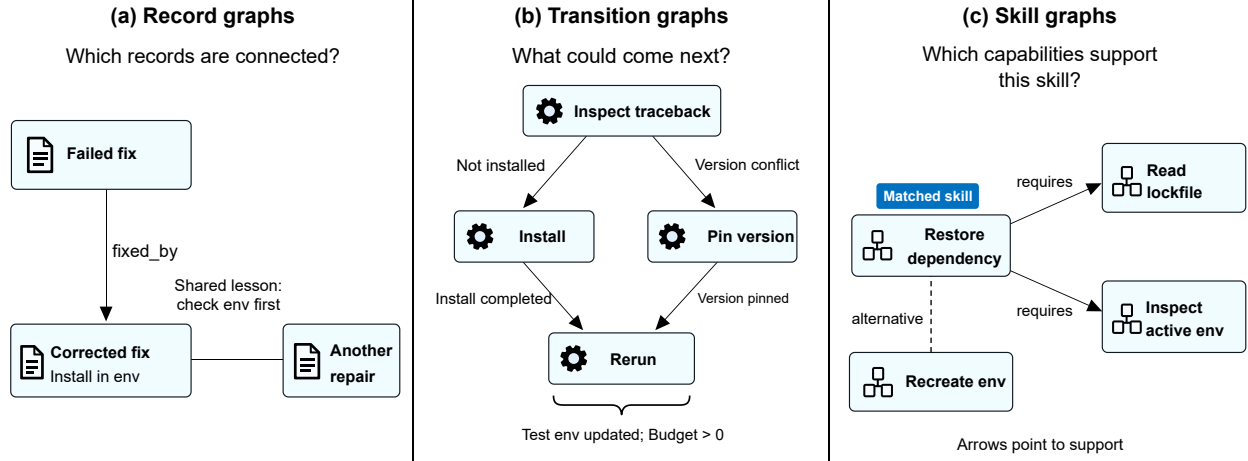


Figure 11 Three uses of graph relations in import-failure repair. (a) Record graphs link a failed fix to its observed correction through `fixed_by`; another relation carries a shared lesson. (b) Transition graphs connect operations through state-dependent candidate continuations. Rerunning the tests requires a completed installation or version pin, an update to the environment the tests use, and remaining budget. (c) Skill graphs connect a matched capability to supporting skills through `requires` relations and to an alternative through a dashed link. Dependency arrows point toward support, not in operation order.

procedural context but may include steps the current state does not need. Whichever design stores it, an observed transition still has to be checked against the current state before it is followed.

Skill graphs preserve what a capability needs and how it relates to alternatives, so a related skill may be required support, a possible substitute, or something to avoid combining with the match. SkillDAG (Bai et al., 2026) distinguishes support from similarity and conflict, while Graph-of-Skills (GoS) (Liu et al., 2026a) recovers prerequisites under a content budget.

In addition, a stored relation needs evidence for the connection it asserts. SkillDAG retains relation edits supported by execution checks. CaSKG (Li et al., 2026h) instead calibrates candidate relations with LLM-scored textual probes, which evaluate the relation without demonstrating causal necessity. The relation’s label therefore needs to be read together with how it was checked.

Some connections require context shared by several skills or by parts of one skill. SkillCAT (Chen et al., 2026e) supplies a selected component with its enclosing skill’s common instructions. HyperSkill (Xu et al., 2026b) uses a *hyperedge*, a single relation joining several nodes, to keep the context and lesson of skills used together in one trajectory, although skills used together need not each require the others. SkillLens (Miao et al., 2026) and HiSkill (Hao et al., 2026) combine decomposition with lateral or transition relations: decomposition locates a part, and the other relations expose its connections.

4.3.2 Unreliable Links and Costly Expansion

Graph access can fail through an unreliable link or an expansion that exposes too much material. An observed correction, a semantic neighbor, and an LLM-assessed dependency have different evidential support, and treating them as equally actionable can introduce misleading guidance.

Stored relations can help, but how access follows them can decide whether they help at all. With GPT-4.1-mini, removing ProPlay’s graph lowers ScienceWorld success from 37.4% to 34.8% (Ma et al., 2026b). In GoS, the direction of propagation makes that difference. Its SkillsBench evaluation uses a 1,000-skill library that adds skills of unreported origin to the curated SkillsBench skills, so its results are complementary evidence. With GPT-5.2 Codex, it reaches an average reward of 34.4, versus 29.3 for the cheaper variant that uses seed retrieval alone and 25.3 when propagation drops its reverse transitions and follows only forward edges (Liu et al., 2026a). Following only forward edges thus does worse than not propagating at all. GoS’s budget-matched offline test on the curated 200-skill library ranks the variants in the same order by how many dependent skill

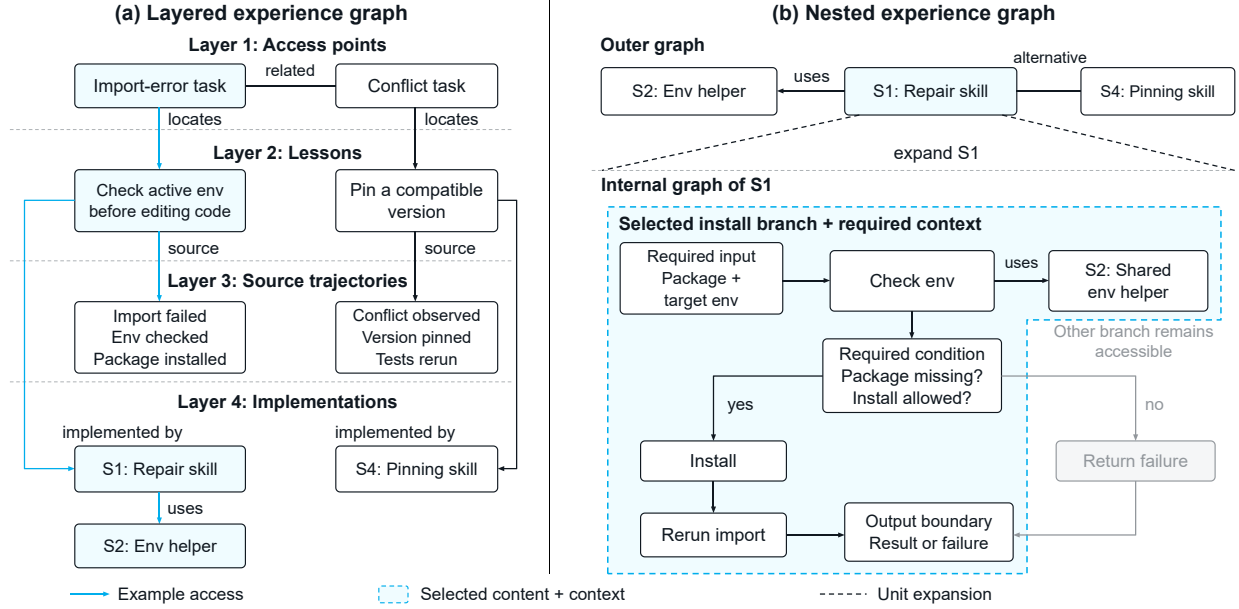


Figure 12 Layered and nested experience graphs for import-failure repair. (a) Layers distinguish kinds of material: a task match locates a lesson, whose links provide access to its source trajectory or implementation S_1 and helper S_2 . (b) S_1 remains connected to other units in the outer graph and expands into an internal graph. The highlighted installation branch retains its required input, applicability check, helper dependency, and output boundary; the gray failure branch remains accessible. Cyan marks example access and selected content. Horizontal dashed lines separate layers; dashed connectors show unit expansion. Repeated identifiers denote the same units across panels.

pairs they recover together: best with reverse-aware propagation and worst with forward-only propagation. SE-GoS (Fu et al., 2026), which revises topology, edge weights, and node descriptions together, points the same way: its own ablation credits the edge weights, not the topology, with the largest gain.

Expansion and construction add costs. More hops expose more material to inspect: under its content budget, GoS uses less than half the tokens needed to load its full 1,000-skill library (Liu et al., 2026a), and G-Memory’s hop comparison shows that wider access need not improve performance (Zhang et al., 2025). Building and checking relations is a separate cost, which SkillDAG reports only for initializing its graph (Bai et al., 2026). These costs are worth paying only when a match needs corrections, continuations, or support it could not otherwise reach, and only if the relations can be checked and expansion kept within an affordable inspection budget.

4.4 Hierarchical Graph Experience Organization

A correct match may still leave the agent needing a lesson’s source trajectory, its implementation, or just one part of a long skill (Q4). Hierarchical graph experience organization keeps this material reachable without returning it all at once. In Figure 12, the agent follows a repair lesson to its implementation S_1 and opens only the installation branch. Layered experience graphs serve the first two needs by linking a unit to material of a different kind, and nested experience graphs serve the third by exposing a unit’s internal parts; the costs of keeping these correspondences usable are examined after both (Table 25 in Appendix B).

4.4.1 Layered Experience Graphs

Layering helps when the matched guidance is right but using it requires a different kind of material. An agent that reaches a lesson through a matching task may then need the trajectory in which the lesson was learned, to judge whether it applies, or the skill that carries it out. Each need has its own kind of link: access-point links route a task to guidance, source links attach guidance to the evidence for interpreting it, and implementation links attach it to the means of carrying it out.

Simpler designs can make this access costly. Copying a shared source or implementation into several lesson records creates consistency work (Section 4.1). Linking the source as one more neighbor without marking it as a source instead leaves retrieval to distinguish a source trajectory from another lesson. Layers keep each kind of link separate, so retrieval can request a lesson’s source or implementation by link type. What makes a design layered is that its links connect different kinds of units, such as a lesson and its source trajectory, not that the layers are stored apart; a record graph instead links units of the same kind, such as one lesson to another.

Each reviewed system provides only some of these correspondences. G-Memory (Zhang et al., 2025) uses task queries to connect returned insights to their interaction context, trainable graph memory (Xia et al., 2025) routes a task toward a strategy through intermediate decision patterns, and PlugMem (Yang et al., 2026a) combines access-point links and source links: shared intents expose prescriptions across tasks, and source links keep their trajectories accessible. Metis (Dai et al., 2026b) instead links plans to the code tools and helper functions that implement them, recovering the means of carrying out guidance where PlugMem recovers the evidence behind it.

4.4.2 Nested Experience Graphs

Nested experience graphs let an agent inspect or use part of a matched unit while preserving the unit’s relations to other units. In Figure 12(b), expanding S_1 exposes the installation branch together with its required input, applicability check, and shared helper. The agent can then check whether installation is warranted and take that branch without loading the alternative failure branch.

In the reviewed designs, expansion serves to check fit, to control detail, or to compose plans. SkillOps (Pu et al., 2026) uses each skill’s internal contract graph over preconditions, operations, artifacts, validators, and failure modes to check fit before use, keeping only candidates whose preconditions hold and chaining skills only when one’s output matches the next one’s input; the skill is still used whole. SkillZip (Tan et al., 2026b) controls detail, presenting a macro as a name, contract, outline, or full source and expanding it together with the dependencies its selected parts need. DocTrace (Zai et al., 2026) composes plans, letting a child question pattern expand into a separately retained reasoning subplan at the point where it fits.

Nested access differs from nearby designs in keeping a unit’s links to other units while exposing its internal graph. Content trees and tree-graph combinations such as SkillLens also relate a routine to its parts. However, they store the parts as separate units cut in advance and return one of them. NSI (Shao et al., 2026b) and Skill-DisCo (Guo et al., 2026b) learn internal graphs but do not relate the enclosing unit to other units.

4.4.3 Stale Correspondences and Partial Expansion

Access across layers and into a unit’s parts fails when a correspondence becomes stale or expansion omits required context: a lesson can cease to describe a revised implementation, and an expanded part is incomplete without the inputs, checks, or helpers it shares with the rest of its unit. The available evidence of benefit concerns layered designs and supports retaining source context while limiting how much of it is exposed. With Qwen-2.5-14B, G-Memory scores lower when using only interactions or only insights than when using both, and its two- or three-hop expansion often scores below one hop, for example falling from 55.24% to 49.79% at two hops for AutoGen on PDDL (Zhang et al., 2025). With an AgentOccam agent on GPT-4o and memory calls run mainly on Qwen2.5-32B-Instruct, PlugMem’s “No Structuring” variant lowers WebArena GitLab success from 51.4% to 41.7% online and from 55.2% to 42.0% offline, but it also removes abstraction and indexing (Yang et al., 2026a).

Reaching a lesson’s source or implementation can therefore help, and the case for layering is strongest when a match requires material that many units share or that is too long to return by default. Nesting is worth its cost when a matched unit must be checked or trimmed and its internal parts remain usable with their context. Evidence on upkeep is thin: for nested designs, SkillOps reports only that a mostly rule-based maintenance pass keeps contracts checked cheaply, on libraries of up to 2,000 skills that pad 229 curated skills with duplicates and synthetic degraded variants.

Table 11 Choosing experience organization by what it takes to find the right unit and by the information a correct match leaves out (questions Q1–Q4 at the start of Section 4). The patterns can coexist: for example, a searchable skill record may link to supporting skills and expose its own internal graph.

Pattern	Useful when	What access returns	Upkeep and failure modes
Flat	To find: a symptom, task, or usage description matches the request. After a match, Q1–Q2: the record itself holds the repair and its conditions.	The matched record: guidance, its conditions, and, where kept, its source trajectory and per-entry signals.	Keys must anticipate later needs and stay current. A missing key fails silently; relations among entries stay implicit.
Tree	To find: many similar records call for a narrower place to search. After a match, Q3: the missing context is a broader group, a containing routine, or a base procedure.	A group of records, a level of guidance, or a variant with its base.	Groups and parent–child links must be maintained. Branch choices can omit records; revising a shared base affects every variant.
Graph	After a match, Q1 or Q3: the repair, a continuation, or a supporting skill sits in another record that would not match on its own.	Connected records, lessons placed on edges, or required skills.	Relations must be built, checked, and updated with their endpoints. Unreliable links and wide expansion mislead or overload access.
Hierarchical graph	After a match, Q4: the match needs its source trajectory or implementation, or must be checked or trimmed before use.	Source context, implementations, or selected parts with their dependencies.	Cross-layer correspondences and part boundaries must stay consistent. Stale links mislead; expansion adds inspection cost.

4.5 When to Add Structure

Whether to add structure depends on why an agent cannot find relevant experience or cannot interpret and use what it finds. The first decision is whether better search keys can solve the problem; if they cannot, the missing context or relation determines what structure to add (Table 11). We examine these choices together with their costs, then ask what the reported ablations actually vary and what a fair comparison of structures would hold fixed.

4.5.1 Better Keys before Added Structure

Better keys are the cheaper remedy and the one to try first: the right record is often missed only because of how it is keyed. A record that holds the fix and its condition needs no further structure; if a new failure does not find it, the first remedy is to key it by the observable symptom, as MemGovern’s repair cards do with error signatures, instead of by the source task. Structure then helps in two situations: before a match, an index tree can narrow where to look when many records share similar keys, as HMT’s stage level does for a web agent; after a correct match, Table 11 identifies which pattern supplies what is still missing. Each added structure should then be compared with the improved-key baseline and kept only if the information it supplies, or the work of finding and interpreting experience it saves, justifies its upkeep.

Organization also determines which information remains accessible after a match. Condition keys make applicability searchable, and contracts in nested graphs make it checkable part by part. Tree and graph relations supply guidance missing from a matched record, while source links keep supporting trajectories reachable. Reliability evaluations can be stored beside paths or transitions, as in ToE and ProPlay (Deng et al., 2026; Ma et al., 2026b). In the terms of Section 2, these structures keep a unit’s conditions c_i , evidence e_i , and reliability r_i reachable when the agent decides whether to use it.

Reaching this information still leaves the agent to judge whether the guidance is reliable and fits the current task. SkillDAG’s execution-checked edits and CaSKG’s probes test particular relations, which says little about how well supported the connected guidance is (Section 4.3.2). The practical test of organization is whether the agent can reach the information it needs to judge fit when selecting and applying experience (Section 5.2.2).

Each added structure also has a price. Explicit relations take work to build and update, but reduce the agent’s need to reconstruct them during a task. Building them pays off only if they are checked during construction and revisited when their endpoints change (Section 6). Inspection cost arises at use time: returning more

related records gives the agent more to read, and reachable material is not necessarily useful, as G-Memory’s hop comparison shows. Organization can keep each returned unit complete for its intended use, as when a nested part arrives with its inputs, conditions, and shared helpers, while how much to return remains a selection decision (Section 5.2). Reported evidence on both costs is thin: whether a structure is affordable depends on how often experience changes and how much material the agent must inspect, yet we found no study that reports how often correspondences become stale.

4.5.2 What Experience-Organization Ablations Vary

The design rules above go beyond the evidence: the comparisons in Sections 4.2–4.4 explain choices within particular systems but do not compare structures. Only two hold the stored content fixed, and both vary a setting within one structure: GoS’s budget-matched offline test changes propagation direction within one graph, and MemTree varies the per-level budget within one tree. The other ablations change what is stored, delivered, or retrieved along with the structure, in three ways. Removing a level removes its content: HMT’s and H²R’s levels take their intents, stages, or lessons with them, and STAIR and SkillX vary which levels are supplied while the tree stays in place. Removing edges removes what is built on them: EXG’s error–repair hints come from its correction edges (Jin et al., 2026), and EMG’s edges carry lessons, with effects of their removal that depend on the backbone (Wang et al., 2026f). The remaining baselines change more than the structure: ExpGraph’s flat baseline also changes retrieval (Feng et al., 2026a), CoEvo-Mem’s variant without memory evolution drops value propagation (Ye et al., 2026a), SE-GoS revises topology, edge weights, and node descriptions together, AgentSkillOS compares tree retrieval with handing the agent the whole pool instead of a well-keyed flat index, and PlugMem’s no-structuring variant also removes abstraction and indexing. For layered and nested designs, no reviewed study compares the design with a simpler one that keeps the same material, or removes only selective expansion while keeping the stored material fixed. The evidence therefore cannot yet say whether structure, apart from the content it holds, explains any of these gains, whether edge types differ under matched retrieval, or when a well-keyed flat repository stops being sufficient as tasks diversify or the repository grows.

A study that attributes a gain to organization should change only the relations, or access to them, while holding the retained guidance and the retrieval budget fixed, and its flat baseline should already use improved keys; otherwise a gain credited to relations may come from better access points. For lessons placed on edges, the control stores the same lessons as independent entries; for layered and nested designs, it keeps the underlying material but removes the links between layers or selective expansion. Running these controls as task diversity and repository size grow would show where the flat baseline falls behind. Reporting the retrieval, token, and update costs, together with whether useful experience reaches the final selection (Section 7.3.1) and whether returned units keep their conditions (Section 7.2.1), would explain the result. Run on a designer’s own tasks, the same comparison shows whether an addition’s benefit justifies its upkeep.

Section 5 turns from what an agent can reach and check in retained experience to how it uses that experience on the current task: when to seek it, which candidates to select, and how to adapt and apply them or proceed without them. Later trajectories then supply evidence for revising both the guidance and its organization.

5 Experience Utilization: Guiding Action with Past Experience

An agent that holds useful experience may still overlook it, follow a familiar procedure under the wrong conditions, or spend more effort consulting experience than the task requires. *Experience utilization* uses retained experience to guide execution or shape behavior learned through training. At runtime, it supplies situated guidance for current decisions, and new observations that contradict this guidance should be able to override it; during training, it can shape trajectories and supervision to develop the executor’s capabilities.

Whenever retained experience might bear on the current step, the agent faces four connected runtime decisions: whether to look for guidance now or keep exploring, which retained experience fits the current task and environment, how to bind it to the objects and tools in front of it, and how the resulting guidance should shape its next action. This section examines them as four questions:

- *Experience triggering*: when should an agent consult experience?

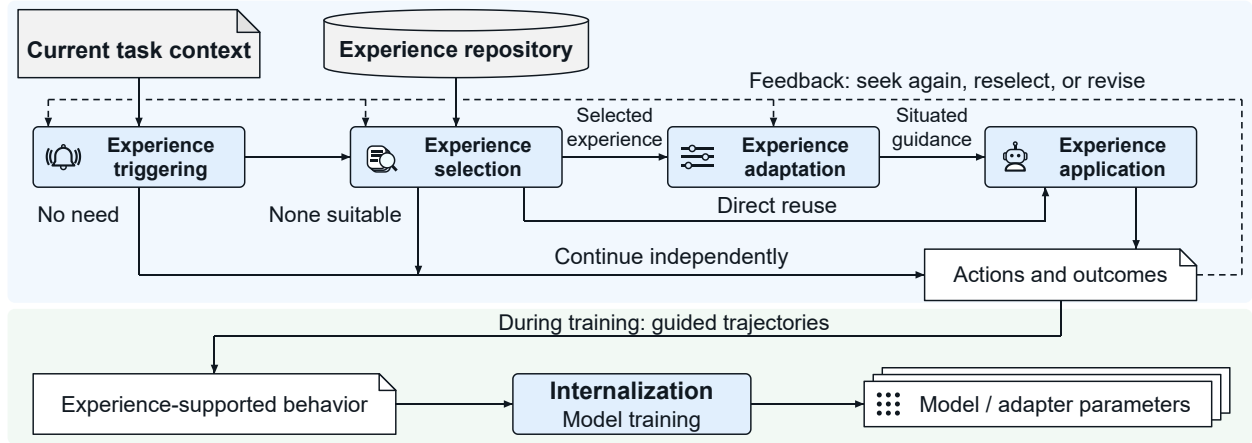


Figure 13 Experience utilization at runtime and during training. The upper workflow connects triggering, selection, adaptation, and application, with paths for direct reuse and for continuing independently without experience. Dashed arrows return feedback for renewed seeking, reselection, or revision. The lower path shows how experience-guided trajectories can support model training to internalize behavior into model or adapter parameters. Runtime decisions need not be separate calls, and revising current guidance does not itself update the repository.

- *Experience selection*: which experience, if any, is worth using?
- *Experience adaptation*: what, if anything, should change before reuse?
- *Experience application*: how should the resulting guidance influence behavior?

Figure 13 connects these runtime decisions, including paths for direct reuse, continuing without experience, and renewed seeking after feedback. The subsections take the four questions in the order listed, and the one on selection also covers how shared experience reaches several agents. Experience internalization, where experience-supported behavior is transferred into model or adapter parameters during training, follows the four runtime questions. The closing subsection compares the evidence for these decisions, which share one budget, and discusses where independent checks of experience-guided actions belong.

5.1 Experience Triggering: When to Seek Experience

Consulting experience can help an agent escape avoidable mistakes, but routine steps or sufficient existing guidance may make another consultation unnecessary. *Triggering* determines when to consult experience: an unexpected error may justify seeking guidance, whereas consulting experience before every command would mostly add cost. A trigger can be described by what it observes, which step it gates (a check for possible need, retrieval itself, or the injection of already retrieved guidance), and when its cost is incurred. Predefined schedules, signals observed during execution, and learned policies are described first and then compared on these properties.

5.1.1 Experience Triggering Mechanisms: Schedules, Signals, and Learned Policies

Triggers differ in what fixes the moment of consultation, which determines how early they can act and how reliably they indicate need. *Schedule-based triggering* fixes the moment in advance; *signal-based triggering* responds to what happens during execution, relying on an observed failure, the agent’s internal state, or the judgment of a coach or of the agent itself; and *learning-based triggering*, discussed last, estimates from outcome data when consultation pays off.

Schedule-based triggering supplies experience at a predetermined point, such as task entry, so that guidance expected to remain useful across several actions is introduced without repeated retrieval decisions. ExpeL (Zhao et al., 2024a) retrieves successful trajectories similar to the new task and supplies them with accumulated insights at task start; the examples vary with the task, but the retrieval point is fixed. Agent Workflow Memory (AWM) (Wang et al., 2025e) instead keeps reusable workflows in prompt memory throughout execution, leaving the executor to recognize when a routine applies. A schedule suits guidance whose span of reuse is

known in advance. However, it cannot recognize when initial guidance becomes insufficient later in the task; signal-based triggering responds to that change.

A failed attempt signals need most concretely, though only after the failure has occurred. In Reflexion (Shinn et al., 2023), a failure triggers reflection rather than consultation: the retained reflection is supplied at the start of later attempts and cannot help the one that failed. By contrast, AMD (Kim et al., 2026b) makes the failure trigger consultation itself: it supplies planning memory before execution and retrieves function-specific hints only when a tool call returns an error, so that the agent can recover within the current episode.

Signals from the agent’s internal state allow intervention before a failed outcome but are less decisive evidence of need, because uncertainty can reflect missing guidance or several reasonable alternatives. ExpSeek (Zhang et al., 2026k) calibrates an entropy interval for each kind of step: a step below the interval receives no guidance, a step above it always does, and a step within it receives guidance with a probability that rises with its entropy. DS-MCM (Sun et al., 2026d) instead compares the semantic diversity of retrieved documents with the agent’s reasoning entropy, which can expose overconfidence as well as doubt; a large deviation triggers a slower monitor that retrieves past successes and failures to diagnose the step and may still withhold guidance.

The decision can also be left to a separate coach or to the agent itself, and the two place the gate at different steps. WebCoach (Liu et al., 2025) retrieves memories at every step and lets a prompted coach intervene only when it detects likely failure, such as loops, or a better workflow in memory, so its gate controls injection rather than retrieval. In the prompting-based variant of ExpWeaver (Zhao et al., 2026c), the agent instead gates retrieval itself by emitting a [Retrieve] token. A reinforcement-learning variant optimizes when the token is emitted and so belongs with the learned triggers discussed next.

Learning-based triggering learns when consulting experience improves task outcomes or efficiency, at the price of outcome data from which that value can be estimated. ProactAgent (Cai et al., 2026) treats retrieval as a policy action and trains when and what to retrieve with a process reward that compares continuations from matched interaction prefixes with and without retrieval. MemCon (Jiang et al., 2026b) instead explores during deployment: a lightweight bandit chooses context-dependent memory operations, such as whether and how much to retrieve, and updates its choices from task success.

5.1.2 Choosing When to Use Experience

Choosing when to use experience requires weighing the time spent retrieving and assessing experience and the extra tokens needed to process supplied guidance against the risk of acting without useful guidance. The triggers above also differ in when this overhead arises: at task entry, after a failure, at every step, or during training.

The right trigger depends on which consultation error the task can least afford: an unnecessary consultation wastes retrieval and context, whereas a missed one can cost far more when the uninformed action is hard to undo. When failures are reported and actions are cheap to retry, as with a tool call that returns an error, a failure trigger suffices. It breaks down when a failure is silent, as when a web form shows a confirmation page although the order was never recorded, and when an action is irreversible, such as deleting or overwriting files, because consultation then belongs before the action. Ambiguous internal-state signals pay off in long tasks where errors noticed late are expensive to unwind, provided that the executor’s token probabilities are accessible.

Where a gate sits, and whose judgment it relies on, also limit what a trigger can know. A gate before retrieval can estimate only whether the executor needs help, not whether the repository holds anything that would help. A gate after retrieval, as in WebCoach, can judge the content but has already paid for retrieval; that cost is worthwhile when injecting unsuitable guidance would cost more than the retrieval itself. DS-MCM stages the two, spending content-aware judgment only on anomalous steps. Triggers that rely on the executor’s own judgment or uncertainty, such as the prompted ExpWeaver variant, AWM’s standing routines, or ExpSeek’s entropy interval, miss confident errors: an agent that is confidently wrong neither asks for help nor crosses an uncertainty threshold. Calibrated and learned triggers are in turn fitted to one executor and repository, so maintenance or a change of executor can move the point at which they should fire.

The reported evidence speaks mainly to how often to consult and whether to consult at all. Within one mechanism, ExpSeek’s GAIA analysis finds diminishing accuracy gains as interventions become more frequent

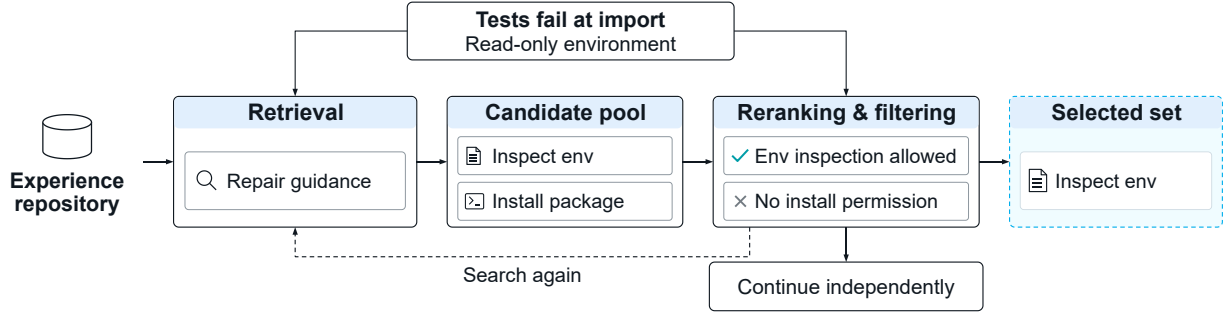


Figure 14 Experience selection for an import failure in a read-only environment. The agent retrieves repair guidance and selects environment inspection over an installation procedure that it lacks permission to run. Selection may instead renew the search or continue independently. Whether a declared dependency is in fact missing remains to be checked.

(Section 7.3.2). Whether consulting at all beats spending the same budget on the task is taken up in Section 5.6. Once assistance is sought, selection determines which experience addresses that need.

5.2 Experience Selection: Which Experience to Use

Once consultation is triggered, the agent must choose which experience, if any, to use. Similarity alone is a poor guide: a similar task may assume unavailable tools, address a different difficulty, or add little beyond the agent’s existing capabilities. Building on the indexes and relations of Section 4, *experience retrieval* constructs a candidate pool, and *experience reranking and filtering* compares candidates by current fit, expected benefit, and cost; when several agents share a repository, selection also decides which recipient receives which entry. Selection returns experience together with its unresolved conditions, or returns none, and an unsuitable pool can prompt renewed search (Figure 14). The discussion follows the same pipeline and ends with multi-agent selection.

5.2.1 Experience Retrieval

Retrieval makes promising experience available under a search budget. Relevant guidance may use unfamiliar wording, sit among superficially similar cases, or be reachable only through a link from a closer match. The candidate pool therefore depends on the request and on how it is matched, routed, and expanded.

A task description states the desired outcome but may not identify the decision that experience should support: “make the failing tests pass” could call for a debugging strategy or for a fix to one specific error. A useful request identifies the current subgoal, relevant observations and attempts, available capabilities, and the kind of help sought, and it preserves the observed symptom without treating a suspected cause as established. Requests therefore often track execution: ExpRAG (Ferraz et al., 2026) refreshes history-based queries during execution, Agent S (Agashe et al., 2025) builds separate task and subtask queries, RaDA (Kim et al., 2024) composes queries from the remaining subtasks of a plan, and Voyager (Wang et al., 2024a) and Agent KB (Tang et al., 2025) add a provisional solution or diagnosis to the request. Such hypotheses should remain revisable, because a mistaken plan or diagnosis can steer search away from the needed guidance. ProactAgent learns the query together with the decision to retrieve, so its gains do not isolate request quality.

Direct matching finds experience whose indexed description expresses the current need. ExpeL and ReMe compare task and usage-scenario embeddings (Zhao et al., 2024a; Cao et al., 2026). The matcher should suit the indexed representation: an exact error message, for instance, may favor lexical matching. In Agent KB’s experiments, hybrid lexical and semantic matching outperforms either alone when experience is distilled into concise summaries of its source trajectories, while the margins narrow once a teacher agent’s critiques of errors refine that experience. SEER adds toolchain coverage and user-intent alignment to trajectory similarity (Cui et al., 2025), although its unordered tool sets leave execution order unchecked.

Routing directs search toward a relevant topic or retrieval channel before candidates are compared. ExpSeek selects likely error topics from the interaction history before reading their experience (Zhang et al., 2026k),

and CoEvo-Mem trains a router that weights dense and sparse retrieval channels (Ye et al., 2026a). This concentrates search effort; however, a mistaken diagnosis can exclude useful experience before matching begins.

Expansion recovers related experience that would not match the request on its own. G-Memory adds the one-hop neighbors of matched queries before ranking and condensing them (Zhang et al., 2025), and EXG adds corrected counterparts through `fixed_by` links (Jin et al., 2026). A request that matches a failure can thus reach its repair even when the repair’s description does not resemble the request.

Candidate coverage limits every subsequent choice, because reranking cannot recover an experience absent from the pool. Reformulation helps when the request misstates the need. When a diagnosis has excluded relevant groups, routing must be broadened instead, and a remedy stored apart from its matching case is reached by expansion; each of these adds search cost. Furthermore, search effort and delivered context need separate budgets: EXG caps candidate discovery and assembles selected hints under a delivery budget, so a broad search need not produce a large executor prompt.

5.2.2 Experience Reranking and Filtering

Retrieval returns candidates, but a high rank reflects only the scoring rule, not whether a candidate fits the current situation or adds anything to what the agent would already do. *Reranking* orders candidates by priority, and *filtering* decides which to keep, possibly none, by asking whether a candidate fits the current context, whether its operating conditions hold, whether its expected benefit justifies the remaining cost, and what it adds to the rest of the selected set.

A matcher tuned for coverage need not rank the final candidates well, so ReMe (Cao et al., 2026) applies an optional LLM reranker conditioned on the task’s context, constraints, and objectives. The query-conditioned reuse (QCR) study by Li et al. (2026g) measures how much such a choice step loses: across WebArena, WorkArena, and AppWorld targets, the five trajectories retrieved from a frozen bank include one that human annotators judge reusable for 97.8% of targets, and summary-based reranking selects a reusable one for 94.8%. These rates show that a choice step can preserve most of the coverage that retrieval provides, provided that compact descriptions keep the conditions and outcomes that distinguish candidates. The targets, however, are variants built from tasks in the bank, so the rates say little about coverage for unrelated new tasks.

A candidate retrieved because it resembles the current task may still be unusable: the Experience-Following study (Xiong et al., 2026) documents errors in retrieved records that the agent replicates, and seemingly correct past trajectories that offer limited or even misleading guidance when replayed. Applicability compares a candidate’s conditions c_i with the available evidence about the task, tools, permissions, and environment, leaving each condition supported, contradicted, or unresolved; the candidate’s execution evidence e_i and reliability r_i describe how well it was supported in the settings where it was formed. A contradicted essential condition excludes the candidate unless the condition can credibly be restored; an installation procedure, for instance, cannot run where the agent may not modify the environment. An unresolved condition, such as whether the tests run in the environment the agent is inspecting, should be checked when the expected benefit justifies it, and otherwise must be passed on to adaptation and experience application as an explicit open check. Systems evaluate fit with varying explicitness: AutoGuide retrieves guidelines for the context it identifies at each step (Fu et al., 2024), Agent KB exposes constraints alongside its workflows (Tang et al., 2025), and SkillLens uses a verifier to accept, decompose, rewrite, or skip partially relevant skill units (Miao et al., 2026), though none of them checks every environmental prerequisite.

Skills that people author from their own practice usually arrive with stated conditions c_i but without recorded execution evidence e_i (Section 2.6), so filtering them rests on those stated conditions and on checks of the current environment. Evidence on this filtering is indirect: SkillsBench compares agents with and without such skills, contributed from public repositories or the contributors’ own domain experience (Section 7.4.1), so its results bear on how authored skills are used as a whole rather than on how they are filtered.

Applicable candidates still differ in expected benefit, failure risk, and adaptation cost, and continuing without experience remains an option when a valid precedent merely repeats what the agent would already do. MACLA (Forouzandeh et al., 2026) combines a Bayesian estimate of procedure success with relevance, risk, and an exploration term, and falls back to the base model when no candidate clears its utility threshold. Outcome-trained selectors such as MemRL and Memento’s parametric variant learn utility values for candidates (Zhang

et al., 2026j; Zhou et al., 2025). Because jointly retrieved memories share outcome-level rewards, however, a high historical utility need not reflect an entry’s marginal contribution (Yang et al., 2026d). Neither selector estimates the cost of checking or adapting a candidate.

Moreover, a set of individually useful entries can fail as a whole, because its entries may repeat guidance, conflict, or jointly omit a dependency. Metis adds the code helpers that selected plans depend on (Dai et al., 2026b), though not checks that the combined steps fit the task, and retrieved-count analyses in ReMe, MemRL, and Memento show that delivering more entries does not reliably help. Weighing candidates both individually and as a set, selection ends in one of three outcomes: a compact set whose expected benefit justifies the remaining costs, a renewed search when the pool misses the need, or continuing without experience.

5.2.3 Selecting Experience for Multiple Agents

When several agents share one repository, selection must also decide which stored guidance each recipient receives. Agents can play different roles, run on different backbone models, or differ in capability: an entry that helps an orchestrator plan may distract an agent executing a subtask, and a constraint learned from one model’s errors may not apply to another.

Existing systems allocate shared experience in three ways. The simplest gives every recipient the same experience without choosing entries for each: G-Memory exposes its linked insights and interaction records to a whole team (Section 4.4), and AMD (Kim et al., 2026b) gives small 4B–8B students a bank that a teacher formed from its own successful trajectories. FORGE (Bogdanov et al., 2026) likewise copies the best-performing instance’s memory to every instance still in training after each stage, a persistent overwrite that also belongs to maintenance (Section 6). LEGOMem (Han et al., 2026a) instead allocates by role, giving full-task memory to the orchestrator and subtask memory to the task agents, and MemCollab (Chang et al., 2026) allocates by source model, keeping for each recipient only constraints whose contrast involved its own backbone.

Evidence on matching guidance to its recipient so far comes from three isolated results. LEGOMem’s placement ablations on OfficeBench show that orchestrator memory matters most, MemCollab keeps model-pair labels because directly reusing another model’s memory sometimes lowered accuracy, and AMD finds on AppWorld that a more accurate teacher did not yield more helpful memory for its 4B student.

5.3 Experience Adaptation: What to Change before Reuse

Experience selected for its fit may still not be usable as stored: it may refer to other objects, assume other interfaces or dependencies, or end with a different completion criterion. *Experience adaptation* instantiates, rewrites, or restructures selected experience for current conditions while preserving guidance supported by its source evidence. If the differences do not affect reuse, direct use avoids unnecessary rewriting and possible distortion.

This view builds on case-based reasoning (Aamodt and Plaza, 1994), whose retrieve–reuse–revise–retain cycle recent work revisits for LLM agents (Hatalis et al., 2025). Agents can adapt experience through reasoning during execution or through a separate transformation step; when adaptation is mixed into execution, identifying what changed and measuring its contribution becomes harder.

Adaptation changes experience in three complementary ways:

- *Contextual grounding* connects references to current entities and operations.
- *Guidance specialization* derives situated guidance from retained content g_i .
- *Procedural reconfiguration* changes steps or dependencies.

They may be needed separately or together. *Feedback-guided revision* cuts across them: it corrects mismatches revealed during use, or prompts reselection or continuing without experience.

Consider a web agent for which selection returns a checkout trajectory recorded on another shopping site. Contextual grounding identifies which panel on the current site plays the role of the earlier site’s shipping form. Guidance specialization reduces the long trajectory to the check that matters now, such as confirming that the item ships to the user’s country before adding it to the cart. Procedural reconfiguration inserts a

Table 12 The three types of experience adaptation and their subtypes, each shown on an online shopping task.

Transformation	Retained experience	Current situation	Adapted guidance for the current task
(a) Contextual grounding: connect references to current entities and operations			
Entity binding	Enter the shipping address in the <i>Ship to</i> form on the checkout page.	The current site collects the address in a <i>Delivery details</i> panel.	Bind the shipping-address role to the <i>Delivery details</i> panel.
Tool mapping	Apply a discount by typing the code into the coupon field and pressing <i>Apply</i> .	The agent works through the store’s API, which offers a call for applying promotions.	Map the apply-discount operation and its code argument to that API call.
Argument instantiation	A retained search skill takes a product name and a size.	The user asks for a “medium” shirt, and the site lists sizes as S, M, and L.	Bind the size argument to the site’s value M, not to the user’s word.
Precondition check	Pay with the saved card on file.	The account holds two saved cards, and the request does not say which to use.	Flag the ambiguous binding; settle which card to use before paying.
(b) Guidance specialization: derive situated guidance from retained content			
Trajectory distillation	A trajectory records searching, opening several listings, adding one item to the cart, and only at checkout finding that it does not ship to the user’s country.	The agent must decide what to check before adding an item to the cart.	Check a listing’s shipping destinations before adding the item to the cart.
Lesson instantiation	Verify that an operation has taken effect before relying on it.	The order form returned a confirmation page; the agent must decide whether the task is complete.	Check that the order appears in the account’s order history before reporting completion.
Merging multiple experiences	Lesson 1: Checkout can fail because the session expired during a long form. Lesson 2: Checkout can fail because the page silently cleared a required field.	Submitting the order fails, and the cause is unclear.	Check <i>both</i> that the session is still valid and that every required field still holds its value before resubmitting.
(c) Procedural reconfiguration: edit retained steps or assemble a procedure from retained parts			
Step insertion	A workflow searches for the item, adds it to the cart, enters the address, and pays.	The current site requires signing in before items can be added to the cart.	Insert sign-in and session-check steps before adding the item, leaving the later steps in place.
Procedure assembly	Several earlier strategies for related tasks: compare prices across sellers; apply a coupon before paying.	No single retained workflow fits: the user wants the cheapest seller and holds a coupon that only some sellers accept.	Assemble a procedure from these parts: list the sellers that accept the coupon, compare their prices after the discount, and check out with the cheapest.

sign-in step if the current site requires one before items can be added. Table 12 breaks these changes into subtypes, each shown on a small shopping case.

5.3.1 Contextual Grounding

Contextual grounding connects the entities and operations referred to in past experience to those available now, while checking the assumptions on which these correspondences depend; block (a) of Table 12 distinguishes binding entities, mapping tools, instantiating arguments, and checking preconditions.

To bind entities, tools, and arguments, systems rebind references through reusable representations, current-state inputs, or accompanying guidance. AWM (Wang et al., 2025e) replaces example-specific values with placeholders that the executor fills from the current observation, ICAL (Sarch et al., 2024) binds objects while generating new code from the current instruction and object state, and SkillMigrator (He et al., 2026a)

separates reusable interaction patterns from site-specific bindings and grounds them against the live page. For tools, PolySkill (Yu et al., 2026b) realizes one abstract skill interface through website-specific implementations, so a common operation survives without assuming that the original calls remain valid, while Agent KB (Tang et al., 2025) describes prompting an LLM to map entities, tools, and workflow steps from a source domain to a target domain. Selecting the right routine does not settle its arguments: SkillWeaver’s error analysis includes correct APIs called with incorrect parameters (Zheng et al., 2025), and DynaSaur (Nguyen et al., 2025) shows how hard-coded values prevent the reuse that parameterized inputs allow.

Even when entities and operations can be mapped, the current context may invalidate an assumption of the retained procedure. CLIN (Majumder et al., 2024) reports repeatedly retrieving stove-based boiling guidance for gallium despite also retaining a warning about insufficient heat sources, so the guidance was applied where its precondition of a sufficient heat source did not hold, and ASI (Wang et al., 2025d) shows skills whose concrete actions become incompatible with another website. Visual Skill Cards (Liu et al., 2026f) keep applicability cues and screenshots alongside a procedure to support this comparison. In practice, however, much grounding is not a separately verified step: AWM, ICAL, and the code-skill systems bind objects and arguments while generating actions. Such binding is safe while the supported conditions hold; once an essential precondition changes, the source evidence no longer covers the adapted guidance, which must be checked in the current setting before it is relied on.

How much grounding remains at reuse time depends largely on the level of abstraction at which experience was retained. A study of memory transfer across coding domains (Kim et al., 2026a) attributes gains to high-level guidance such as validation routines, while overly specific traces can cause negative transfer. A comparison of induction granularity (Feng et al., 2026b) finds that subtask-level skills transfer more reliably than task-level skills, with transfer depending on both task fit and breadth of applicability. These findings motivate separating reusable guidance from source-specific bindings during formation (Section 3), while making the remaining assumptions explicit during utilization. The preferred level of abstraction nevertheless depends on what differs between source and target. Removing source bindings helps when experience moves across tasks or domains. A recipient with different or weaker capabilities, however, may still need executable detail or concrete examples. The multi-agent results in Section 5.2.3 show that guidance useful to one recipient may not help another, and leave open which level of abstraction each recipient needs.

5.3.2 Guidance Specialization

Guidance specialization derives situated guidance for the current decision from selected experience by distilling a trajectory, instantiating a lesson, or merging several experiences (block (b) of Table 12). Each can improve relevance but also discard a necessary condition, so its evaluation must isolate the rewriting step from other changes and check that the source conditions survive.

The three operations start from different inputs: distillation condenses selected trajectories, instantiation fills a stored lesson with details of the current step, and merging combines several experiences, sometimes of different forms. QCR (Li et al., 2026g) distills a selected trajectory into reuse notes that name its reusable structure, the bindings to recover, applicability conditions, and verification requirements. AdaMEM (Zhang et al., 2026n) distills during execution, synthesizing temporary state-specific strategies from retrieved successful trajectories. ExpSeek (Zhang et al., 2026k) instantiates stored behavior–mistake–guidance triplets into guidance for the current step. ReMe (Cao et al., 2026) rewrites several retrieved experiences into guidance for the current task. ToolAtlas (Fang et al., 2026b) and WebCoach (Liu et al., 2025) merge experiences of different forms, such as tool-use traces with capability entries or episode summaries with the current partial trajectory, though both change retrieval and presentation together.

Specializing one selected trajectory to the current query can raise success while lowering cost. With DeepSeek-V4-Pro on constructed target variants across WebArena, WorkArena, and AppWorld, QCR’s query-conditioned notes raise average task success by 10.7 percentage points over supplying the full trajectory and by 14.4 points over a generic summary. Online tokens, including note generation, fall by 48.9% relative to the full trajectory. AdaMEM instead refreshes state-specific strategies repeatedly, while ExpSeek couples selection with guidance generation. These mechanisms make two distinct choices: which source to rewrite and how often to refresh the result. Section 7.3.2 compares the controls needed to isolate their effects and check whether rewriting preserves source conditions.

Specialization fails when rewriting drops an applicability condition or a check that the source carried. Direct supply avoids this separate rewriting step and leaves interpretation to the executor: ExpeL (Zhao et al., 2024a) supplies lessons from earlier trials, and Memp (Fang et al., 2026a) supplies trajectories and procedural guidance. The choice is a trade-off: specialization pays a generation cost and risks dropping conditions in exchange for guidance fitted to the current decision, and refreshing it is warranted only when the decision state changes what guidance is needed.

5.3.3 Procedural Reconfiguration

A retained procedure may identify the right objects and provide relevant guidance yet still fail under the current execution requirements. A checkout workflow recorded on one shopping site, for example, may omit the sign-in step that another site requires before an order can be placed. *Procedural reconfiguration* addresses this mismatch by changing the steps, control flow, or dependencies of a retained procedure, or by assembling a current procedure from parts of several. Its result is still guidance whose steps come largely from retained experience; writing a new plan that experience only informs is experience-guided generation, and following retained steps unchanged is experience-guided procedure reuse (Section 5.4). Editing a single retained procedure is the simpler case and comes first, followed by assembling one from several and by the dependency checks that both require.

Editing a retained procedure inserts, reorders, or replaces steps so that each step’s requirements hold under current conditions, as when a sign-in step is inserted before checkout while the later steps stay in place. XSkill (Jiang et al., 2026c) rewrites skills derived from successful trajectories at inference time, removing irrelevant content and adjusting workflow steps and code templates to the current task.

When no single retained workflow fits, a procedure can instead be assembled from retained parts. ProPlay (Ma et al., 2026b) assembles a stage sequence for the current task from retained stages and observed transitions ranked by their past contribution to success, including some untried transitions for exploration, and supplies the sequence as a soft prior that the agent may depart from. The structure of the result comes from retained stages, while some joins between them remain untested hypotheses.

Whether a procedure is edited or assembled, each step’s preconditions must hold when it executes, whether established by the initial state or by earlier steps. If an inserted or reordered step invalidates a condition needed later, the adapted procedure must re-establish it before the dependent step executes; a session opened by an inserted sign-in step, for instance, must still be valid when the order is placed. When parts acquired separately are joined, their intermediate states must also be compatible, or additional actions must bridge the gap; ProPlay’s untried transitions are joins of this kind, left for execution to test. Checking such compatibility in general remains open (Section 9.4). Feedback-guided revision, discussed next, uses execution feedback to identify and correct mismatches that remain after these changes.

5.3.4 Feedback-Guided Revision

An initial adaptation can still be wrong, and execution feedback is often the first sign of it. *Feedback-guided revision* changes the current guidance or procedure in response to execution feedback, which first requires locating what went wrong.

When retrieved experience guides program generation, execution can expose mismatches that were not apparent beforehand. Voyager (Wang et al., 2024a) and ICAL (Sarch et al., 2024) revise programs built from retrieved skills or examples using execution errors and critique, and AdaPlanner (Sun et al., 2023) regenerates its plan after failed assertions and chooses a resumption point, so that execution continues from the intermediate state already reached without repeating actions that have taken effect.

Before changing anything, however, the discrepancy must be traced to the adapted guidance and the actions actually taken. The kind of discrepancy indicates which adaptation step to revisit: grounding for an incorrect argument, specialization for an omitted requirement, and reconfiguration for a misplaced prerequisite. If the guidance already included a check that the executor skipped, it is the execution that needs correcting. In addition, tracing has to confirm what actually happened, because a tool can return successfully without producing the intended effect.

Table 13 The five functions of experience application, each with an example from the reviewed systems.

Function	Question addressed	Example of experience use
Interpretation	What might explain the observations?	OpsMem updates a graph that links candidate causes of a failure to supporting and contradicting evidence, which the agent consults when explaining new symptoms (Sun et al., 2026c).
Generation	What new content should the agent write?	Buffer of Thoughts retrieves a solution template distilled from earlier problems; the template shapes the solution, and the current question supplies its content (Yang et al., 2024).
Decision-making	Which option should be taken, or should the agent continue?	To find a first purchase date, a ReasoningBank strategy directs the agent to the full order history; the date itself comes from the records it then reads (Ouyang et al., 2026).
Procedure reuse	How can retained steps perform the operation?	An AWM agent follows the first steps of a “find a place by its name” workflow and then adds steps of its own to obtain the place’s zip code (Wang et al., 2025e).
Evaluation	Has the required outcome occurred?	TRACE compiles earlier user corrections into programs that check the agent’s later actions (Zhou et al., 2026d).

Revision also needs a stopping rule, because some mismatches cannot be revised away: if required access cannot be obtained, rewriting alone cannot make the procedure applicable. Continued failure or an unmet prerequisite should therefore prompt reconsideration of the selected experience, and an exhausted revision budget should end further edits. The experience application functions considered next explain how guidance influences behavior and yields the feedback used in these decisions.

5.4 Experience Application: How Guidance Shapes Behavior

Guidance that reaches the agent can affect its behavior in several ways: the same lesson can help explain a failure, shape a new patch, or decide whether to retry, and each contribution fails in a different way. *Experience application* describes what retained experience contributes to a current operation. We distinguish five experience application functions:

- *Interpretation* helps explain the task or observations.
- *Generation* constructs candidate content or a new solution.
- *Decision-making* chooses an option or whether to continue.
- *Procedure reuse* follows retained steps to perform an operation.
- *Evaluation* checks proposed work or its results.

These functions apply to language tasks and interaction with an environment. They describe contributions to behavior, and one system can make several of them in any order. Where a system’s memory also holds facts, preferences, or documents without execution grounding, we discuss only its trajectory-derived guidance. The subsection first shows that the function depends on how the agent uses the supplied content, then compares how existing methods realize each function.

The same experience can guide different operations. An agent may use a saved workflow to write a new plan, follow its steps directly, or check whether a proposed plan omits a requirement; these uses contribute to generation, procedure reuse, and evaluation, respectively. The experience application function therefore depends on what the agent does with the supplied content, and a workflow placed in context may not be followed at all. When a system uses experience in several ways, we describe each contribution and put the dominant one first. Table 13 gives one example of each function from the reviewed systems.

Experience-guided interpretation uses retained experience to help an agent understand the current task, observations, or failure, contributing an explanation of the situation that informs subsequent work. Under incomplete information, Agent-Pro (Zhang et al., 2024a) retains guidelines and descriptions of the game world and opponents from completed games and uses them to reason about hidden information in later ones.

For failures, earlier diagnoses connect symptoms to possible causes: OpsMem (Sun et al., 2026c) updates a graph linking candidate causes to supporting and contradicting evidence, DS-MCM (Sun et al., 2026d) uses summaries of earlier searches to explain unexpected observations, and FLASH (Zhang et al., 2024b) reads hindsight notes written about earlier failed diagnoses, each kept only after a retry showed that it corrected the failure. Similar symptoms can have different causes, however, and a task-level result such as Agent-Pro’s game performance leaves the correctness of the explanation unchecked. The distinctive question for this function is whether experience helps explain the same observations correctly, especially when a familiar explanation conflicts with current evidence.

Experience-guided generation occurs when retained experience shapes new content, such as an answer, a recommended item, a search query, a plan, or a program, while the current request and evidence supply the facts; it helps most when it narrows an open-ended output space. The boundary with decision-making is whether experience helps compose an output or chooses among options already available: writing a new search query is generation, whereas choosing which of several available sources to inspect is decision-making. Systems that supply example trials, insights, notes, or workflows from which the executor composes its plan or next action, such as ExpeL and AWM, therefore contribute mainly to generation, whereas the ReasoningBank strategies discussed below settle such a choice. AWM’s workflows can also be followed step by step, a use discussed under procedure reuse below.

The reviewed systems narrow three kinds of output space, and in each the experience fixes a different share of the output. The first is what to answer or propose. Buffer of Thoughts retrieves a solution template distilled from earlier problems and adapts it to the current question (Yang et al., 2024), so the experience fixes the shape of the solution while the question supplies its content; reflections on failed attempts and feedback on earlier recommendations narrow answers and proposals in a similar way (Shinn et al., 2023; Gupta et al., 2024; Zhu et al., 2025; Xi et al., 2024). The second is how to search: lessons from earlier unhelpful searches tell the agent how to reword its next query, leaving the answer to whatever evidence that query returns (Gupta et al., 2024; Zhao et al., 2024a; Fan et al., 2026a). The third is how to structure a plan or program, where experience can fix either an outline or a whole procedure. AdaPlanner drafts subgoals from earlier plans and then revises them from environment feedback (Sun et al., 2023), whereas EGuR writes a complete solution strategy as code conditioned on retained strategies (Stein et al., 2025); task summaries, repair lessons, and annotated examples likewise guide new plans and code without fixing them (Agashe et al., 2025; Chen et al., 2025b; Sarch et al., 2024).

Errors in retained guidance pass into generated content when nothing independent checks the output; Reflexion’s self-generated unit tests, for example, may be incomplete. Generated content therefore needs a check that does not rely on the same guidance, scored on correctness, relevance, or feasibility. A better output can be credited to experience only if nothing else changed: APEX (Ding et al., 2026), which supplies earlier search steps and skill documents to a research planner, also updates the planner’s parameters, so its gain mixes the supplied guidance with that training.

In *experience-guided decision-making*, retained experience settles a choice: which answer, strategy, tool, or plan to take, or whether to continue, change course, or stop. For initial choices, R2D2 (Huang et al., 2025) ranks routes built from a replay graph of earlier observations and actions, and SkillOpt (Yang et al., 2026e) supplies written decision rules learned from training trials. ReasoningBank (Ouyang et al., 2026) supplies strategies that tell the agent which source to inspect: to find a first purchase date, the guidance directs it to the full order history, and the date itself comes from the records it then reads. For decisions later in a task, MACLA (Forouzandeh et al., 2026) stores meta-procedures whose rules decide whether to continue, skip, repeat, or stop a procedure. Such guidance can help avoid repeated failures but can also preserve a poor choice, since similar queries encourage imitation of retrieved experience, including its errors (Xiong et al., 2026); evaluation should therefore count unnecessary retries and premature stops, which an aggregate success score can hide.

In *experience-guided procedure reuse*, retained experience performs a current operation directly: the agent follows steps, dependencies, or branches kept from earlier task trajectories; calling an ordinary tool does not qualify. A procedure can be followed as a textual workflow or called as saved code, and the two leave the model different room to inspect and alter individual operations. AWM (Wang et al., 2025e) supplies workflows from which the model generates each action, so following and extending a routine can mix, as

when an agent follows the first steps of a “find a place by its name” workflow and then adds steps to obtain a zip code. SkillWeaver (Zheng et al., 2025) instead exposes saved Python routines that carry out several browser operations in one call, as do libraries of accumulated or tested functions for computation, document processing, and environmental actions (Nguyen et al., 2025; Yuan et al., 2024; Wang et al., 2024c; Zhao et al., 2024b). Retained conditions can also control a workflow’s path: Skill-Pro specifies activation, execution, and termination conditions (Mi et al., 2026), and MACLA’s branches let the current state choose among existing steps. A call avoids repeated model decisions but may execute several steps before exposing a wrong assumption, so a lower call count is a saving only if inputs and starting conditions are checked and recovery from failure stays affordable.

In *experience-guided evaluation*, retained experience judges an existing answer, program, proposed action, or completed attempt, producing a critique, score, or check result; it evaluates current work, whereas Section 7 evaluates experience systems themselves. Its mechanisms bind the agent to different degrees. Past cases can inform scores, as when CRAVE’s critic scores other agents’ suggestions (Zhu et al., 2025) or Critic Experience Bank scores a proposed action from earlier productive and unproductive actions (Zeng et al., 2026), but such scores only inform a later choice, and their retrospective labels can be wrong. Earlier evaluation errors can also revise the scoring criteria themselves, as in AutoCalibrate and Reflect-and-Revise (Liu et al., 2024; Harada et al., 2026), whose agreement with human scores is shown only in the text-scoring settings where the rules were revised. Executable checks can block covered actions: SkillSentry checks dependencies and required steps using skill documents and action patterns from earlier runs (Lu et al., 2026a), and TRACE compiles earlier user corrections into programs that check later actions (Zhou et al., 2026d), yet both cover only patterns seen before. Experience-guided evaluation therefore helps where earlier mistakes recur in recognizable form, and it is weaker as an independent safeguard when it shares the premise of the action it checks (Section 5.6).

5.5 Experience Internalization: From Explicit Guidance to Model Parameters

Runtime utilization pays for experience at every use: each consultation spends retrieval, context, and adaptation effort, and behavior depends on continued access to the repository. When the same guidance helps repeatedly, the executor may instead learn the supported behavior. *Experience internalization* transfers behavior supported by explicit experience into model or adapter parameters during training; it is a training-time route for utilization. We first examine learning from trajectories shaped or supplied by experience and distilling experience-conditioned policies. We then discuss how training can progressively withdraw guidance, what to internalize, and what evidence shows that experience-assisted training produced a capability.

The first route trains the model on trajectories that explicit experience shaped or that the agent retained as experience, so that it reproduces useful behavior without the original guidance. EMPO² (Liu et al., 2026e) uses self-generated tips from earlier attempts during further execution and then trains the policy on some of the resulting trajectories with the tips removed from its input, reinforcing higher-reward actions; its main ScienceWorld and WebShop evaluation withholds memory. PEAM (Guo et al., 2026a) instead consolidates retained trajectories. A slower deliberative module produces verified successful Minecraft trajectories and failures paired with later corrections, and keeps them in an external episodic store; PEAM then trains skill-specific low-rank adaptation (LoRA) adapters, small trainable parameter sets attached to the model, with behavioral cloning on the successes and preference optimization on the failure–correction pairs.

The second route trains a student to match the action probabilities of a teacher that sees additional experience; the teacher can be the same model under a richer prompt. What is internalized then depends on what the teacher sees and supervises, and on what guidance the student still receives. EDGE (Xie et al., 2026a) enables distillation only when guided attempts outperform unguided ones from the same policy, and tests the resulting policy without external experience. Skill0.5 (Zhu et al., 2026a) distills only general strategies, while the student keeps retrieving task-specific skills at deployment, so its results support combining learned general behavior with continued explicit guidance.

Training can also withdraw guidance progressively, with a schedule that determines how much external guidance the policy receives as it learns. SKILL0 (Lu et al., 2026c) ranks skill files by how much they still help on validation tasks and supplies only helpful files within a budget that decreases to zero during policy optimization. Its skill-removal comparisons support reduced dependence in the tested settings.

Some neighboring methods update parameters around explicit experience while keeping it in use, so their gains show internalization only if use with and without the guidance is compared. SEAM (Li et al., 2026e) trains a small generator that supplies textual guidance to a frozen executor (Section 2.6), and Evolving-RL (Fan et al., 2026b) trains one policy both to extract skills and to solve tasks with them (Section 3.5); only Evolving-RL’s evaluation with and without accumulated skills separates assisted use from behavior retained in the policy.

We recommend internalizing stable, broadly useful behavior while keeping task-specific conditions, rare exceptions, and the source evidence needed for correction explicit, because explicit guidance can still be inspected, corrected, and supplied again, whereas behavior moved into parameters cannot be revised entry by entry. The evidence for the two halves of this recommendation is uneven. On what to internalize, a study of three rounds of internalization for web reasoning, mainly with a Qwen3-4B backbone, finds that gains from distilling instance-level experience, or from injecting experience once for a whole trajectory, fade in later rounds and can fall below the base model. Principle-level experience injected step by step and distilled from successful teacher trajectories instead sustains gains on WebWalkerQA and GAIA, and step-wise injection also preserves the model’s ability to benefit when that round’s experience is supplied again (Chen et al., 2026c). The case for keeping guidance explicit rests mostly on design reasoning, although Skill0.5 follows it by keeping task-specific skills retrievable at deployment.

Testing a trained policy with guidance withdrawn and supplied again shows its remaining dependence on runtime support. Attribution to experience-assisted training requires the matched training comparison in Section 7.4.4. EDGE’s GRPO baseline approximates this control; EMPO² and SKILL0 mainly use mode and curriculum ablations to credit training components. Section 9.5 considers continued changes to explicit experience and the executor.

5.6 Runtime Choices under One Budget

The four runtime decisions are distinct, but they share one budget, and a risk left unresolved at one carries into the next. Seeking, selecting, and adapting experience all consume resources before the guidance helps with the task, and a condition left unchecked during selection can resurface as a failure during experience application. Inexpensive checks that could change the reuse decision therefore belong before costly actions. The decisions also depend on different things: the trigger on what the system can observe and afford, and a separate adaptation step mainly on whether the stored experience can be acted on directly.

Table 14 sets the options reviewed in this section against the strongest comparison reported for each decision and the comparison that is still missing. The evidence is uneven across the rows. The clearest results compare variants of reranking and adaptation within single systems, such as QCR, whereas triggering, multi-agent allocation, and feedback-guided revision have no comparison of alternatives under one budget.

Table 14 Evidence on the runtime decisions of experience utilization and on experience internalization. For each decision, the table lists the options reviewed in this section, the strongest comparison the reviewed studies report, and a comparison that is still missing.

Decision	Options in the reviewed systems	Strongest reported comparison	Comparison still missing
Experience triggering (§5.1)	Fixed schedule (ExpeL, AWM); observed failure (AMD); internal-state signal (ExpSeek, DS-MCM); coach or agent gate (WebCoach, ExpWeaver); learned policy (ProactAgent, MemCon)	Within ExpSeek on GAIA, accuracy gains diminish as interventions become more frequent; an executor without experience given a comparable budget can match methods that accumulate experience (Hajimiri et al., 2026).	Signal sources under a common budget; counts of applicable guidance left unconsulted (§7.3.1); whether calibrated and learned triggers must be refitted after maintenance or a change of executor.
Experience retrieval (§5.2.1)	Requests that track execution (ExpRAG, Agent S, RaDA) or carry a provisional solution or diagnosis (Voyager, Agent KB); lexical, semantic, or hybrid matching (ExpeL, ReMe, Agent KB, SEER); routing (ExpSeek, CoEvo-Mem); expansion through links (G-Memory, EXG)	In Agent KB, hybrid lexical and semantic matching outperforms either alone on concise experience summaries, with narrower margins once teacher critiques refine the experience.	A test of request quality on its own; ProactAgent learns the request together with the decision to retrieve.

Table 14 continued

Decision	Options in the reviewed systems	Strongest reported comparison	Comparison still missing
Experience reranking and filtering (§5.2.2)	Reranking on current constraints (ReMe, QCR); applicability checks (AutoGuide, Agent KB, SkillLens); estimated or learned utility (MACLA, MemRL, Memento)	In QCR, the five trajectories retrieved from a frozen bank include a reusable one for 97.8% of targets, and summary-based reranking selects one for 94.8%, on targets built from the bank’s tasks.	Utility estimates that separate an entry’s marginal contribution from that of memories retrieved with it (Yang et al., 2026d); the cost of checking or adapting a candidate.
Selection for multiple agents (§5.2.3)	The same experience for every recipient (G-Memory, AMD, FORGE); allocation by role (LEGOMem) or by source model (MemCollab)	Three isolated results: LEGOMem’s placement ablations on OfficeBench, MemCollab’s observation that reusing another model’s memory sometimes lowered accuracy, and AMD’s finding on AppWorld that a more accurate teacher did not yield more helpful memory for its 4B student.	Allocation policies set against one another on one benchmark and backbone.
Experience adaptation (§5.3)	Contextual grounding (AWM, ICAL, SkillMigrator, PolySkill); guidance specialization (QCR, AdaMEM, ExpSeek, ReMe); procedural reconfiguration (XSkill, ProPlay); feedback-guided revision (Voyager, ICAL, AdaPlanner)	With DeepSeek-V4-Pro, QCR’s query-conditioned notes raise average success by 10.7 points over the full trajectory and by 14.4 over a generic summary, using 48.9% fewer online tokens than the full trajectory.	Feedback-guided revision against one-time adaptation, reselection, or continuing without experience under a matched total budget; SkillAligner (Li et al., 2026b), for instance, adapts guidance once and leaves online revision to future work.
Experience application (§5.4)	Interpretation (Agent-Pro, OpsMem, DS-MCM, FLASH); generation (Buffer of Thoughts, AdaPlanner, EGuR); decision-making (R2D2, ReasoningBank, MACLA); procedure reuse (AWM, SkillWeaver, Skill-Pro); evaluation (Critic Experience Bank, AutoCalibrate, SkillSentry, TRACE)	Mostly task-level outcomes; AutoCalibrate and Reflect-and-Revise report agreement with human scores, but only in the text-scoring settings where their rules were revised.	Function-level measures, such as whether experience yields a correct explanation of the same observations, and counts of unnecessary retries and premature stops (§7.3).
Experience internalization (§5.5)	Training on experience-shaped trajectories (EMPO ² , PEAM); distilling experience-conditioned policies (EDGE, Skill0.5); withdrawing guidance during training (SKILL0)	Over three rounds of internalization for web reasoning, mainly with Qwen3-4B, principle-level experience injected step by step sustains gains on WebWalkerQA and GAIA, whereas instance-level experience fades (Chen et al., 2026c).	Which guidance should stay explicit after training; matched training comparisons, which EDGE’s GRPO baseline only approximates (§7.4.3).

For practice, an even more basic comparison sets using experience at all against spending the same budget on the task itself. On largely self-contained web tasks, giving an executor without experience more interaction steps can match or exceed online skill and memory augmentation at approximately matched token cost (Hajimiri et al., 2026). The study does not vary how often tasks recur, but its result suggests recurrence as a practical criterion for deciding whether to invest in experience: repeated conditions offer more opportunities to repay experience acquisition, whereas an isolated task may benefit more from additional attempts or checks. Section 7.4.2 details the comparison and separates recurring online costs from offline preparation that can be spread over later uses.

Wherever checks are placed, a task outcome should be checked independently of the explanation that experience proposed. If experience attributes a failed order to an expired session, for example, the check confirms that the order now appears in the account history instead of accepting a successful sign-in. When the same experience shapes both the action and the check, the check can inherit the action’s mistaken premise, and assigning it to another model does not remove that dependence. Intermediate checks belong where conditions can change or an error would become costly to correct. When a check fails, the tracing described under feedback-guided revision (Section 5.3.4) applies, so enough of the trajectory should be retained to support it.

Everything runtime utilization produces acts within the current episode and leaves retained experience unchanged; Section 6 turns to the decisions that change it.

6 Experience Maintenance: Deciding What to Retain, Revise, or Retire

Each episode leaves two kinds of input for later learning: new candidate experience proposed by formation, and evidence about experience already retained, such as an adaptation that worked, a failure that exposed a missing condition, or a sign that the agent now succeeds without it. Runtime experience utilization uses such evidence to guide current decisions (Section 5); deciding whether it should change what is retained is a separate problem, and simply appending each episode’s candidates accumulates noise alongside useful guidance. *Experience maintenance* makes these persistent decisions: it re-evaluates what should be retained, combined, corrected, or retired so that later reuse remains reliable and efficient. In the notation of Section 2.3, maintenance implements Update in Equation 4: it decides which candidates in $\mathcal{E}_k$ enter R_{k+1} and how retained units change. How retained experience is structured and accessed remains a matter of organization (Section 4). Some systems also use feedback to revise the procedures that form, maintain, or retrieve experience (Section 6.5); what an update can claim to improve depends on whether it changes retained units or such a procedure (Section 2.5).

Different evidence calls for different operations. A candidate supported by a successful trajectory raises the question of *admission* (Section 6.1): whether its support and added benefit justify retention. Repeated or complementary evidence from later trajectories calls for *consolidation* (Section 6.2), which combines related experience while keeping distinct situations apart. A new case that exposes a missing condition or a wrong step calls for *refinement* (Section 6.3), and a sign that the agent now succeeds without a unit calls for *retirement* (Section 6.4), which reduces its availability. The operations are alternative decisions, taken as evidence arrives and in no fixed order. Safeguards apply to all of them, because a mistaken update can spread into experience derived from it (Section 6.6).

We sort the evidence for these operations into three check levels. A *support* check examines content against recorded evidence, such as its source trajectory, related entries, or a later failed trajectory, but does not measure what using it changes; correct guidance may still not help. A *paired comparison* measures the local gain from using it. A *collection-level test* evaluates it together with the experience it would join, because a unit that helps on its own may add nothing, or cause harm, alongside the units it joins, and an update can reduce the value of units it did not edit. The levels are not a fixed sequence: a support check usually comes first because it is cheap and avoids running unsupported procedures, while a single replay can serve as both a paired comparison and a collection-level test. These check levels serve decisions about what to retain; what a study can claim is judged instead by the three evaluation targets of Section 7 and the improvement claims of Section 2.5 (Table 8).

Figure 15 shows how admission and updates to retained units change a shared experience repository, whose later reuse supplies evidence for further maintenance decisions. The following subsections examine each operation in turn; Section 6.5 then asks how a system chooses among them and aligns them with formation and retrieval, and Section 6.6 examines the safeguards. Conversational-memory studies supply adjacent mechanisms, labeled where introduced; their recall or QA gains do not validate trajectory-derived guidance.

6.1 Experience Admission

Experience admission decides which candidates become persistent resources for reuse. Even a successful trajectory can yield guidance that is narrow, unsupported, or already available. Deciding admission means settling when enough evidence has arrived to justify review, what checks a candidate needs, and how strict the gate can be before its cost and lost coverage outweigh the errors it prevents. How candidates are proposed is the subject of Section 3.

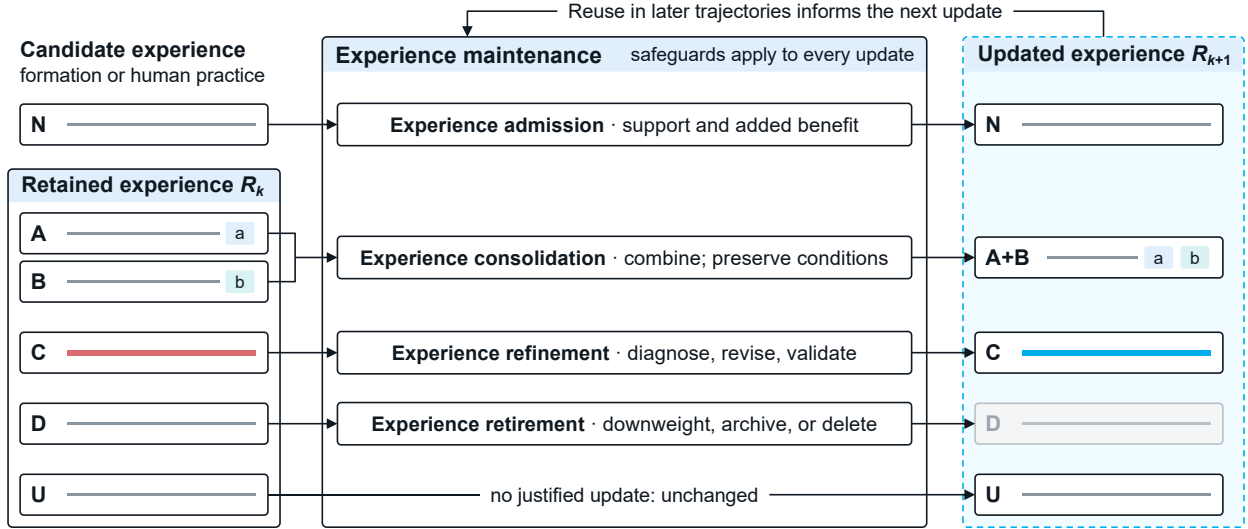


Figure 15 Experience maintenance admits candidates (N), consolidates related units (A and B), refines units (C), or retires them (D). Consolidation preserves applicability conditions (a and b), while units without a justified update remain unchanged (U). Safeguards apply to every update, and reuse in later trajectories informs subsequent maintenance.

6.1.1 When to Review New Evidence

Reviewing early makes guidance available sooner, at the price of extraction and evaluation calls spent on incomplete or repetitive evidence; waiting for a clearer account of the outcome lets mistakes repeat in the meantime. In most systems, one event starts both candidate extraction and the retention decision, and that event varies widely. Mem0 starts at each new interaction for conversational facts (Chhikara et al., 2025), and MUSE at task or subtask boundaries, admitting a standard operating procedure after verified subtask success (Yang et al., 2025a). Reflexion starts at informative events, reflecting after repeated action–observation loops and appending the reflection without a separate review (Shinn et al., 2023). The adjacent RecMem waits until enough related evidence accumulates (Dai et al., 2026a), and Live-Evo and EvolveR run scheduled update phases (Zhang et al., 2026m; Wu et al., 2026b). These timing rules can be combined: evidence is recorded as it arrives, completed subtasks and consequential failures are reviewed promptly, and repetitive material waits for a scheduled pass.

6.1.2 Evaluating Candidate Experience for Admission

Formation produces candidate precedents, lessons, or procedures (Section 3); admission decides which of them to retain. Checks run during formation, such as executing and refining a candidate procedure (Section 3.4.3), supply evidence that admission weighs; admission decides whether that evidence suffices and may add checks of its own, which is why a system such as Voyager appears in both sections. Candidates may be accepted, deferred for more evidence, or rejected. Table 15 compares complementary checks by the evidence they require and the errors they can expose, and each group heading names the check level, defined at the start of this section, to which its checks belong. Scores that predict novelty or utility belong to none of these levels and are marked separately as predicted screening.

Skills that people author from their own practice reach admission without passing through formation and usually without recorded trajectories (Section 2.6), so support checks against source trajectories cannot run; admission can check such skills only against their stated conditions or by replaying them, and later use gradually supplies the execution evidence they lack. Systems differ in whether they maintain such skills at all: SkillCAT merges replay-checked patches into a human-written spreadsheet skill (Chen et al., 2026e), whereas Hermes Curator leaves user-written skills untouched (Nous Research, 2026).

Support checks examine a candidate against its own origin and cost little beyond verification calls or repeated solving. A success gate admits a candidate only if its source trajectory ended in an outcome judged successful,

Table 15 Admission checks differ in the feedback they require and in what passing them establishes. Groups follow the order of the text; example systems are listed with the settings in which they were evaluated.

Check	Required feedback	What it checks	Cost and blind spot	Example and setting
Evidence and consistency (support checks)				
Success gate	Observed outcome and completion evaluation	The source attempt achieved its goal	Execution and evaluation; success does not validate the extracted explanation or transfer, and the gate excludes failure-derived experience	Voyager: Minecraft; MUSE: workplace tasks (Wang et al., 2024a; Yang et al., 2025a)
Solution agreement	Separately constructed solutions	Agreement among independently built solutions	Repeated solving; shared extraction errors and missing premises can survive	AdmitOR: optimization problems (Lian et al., 2026)
Source support	Source context	Whether the content follows from the context it came from	Verification calls; useful implicit facts can be rejected	ConsistencyGate: conversational memory (Zhang and Li, 2026)
Novelty and utility (predicted screening)				
Novelty / utility score	Existing entries and estimated future value	Added information and predicted usefulness	Search and scoring; estimated value is not an observed benefit	EvolveR: search-based QA; A-MAC: conversational memory (Wu et al., 2026b; Zhang et al., 2026d)
Replay-based validation (paired comparison and collection-level test)				
Paired replay	Comparable runs with and without the candidate	Measured benefit after reuse costs	Repeated execution; source-task gains can remain local	SEDM: fact verification and multi-hop QA (Xu et al., 2025a)
Held-out collection test	Validation tasks and the existing skill collection	Candidate value within the deployed collection	Multiple task runs; selection can overfit validation tasks	EvoSkill: document and search-augmented QA (Alzubi et al., 2026)

as when Voyager stores code after a critic classifies the task as complete (Wang et al., 2024a), OS-Copilot evaluates reusable tools with critic feedback (Wu et al., 2024b), or MUSE inserts procedures after verified subtask success. A task can succeed for reasons other than the retained guidance, however, and agents can confidently retain an incorrect explanation of a failure (Dixit et al., 2026). Without a reference answer, AdmitOR accepts, defers, or escalates candidates by the agreement of solutions from different model families and solver stacks (Lian et al., 2026), although agreement among solvers checks only the solution to the extracted problem, not whether that problem’s formulation matches the intended task. ConsistencyGate admits an extracted conversational fact when repeated LLM support scores against its source exceed a threshold (Zhang and Li, 2026); the corresponding check for trajectory-derived guidance would connect the recommended action and its conditions to the observed trace.

Predicted screening addresses a different risk: a supported candidate may add little when equivalent guidance is already retained. EvolveR attaches new source evidence to an existing principle instead of adding another entry (Wu et al., 2026b), and A-MAC scores conversational memories by predicted usefulness, novelty, recency, and related features (Zhang et al., 2026d), but predicted utility is not an observed improvement during reuse.

Replay asks whether the candidate changes task performance. Live-Evo reruns the source task with candidate experience and keeps improvements (Zhang et al., 2026m). SEDM runs paired replays with and without each candidate and scores reward gain after latency and token costs (Xu et al., 2025a); its ablation, with GPT-4o-mini on FEVER fact verification and HotpotQA, shows that the whole admission pipeline outperforms no memory, but it neither compares admission with unfiltered memory nor reports the cost of the validation runs. How far such evidence extends depends on the validation set: source-task replay establishes a local gain, UMEM evaluates marginal utility over related queries (Ye et al., 2026c), and EvoSkill tests a proposed skill together with existing skills on held-out validation tasks while keeping the final test set outside evolution (Alzubi et al., 2026).

Passing a cheaper check leaves some questions unresolved, so the further review a candidate receives should grow with its consequences. A failed check justifies rejection only when it undermines the candidate’s support; failure-derived experience that fails a success gate should instead be deferred or reviewed more closely (Section 6.1.3). The experience form also changes what a passed check establishes: a verified outcome supports a precedent under its recorded conditions, whereas a lesson also needs its explanation and scope to follow from the trajectory, and a procedure needs its prescribed actions checked on new inputs. The same trajectory can therefore justify retaining the recorded case while a broad rule drawn from it stays provisional.

6.1.3 Balancing Selectivity, Cost, and Coverage

Stricter admission can exclude unreliable guidance, but a gate on source success also discards useful failure-derived guidance and rare recovery procedures; the source outcome alone is therefore a poor proxy for a candidate’s value. Memento retains both successful and failed cases for later selection (Zhou et al., 2025), and PREPING lets trajectories from feasible practice tasks inform solver memory even when the attempt fails (Choi et al., 2026). In ReasoningBank’s WebArena-Shopping ablation with Gemini-2.5-flash, extracting lessons from failures as well as successes raises success from 46.5% to 49.7% (Ouyang et al., 2026), although that result concerns which outcomes supply lessons rather than how a gate filters them.

Review effort should match the decision: check support, applicability, and duplication first, and reserve expensive replays for valuable or consequential candidates, such as a procedure that directly controls tool calls. Where feedback is sparse or the environment cannot be replayed, a provisional status, as in MemGuard (Wang et al., 2026b), records the gap so that the candidate can be reviewed again later instead of being forced into acceptance or rejection. Coverage that a strict gate loses becomes visible only when selection policies are compared on the same candidate set and omitted task types and recovery cases are inspected. The Experience-Following study (Xiong et al., 2026) makes such a comparison, admitting every trajectory or admitting by a coarse or a strict evaluator on the same task stream (Section 6.7); PREPING instead puts what its gate rejects to use, letting rejected practice outcomes guide further task generation. Admission is therefore best evaluated by the reliability and coverage of retained guidance together with review cost and availability delay.

6.2 Experience Consolidation

Useful experience can still overlap: repeated tasks produce equivalent guidance, while related tasks contribute different parts of a procedure. *Experience consolidation* integrates such experience while preserving the distinctions needed for correct reuse; unlike internalization (Section 5.5), it changes explicitly retained experience. Whether the resulting compactness is worth the information it may remove depends on what an integration changes and on how much retained experience it reconsiders at once, the two questions the following subsections address.

6.2.1 Integrating Repeated and Related Experience

Consolidation changes retained experience in three distinct ways, and each can lose different information. *Deduplication* combines equivalent experience into one entry and requires a check of equivalence; *abstraction* extracts shared guidance from related experience and requires that the conditions distinguishing the original cases survive; and *structural integration* reconciles edits to one procedure or factors out components shared across procedures, and requires compatible interfaces and dependencies.

Deduplication has to compare conditions as well as content, because equivalent wording is insufficient if two entries apply under different conditions. EvolveR (Wu et al., 2026b) deduplicates during the novelty screening described under admission (Section 6.1.2): it checks newly distilled principles for semantic equivalence with retained ones and attaches the new source trajectory to a matching principle instead of adding an entry. Abstraction during consolidation revisits experience already retained, which distinguishes it from abstraction during formation, such as Controlled Self-Evolution’s extraction of global experience from selected optimization steps (Hu et al., 2026b). MUSE generalizes procedures after a task (Yang et al., 2025a), and DeltaMem (Tan et al., 2026a) stores base experience in tree roots and later variations in residual nodes, distilling a chain into a self-contained root after enough successful uses. Related trajectories can, however,

share steps without supporting the same unconditional rule, so the shared guidance must preserve differing prerequisites, parameters, and recovery actions.

Structural integration applies mainly to procedural-level experience. Trace2Skill (Ni et al., 2026) merges patches proposed from successful and failed trajectories against the same skill, with programmatic checks for invalid references, overlapping edits, and format; these catch inconsistent edits but leave the merged behavior untested. SkillZip (Tan et al., 2026b) factors recurring sections into reusable macros that preserve interfaces, dependencies, and verification steps, with source pointers for expansion. Factoring out a shared routine changes the dependencies of every procedure that calls it, so a later edit to the routine can affect several otherwise unchanged skills.

Each kind of integration also leaves a different regression to check. After deduplication, one entry carries the guidance its duplicates gave; after abstraction, the agent must bind a shared rule to a particular case, so omitted parameters can change behavior even when the rule reads well; and after structural integration, every caller depends on the shared routine. Regression checks should therefore follow the kind of integration: earlier equivalent uses after deduplication, cases exercising alternative bindings after abstraction, and affected callers after structural integration. Each piece of original evidence should stay linked to the use it supports instead of being pooled into a single support count.

6.2.2 Targeted and Region-Level Experience Consolidation

Update scope determines how much retained experience is reconsidered together, which affects processing cost, validation effort, and recovery when consolidation removes a useful exception. A targeted update keeps unit boundaries intact: each ExpeL extraction round can add, edit, or vote on particular insights in the current list (Zhao et al., 2024a), which keeps changed units easy to inspect and restore but can leave complementary guidance fragmented. Region-level replacement reorganizes related experience jointly: Auto-Dreamer (Ye et al., 2026b) separates per-session writing from slower consolidation of a selected region and its linked trajectories, for example synthesizing a shared procedure from five task-specific ScienceWorld notes alongside a warning drawn from failed trajectories.

We recommend a targeted update by default when new evidence clearly matches an existing unit, and reconsidering a region when the needed guidance is spread across several units or repeated local edits obscure their common structure. The only same-evidence comparison we found is coarser: ACE (Zhang et al., 2026i) adapts a playbook offline on AppWorld with DeepSeek-V3.1 and reaches 76.2 task goal completion on AppWorld’s test-normal split with itemized incremental updates, against 67.3 when the playbook is rewritten as a whole and 63.7 for ReAct without adaptation. A region-level replacement requires checking coverage across the whole replaced collection, including exceptions uncommon in the batch. It also needs an explicit record of which original conditions and exceptions it preserves, changes, or omits, so that checking does not rely on the consolidator’s summary of its own work.

Consolidation is worthwhile only if it improves later behavior, and three checks test this. The first is whether details that change actions survive: Auto-Dreamer’s ALFWorld analysis reports that trained consolidation lost useful location details and hurt one task family despite stronger performance elsewhere (Ye et al., 2026b), which argues for keeping source links and a recoverable prior version. The second compares the original and consolidated repositories under the same reuse budget, with consolidation cost included, because consolidation can help under tight answer-time budgets while raw evidence can be preferable when more of it fits (Kang et al., 2026a). The third looks for regression across successive updates: besides the ARC-AGI decline under per-problem rewriting (Section 3.3), memory utility in ScienceWorld and WebShop agents first rises and then declines as consolidation continues (Zhang et al., 2026a). A single rewrite can also collapse a repository: in an AppWorld case study of Dynamic Cheatsheet (Suzgun et al., 2026), one full rewrite shrank an 18,282-token context to 122 tokens and accuracy fell from 66.7 to 57.1, below the 63.7 of the agent without adaptation (Zhang et al., 2026i).

6.3 Experience Refinement

Later trajectories can reveal missing prerequisites, exceptions, or mistaken explanations in retained experience. Furthermore, tools and environments can change its validity. *Experience refinement* uses this evidence to

re-evaluate reliability, clarify applicability, or correct action guidance. Refinement has to repair the implicated part while preserving supported behavior: rewriting after every failure can damage valid instructions when retrieval or execution caused the problem. Repair therefore starts from diagnosis, which determines what targeted revision changes and what validation must recheck; signals that repair is the wrong response close the subsection.

6.3.1 Diagnosing the Problem

Before editing anything, maintenance must decide whether retained experience contributed to a failure and, if so, whether the guidance, its stated conditions, or the way the agent applied it requires attention, because a failure after retrieval can also originate in selection or execution. Attribution narrows the target: SkillsVote attributes trajectory outcomes to skill use, exploration, and environmental factors (Liu et al., 2026b); EmbodiSkill edits a skill’s body for a content defect but only emphasizes existing guidance when valid instructions were not followed, and abstains when evidence is insufficient (Ju et al., 2026); and SkillAdaptor locates the earliest actionable fault and the contribution of retrieved skills (Yu et al., 2026e). Workflow structure further separates a faulty rule within one component, which Workflow-Localized Mechanism Learning repairs in the resource itself, from an incorrect relation between components, which it repairs in the composition instructions (Lin et al., 2026b). Repeated outcomes must be read against the conditions that produced them, as when MACLA records the situation of each use while updating success estimates (Forouzandeh et al., 2026), because pooling outcomes before and after a tool change can hide a version-specific failure.

Diagnosis also has to tell apart three kinds of change, each with its own check. A *tool or environment change* is checked against the instructions and prerequisites, *executor improvement* by re-evaluating added benefit, and *changing task demand* by checking which current tasks the unit still serves. The latter two may justify retirement without any content repair (Section 6.4).

6.3.2 Targeted Revision

The diagnosis determines which part of the experience to revise: its reliability evaluation r_i , its applicability conditions c_i , or its guidance g_i (Equation 1), taken in that order below. Whichever component is revised, the required validation effort depends on how the proposed change affects behavior and how widely other experience depends on it.

- *Reliability update*: when support weakens but no instruction is demonstrably wrong, the reliability estimate should be revised or the experience marked uncertain, as MACLA updates each procedure’s success estimate from the outcomes of its uses (Forouzandeh et al., 2026) and MemGuard keeps unverified entries in a provisional state (Wang et al., 2026b). We recommend that units prescribing conflicting actions under apparently identical conditions both be kept, marked as conflicting, and restricted until a discriminating test run or external check resolves them; recency or repeated wording should not decide which rule wins.
- *Applicability clarification*: when differing conditions explain an apparent conflict, the repair is to clarify applicability, as when a procedure for an older interface remains scoped to that version while an updated procedure serves the new one. SkillOps (Pu et al., 2026) makes such requirements explicit through skill contracts and repairs observable contract defects, such as missing checks and type mismatches, though not arbitrary environmental drift. Narrowing applicability requires evidence for the distinction, not merely wording that removes an apparent contradiction.
- *Action-guidance correction*: if the action is wrong even when its conditions hold, the instruction itself must change. SkillRevise (Liu et al., 2026d) records suspected defects, verification requirements, and behavior to preserve, and proposes edits with observable behavioral checks; the result remains a candidate until validation supports persistent replacement.

6.3.3 Validating and Retaining the Revision

Validation asks whether the candidate repairs the diagnosed problem, preserves earlier supported uses, and allows recovery if a regression appears. These checks determine whether to retain the revision and what records to keep for later review.

Validation first tests whether the revision repairs the diagnosed failure. MUSE-Autoskill uses unit-test feedback to trigger refinement (Lin et al., 2026a), SkillRevise re-executes candidates within a bounded episode, and Workflow-Localized Mechanism Learning compares original and revised skills on the same batch. A successful retry supports the candidate under that test’s conditions, not every part of the diagnosis.

Previously supported uses must then be rechecked, and systems differ in how much of that use they cover. SkillAdaptor commits a change only if the aggregate outcome under the candidate collection does not decrease, which can still hide regressions on individual tasks. SkillCAT (Chen et al., 2026e) merges a patch into its base skill only when replaying the patched copy on the patch’s own source tasks repairs a failure or preserves a success, so its check covers each patch’s source tasks rather than the merged skill. SkillOpt (Yang et al., 2026e) requires an improved held-out selection score for replacement while keeping a separate final test split.

Keeping the previous version, test conditions, and acceptance rationale makes later regressions easier to investigate and lets a harmful revision be undone without discarding useful evidence. WikiSkill (Tang et al., 2026) applies such an undo at validation time: it rolls back proposed skills that fail validation while keeping the accumulated wiki knowledge.

Diagnosis and validation outcomes also indicate when editing a unit will not help. If diagnoses repeatedly abstain or disagree, collecting discriminating trajectories may be worth more than another edit. Successive patches that mainly add exceptions suggest that the unit’s conditions may no longer describe a coherent case, and splitting it or reconstructing it from its supporting trajectories may be easier to validate. If the guidance remains correct but the executor or workload no longer needs it, the decision becomes one of retirement (Section 6.4).

6.4 Experience Retirement

Experience can remain correct while contributing little to the agent’s current tasks, because the tasks have changed, the agent has improved, or other experience supplies the same guidance. Keeping everything active adds storage and maintenance costs and complicates retrieval. *Experience retirement* persistently reduces the availability of experience whose expected benefit no longer justifies its cost, ranging from lower retrieval priority through archiving to deletion; memory studies often call this progressive reduction forgetting. Retirement need not entail deletion, and it is more than omission from one retrieved context. Deciding to retire experience requires knowing what gives it continued value and what evidence shows that this value has declined; only then can downweighting, archiving, or deletion be chosen.

Deciding whether to retire experience starts from what future tasks would lose without it. That loss depends on three factors, each of which can change while the guidance remains correct. First, current tasks may no longer need the unit, although a rarely used recovery procedure can still prevent a costly failure. Second, the agent may now succeed without it: EDGE (Xie et al., 2026a) compares rollouts with and without an experience item during agentic reinforcement learning and deletes items whose moving-average gain falls below a threshold. Third, other experience may already cover the unit. A seemingly redundant unit may still hold an extra prerequisite or recovery step, and two procedures that each appear to cover the other cannot both be discarded. What retention should preserve is coverage, which a count of individually valid units does not measure. Correctness and added benefit are therefore separate considerations; internalization, for instance, can reduce added benefit (Section 5.5) without justifying deletion of the source evidence needed for rare exceptions.

Low benefit is hard to distinguish from limited opportunity to contribute, because rarely retrieved experience may serve infrequent tasks or be overlooked by the retriever. Age and retrieval frequency, as in the temporal decay and recall-based reinforcement of MemoryBank, an adjacent conversational-memory system (Zhong et al., 2024), are cheap but are best read as nominations for review. Observed use is closer to utility: ReMe prunes experience with insufficient utility only after a minimum number of retrievals (Cao et al., 2026), and MACLA combines reliability, usage, and age, but neither a successful nor a failed use shows whether the experience was needed. Two studies test such signals against keeping everything. SSO prunes skills whose execution stops leading to high later reward, and removing this pruning lowers its performance on a ScienceWorld task and a NetHack task (Nottingham et al., 2024). With GPT-4o-mini, deleting records according to how they fared when later retrieved raises success with a smaller memory in three of four agents

when a strict evaluator judges outcomes, for example from 38.67% to 42.06% for EHRAgent (Xiong et al., 2026).

Comparisons of runs with and without the experience on comparable tasks and budgets give the strongest evidence; EDGE and Live-Evo already retire or downweight experience this way, but only on tasks where it is currently supplied, and the natural next step is to extend the comparison to the exceptions and recovery cases that retirement could leave uncovered. The time horizon also affects the decision: a procedure of little current benefit may be costly to reacquire when an older environment returns, so a retention comparison needs to state the expected workload and to separate the cost of keeping a record from the cost of making it routinely retrievable.

Downweighting, archiving, and deletion save different costs and preserve different routes to later reuse, so the choice depends on both the evidence of declining value and the consequences of a mistake. Downweighting keeps experience accessible: Live-Evo (Zhang et al., 2026m) updates persistent weights from the gap between memory-guided and memory-free performance and combines them with similarity during retrieval, although the resulting decline in retrieval frequency is not independent confirmation of declining value. Archiving removes experience from routine access but keeps a recoverable copy, as the Curator in Hermes Agent does for inactive agent-created skills (Nous Research, 2026); a restored procedure still needs revalidation under current tools and tasks. Deletion, as in ReMe’s utility-based pruning or SkillOps’s removal of retired skills and their graph edges, is irreversible and is safe only after checking whether exceptions or dependent procedures would lose necessary support; influence already propagated into derived experience requires the recovery measures of Section 6.6.

6.5 Coordinating Experience Maintenance

Which operation to apply, and whether it helps, also depends on the components around maintenance: formation supplies the candidates, and retrieval and selection determine which experience receives feedback. The same repeated failure may therefore come from choosing the wrong operation, from a poor candidate, or from retrieval that never surfaced the right experience. Each source calls for its own remedy: a learned operation policy, coordination with formation, or coordination with retrieval. In contrast to an edit to one retained unit, each remedy changes many later decisions. Much of the evidence here comes from adjacent conversational-memory systems.

A learned policy chooses whether and how to change retained experience, including leaving it unchanged, so its feedback must distinguish selecting an appropriate operation, executing it correctly, and improving later tasks. In adjacent memory-management policies, Memory-R1 learns ADD, UPDATE, DELETE, and NOOP decisions from downstream answer correctness (Yan et al., 2026c). TARL, by contrast, supervises whether an operation produces the intended repository state (Xiao et al., 2026). For trajectory-derived experience, CODESKILL trains add, merge, or drop decisions for coding skills from verifier gains over a no-skill baseline (Li et al., 2026f), though one candidate at a time, which leaves interacting updates unresolved.

When several updates precede one outcome, the policy also needs to know which of them deserves the credit, and among the reviewed work only adjacent memory-management systems address this. AgeMem assigns the final outcome to all preceding memory actions (Yu et al., 2026d), whereas Fine-Mem and Mem-T use the memories cited in reasoning or retrieval traces to narrow credit (Ma et al., 2026a; Yue et al., 2026), although a memory can appear in a trace without having helped.

Operation selection cannot compensate for repeatedly generating incomplete or overly specific candidates, so formation and maintenance can share feedback. UMEM optimizes extraction and management jointly by the marginal utility of memory over related queries (Ye et al., 2026c), which makes the chosen query neighborhood a substantive design choice. CoMAM coordinates extraction, profile, and answering agents in conversational memory with local and weighted global rewards (Mao et al., 2026), and MemSkill revises a shared bank of memory-processing routines, so that one update affects many later units, although its routine-evolution ablation comes from conversational memory (Zhang et al., 2026f) (Table 26, Appendix C). Revised processing procedures, including Auto-Dreamer’s learned consolidator, are worth their cost only if they produce better later experience than fixed procedures under matched evidence and budgets.

Repository updates also change what retrieval can return, while retrieval controls which experience receives

later feedback. Revising, merging, or removing experience therefore has to update its indexes and relations, as A-MEM and SAGE do when writing general memories (Xu et al., 2025b; Wang et al., 2026c). Before low usage counts as grounds for retirement, maintenance also needs to distinguish experience that was unavailable, retrieved but not selected, or applied without benefit. When the repository and the retriever adapt together, the schedule matters: ExpGraph pairs graph maintenance with a learned retrieval copilot (Feng et al., 2026a), and CoEvo-Mem alternates router and graph updates during training before freezing both (Ye et al., 2026a). Scheduling also sets how long new experience waits before it can be retrieved; in an adjacent study, delaying buffer flushes lowered answer quality (Zhou et al., 2026b), so update policies should be compared on processing cost, availability delay, and downstream quality together.

6.6 Safeguarding Experience Evolution

Updates that improve performance can still spread unsupported guidance: it can survive admission, spread through reuse or consolidation, and influence later guidance after its source is removed. Limiting contamination comes first; provenance records then let a later error be traced to what it affected, which selective recovery needs once a mistaken unit is found. Sections 9.6 and 9.7 develop the open problems.

Contamination has to be checked wherever experience is written, derived, or activated, because reuse spreads a mistaken unit, and the units derived from it can look like independent corroboration although they all repeat one source. Reported attacks enter at each of these points: through optimized triggers that promote retrieval of malicious demonstrations (Chen et al., 2024b), ordinary queries that induce malicious memory writes (Dong et al., 2025), reflection that distills locally successful adversarial solutions into domain-level rules (Wang et al., 2026d), and malicious procedure templates that persist across sessions (Srivastava and He, 2025); a further taxonomy separates threats that act directly, through composition, or after remaining dormant (Gao et al., 2026b). EvoBreak (Yan et al., 2026a) needs even less access: its adversary can submit tasks and read the experience generated from those interactions but cannot edit that experience directly. Through tasks alone, it induces units that model evaluators mostly judge harmless individually but that elicit harmful behavior when a later query activates them together. In SE-Agent and ReasoningBank, attack success rises as more induced experience is retained.

Formation can also hide contamination and become its target. In SkillJack (Ying et al., 2026), a model judge flags 98.5% of poisoned trajectories but only 11.4% of the skills SkillX extracts from them, and 80.0% of skill-mediated attacks persist after the poisoned records are deleted. PoisonedEvolution (Chen et al., 2026b) instead crafts trajectories that make a target behavior look causally useful, recurrent, and general to the skill evolver, embedding it in 91.0% of trials on SkillClaw and 61.5% on Trace2Skill when attackers supply 10% of the records, so a support check is itself an attack surface. Defenses therefore have to reach beyond the write interface, to derived guidance and to the combinations that a later query is likely to activate. Defenses such as A-MemGuard, which validates agreement across reasoning paths and consults a separate store of failure lessons (Wei et al., 2026), have been evaluated only against specific attacks, so their protection beyond those attacks is unknown.

An update history records what changed and why, whereas derivation provenance records which sources support the result; after a merge, the first does not reveal which source supports each resulting claim. ByteRover records reasons for curation operations in conversational memory (Nguyen et al., 2026); derivation provenance, however, should keep the execution evidence e_i of a derived unit (Equation 1) tied to the trajectories and counterexamples that actually support it. MemLineage (Ouyang and Hou, 2026), an adjacent memory-security system, records a weighted graph of which retrieved entries influenced each new memory and refuses sensitive actions whose justification descends from an external source; it is evaluated against memory-poisoning attacks, not for the quality of the guidance it keeps. Source authority must also survive rewriting: *memory provenance laundering* occurs when consolidation turns a low-trust external observation into apparent user history or workflow support (Xu et al., 2026a), so repeated retention must not by itself give rewritten text authority. Traceability has privacy costs as well, since membership inference can reveal what agent memory retained (Chen et al., 2026d), so maintenance needs enough evidence for investigation without keeping all source data indefinitely.

When guidance loses support, recovery must find its potentially affected descendants, check their remaining support, and restrict, remove, or reconstruct them, stopping unsupported influence while preserving justified

guidance. MemoRepair (Zhao et al., 2026e) withdraws affected descendants and republishes only validated successors whose required predecessors have also been repaired, but its experiments support cascade repair only when influence provenance is complete. Descendants may also be written by the agent itself: in EVOMAL (Wu et al., 2026c), coding agents on SWE-bench Verified tasks author new malicious skills using retrieved malicious skills as templates, and for three of six models the infection persists after the planted skills are withdrawn, so revoking the original units does not remove what was derived from them. Selective repair also needs a clear unit to revoke: if an overgeneralized rule such as “resubmit the form after every failed payment” enters a derived skill, repair should remove that branch while keeping the checks that other trajectories support, and then recheck the skill as an executable whole.

Repairing stored experience does not correct an execution that already consumed it. Dependency-guided rollback and execution-state unlearning replay the affected computation from recorded dependencies (Yu et al., 2026a; Yao et al., 2026), but they assume that the faulty memory has been identified and cannot reverse side effects. Because deleting a unit leaves its earlier effects in place, evaluation has to distinguish deleting experience from removing its harmful guidance while preserving valid experience, as MemSecBench’s selective-repair measure does (Chen et al., 2026h), and to report residual harmful influence alongside preserved capability and repair cost (Section 7.5.3).

6.7 Unresolved Trade-offs in Experience Maintenance

Before changing retained experience, a system needs to check that the change has support, helps on the affected tasks, and remains useful alongside the rest of the repository. These are the three check levels defined at the start of this section, and each has a counterpart among the evaluation protocols. A support check corresponds to a unit-level audit of evidential support (Section 7.2.1), and a paired comparison to a local intervention such as the four-condition replay (Section 7.3.3). A collection-level test either compares units separately and together (Section 7.3.3) or audits the repository as a whole (Section 7.2.2). Table 16 maps problem signals to maintenance choices and the evidence each requires, and Table 26 in Appendix C records representative systems beyond admission. Admission and repair results expose the trade-offs discussed next, and four open questions close the section.

Admission quality can determine whether updating helps at all. The Experience-Following study (Xiong et al., 2026) compares updating policies for four agents with a fixed copy of the same starting memory: adding every trajectory ends below the fixed-memory baseline, whereas selective addition with a strict evaluator based on ground truth ends above it. The reported stream curves also show benefits from selective updating. The practical constraint is the feedback needed for that selectivity: deployed agents rarely have those labels. Successive updates can reverse an early gain, as the consolidation results in Section 6.2.2 show.

Regression and recovery results show what maintenance must preserve alongside the targeted change. Auto-Dreamer identifies a task family harmed by consolidation, while MemoRepair checks reconstructed successors under assumptions about provenance and validation coverage. These results motivate testing unaffected behavior and keeping the support needed to repair descendants. Section 7.5 specifies how update and mechanism comparisons can establish benefits beyond an individual accepted repair.

Several choices discussed in this section have not been compared directly. The review triggers of Section 6.1.1 have not been compared on a shared task stream, and the only same-evidence comparison of update scope sets itemized updates against rewriting a whole playbook, not against replacing a selected region (Section 6.2.2). For learned maintenance policies, the credit-assignment schemes of AgeMem, Fine-Mem, and Mem-T have not been compared on a shared update stream, and operation policies would also need a shared candidate stream, because a policy can look better simply by receiving cleaner candidates or a stronger retriever (Section 6.5).

The first of the four open questions is how broadly to update. A repeated failure can originate in a particular instruction, a missing applicability condition, or the procedure that extracts experience. Editing each affected entry separately may leave a shared formation defect intact; revising a whole repository for a local execution lapse can introduce unnecessary regressions. Choosing between targeted and broader updates means asking how many decisions share the implicated assumption and what evidence can test them. A useful research direction is diagnosis that selects both an operation and its scope, then checks the targeted repair together with behavior expected to remain valid.

Table 16 Experience maintenance choices across operations and cross-cutting concerns. Required evidence is labeled by the three check levels defined at the start of Section 6: support, paired comparison, and collection-level test; predicted screening and comparisons between procedures are labeled separately.

Operation or concern	Problem signal	Maintenance choice	Required evidence	Main risk
Admission	A new candidate is proposed	Accept supported value; defer uncertainty; reject unsupported content	<i>Support:</i> source support and applicability. <i>Predicted screening:</i> novelty and utility against existing entries. <i>Paired or collection:</i> reuse validation where feasible	Excluding rare useful guidance or accepting a false explanation
Consolidation	Equivalent or fragmented guidance accumulates	Merge equivalents; abstract or share structure while keeping distinctions	<i>Support:</i> equivalence for deduplication; preserved distinctions and dependencies for broader integration. <i>Collection:</i> joint reuse checks	Compression removes an action-changing condition
Refinement	Contradictory outcomes or changed prerequisites	Diagnose; adjust reliability, scope, or actions; validate before replacement	<i>Support:</i> trajectories and environment conditions for diagnosis. <i>Paired:</i> targeted tests of the repair. <i>Collection:</i> recheck of earlier supported uses	Repairing the wrong cause or breaking valid behavior
Retirement	Suspected loss of added benefit	Downweight, archive, or delete according to evidence and recoverability	<i>Paired:</i> benefit to the current agent, judged after checking exposure opportunities. <i>Collection:</i> remaining coverage	Confusing poor access with low value; losing a rare recovery
Maintenance coordination	Repeated formation defects or mismatched access	Change operation selection, formation, or retrieval, depending on where the failure originates	<i>Procedure comparison:</i> revised versus unchanged component on the same candidates, tasks, and budgets. <i>Collection:</i> task outcomes linked to updates and use	Delayed rewards obscure which change helped
Evolution safeguards	A premise loses support or contamination is detected	Restrict affected guidance; trace, repair, and validate supported successors	<i>Support:</i> source authority, derivation links, independent support. <i>Paired:</i> checks of repaired successors	Missing descendants or removing independently valid guidance

Coverage across the collection raises a second question, because entry-level scores can miss interactions, as the mutually covering procedures in Section 6.4 show. A consolidated rule may shorten retrieval yet obscure an exception needed for a rare failure. Joint validation should test the proposed collection under the same retrieval and context budgets, including cases that exercise the distinctions at risk. Source links support recovery only when the system recognizes that the compact guidance is insufficient and can recover the missing evidence. What a joint test must cover is itself unsettled: no system we reviewed specifies which interactions it should include, or how to find them before a rare failure does. Nor does any detect, within a limited validation budget, that a consolidated version has dropped action-changing information before it replaces the original.

Third, maintenance cannot read value directly from usage data that its own earlier decisions have filtered. A unit that was never admitted or rarely retrieved has little use evidence, and a retired unit gathers none. A poorly exposed recovery procedure may therefore look useless because earlier decisions kept it from being tried, which is why ReMe prunes only after a minimum number of retrievals (Cao et al., 2026) and why retrieval frequency is better read as a nomination for review (Section 6.4). Correcting for this requires checks that distinguish retrieval failure from weak guidance and measure value for the current executor; Section 7.5 develops the controlled comparisons and longitudinal protocols involved.

The last question is where to spend verification effort: which unit, and which operation, deserves the next check. A local edit can later guide many tasks and become part of several skills, so the consequences of an

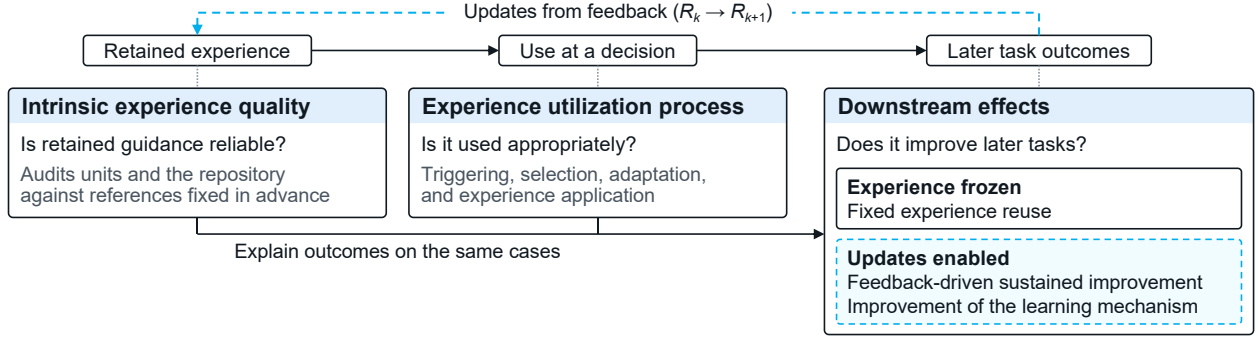


Figure 16 Evaluation of agent experience across three targets: intrinsic experience quality, the experience utilization process, and downstream effects. Quality and process measurements help explain outcomes on the same cases, while downstream evaluation distinguishes fixed experience reuse from improvement with feedback-driven updates.

update depend on both its uncertainty and how widely other experience comes to rely on it. Retained versions, limited initial reuse, or additional checks can contain this reach while evidence accumulates, at the cost of delayed benefit and maintenance effort. Effort spent on one operation also shifts costs to another: a strict admission gate beats adding every trajectory in the comparison above but discards rare recovery procedures (Section 6.1.3), and consolidation saves context but can drop an action-changing detail (Section 6.2.2). Optimizing write count, repository size, or one update’s score separately can therefore shift work elsewhere without improving subsequent task solving. Neither how widely other experience depends on a unit nor the costs and benefits of checking can be read from a single update: the first becomes visible only after reuse, and the second accrues over the whole task stream. Section 9.6 extends this problem to successive update rounds and experience derivations.

7 Evaluation of Agent Experience

Task success alone does not show whether stored experience is correct or whether the agent benefited from using it. An agent may ignore a useful lesson or complete a task despite errors in a stored procedure. Moreover, a higher task score may result from additional model calls rather than from using experience. Evaluating experience therefore requires answering three questions separately: whether the retained guidance is reliable, whether the agent uses it appropriately, and whether it improves later tasks. These questions define three targets: **intrinsic experience quality**, **the experience utilization process**, and **downstream effects** (Figure 16).

Every later comparison shares the scoring and feedback-access rules set out in Section 7.1; Sections 7.2 to 7.4 then evaluate retained experience, how agents use it, and its effects on later tasks. Downstream evaluation spans two subsections: Section 7.4 holds experience fixed during evaluation, and Section 7.5 compares an agent across updates for feedback-driven sustained improvement and improvement of the learning mechanism. The closing subsection (Section 7.6) connects the three targets to explain an outcome and, for each improvement claim, separates the evidence it requires from the designs and diagnostics that help obtain or explain that evidence.

Each protocol is identified as implemented in prior work or proposed by this survey, and Table 28 in Section D records each study’s evidence role under the scope of Section 2.6. Section 8 then examines the application settings in which these evaluation targets acquire domain-specific constraints.

7.1 Scoring Reliability and Feedback Access

Evaluation may overestimate the value of experience for two reasons. First, a scorer may reward wording or format that does not reflect actual quality or task performance. Second, experience may have been revised using feedback from the same tasks later used to evaluate it. Reliable comparisons therefore require both a checked evaluator and a clear account of which feedback the system could access. We first examine how to validate scores, then specify the feedback permitted during acquisition, mechanism selection, and independent

testing. These requirements apply to experience quality, the utilization process, downstream outcomes, and learning across updates.

Every scorer, scripted or model-based, has blind spots that can favor one condition over another. Executable verifiers cover only their programmed checks, as in SkillsBench (Li et al., 2026d); Trust-Memevo’s trust-worthiness scores depend on the evaluator model, criteria, and prompt (Cheng et al., 2026). SkillFlow’s verifier-derived rubrics inherit verifier coverage and correctness limits, while model-generated skill patches add a distinct model-and-prompt dependency (Zhang et al., 2026o). Using the same model family to write and score experience can reward evaluator-preferred wording without improved behavior, and repeated updates can compound the problem: in SkillLearnBench (Zhong et al., 2026), further iterations improve generated skills when feedback comes from outside the agent, whereas self-feedback alone leads them to drift. Scoring models, criteria, and prompts should therefore stay fixed across conditions, so that a score difference cannot come from the scorer; agreement with independently established labels on a subset, together with independently verified outcomes where possible, shows how far model-based scores can be trusted.

Scripted checks also need auditing: an audit of publicly released computer-use trajectories scored as failures found evaluator false negatives and broken tasks among them (Dong et al., 2026) (Section 3.1.2). Such audits are informative only when they use complete trajectories, distinguish invalid tasks from agent failures, and document their coverage and agreement. The distinction between invalid tasks and agent failures matters for experience evaluation as well: a failed task check can indicate a bad repair, an incomplete verifier, or a broken environment. These possibilities should not all be fed back as evidence that the retained lesson was wrong. Where an evaluator’s result is disputed, recording the disagreement and the evidence used to resolve it keeps the score from changing silently in one condition.

A later test confirms nothing independently if its feedback already shaped the experience or the mechanisms being tested. Our feedback-access rules require a study to disclose which tasks, target answers, and verifier signals were visible during *acquisition* (forming and admitting experience) and during any search over the procedures that form, organize, utilize, or maintain it. In a declared online-learning stream, feedback from a completed task may inform later tasks, provided that performance on each task is reported before the update it triggers and the permitted feedback is specified. Held-out generalization targets must remain unavailable to acquisition, mechanism selection, and tuning. A frozen replay, which reruns earlier tasks with updates disabled, may revisit acquisition tasks if the overlap is disclosed, but its outcomes must not update the repository or select mechanisms. The comparisons across updates in Section 7.5 rely on these rules most heavily, because they combine scores taken along the stream with checkpoint audits whose outcomes must not feed back into learning.

7.2 Evaluating Intrinsic Experience Quality

Intrinsic experience quality asks whether retained guidance is supported and usable, whatever any one executor achieves with it. Task success cannot answer this, because an executor may work around flawed guidance or fail to use sound guidance. We therefore audit stored experience directly, first as individual units and then as a repository, through the seven audit questions in Table 17. Unit-level questions ask whether guidance is correct, supported by its source, complete for a decision, and scoped appropriately; repository-level questions ask what guidance is missing, whether its basis and validity status survive revision, and whether applicable units agree. Each question mainly examines one or two components of the experience unit $E_i = \langle g_i, c_i, e_i, r_i \rangle$ (Section 2.2), as Table 17 marks; the questions are not additional requirements for qualifying as experience. Each question is reported separately, because in a single quality score a strength on one question could hide a defect on another.

Every audit below compares stored experience with a reference derived from task requirements, tool specifications, or independently verified cases, not from the experience itself, which would let it grade itself. The reference, counting unit, and denominator are declared before the comparison, and unresolved cases are reported alongside any rate, so that padding the experience with easily supported claims or quietly shrinking the audit population cannot inflate a rate. A resulting figure describes its declared audit population, and systems audited against different references cannot be ranked on that basis alone.

Table 17 Audit questions and representative metrics for intrinsic experience quality. Parentheses give the components of $E_i = \langle g_i, c_i, e_i, r_i \rangle$ that each question mainly examines; no reviewed protocol audits whether the reliability evaluation r_i matches the strength of its evidence. *Prior* marks implemented metrics and *Proposed* marks metrics or audits defined in this section; Section 7.6 discusses which prior metrics examine trajectory-derived experience directly.

Audit question (components)	What is audited	Representative metrics
Unit-level questions		
Correctness (g_i)	Whether claimed actions and effects hold under their stated conditions, evaluated against task requirements, reference outputs, or an action model.	<i>Prior</i> : tests generated with each skill in MUSE-Autoskill (Lin et al., 2026a); reference-output tests such as EvalPlus (Liu et al., 2023); action-model checks such as Plan-Bench (Valmeekam et al., 2023).
Evidential support (e_i)	How far the unit’s execution evidence supports each audited claim, separating supported, contradicted, and insufficiently evidenced claims.	<i>Prior</i> : Honest Lying check that retained reflections name the task’s target object (Dixit et al., 2026); HaluMem Memory Accuracy (Chen et al., 2025a). <i>Proposed</i> : shares of supported, contradicted, and uncheckable claims.
Actionability (g_i)	Whether the experience and its permitted supporting sources answer each decision an applicable context requires.	<i>Prior</i> : ACES per-criterion pass/fail and scores (Kevin et al., 2026).
Applicability (c_i)	Over- and under-inclusion of the stated scope relative to contexts labeled against independent requirements.	<i>Prior</i> : OEP check for one attacker-specified overgeneralization (Wang et al., 2026d). <i>Proposed</i> : shares of wrongly admitted and wrongly excluded contexts.
Repository-level questions		
Coverage (g_i, c_i)	Weighted share of independently specified requirements verified as covered by stored experience, including necessary conditions.	<i>Prior</i> : HaluMem Weighted Memory Recall (Chen et al., 2025a). <i>Proposed</i> : share of requirement weight verifiably covered.
Validity (e_i, r_i)	Whether invalid guidance is still presented as valid, and required valid content remains available with its evidence, against fixed reference lists.	<i>Prior</i> : MemSecBench Selective Repair Success Rate (Chen et al., 2026h). <i>Proposed</i> : audit against two lists fixed before the update.
Coherence (g_i, c_i)	Share of audited unit groups with overlapping conditions that give verified conflicting guidance for the same decision.	<i>Prior</i> : ConInstruct conflict-detection precision, recall, and F1 (He et al., 2026b).

7.2.1 Evaluation of Experience Units

Unit-level evaluation asks whether one piece of guidance can be trusted and used on its own, before any repository-level effect enters. The four questions run from what the guidance claims to what an agent can do with it, and each needs a different reference; a successful source trajectory alone answers none of them completely.

Correctness asks whether retained claims and prescribed operations hold under their stated conditions; a fully specified procedure can still act on the wrong account or file. A failure counts against a claim only when its prerequisites held, the guidance was followed, and the result contradicted the claimed effect, so a failed task does not refute a lesson that only recommends a check. Existing checks test claims against references other

than the experience’s own assertions, each for a narrow object. MUSE-Autoskill compares skill outputs with tests generated alongside the skill (Lin et al., 2026a), but how strict generated tests are depends on how their author read the requirement: in CoEvoSkills, a skill passed all 15 tests written by a separate verifier that matched results within 1%, yet failed the hidden benchmark test’s exact five-decimal match (Zhang et al., 2026e). EvalPlus checks code against reference implementations on additional admissible inputs (Liu et al., 2023), and PlanBench checks natural-language plans against an action model that separates illegal steps from executable sequences that miss the goal (Valmeekam et al., 2023). Whatever the reference, results should be counted per tested claim; counted per test input, many tests of one claim can hide claims that were never examined.

Evidential support asks how well a unit’s execution evidence e_i warrants each retained claim; whether the unit is grounded in trajectories at all is the inclusion criterion of Section 2.1, so a weakly supported reflection is still experience, only less reliable. The audit must separate claims with different evidential demands. For the dependency-repair lesson of Section 2.1, “the dependency was absent,” “its absence caused this failure,” and “check dependencies in comparable failures” are an observed outcome, a diagnosis, and provisional guidance. Implemented checks each examine one part of this link. Honest Lying checks whether reflections retained across attempts mention the task’s target object (Dixit et al., 2026), BenchTrace compares a predicted failure explanation with an annotated diagnosis (Huang et al., 2026), and HaluMem grades conversational memories claim by claim against their source dialogue (Chen et al., 2025a), a design that carries over to experience. For trajectory-derived experience, an audit can report the shares of audited claims that the linked trajectories support, contradict, or cannot check, separately for observations and for inferred explanations and guidance. Pooling them would let many supported observations conceal unsupported inferences, and counting unchecked claims as contradicted would penalize guidance that the trajectory simply did not test.

Actionability asks whether guidance supplies enough information to determine what to do, avoid, or check in an applicable context. The audit fixes the executor, tools, and permitted supporting sources and lists the decisions an applicable context requires before any experience is inspected, so that a detail supplied by tool documentation is not counted as a gap and a shorter explanation is not mistaken for missing information. For a checkout procedure, these decisions might concern which account and payment method to use, what counts as a confirmed order, and which steps need the user’s approval. The implemented metric instead rates how well instructions are written: ACES has an evaluator model score skill instructions for clarity, documented inputs and outcomes, workflow completeness, and failure responses (Kevin et al., 2026). Running the executor with the experience supplied directly can reveal remaining gaps, but a failed run also reflects how well the executor follows guidance: in SLBench, some violations persisted even after rewrites that made the intended safe action clear to human readers (Chen et al., 2026g).

Applicability asks whether a unit’s stated scope matches the conditions under which its guidance actually holds; a lesson to retry a timed-out request, for instance, holds on a search page but not on a payment page, where a retry can charge the user twice. Existing checks are narrow. OEP has an evaluator model inspect retained content for one attacker-specified overgeneralization (Wang et al., 2026d), while behavioral tests in SkillEvolBench and Skill-Use embed a skill need in broader requests or test abstention on requests that do not need the skill (Lei et al., 2026; Han et al., 2026b). A sharper test pairs contexts. With the experience version fixed, one pair changes surface details while preserving reuse conditions, and another changes a decisive condition, such as whether the request submits a payment, while keeping the failure superficially similar; applicability is labeled from reference requirements before the unit is scored. The results can be summarized by how often the stated scope admits contexts in which the unit does not apply and how often it excludes contexts in which it does (Table 19); a deliberately narrow, correct scope lowers coverage but is not false guidance.

7.2.2 Evaluation of the Experience Repository

Individually sound units can still fail as a collection: required guidance may be absent, supporting evidence or validity status may be lost during revision, and units may prescribe incompatible actions. Repository checks examine these problems on the stored units themselves, separately from retrieval and later behavior, so that a repository defect is not mistaken for a unit defect or a utilization failure. They proceed from what is missing, to what revision has lost or left standing, to whether applicable units agree.

Coverage asks what guidance the repository is missing, and because missing units cannot be inspected, the audit starts from target tasks and decisions defined independently of the repository: if target web tasks fail both from expired sessions and from changed page layouts, experience covering only the former leaves a gap. HaluMem’s *Weighted Memory Recall* for personal memories (Chen et al., 2025a) shows how importance weights and partial credit can enter such a measure. No reviewed protocol applies such a measure to experience; for it, the natural measure is the share of reference requirement weight that stored units verifiably cover together with their necessary conditions, counting each requirement once however many units support it.

Repository *validity* asks whether revision keeps each unit’s supporting evidence and current valid or invalid status, so that outdated guidance is not still presented as valid: storing a corrected checkout procedure is insufficient if the old version, which skipped the address check, remains available as guidance. Existing audits cover parts of this problem. MEMAUDIT measures, for factual memory under a storage budget, how completely the evidence for current, superseded, and deleted facts is kept relative to the best attainable selection (Bhargava and Barrento, 2026). MemSecBench counts a repair after poisoning as successful only if the targeted content is no longer presented as guidance and specified benign content survives, so clearing the store cannot pass (Chen et al., 2026h). An experience audit can keep this two-sided requirement beyond one repair case by fixing, before the update, one list of known-invalid guidance and another of content that must survive, so that deletion cannot quietly shrink the audit population.

Coherence asks whether units that apply to the same decision under overlapping conditions prescribe conflicting actions without a resolving exception or priority; two permitted alternatives, or rules separated by stated conditions, do not conflict. Implemented diagnostics locate candidate conflicts or calibrate detectors but do not estimate how often a repository’s applicable units actually disagree. SkillAligner flags tasks that succeed without retrieved skills but fail with them (Li et al., 2026b), ConInstruct scores conflict detectors on seeded, human-verified conflicts (He et al., 2026b), and TANGLE tracks whether curated conflicts remain visible in returned personal memories (Yang et al., 2026b), which cannot distinguish a resolved conflict from one that retrieval failed to return. An experience audit therefore compares stored units before and after revision and traces each drop in the conflict count to a recorded resolution, because unchecked deletion, or a condition added only to make two instructions appear compatible, can hide disagreement while losing valid guidance.

7.3 Evaluating the Experience Utilization Process

Stored experience that passes these checks may still be overlooked, selected for the wrong situation, or altered inappropriately before use. Process evaluation asks where reuse succeeds or fails by following the four utilization questions in Section 5: experience triggering, experience selection, experience adaptation, and experience application. We first evaluate whether the agent seeks and selects suitable experience, then whether adaptation preserves its conditions and application produces the intended behavior. Use records and controlled interventions help connect supplied guidance to observed actions. A justified decision to continue without experience also counts as a correct outcome. Here the repository is given; Section 7.5 examines what changes across repository updates.

7.3.1 Scoring Experience Triggering and Experience Selection Decisions

Triggering asks whether consulting experience is useful at a decision; selection asks which available units are relevant, under what conditions, and at what expected benefit and cost. Evaluation must count both unnecessary invocations and missed opportunities relative to the repository available at that moment, because retrieval frequency alone cannot distinguish useful assistance from repeated unsuccessful searches. We first score triggering at a named gate, including missed consultations, and then separate what the candidate pool holds from what selection delivers.

Triggering evaluation must name the decision being gated, because the three gates in Section 5.1—detecting a possible need, running retrieval, and injecting retrieved guidance—have different costs and failure modes; WebCoach, for instance, retrieves at every step but lets a coach decide whether to intervene, so its intervention rate does not measure retrieval frequency (Liu et al., 2025). Separating the calls, tokens, and latency incurred before and after the named gate would show what each gate costs; the reviewed protocols report only module-level or aggregate costs.

Missed consultations need a count of their own. Skill-Use scores the triggering decision itself, separating triggering from prescribed-action compliance and prohibited-action avoidance and testing correct non-triggers on look-alike tasks that do not need the skill (Han et al., 2026b); because it compares completion with a no-library run only where the skill was triggered, tasks lost to missed triggers go unmeasured. To count them, an evaluation can annotate sampled decision points at which the repository held applicable guidance calling for a different action and report the share at which experience was not sought. This share measures missed consultation; the replay in Section 7.3.3 tests the benefit lost.

A candidate pool may contain useful experience even when selection misses it, so selection evaluation needs one score for what the pool holds and another for what is delivered. The query-conditioned reuse (QCR) study separates candidate coverage from final selection in this way and finds that, in its frozen bank, the choice step loses little of the coverage that retrieval provides (Li et al., 2026g) (Section 5.2.2). Because delivering more of the retrieved entries does not reliably improve performance (Cao et al., 2026; Zhang et al., 2026j; Zhou et al., 2025), delivered-set size belongs alongside relevance in a selection report. Benchmarks that isolate ranking over a public skill corpus or evidence gathering from web-agent histories (Kang et al., 2026b; Wu et al., 2026a) show how to score these steps, although their references are annotated relevance and answers, not the validity or utility of individual experience units. Comparing fixed task-skill assignments, as in AFTER (Belikova et al., 2026), with ordinary retrieval would locate selection losses more directly.

Two further controls test selection beyond a single fixed bank. Selection should be tested as the repository grows, keeping the required evidence fixed while irrelevant content accumulates, as a scale-conditioned memory study does for conversational memory (Shao et al., 2026a). A curated reference can also separate losses in encoding and routing from failures in later use: EvoAgentBench (Gao et al., 2026c) backs every test task with trajectory-derived Ability units on the training side and compares automatic methods with a curated reference that supplies the matching Ability content. Across two scaffolds and three backbones, the curated content transfers across model families, while no automatic method keeps a positive gain in every setting, a pattern consistent with losses in encoding and routing rather than in the executor’s use of matching content.

7.3.2 Controls and References for Experience Adaptation and Experience Application

Adaptation and application turn selected experience into behavior, and a final score does not show which of the two steps went wrong. Adaptation evaluation asks whether rewritten guidance fits the target context while preserving required conditions, since an adapter that drops a precondition can turn a sound lesson into a harmful instruction; application evaluation asks whether that guidance informs the intended behavior, including justified rejection. The two steps are tested differently: adaptation against controls matched to what an adapter changes, and application against a function-specific reference for each of the five experience application functions.

For experience adaptation, the cleanest comparison fixes the retrieved candidates and varies only what is delivered. SkillAligner compares raw supplied skills with guides adapted to the task, execution interface, and other skills, including guide generation without retrieved skills, and has an evaluator model label the mismatches behind tasks that raw skills harmed (Li et al., 2026b). QCR fixes the candidate pool, selected trajectory, target state, executor, and tool budget while comparing query-conditioned notes with full trajectories and generic summaries (Section 5.3.2), and its human annotations also track whether stale values were correctly rebound (Li et al., 2026g).

AdaMEM adds a control for the guidance-generation step itself (Zhang et al., 2026n): on ALFWORLD, refreshing guidance at every step improves on supplying retrieved trajectories directly at a substantial token cost, and removing experience while keeping strategy generation lowers success sharply, showing that the retrieved content adds more than the extra generation. ExpSeek’s gain over direct reuse of the most similar experience comes with more time per question and changes selection and computation as well as rewriting (Zhang et al., 2026k); its GAIA analysis also finds diminishing accuracy gains with more frequent interventions.

Each control excludes one alternative explanation for a gain from rewriting. A generic summary tests whether shortening alone helps; generation without experience tests whether the extra reasoning step explains the gain; and fixing selection keeps the adapter from gaining simply by finding a better precedent. A gain can therefore be attributed to rewriting only against the control that matches what the adapter changes.

These comparisons leave unverified whether adaptation preserves the source applicability conditions. A preservation check on adapted guidance declares which properties may change and which preconditions must remain satisfied, then compares original and adapted guidance at the same decision state. It scores the intended action effect, including justified rejection; textual similarity would miss a fluent rewrite that drops the precondition blocking an unsafe action.

Each experience application function needs its own fixed reference, because task success cannot show which function experience changed. Among the reviewed protocols, three functions have such references, each for a narrow object. For experience-guided procedure reuse, SLBench tests prerequisites and exceptions within one supplied skill through executable cases (Chen et al., 2026g); for experience-guided generation, Mem2ActBench scores memory-based tool arguments with the reference tool supplied (Shen et al., 2026b); and for experience-guided evaluation, AutoCalibrate and Reflect-and-Revise measure how well revised scoring rules agree with human ratings of text (Liu et al., 2024; Harada et al., 2026).

Experience-guided interpretation and experience-guided decision-making have no comparable measure in the reviewed protocols. Agent-Pro, for instance, reports game performance, which leaves better beliefs entangled with other effects (Zhang et al., 2024a). Separate tests could locate whether experience changes understanding or choice. An interpretation test fixes the observations of a failed task and scores the agent’s diagnosis against verified evidence; a decision test instead fixes that diagnosis and the available actions and scores the choice. Both tests hold the function’s inputs fixed and vary only the targeted use of experience. Evaluation can begin with the system’s claimed main function and test its interactions with other functions after establishing its individual effect.

7.3.3 Attributing Behavior to Supplied Experience

The measurements above locate where a failure could arise; testing which steps contributed requires recording the use process and then intervening on it. A trace record of the triggering decision, selected unit identities and versions, original and adapted contents, resulting application, and any explicit rejection connects the measurements, as SkillAligner’s logs of skill identifiers and generated guides and AFTER’s links from trajectories to versioned skill files begin to do (Li et al., 2026b; Belikova et al., 2026). The record needs both identity and content, because an unchanged name can conceal a revised instruction. Such a record locates candidate failures; measuring their contribution takes an intervention.

Implemented interventions yield local proxies or sensitivity measures; none estimates net benefit. ProactAgent compares retrieval with continued execution from the same interaction prefix to build training-time process rewards (Cai et al., 2026), a proxy that requires exact state restoration and repeated continuations to interpret, whereas TAME’s model-generated contribution score (Cheng et al., 2026) is an attribution estimate without any intervention. Faithful Self-Evolvers removes, shuffles, corrupts, or replaces raw and condensed experience within frozen-backbone frameworks and finds that raw-trajectory perturbations generally change behavior more than perturbations of condensed guidance (Zhao et al., 2026b); such sensitivity shows dependence, not benefit, because following harmful guidance also changes behavior. For net contribution, we propose a *four-condition replay* that repeats one decision with the original unit, no unit, an independently checked replacement, and an irrelevant length-matched unit, holding the executor, state, and delivery interface fixed (Table 19).

A unit’s contribution also depends on the units around it and on what later updates do with it. Knockout comparisons on identical examples, as in Coalition-Aware Skill Reliability for MemSkill’s memory-writing routines, show contributions that change with bank composition and can reverse direction across domains (Zhao et al., 2026a), so retained units should be compared separately and jointly. In addition, a retrieval intervention can change future contents: periodic and once-per-episode retrieval produce different stored experience and final repositories in a controlled reuse study (Hu et al., 2026a), so a fixed-repository replay isolates the immediate effect, while matched streams with updates enabled measure the combined effect. Even a capacity control leaves contents free to differ: RoMeRL fixes how many entries are stored per task, but the compared methods still store different contents, so the comparison measures each method together with what it comes to store (Yang et al., 2026d). Decision-level effects therefore cannot be added into a claimed total; Section 7.4 evaluates the resulting task benefit and full cost.

7.4 Evaluating Downstream Effects: Task Benefits and Transfer

Quality and process checks explain whether experience is reliable and how it is used. Evaluation of downstream effects asks whether that use improves tasks and whether the benefit justifies its cost and survives changes in tasks or operating conditions. This subsection evaluates fixed experience reuse along its two routes. In runtime reuse, the executor stays fixed and only its access to a frozen repository changes at inference; in internalization, frozen experience trains the policy, and the comparison varies training rather than inference-time access before testing what remains without the original guidance (Section 7.4.4). Letting the repository or its processing mechanisms change is the subject of Section 7.5. The subsection begins with immediate effectiveness, weighs it against efficiency and amortized cost, turns to transfer and harmful reuse, and closes with internalization.

7.4.1 Immediate Effectiveness of External Reuse

Immediate effectiveness asks the most basic downstream question: does access to a frozen repository change which tasks the agent solves? A positive answer needs more than a higher mean score. Experience may help some tasks while harming others or raise success only by consuming more resources, and a success may be credited to experience the agent never used. The matched access/no-access comparison and its implemented examples are described below, together with the outcomes a report should include so that the measured benefit matches the claimed one.

The central comparison fixes the executor checkpoint, task instances, tools, environment configuration, and resource ceiling, while enabling or disabling access to a frozen repository. It estimates the total effect of the repository and its utilization interface. Paired runs show which tasks the repository changes from failure to success and which it changes the other way, through guidance that was unnecessary or did not apply. A correct lesson may add little if the executor already performs the check; conversely, success alone does not show that supplied guidance was used.

SkillsBench implements a related comparison under the same model-harness configuration, scored by deterministic verifiers (Li et al., 2026d). Its skill packages were authored by contributors drawing on public repositories or their own domain experience rather than formed from the agent’s trajectories, and its contrast jointly measures skill text, bundled scripts or assets, discovery, and induced execution. Neither the central comparison nor SkillsBench isolates one unit or utilization step; those interventions belong to Section 7.3.3.

Method papers apply the frozen-repository design to experience that the system forms from trajectories. ExpeL retains training trajectories and the insights extracted from them, freezes both under fixed extraction and retrieval procedures, and compares held-out success with retrieval and insight ablations (Zhao et al., 2024a). Such results support fixed experience reuse.

The outcome reported should match the benefit claimed. BenchTrace separately measures recurring failures and later task progress (Huang et al., 2026); avoiding a failure is not equivalent to completing the task. Matched task-level transitions should therefore be reported, splitting tasks into those solved in both conditions, only with experience, only without it, and in neither, because gains and harms can cancel in the mean; repeated runs show how much of each split is stochastic variation. Unchanged success can likewise conceal reduced cost or improved intermediate progress. Even with fixed resource ceilings, enabling experience may change actual tokens, reasoning steps, tool calls, and latency, which belong alongside the result.

A success rate also omits progress on unsolved tasks and how experience shaped the agent’s actions. MemoryArena reports the fraction of subtasks completed across interdependent sessions alongside task success (He et al., 2026d). Neither that measure nor the outcome split above reveals whether the agent imitated similar stored trajectories, errors included (Section 5.2.2) (Xiong et al., 2026).

7.4.2 Efficiency and Amortized Cost

Experience is worth its cost when it yields more benefit than spending the same budget on additional task solving. The comparison is easy to bias: preparation happens before the evaluated tasks, and retrieval and adaptation add calls that per-task counts may miss. If the unaugmented baseline receives fewer resources, a gain may reflect the extra budget rather than experience. We state what the account must include and what budget the control should receive.

Counting only the tokens spent on evaluated tasks overstates the efficiency of experience, because much of its cost is paid elsewhere. Over a common time horizon, the account should include formation, exploration, organization and search, delivered context, adaptation, execution, and maintenance. When policies or processing mechanisms are learned, it should also include policy training, mechanism generation, search, optimization, selection, validation, and the evaluator feedback used during optimization. One-time preparation and recurring use should be reported separately; for an offline repository, the report should give both the unamortized cost and the number of later uses assumed for amortization, since a cost spread over an unstated number of uses cannot be checked. For an online learner, experience acquisition—forming candidate guidance from trajectories and admitting it to the repository—happens inside the observed stream, so its cost already appears in each task’s account.

The no-experience control should receive the corresponding budget for task solving, such as additional attempts or checks. Success, tokens, tool calls, latency, and the cost of failed runs should be reported together, because an agent that fails early spends less without being more efficient. This cost definition also applies to the update comparisons in Section 7.5.

A budget-constrained web-agent study illustrates this control (Hajimiri et al., 2026). It counts acting-agent and auxiliary tokens for workflow induction, skill synthesis, retrieval, and verification in online AWM, ASI, and ReasoningBank. The base agent instead receives 15 interaction steps, against their 10, and rule-based pruning of duplicated accessibility-tree text. Across four WebArena domains and each of three backbones, it matches or exceeds the online methods’ aggregate success, mostly with fewer tokens; it remains competitive on WorkArena-L1, tested with one backbone. Matching is approximate, and the tasks are largely self-contained, placing little demand on cross-episode memory. The finding therefore concerns online acquisition within the measured stream; offline methods such as SkillWeaver still require separate reporting of preparation cost and its amortization.

7.4.3 Transfer and Harmful Reuse

Transfer is often reported as a single score on a new benchmark, yet several relations between the source and the target of experience can change at once, each helping or harming reuse for different reasons. A stored checkout procedure may carry over to a new store but mislead on a site with a different payment flow, or work for the agent that formed it but confuse a different executor, so harmful reuse is treated here together with transfer. We first separate the transfer relations a report should state, then review studies that vary the source domain, producer model, and executor architecture.

A transfer claim should state which of three separable relations changed between acquisition and reuse. The *task relation* distinguishes another attempt at the same task from a new instance, family, or domain; same-task retry shows cross-episode reuse, not cross-task transfer. The *environment and interface relation* covers tools, schemas, permissions, state distributions, and versions. The *producer-executor relation* records whether the agent that formed experience also reuses it and, if not, how the reusing executor differs in model or architecture. The relations can change together, but a cross-model result need not establish robustness to a new interface. A transfer report should also state sources, target split, repository update permissions, and acquisition overlap, as the feedback-access rules in Section 7.1 require, because overlap between acquisition and target tasks can make same-task reuse look like transfer. Paired helped/harmed tasks reveal effects hidden by average transfer scores, including outdated prerequisites, distracting precedents, and overconfident imitation.

Memory Transfer Learning constructs coding experience from benchmarks other than the target, compares trajectories, workflows, summaries, and insights, and separately tests experience from another producer model (Kim et al., 2026a). Cases solved without memory but failed with it expose domain-mismatched precedents, misplaced confidence in remembered validation routines, and misapplied best practices. Changing format changes delivered content as well as abstraction, so the representation comparisons apply only to the settings tested.

The effect of experience also depends on executor architecture. In a machine-learning-engineering study, debugging experience supports sequential refinement; within tree-based search it improves procedural stability while reducing exploration diversity (Zhao et al., 2026d). Part of this reuse stays within one coding run, which

makes the study adjacent evidence (Section 8.6); it nonetheless shows why the producer–executor relation should record executor architecture.

7.4.4 Internalization and Guidance Removal

Internalization asks what behavior learned with experience remains useful after its original guidance is withheld (Section 5.5). Its necessary comparison varies training rather than inference-time access, because removal alone cannot distinguish internalization from pre-existing ability or ordinary training. Freezing the source experience can isolate the training contribution of a fixed collection; when experience and the policy are updated together, the longitudinal controls in Section 7.5 and the co-evolution design in Section 9.5 also apply.

Two comparisons are needed. The first trains with and without experience from the same initial policy, matching tasks, permitted feedback, and total training budgets, and then tests both without the target guidance; this training-time comparison measures the effect of experience-supported training and is the necessary evidence that behavior has been internalized (Table 18). The second tests the same experience-trained policy with and without that guidance to measure its remaining inference-time dependence.

Complete removal concerns the designated external experience, while task instructions, observations, interaction history, and tools remain available. EMPO² evaluates its trained policy without retained reflection tips, SKILL0 without external skill prompts, and EDGE with and without retrieved experience (Liu et al., 2026e; Lu et al., 2026c; Xie et al., 2026a); EDGE’s component ablations concern task performance and would need to be crossed with the removal condition to show what each component contributes to retained behavior.

Selective removal and modular execution require narrower claims. Skill0.5 removes general skill guidance while continuing to retrieve task-specific skills (Zhu et al., 2026a), so evaluation should report these access conditions separately. PEAM routes familiar skill categories through learned LoRA adapters but falls back to a slower deliberative module when no adapter applies or verification fails (Guo et al., 2026a), so its adapter path and full system need separate evaluation, including fallback frequency, since a success after fallback says nothing about the adapter. Every condition should state which learned modules remain, because a retained module can supply what the removal appears to test.

Finally, internalized behavior should be tested under changed applicability conditions as well as on the training distribution; for example, a learned habit of retrying failed requests should still be withheld on pages where a retry repeats a payment. Its cost account covers experience acquisition, training, and inference, as in Section 7.4.2.

7.5 Evaluating Downstream Effects: Sustained Improvement and Improvement of the Learning Mechanism

When an agent updates its experience, evaluation must track whether those updates keep improving performance on subsequent tasks. If the agent also changes how it learns, the revised mechanism needs a separate comparison of subsequent learning. These are the two claims of feedback-driven sustained improvement and improvement of the learning mechanism defined in Section 2.5. Closed-loop learning (Section 2.3) changes the repository from R_k to R_{k+1} , and a system may also revise its *processing mechanisms*, the procedures that form, organize, utilize, or maintain experience. A task stream supplies successive tasks, and experience acquisition forms and admits guidance from their trajectories. Multiple sessions, repeated queries to a fixed repository, or added distractors provide no evidence of learning by themselves. Building on the outcome comparisons in Section 7.4, the subsection moves from comparing permitted updates at frozen checkpoints and along the task stream, through detecting regressions, to testing whether a revised mechanism improves subsequent learning.

7.5.1 Comparing Learning Configurations at Frozen Checkpoints

Learning across a task stream is usually reported as a score that rises over the stream. However, such a score mixes what the agent has learned with how difficult the current tasks are and what feedback they happen to expose. One way to separate these effects is the *frozen-checkpoint audit* we propose: it first specifies what each compared configuration may update and then pauses updating at common checkpoints to measure every

configuration on the same audit tasks. When updates cannot pause, scoring each streamed task before the agent updates on it offers a lighter alternative with narrower conclusions (Section 7.5.2).

A learning comparison must first fix what each configuration may change, because narrowing one retained lesson and revising the procedure that extracts future lessons are different interventions that support different improvement claims. Splitting the closed-loop learning of Section 2.3 by whether processing mechanisms may change, and adding a no-experience control and a fixed repository, yields four evaluation conditions: no cross-episode experience, a fixed repository, an updated repository under fixed mechanisms, and updates to both repository and mechanisms. The three configurations with experience share the starting repository and processing mechanisms, so the three adjacent contrasts in this list test fixed experience reuse, continued content updates, and permission to change mechanisms (Figure 17(a)). Any changed mechanism must be identified and versioned, because a growing repository or a fixed updater run many times can resemble a changed mechanism.

A checkpoint comparison is fair only if each configuration reached its checkpoint under matched conditions, so schedules and comparators should be declared in advance and the path to each checkpoint recorded. Each checkpoint records repository and mechanism versions, tasks and feedback observed since the preceding checkpoint, resources consumed, and when updates became accessible. External task streams and permitted feedback are matched across conditions, and the feedback actually observed is recorded as well, because different actions can expose different evidence. Resources follow the total-budget accounting in Section 7.4.2, including preparation and mechanism optimization when applicable.

The audit must not change what it measures, so updates are disabled while paired audit instances measure each checkpoint on previously supported task families, held-out targets, and unfamiliar compositions or environments. Successive checkpoints share these audit tasks, or use tasks matched to them family by family, so that a change between checkpoints cannot come from a different audit set. Audit tasks use task-specific independent verification where available, and their outcomes stay outside updating and selection under the feedback-access rules of Section 7.1. Repeated independent runs should support uncertainty intervals for checkpoint differences.

The audit reports three distinct outcomes. *Capability acquisition* identifies newly solvable cases relative to the preceding frozen checkpoint or another predeclared baseline. *Checkpoint retention* measures preservation of capabilities established at an earlier frozen audit; it is different from retaining an experience unit or reusing a library without updates. *Generalization* measures benefit on targets excluded from acquisition against a declared no-experience or other appropriate baseline. New successes and regressions are reported separately, so an unchanged mean cannot conceal lost capabilities.

7.5.2 Comparing Learning Configurations along the Task Stream

When pausing updates for a separate audit is impractical, evaluation can compare the updating agent with a frozen copy along the task stream. Each task is scored before the agent updates on it (Section 7.1), and the frozen copy attempts the same task while keeping the starting repository. Before running the study, specify the comparable task interval, task families, and windows over which further contributions from multiple updates will be assessed. Report the difference from the frozen copy and changes between update stages, with uncertainty, acquisition, retention where tested, regressions, and costs. Evidence may include plateaus or local deterioration; the claim requires identifiable later contributions, not a monotonically increasing advantage or only persistence of an early gain. A high stream average alone cannot identify those contributions.

Task comparability matters because changes in demand can mimic or conceal learning. An agent that learns a check for one kind of failure early and then meets that failure more often in later windows can gain on the frozen copy even if no later update adds capability. Conversely, a shift toward tasks that experience cannot help can hide genuine later gains. Randomizing task order makes windows comparable in expectation; when order cannot be controlled, reporting the difference within each task family removes shifts in the mix of families. Without control for changing task demands, the comparison establishes only the relative benefit of updating on that particular stream.

The stream comparison also leaves earlier capabilities and held-out transfer untested. Because tasks are scored before the agent learns from them and are not revisited, this design cannot measure checkpoint retention or

generalization to held-out targets. Those questions still require the frozen-checkpoint audit.

Existing stream protocols each implement part of this audit. SEAGym (Zheng et al., 2026) comes closest: it saves snapshots of an updating harness, comparing ACE, Training-Free GRPO, and AHE, and scores them on held-out in-distribution and out-of-distribution views and on replayed training tasks, finding that frequent updates can fail to improve held-out performance and that useful intermediate snapshots can collapse later. It audits one updating configuration at a time, however, so the four configurations of the audit are never compared at common checkpoints. Among other benchmarks, SkillFlow and StreamBench track scores along updating streams (Zhang et al., 2026o; Wu et al., 2024a); AgentStream varies whether tasks arrive isolated, sequentially, or interleaved, and finds that method rankings change with model and stream composition (Yan et al., 2026b); and AgentCL and SkillEvolBench update experience during acquisition, then freeze it for replay and for held-out or shifted deployment (Shu et al., 2026; Lei et al., 2026).

Method papers with online accumulation, such as Voyager, AWM, and ReasoningBank, keep their extraction, retrieval, and addition rules fixed and report cumulative or streamed scores (Wang et al., 2024a, 2025e; Ouyang et al., 2026), which are compatible with feedback-driven sustained improvement but are not a test of it. AWM and ReasoningBank start from empty repositories, so their agents without experience already serve as the frozen starting copy; what they lack is the window-by-window comparison described above. Both report averages over the whole stream, and their cumulative success curves cover only a single WebArena subset, with no baseline curve in AWM’s case; such curves mix later gains with task order and difficulty, and neither study reports regressions.

Gains on cases that a no-memory baseline already solves and on cases it fails must be reported separately, because experience can move the two in opposite directions. A controlled reuse study with raw-trajectory memory compares acquiring experience on other tasks with acquiring it on the target tasks themselves (Hu et al., 2026a). Cross-task acquisition solves slightly more of the cases the no-memory baseline already solves but sharply fewer of the cases it fails; the result comes from one ALFWorld direction with two runs per condition, and insight memory largely removes this asymmetry. Separate reporting matters along other lines as well: EvoMemBench reports reuse within an episode and across episodes separately (Wang et al., 2026h).

Recent benchmarks also vary task order and feedback type directly. PATH-Bench (Yang et al., 2026c) builds sequences with controlled helpful or interfering histories around recurring probe tasks to measure forward transfer, backward transfer, and forgetting, and in its preliminary LifelongAgentBench study two orders of the same task set change the ranking of agents. Evo-Memory (Wei et al., 2025) runs its tasks from easy to hard and from hard to easy, and stores either successful experience only or successes together with failures; both choices change results more for simple retrieval baselines than for its refining agent.

7.5.3 Evaluating Regression and Error Propagation

Update robustness asks whether later evidence corrects misleading experience while preserving previously supported capabilities. Ordinary-update regressions and deliberately misleading content need separate tests because they assume different error sources. A correction that narrows an overly broad rule, for instance, should also be checked on cases where the rule’s original conditional guidance remained useful.

For ordinary updates, before/after tests should cover both the cases affected by new evidence and previously supported cases, and a log of admission, consolidation, refinement, and retirement connects observed changes with the maintenance decisions in Section 6. Trust-MemEvo covers the preservation side by freezing benignly evolved memory for trustworthiness tests (Cheng et al., 2026), and Yu et al. (2026c) continue memory evolution on a new task stream while testing held-out probes of the old tasks: naive updates lower retained performance, and protecting memories with repeated self-verified support reduces the average retention gap from 2.3% to 0.7%. Stress tests instead inject misleading content to expose propagation paths: MemEvoBench measures attack success and misleading-memory detection across rounds (Xie et al., 2026b), OEP measures the later effects of induced over-generalized rules (Wang et al., 2026d), and RoMeRL corrupts a fraction of early entries and finds that broader exploration can keep crediting unusable content over ten rounds (Yang et al., 2026d). Successful repair, collateral regression, and the continued influence of experience already shown to be invalid need separate reports, because a repair can succeed on its target while damaging valid content elsewhere.

7.5.4 Evaluating Improvements to the Learning Mechanism

When a learning mechanism is revised, the question is whether the agent subsequently learns better or merely uses what it already has better, and a higher score on the repository available at revision time cannot tell the two apart. Existing studies offer only partial evidence on this point, which motivates the comparison from a shared starting state proposed here; a separate question is how to establish that a revision originated from the agent’s own improvement process.

Each reviewed mechanism comparison is limited in scope or in what it can attribute to the mechanism. MemSkill compares evolved memory-processing routines with fixed operations, but its routine ablation uses conversational memory (Zhang et al., 2026f), and CoEvo-Mem tests co-adaptation of a retrieval router and a memory graph during training and then freezes both (Ye et al., 2026a). MemEvolve starts every candidate architecture from empty memory, so architectures are compared together with the experience each builds, and a difference between them may come from forming experience or from using it (Zhang et al., 2026c). DGM and Hyperagents go beyond explicit experience to agent code: DGM supports modification benefits within coding (Zhang et al., 2026g), and Hyperagents transfers task-agent and meta-agent implementations together, a system-transfer result rather than the substitution of one mechanism (Zhang et al., 2026h).

To separate immediate use from subsequent learning, we propose a *shared-start mechanism comparison*, summarized in Figure 17(b), which runs both mechanisms first without updates and then with them from a common starting state. In the first step, the original and revised mechanisms run from the same repository and executor state on matched held-out tasks with experience updates disabled, which estimates their immediate difference in using existing contents, including triggering, selection, adaptation, and application. In the second step, both return to that shared starting state and acquire experience on matched new task streams, each branch keeping its mechanism version fixed while its repository evolves.

The shared starting state requires compatible representations and interfaces that expose the same retained information to both mechanisms. If that substitution is undefined, an empty-start variant gives each mechanism an empty store in its own representation, keeps the same executor, and compares acquisition on matched streams. Because this variant compares each mechanism together with the content it builds, it cannot identify an immediate-use effect on identical retained contents. Both variants use the stream controls and frozen audits above and count total resources as in Section 7.4.2.

The comparison reports absolute held-out scores, each mechanism’s change from its own pre-acquisition score, and the difference between those two changes (a difference-in-differences contrast), because final scores and gains from each starting score can rank the mechanisms differently. The area under the checkpoint learning curve per unit of cost additionally shows when gains occur. These measurements help distinguish an initial advantage in using the shared repository from subsequent learning; the learning effect itself still combines building different repositories with using them differently.

The example scores in Figure 17(b) show why both views are needed. The original mechanism moves from 60 to 70 while the revised mechanism moves from 70 to 72: final scores favor the revised mechanism, whereas gains from their respective starting scores are 10 and 2. Neither comparison alone establishes which mechanism learns better; task composition, uncertainty, ceilings, and cost still matter.

If representations and interfaces are compatible, running each resulting repository under both mechanisms with updates frozen helps locate differences in retained experience, its use, or their interaction. Retention, regression, and transfer should accompany average gains; independent runs should expose stochastic variation, and ceiling effects should be checked. Changes to the executor, tools, or environment likewise broaden the comparison unless further controls isolate the processing mechanism.

A mechanism gain shows that the agent improved its own learning only if the revision arose from the agent’s prior improvement process, persisted into later learning rounds, and produced independently evaluated benefits on subsequent held-out learning. The record must therefore state who or what generated the revision, which trajectories informed it, and how it was selected; an externally designed revision supports a comparison of the two mechanisms, not a claim that the agent improved its own learning. Repeating this test across further mechanism revisions can support repeated improvement of the learning mechanism within the tested tasks, budgets, and feedback conditions, which is still short of open-ended improvement.

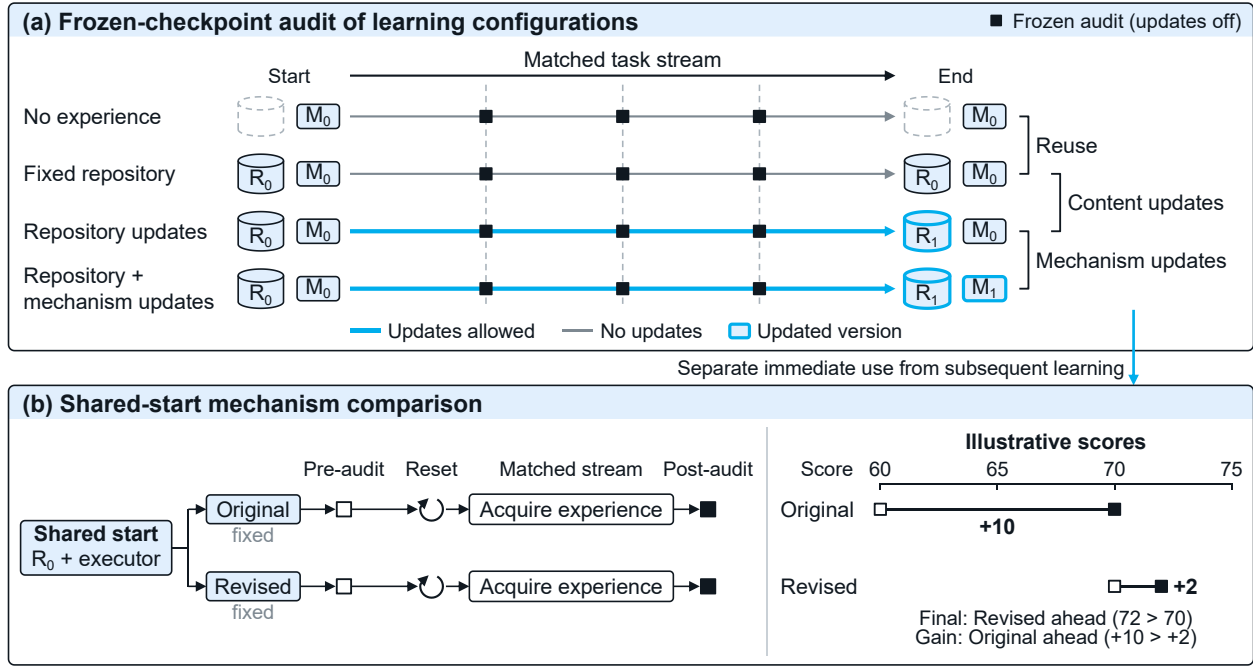


Figure 17 Evaluation protocols for experience-driven improvement. (a) Frozen-checkpoint audits compare four configurations along matched task streams to distinguish the effects of experience reuse, repository updates, and mechanism updates. (b) A shared-start comparison evaluates original and revised mechanisms before and after experience acquisition, separating immediate use of existing experience from subsequent learning. Its example scores show that final performance and learning gains can rank the mechanisms differently.

7.6 Matching Evaluations to Improvement Claims

A study may establish that a fixed repository helps while leaving continued learning untested. Choosing an evaluation therefore starts with the intended improvement claim and the evidence still needed to support it. We first examine what the reviewed studies establish and where their comparisons fall short. For each improvement claim, Table 18 then separates the evidence that is necessary from the diagnostics that help explain it, and a worked diagnosis shows how measurements of the three targets connect. The subsection closes by collecting the proposed measurements and indexing the reviewed protocols.

Overall, the reviewed quality checks each cover one narrow property of experience, and none examines all four components of a unit. MUSE-Autoskill tests generated skills, Honest Lying checks whether retained reflections name the target object, OEP checks for an induced overgeneralization, and SkillLearnBench has a model judge score generated skills against reference key points; these are the direct checks listed in the intrinsic-quality group of Table 28. Additional diagnostics examine code, plans, personal memories, authored skills, and instructions. These checks leave the calibration of a unit’s reliability evaluation r_i against its evidence unresolved, while the process protocols in Section 7.3 leave experience-guided interpretation and decision-making without function-level measurements.

Downstream comparisons show that benefits depend on the source of experience, executor architecture, and which tasks a baseline already solves (Sections 7.4.3 and 7.5.2). Longitudinal evidence is partial but informative. Experience-Following compares selective updates with a fixed starting memory on the same streams; its curves support benefits within those tested workloads, while leaving independent checkpoint retention and held-out transfer untested (Xiong et al., 2026). SEAGym audits snapshots of one updating configuration, and PATH-Bench and Evo-Memory vary task order. None of the reviewed protocols implements the frozen-checkpoint audit proposed here, which compares all four configurations at common checkpoints and reports acquisition, retention, regressions, costs, and uncertainty together (Section 7.5). Mechanism comparisons also remain limited: some test conversational-memory routines, some change mechanisms together with their repositories, and some transfer whole systems.

Table 18 Evidence requirements by improvement claim. Necessary evidence states what a comparison must establish; recommended designs provide ways to obtain it, and diagnostics locate causes or test additional properties. The indented internalization row gives the training-time route to fixed experience reuse.

Claim	Necessary evidence	Recommended designs and diagnostics
Fixed experience reuse	Runtime route: matched access/no-access comparison with a frozen repository and fixed executor and processing mechanisms; task-level gains and harms under stated resources and conditions (Sections 7.4.1 to 7.4.3).	Audit unit quality and, when claimed, repository coverage, validity, or coherence; trace triggering, selection, adaptation, and application; use local interventions to locate effects (Sections 7.2 and 7.3).
Internalization (training-time route to fixed experience reuse)	Matched training with and without the frozen experience from the same initial policy, tested without the guidance (Section 7.4.4).	Test the trained policy with and without the guidance to measure remaining dependence; state which learned modules remain in each condition and test changed applicability conditions.
Feedback-driven sustained improvement	Independent evidence that multiple updates add capability over a predeclared comparable task interval, against a copy with the starting repository fixed, under comparable feedback and budgets; versioned updates, acquisition, retention, regressions, costs, and uncertainty. Plateaus and local regressions are allowed; retained early gains alone are insufficient. Add a no-experience control for an absolute-benefit claim (Section 7.5.2).	Use the proposed frozen-checkpoint audit with a shared audit set, or score streamed tasks before updating and report the difference from the frozen copy by window, within task families, with regressions. Trace maintenance decisions and changing quality; test invalid-experience persistence, repair, and collateral damage for update-robustness claims (Section 7.5.3).
Improvement of the learning mechanism	Identified, persistent revisions compared with the original mechanism on subsequent learning under comparable tasks and budgets. Autonomous-improvement claims also need evidence that the evaluated process produced the revisions; sustained claims need independently verified gains across further revisions (Section 7.5.4).	Use the proposed shared-start comparison to separate initial use from subsequent learning. Cross-evaluate repositories and mechanisms when compatible; examine transfer, ceilings, retention, and regressions. Uncontrolled executor or environment changes support broader system-level conclusions.

Linking quality, process, and outcome measurements on the same tasks helps locate why experience helped or harmed an agent. Suppose that enabling a frozen repository leaves mean success unchanged. First isolate tasks solved without experience but failed with it, such as dependency repairs that attempted an installation in a read-only environment (Section 7.4.1). Use records then identify the selected unit, its version, and any changes made during adaptation (Section 7.3.3). If the stored unit permits installation in read-only environments, the applicability audit identifies a defect in its scope (Section 7.2.1). If the stored unit excludes those environments but the adapted text drops the restriction, the defect lies in adaptation (Section 7.3.2). A four-condition replay with an independently checked replacement that withholds installation can then test how the unit’s content affected the outcome.

None of the reviewed protocols runs this diagnosis end to end. SkillLearnBench scores generated skills, the resulting trajectories, and task outcomes on the same tasks, but a model judge scores the skills and trajectories against reference key points and oracle trajectories. SkillAligner instead has an evaluator model label the causes of harmed tasks, without fixed references (Zhong et al., 2026; Li et al., 2026b). Linking audited unit quality to downstream gains and harms on the same cases, against fixed references, remains an open gap.

Beyond the outcome, cost, and uncertainty controls in Section 7.4, which diagnostics a study needs depends on how specific its claim is. A frozen-repository benefit claim needs matched access/no-access comparisons. A claim that guidance is applied only where its conditions hold also needs applicability tests and use records showing where the unit was applied or withheld. Table 18 distinguishes the necessary evidence from further diagnostics.

Table 19 Measurements proposed in this section, by the evaluation question each answers and where each is defined.

Evaluation question	Proposed measurement	Defined in
Does a unit’s evidence support its claims?	Shares of audited claims that the linked source trajectories support, contradict, or cannot check, reported separately for observations and for inferred explanations and guidance.	§7.2.1
Does the stated scope match where the guidance holds?	Paired contexts that preserve or change one decisive reuse condition, labeled from reference requirements, with the share of reference-inapplicable contexts that the stated scope admits and the share of reference-applicable contexts within the intended target set that it excludes; unresolved and unlabeled contexts are reported separately.	§7.2.1
What guidance is the repository missing?	Share of reference requirement weight that stored units cover together with their necessary conditions, counting each requirement once.	§7.2.2
Does an update remove invalid guidance and keep valid content?	Two lists fixed before the update: known-invalid guidance that must no longer be presented as valid, and required content that must survive with its conditions and evidence.	§7.2.2
What does triggering cost?	Calls, tokens, and latency incurred before and after the named triggering gate.	§7.3.1
How often is applicable guidance not sought?	Share of annotated decision points with applicable, action-changing guidance at which experience was not sought.	§7.3.1
Does adaptation preserve the source conditions?	Original and adapted guidance compared at the same decision state against declared preconditions, scoring the intended action effect, including justified rejection.	§7.3.2
Does experience change interpretation or choice?	Observations, or options and their evidence, held fixed while the interpretation or choice is scored against an independent reference.	§7.3.2
What does a unit contribute to one decision?	<i>Four-condition replay</i> : one decision repeated with the original unit, no unit, an independently checked replacement, and an irrelevant length-matched unit; the first two estimate the unit’s effect, and the replacement tests whether its content limits the outcome.	§7.3.3
Do repository updates keep producing gains?	<i>Frozen-checkpoint audit</i> : updates paused at common checkpoints and every configuration measured on the same audit tasks, reporting capability acquisition, checkpoint retention, and generalization.	§7.5.1
Does a revised mechanism make later learning more effective?	<i>Shared-start mechanism comparison</i> : original and revised mechanisms run from a common state, first without and then with updates, reporting final scores, gains from each starting score, and their difference-in-differences.	§7.5.4

Table 28 in Section D indexes the representative protocols discussed in this section, grouped by primary evaluation question, with each work’s evaluated object, measured targets, and comparison basis. Evidence roles follow Section 2.6, with one addition, Boundary, for system-level comparisons whose changes extend beyond explicit experience to agent code or the surrounding harness.

For the four authored-skill studies marked Direct[†], Table 27 in Section D separates their reported sources from our reading of practice origin.

Table 19 collects the measurements proposed in this section by the question each answers. Two of them target gaps named in this subsection: tests that hold their inputs fixed and score interpretation or choice, and the frozen-checkpoint audit, which compares configurations at common checkpoints. Three others answer gaps found in Sections 7.2 and 7.3: applicability tests change a decisive reuse condition, adaptation tests check that preconditions survive rewriting, and the four-condition replay tests what one unit contributes to a decision. None of them checks whether a unit’s reliability evaluation r_i matches the strength of its evidence; that gap

remains open.

These recommendations connect partial measurements; no comprehensive benchmark suite yet implements them together. Section 8 adds application constraints, and Section 9.8 considers how experience changes the evidence available for later learning.

8 Applications of Agent Experience

Application studies test experience under the constraints of particular domains. In each setting, the four decisions named in the introduction recur: whether expected reuse justifies its cost, what to retain, how strongly experience should influence a task, and when to revise it. In computer use, for example, repeated operations on the same website can justify retaining a tested workflow, but the workflow still has to be checked against the page the agent currently sees.

We examine seven representative applications: question answering, software engineering, computer use, recommender systems, embodied control, scientific research, and financial trading and forecasting. For each, we describe the task difficulties that motivate experience reuse, how existing methods retain and use experience, and what their reported results establish. Working state kept only within one investigation, repair, or research workflow (Li et al., 2026i; He and Roy, 2026; Ghareeb et al., 2025; Weidener et al., 2026) counts as experience only if it is retained to guide later episodes (Section 2.6), so such systems are set aside. Section 8.8 then brings the findings together, and Section 8.9 relates them to the four design decisions.

8.1 Question Answering

Question-answering agents, whether they handle mathematical, database, or clinical questions, must decide how to solve a problem, which evidence to gather, and when to use a specialized tool. Similar reasoning or search difficulties recur even when the requested answer changes, so experience can preserve a useful solution strategy while the agent still obtains the facts needed for the current question. Reference answers make many of these outcomes straightforward to check, although research reports are harder to judge. Experience appears below in three roles that match these decisions: reusable solution strategies, guidance for evidence collection in longer investigations, and guidance for specialized tool use.

For questions answered from supplied information, the reusable content is a way of solving the problem rather than an answer to repeat. Buffer of Thoughts distills solution templates from earlier problem-solving processes and adapts a retrieved template to the current question (Yang et al., 2024). Dynamic Cheatsheet instead accumulates strategies and code while answering test questions, judging its own solutions without reference answers before updating its memory (Suzgun et al., 2026). With GPT-4o on Game of 24, success rises from 19% with the same instructions but an empty memory to 99% once it stores and reuses a Python solver. Most of this gain comes from one reused program, so here a procedure for a recurring subproblem is worth more than retained answers.

Multi-hop QA additionally requires connecting evidence from several retrieval steps. On HotpotQA, Reflexion retains feedback-derived reflections for later attempts at the same question, whereas ExpeL collects successful trajectories and semantic lessons across training questions and retrieves experience when answering unseen questions (Shinn et al., 2023; Zhao et al., 2024a). ExpeL’s experience can guide question decomposition or a change of search keywords. With gpt-3.5-turbo answering and GPT-4 extracting the lessons, ExpeL answers 39% of held-out questions, compared with 28% for a ReAct agent without experience, with its experience frozen before testing; Section 3.5 compares the contributions of its lessons and trajectories. In contrast, Reflexion’s gains accrue over repeated attempts at the same questions. Both methods reuse experience across episodes, but their evaluations cover different scopes: recovery on a previously failed question and transfer to other questions.

Longer research-style investigations can benefit from experience that redirects a search while it is under way. ExpSeek derives corrective guidance from paired successful and failed search trajectories and supplies it at steps where the agent’s uncertainty, measured as entropy, is high (Zhang et al., 2026k). With Qwen3-8B agents, this guidance raises average accuracy on four web-search benchmarks from 32.2% for a ReAct agent without experience to 41.5%. Experience is formed before testing, and the guidance is written by a larger

Qwen3-235B model, so the comparison combines stored experience with stronger guidance generation. Entropy separates correct from incorrect steps only moderately better than chance in the study’s analysis: uncertainty can prompt a check, but current evidence must determine whether to change the search route.

At the planning level, retained cases can guide how a deep-research agent organizes an investigation, although adding more cases does not keep helping. In Memento’s DeepResearcher comparison with a GPT-4.1 planner and o4-mini executor, case-based reasoning raises average F1, which balances recovery of the reference answer with the accuracy of the generated content, from 59.9 to 66.6 (Zhou et al., 2025). Additional retrieved cases eventually produce a plateau or slight decline, motivating selective access rather than maximizing the amount supplied.

Specialized tools make a further kind of experience reusable: how to form a tool call and correct it from the feedback it returns. MERIT’s database agent retrieves complete interactions with distilled lessons before a new request and shorter state–action–observation examples during execution (Wang et al., 2026g). With a GPT-5 agent on the interactive text-to-SQL benchmark BIRD-Interact, success on initial requests rises from 23.3% without memory to 29.2% when BM25 retrieves from memory written by the same procedure, and to 33.0% with MERIT’s trained retrieval. The stored content thus accounts for a substantial gain before learned retrieval adds more. For tool-using agents, this comparison motivates checking the value of the retained examples before investing in a more complex selector.

Clinical QA similarly couples reasoning with tool selection and checking. ReflecTool retains successful tool-use trajectories and tool-specific experience formed by comparing earlier attempts with reference answers (Liao et al., 2025). On later questions, stored trajectories guide action generation and accumulated experience helps evaluate candidate actions. On a subset of its clinical benchmark with Qwen2-72B, removing the stored experience lowers average performance by nearly 4 or 7 points, depending on how candidate actions are verified. The experience is prepared in advance and frozen during inference on its medical QA, image, and health-record benchmarks, and these benchmark results do not show clinical effectiveness in deployment.

8.2 Software Engineering

In software engineering, agent experience supports repository-level issue repair and environment setup. Repair trajectories expose explored files, rejected edits, dependency failures, and test outcomes, and tests let the current patch be checked by running them; yet similar symptoms can have different causes across repositories. Experience helps here when it preserves a useful diagnostic decision without copying a patch that does not apply. For issue repair, we ask what to retain, how experience selection affects its usefulness, and whether retained experience helps diagnosis across repositories.

Repair experience can come from agent trajectories or from public issue records, and it is easier to select when it distinguishes the subtask that needs help. SWE-Exp extracts comprehension and modification experience from agent trajectories, while MemGovern converts GitHub records into searchable experience cards (Chen et al., 2025b; Wang et al., 2026e). MemGovern’s offline cards improve SWE-Agent repair results at additional token cost, although the reported de-duplication leaves unclear whether benchmark issue records were excluded. Subtask-level memory further separates localization, diagnosis, modification, and verification (Shen et al., 2026a). Starting empty and updating after each issue in a single pass through SWE-bench Verified, it raises mean Pass@1 by 4.7 points over the memory-free agent across four backbones; instance-level memory slightly lowers or fails to raise success on the two Claude backbones. This comparison favors indexing the repair decision that experience can inform over matching whole issues by similarity.

Retained repair experience can also hurt when the agent selects the wrong precedent. SWE-ContextBench pairs issues with related earlier issues whose Claude Code trajectories form a fixed context pool (Zhu et al., 2026b). On its 99-task Lite split with Claude Sonnet 4.5, letting the agent choose from trajectory summaries lowers resolution from 26.26% without supplied experience to 22.22%, whereas supplying the summary of the truly related issue raises it to 34.34%. The pool already contains useful experience; the practical bottleneck is recognizing the relevant issue, not acquiring more records.

Comprehension experience improves repair even when it comes only from other repositories, according to the first version of SWE-Exp, which collects experience once before testing and excludes experience from the target repository before retrieval. With DeepSeek-V3-0324 on SWE-bench Verified, full SWE-Exp resolves

41.6% of issues in one attempt (Pass@1), versus 38.4% without comprehension experience (Chen et al., 2025b, arXiv v1, Table 2). Only the first version supports this cross-repository claim, because the second permits earlier experience from the same repository.

Diagnostic experience can change which component the agent investigates. In one issue reported for SWE-Exp, converting an enumeration value to a string does not yield its stored value. Without comprehension experience, the agent patches model-field preparation code; with it, the agent turns to the enumeration’s own string conversion and writes a patch for that component instead of copying one from an earlier issue (Chen et al., 2025b).

Environment setup is a second software subtask in which retained experience helps, though not uniformly across languages. EnvPilot (Chen et al., 2026a) distills setup trajectories into problem–solution–action entries and uses a repository profile to retrieve relevant setup guidance; in its example, an entry links a missing Rust runtime library to an installation command. EnvPilot counts setup as successful when the repository’s tests run to completion, whether or not they pass. On AES-Bench’s 112 repositories across nine languages, the DeepSeek-V3-0324 ablation reports 75.00% setup success with full memory versus 62.50% without it, with memory built from repositories disjoint from AES-Bench and frozen during evaluation. The full method uses fewer calls but more tokens in this comparison, and its C/C++ results show that retrieved guidance can also hurt.

8.3 Computer Use

Computer-use agents perform browser and office operations through websites and graphical interfaces, completing tasks or producing artifacts such as documents and spreadsheets. Operations on the same sites and applications repeat often, which creates opportunities to retain a workflow or callable skill, and interface state lets each step be checked. Execution nevertheless depends on the current interface and the intended output. After showing how practiced operations become reusable procedures, the subsection asks how those procedures cope with changing interface state and with the artifact and role a task requires.

8.3.1 Reusing Practiced Operations

Practiced operations can be retained either as workflows that guide each next action or as code that executes several steps at once. Agent Workflow Memory (AWM) induces workflows from demonstrations or online trajectories; in its main in-context setting, workflow descriptions and steps guide action selection from current observations (Wang et al., 2025e). SkillWeaver packages successful website interactions into Python functions and tests and refines them through further execution (Zheng et al., 2025). Procedures authored by people from their own task practice also count as experience: CUA-Skill encodes human computer-use knowledge for common Windows applications as parameterized skills, each with an execution graph that combines interface actions with steps run directly (Chen et al., 2026f). These approaches differ in how much of an action sequence is fixed before the next model decision.

Online procedure acquisition can raise task success; however, it incurs construction costs within the task stream. With Claude 3.5 Sonnet on WebArena, ASI’s programmatic skills that pass execution checks reach 40.4% success, against 36.3% for AWM’s textual workflows on the same backbone and 32.7% without induced procedures, and solve tasks in fewer steps (Wang et al., 2025d). Both methods update along the evaluated stream; the budget comparison in Section 7.4.2 shows why acting steps alone can understate total cost.

Online acquisition can also keep lessons and procedures side by side. Mobile-Agent-E (Wang et al., 2025a) maintains Tips, natural-language lessons about operating apps, and Shortcuts, sequences of atomic operations whose stated preconditions the agent must verify before calling them; two reflector models update both after each task. Its Mobile-Eval-E benchmark runs 25 multi-app tasks on real devices and scores each by the share of task-specific rubric items that a human evaluator judges fulfilled. Letting the five tasks of each scenario share and update this memory raises that score from 75.1% to 86.9% with GPT-4o and from 75.5% to 83.0% with Claude-3.5-Sonnet, but only from 70.9% to 71.2% with Gemini-1.5-Pro. The reflectors also read the queries of the scenario’s remaining tasks when they update memory, so the evaluated tasks partly shape the experience used on them.

Skill libraries move acquisition cost before testing, into agent exploration or human engineering. SkillWeaver’s GPT-4o agent achieves 29.8% WebArena task success with synthesized skills versus 22.6% with basic browsing actions, and skills synthesized by a strong agent also help weaker executor models (Zheng et al., 2025). The skills are synthesized in an exploration stage before testing, which entails 160 exploration iterations per website, each attempting or testing a skill, so this upfront work belongs in any efficiency comparison. Authored skills replace that exploration with human engineering. On WindowsAgentArena with GPT-5 as planner, CUA-Skill’s agent succeeds on 50.3% of tasks with its skill library, compared with 34.6% for a no-skill baseline on the same backbone, and the gains are smaller with GPT-4o and Qwen3-VL-32B, again with a fixed library (Chen et al., 2026f).

Whether transferred workflows help depends on the unit of success. On Mind2Web, AWM’s workflows improve step success over MindAct across task, website, and domain splits, while full-task success stays below 5% and does not consistently exceed that baseline (Wang et al., 2025e). Reusing local operations can therefore help without reliably completing the whole workflow.

8.3.2 Coping with Interface Changes and Task Roles

A procedure that works on one interface and for one task can fail when the interface changes or when another role uses it for a different purpose. State-dependent operations can require more structure than a fixed sequence, and the open question is whether that structure still helps after the interface changes. EAM constructs a GUI logic graph from exploration and searches it with learned value estimates (Qin et al., 2026). Evaluated as an integrated planner, with a UI-TARS-2B executor, a fine-tuned Qwen2.5-3B path extractor, and one GPT-4o call to filter extracted paths, it reaches 52.6% task success on AndroidWorld, compared with 6.9% for UI-TARS-2B alone and 40.5% for the GPT-4o-based M3A agent. The graph is built before testing. Its authors also note that the graph assumes a relatively static interface and may become outdated after significant application updates.

Carried unchanged across an interface shift, retained interface experience yields small or mixed gains. MAGNET targets such shifts directly: it links visual examples to functional descriptions and retains workflows for related task intents (Sun et al., 2026b). Its offline evaluation scores step success, the share of steps whose predicted action matches the reference. On subsets with shifted templates, apps, or domains, MAGNET’s Qwen2.5-VL-32B results show that the visual–functional links add less than one point of step success across most shifts, while retained workflows range from slight losses under template and app shifts to moderate gains under domain shifts.

When the experience is instead updated from trajectories in the new environment, task success rises over successive passes. MAGNET’s online evaluation uses the same Qwen2.5-VL-32B backbone, warm-starts it with memory from another mobile dataset, and updates that memory after each pass over the same 61 AndroidWorld tasks. Task success rises from 31.14% to 40.98% over three passes, while the share of retrieved entries drawn from the original memory falls below one third (Sun et al., 2026b). Because the tasks are revisited, this gain shows adaptation to the new environment rather than transfer to new tasks.

A reusable procedure must preserve the intended artifact and professional role, not only the interface operations that produced it. AFTER is a benchmark of 382 workplace tasks across six professional roles that tests whether procedural skills, reusable instructions for operations such as processing PDF documents, transfer across tasks, roles, and backbone models (Belikova et al., 2026). With a Qwen3.5-35B-A3B solver, evolving the PDF skill on one role’s tasks raises accuracy over the skill before evolution by 11.7 points for project managers and 6.2 points for data scientists, a before-and-after comparison of a single evolution round, too short to show sustained improvement. Applying either evolved skill to the other role instead lowers accuracy by 4.8–7.5 points relative to that unevolved skill; the authors attribute the loss to the two roles using the same skill for different purposes, such as executive summaries versus data ingestion.

8.4 Recommender Systems

A recommender agent selects items that fit a user’s current request or next need, and similar requests recur while the preferences behind them vary. Whether a selection worked is known only from the user’s response, so feedback depends on users, with no check the agent can run itself; current evaluations replay recorded

responses offline. In recommender systems, experience links how candidates were selected to the feedback those selections received, which distinguishes it from a preference record stating what a user likes. Such experience can be shared across users, aimed at the recommender’s own errors, or kept for each user; after comparing the three, the subsection asks what offline replay can establish.

Experience shared across users carries lessons about how to recommend that no single user’s history reveals. CRAVE builds experience banks by reflecting on earlier movie-selection trajectories and historical user feedback, then retrieves experience to guide candidate generation and evaluation for new queries (Zhu et al., 2025). A separately trained retriever matches the current preference pattern to stored experience. In offline movie recommendation on ReDial and Reddit-v2 with GPT-4o, CRAVE places more of the movies that users accepted near the top of its recommendation lists than the same model without experience, and more than prompts augmented with retrieved movie descriptions or with global rules summarized from the training conversations. On Reddit-v2, however, replacing the fine-tuned retriever with an untuned one substantially lowers performance, so the gain reflects stored experience together with trained retrieval.

Shared experience can also take the form of reasoning guidelines. MemoCRS (Xi et al., 2024) keeps a small set of guidelines shared by all users, initialized with hand-written generic reasoning rules such as “Let’s think step by step” and revised from the LLM’s earlier recommendation reasoning and users’ written responses to its recommendations. In its GPT-4 ablation on the conversational datasets ReDial and TGRDial, guidelines revised from users’ responses rank target items higher than either no guidelines or the generic initial rules: removing them lowers rank-sensitive scores, for example NDCG@20 on ReDial from 0.1308 to 0.1164, and replacing them with the initial rules lowers NDCG@20 on both datasets. On ReDial, however, the share of target items within the top 20 is slightly higher without the guidelines.

Experience can also target the recommender’s own recurring mistakes, whoever the user is. Re2LLM (Wang et al., 2025c) has GPT-4 rank candidate items for training sessions in session-based recommendation, reflects on sessions in which the target item was missed, and turns the inferred causes into short imperative hints. A hint enters the knowledge base only if adding it to the prompt lets the model recover the missed target for its source session and if it differs from the hints already stored; a lightweight retrieval agent trained with reinforcement learning then chooses which hint, if any, to supply for each new session. With the hint base fixed at test time, Re2LLM outperforms conventional and LLM-based session recommenders on MovieLens and Amazon video-game data in a few-shot setting. Its ablations show why selection matters: hints drawn at random perform worse than no hints, and supplying all hints at once improves little on supplying none.

Experience kept for each user borders on preference memory. SAGER (Tao et al., 2026) maintains for each user a natural-language policy skill of preferences, behavioral patterns, and ranking criteria, initialized from population templates or item statistics and revised after each interaction by contrasting the item the user accepted with the candidates the agent had ranked; a short extract of the skill then breaks near-ties in later rankings. Although these skills resemble preference memory, recording mainly positive preferences and style, the evaluation isolates their role in ranking. With gpt-4o-mini on four offline datasets, skills evolved during warm-up rounds and then frozen raise top-1 hit rate over the same agent with only its preference memory by 1.9% to 14.9% in relative terms, with the largest gain on the densest dataset.

These studies evaluate by replaying recorded conversations or interaction logs, which limits what their results establish. CRAVE builds its experience banks from training conversations before testing, and Re2LLM and SAGER fix their experience during testing, so their results support fixed experience reuse on these datasets; none follows satisfaction over a long-running user relationship. MemoCRS revises its guidelines from users’ responses but does not state whether revision continues on the test conversations, so its result may already include learning from the conversations on which it is scored. Their gains also combine experience with other components, such as CRAVE’s trained retriever, Re2LLM’s learned hint selector, and MemoCRS’s collaborative knowledge from an expert recommender. Because success is judged against a recorded choice, an evaluation cannot credit a suggestion the user was never shown, and experience formed from such logs inherits the logged candidates’ blind spots.

8.5 Embodied Control

In embodied and interactive environments, a familiar instruction may conceal different objects, resources, or initial states, so the consequences of an action depend on the current state. Experience is useful when it helps the agent choose executable behavior under those conditions, and task success after each attempt provides a check, although most evidence comes from simulators and physical tests remain few. We compare code skills with environmental lessons and review the thin evidence from physical robots, then examine how stored skills contribute to replanning and what feedback is still needed within an attempt.

Embodied agents retain experience either as executable skills or as textual lessons about how the environment behaves, and the two forms have so far been tested in different ways. Voyager grows an executable skill library through exploration and feedback in Minecraft (Wang et al., 2024a). With GPT-4, its exploration without the library tends to plateau in later stages. In a new world, the learned library lets it solve four unseen tasks in fewer prompting iterations than the same agent without the library, for example 19 versus 36 for crafting a diamond pickaxe, which the agent without the library completes in only two of three trials. The library is built during earlier exploration and carried into the new world. However, the paper does not state whether skills added while solving one unseen task remain available in later trials, which would let the library grow during testing. CLIN instead revises textual lessons about causal relations in ScienceWorld, such as which objects and locations a task needs (Majumder et al., 2024). With GPT-4, a summary of lessons from ten training environments raises the average starting score on unseen environments from 48.6 to 52.7 on ScienceWorld’s 100-point scale, measured before the lessons are revised in the new environment. These studies differ in environment, metric, and task design, so they cannot say which form transfers better; each form helped in its own setting, the skill library by reducing the prompting iterations needed on unseen tasks and textual lessons by informing planning about what the environment contains.

Evidence that retained experience helps physical robots remains thin. On a legged manipulator with GPT-4o, Qu et al. (2026) summarize lessons from the robot’s own attempts into a long-term memory, together with 96 entries from simpler instructed tasks, and keep it frozen during evaluation; across 12 real-world scenarios, eight of them unseen, retrieving from this memory yields 80% single-trial success, against 22% for COME, a prior system without long-term memory. The baseline, however, is a different system, and most memory entries come from simpler instructed tasks instead of the robot’s own attempts.

The contribution of stored skills is measured most directly where they inform replanning after a failure. ViReSkill, evaluated in manipulation simulators and on a physical robot, retains successful plans and control code, reuses them on later encounters, and consults visual observations to replan after failures (Kagaya et al., 2025). In its LIBERO-10 simulation ablation, enabling skill transfer during replanning raises success from 50% to 66%, whereas the other reported LIBERO subsets show small or no differences (Kagaya et al., 2025). Because the skill memory starts empty and accumulates over ten iterations on the same LIBERO tasks, with success computed from the memory saved after each iteration, this ablation measures gains on revisited tasks rather than fixed experience reuse on new ones. Perception in ViReSkill is open-loop within an individual attempt: visual feedback reaches the agent only between attempts, so a reusable procedure does not replace the feedback needed to respond to changing state.

8.6 Scientific Research

Scientific agents choose research directions and turn them into experiments under limited resources. Their feedback distinguishes several outcomes: code that cannot run, a method that underperforms a baseline, and a completed study that receives a favorable evaluation. Experience must retain these distinctions to help later research choices. Model development for recurring machine-learning tasks is the most directly testable part of this work, so we begin there with reuse of successful modeling solutions. From there the discussion widens to experience for ideation and experimentation, to how the executor’s search procedure changes the effect of reuse, and to how failed attempts should be interpreted.

For recurring machine-learning tasks, a previous executable solution can guide both model choice and implementation. DS-Agent retains successful task descriptions and Python scripts, then retrieves a similar case and adapts its code without iterative experimentation at deployment (Guo et al., 2024). Across 18 deployment tasks covering tabular, text, and time-series problems, drawn from the same distribution as its 12

development tasks, GPT-4 produces runnable programs on the first attempt in 99% of cases with retrieval, compared with at most 60% without a case or with a random one. This comparison supports retaining task context alongside executable solutions so that retrieval can identify a relevant starting point.

Research agents need different experience for choosing a direction and for implementing it. EvoScientist separates ideation experience about promising or unsuccessful directions from experimentation experience about implementation and training (Lyu et al., 2026). With Claude-4.5-Haiku implementing proposals for 30 research queries, distilling earlier outcomes into experimentation experience raises mean code-execution success across four stages from 34.39% to 44.56%. The source of that experience relative to the evaluated queries is unspecified, limiting conclusions about transfer. The ideation comparison instead uses an LLM to judge novelty and feasibility, so the two experience stores are tested against different outcomes. Keeping those outcomes distinct helps avoid treating an implementation failure as evidence against a research direction.

The effect of experience also depends on whether an agent keeps revising one solution or explores several candidate solutions. A controlled machine-learning-engineering study examines memory that stores coding errors and their fixes. This memory supports OpenHands-style sequential refinement, in which the agent repeatedly modifies one solution. In AIDE-style tree search, where the agent explores multiple candidate solutions, the same kind of memory makes the procedure more stable but reduces the variety of solutions explored (Zhao et al., 2026d). Some of the memory is accumulated and reused within a single coding run, and retrieval is organized by task. The result therefore shows how memory interacts with the agent’s search procedure, not whether experience from one task helps with a different task.

The reliability of negative research experience depends on the rule that declared the attempt a failure. EvoScientist treats a proposal as failed when executable code cannot be found within the allotted budget (Lyu et al., 2026). That rule can end an unproductive attempt, but its implication for the direction remains conditional on implementation and resources. Omitting these conditions from negative experience could later suppress a viable direction, a risk the study does not measure.

8.7 Financial Trading and Forecasting

In trading and forecasting, the outcome of a decision arrives later, and an agent must connect it to the conditions under which the decision was made. Experience can then help the agent reconsider a strategy when similar conditions recur, provided that noisy outcomes are judged over many decisions rather than one. Trading comes first, with how its decisions are reused and what historical backtests establish; forecasting follows, where feedback arrives only after the source episode has ended.

For later review, a retained trading decision needs both the information available when it was made and its subsequent outcome. TradingGroup summarizes decisions from the preceding 20 trading days into successful and failed cases, then combines this experience with current news and financial reports (Tian et al., 2025). Its backtests on five stocks report improved return and risk metrics on four when retrieval and reflection are added to the risk-control modules. That comparison, however, adds retrieval and reflection together.

Holding experience fixed across market periods tests a different form of reuse. FinCon’s manager revises investment beliefs from January to October 2022 and then trades from October 2022 to June 2023 with those beliefs fixed (Yu et al., 2024). With GPT-4-Turbo on GOOG, median cumulative return over five runs rises from −11.9% without belief updates to 25.1% with them. This is a historical backtest, not prospective trading evidence; a retained strategy still needs its market conditions checked before reuse.

Forecasting makes delayed feedback explicit, because a prediction can be scored only after its outcome resolves. Live-Evo separates an Experience Bank from a Meta-Guideline Bank, compiles selected experience into guidance, and adjusts experience weights using subsequent feedback (Zhang et al., 2026m). Its principal evaluation follows ten weekly batches of new Prophet Arena questions, with experience updated online between batches. With GPT-4.1-mini, Live-Evo lowers the average Brier score, the squared error of predicted probabilities where lower is better, from 0.19 for the same search agent without experience to 0.14, and fixing all experience weights raises it to 0.17. The authors note that the framework relies on dense feedback and may apply less well where feedback is sparse or subjective.

8.8 Comparing Application Conditions

The applications above report gains from experience under some conditions and losses under others, and they differ in how widely the experience can be reused and in how firmly its benefit is established. This subsection asks which application conditions explain these differences, so that it is clear where experience is likely to pay off and what a reported gain shows. We compare the applications on four conditions, summarized in Table 20: how often tasks or subproblems recur, which environmental changes can invalidate earlier guidance, how outcomes can be checked, and when feedback becomes available. Recurrence and environmental change determine how widely experience can be reused, whereas outcome checking and feedback timing determine how firmly a gain can be established. In short, experience is most useful where tasks recur in a stable environment, and its benefit is best established where the outcome that matters can be checked directly. The closing paragraphs distinguish the improvement claims supported by these evaluations, and Section 8.9 relates the comparison to the design decisions.

Recurring tasks make experience worth retaining, but only while the conditions under which it was formed continue to hold. Recurrence is most direct in computer use, where operations repeat on the same sites and applications, and in recurring modeling tasks; in software engineering and embodied control, a familiar symptom or instruction can hide a different cause or initial state. The clearest reported losses occur when experience crosses a change in these conditions: AFTER’s evolved PDF skill falls below the unevolved skill when moved to another professional role, MAGNET’s retained workflows lose slightly under template and app shifts, and EnvPilot’s retrieved setup guidance lowers success on C/C++ repositories (Belikova et al., 2026; Sun et al., 2026b; Chen et al., 2026a). Frequent recurrence therefore justifies retaining experience only if its applicability conditions are recorded and rechecked before reuse.

How firmly a gain is established depends first on whether the outcome that matters can be checked. Question answering, software engineering, and computer use check outcomes against reference answers, tests, or interface state, and they provide the clearest controlled comparisons, in which experience built before testing is compared with the same agent without it, as in ExpeL, SWE-Exp, EnvPilot, and SkillWeaver (Zhao et al., 2024a; Chen et al., 2025b, 2026a; Zheng et al., 2025). Such checks confirm the current outcome; whether the diagnosis behind a passing repair will apply to the next repository needs a separate test. Embodied control can also check task success, but mostly in simulators. Where the outcome depends on users, markets, or research judgment, evaluations check a stand-in for it. The reviewed recommender and trading systems replay recorded user responses or historical prices, so they leave untested how a user would react to an item never shown or how the market would respond to the agent’s own trades (Zhu et al., 2025; Wang et al., 2025c; Tian et al., 2025). In scientific research, whether code runs can be checked, but a failed run may reflect its budget, and no test settles whether a direction has research value (Lyu et al., 2026).

An outcome that arrives later can still provide an independent reference for evaluating a decision. Live-Evo scores forecasts once their questions resolve and compares its agent with the same agent without experience over ten weekly batches (Zhang et al., 2026m). Trading backtests instead replay historical prices; their limitation is that they do not test decisions prospectively. Neither kind of study varies how long feedback takes to arrive, so the effect of the delay itself remains unmeasured.

Frozen-repository comparisons in question answering, software engineering, and computer use support fixed experience reuse under their tested conditions. Buffer of Thoughts and Memento leave the repository’s status during the cited comparisons unclear (Yang et al., 2024; Zhou et al., 2025). Dynamic Cheatsheet, ASI, Mobile-Agent-E, MERIT, and MAGNET report aggregate scores or gains on revisited tasks; MERIT also leaves initialization and write order unspecified (Suzgun et al., 2026; Wang et al., 2025d,a, 2026g; Sun et al., 2026b). Two application studies offer more direct longitudinal evidence. Live-Evo compares weekly performance with a no-experience control over ten weeks, but its fixed-weight ablation still permits experience additions. Subtask-level memory starts empty and reports windowed gains over a no-memory baseline in a representative shuffled software-repair run, including an early negative window followed by later gains (Zhang et al., 2026m; Shen et al., 2026a). Both leave task-level regressions and measured resource costs unreported; the latter’s plotted longitudinal result also needs uncertainty across runs (Section 7.5).

Attribution also matters when choosing which mechanism to adopt. CRAVE’s fine-tuned retriever, Re2LLM’s learned hint selector, and ExpSeek’s larger guidance model contribute alongside retained experience (Zhu et al., 2025; Wang et al., 2025c; Zhang et al., 2026k). MERIT’s comparison with BM25 shows how to distinguish

Table 20 Representative applications compared on the four reuse conditions. Condition entries summarize the reported evaluation settings and the analysis in each subsection.

Application	Retained experience	Representative work	Recurrence	Environmental change	Outcome checking	Feedback timing
Question answering	Solution templates, search lessons, tool-use procedures	ExpeL (Zhao et al., 2024a) Memento (Zhou et al., 2025) Dynamic Cheatsheet (Suzgun et al., 2026) MERIT (Wang et al., 2026g)	Reasoning and search difficulties recur across questions	Facts, sources, and databases differ across questions	Reference answers; report quality is harder to check	Per question, from answers or query results
Software engineering	Diagnostic precedents, modifications, setup strategies	SWE-Exp (Chen et al., 2025b) SWE-ContextBench (Zhu et al., 2026b) EnvPilot (Chen et al., 2026a)	Similar symptoms recur, often with different causes	Versions, dependencies, and languages	Executable tests; they do not validate transfer of a diagnosis	Per attempt, from tests or setup runs
Computer use	Workflows, callable and authored skills, interaction corrections	AWM (Wang et al., 2025e) SkillWeaver (Zheng et al., 2025) MAGNET (Sun et al., 2026b) CUA-Skill (Chen et al., 2026f)	Repeated operations on the same sites and applications	Interface updates and professional roles	State and artifact checks; clicks alone are insufficient	Per step, from interface state
Recommender systems	Shared selection lessons and guidelines, error-correcting hints, per-user decision principles	CRAVE (Zhu et al., 2025) MemoCRS (Xi et al., 2024) Re2LLM (Wang et al., 2025c) SAGER (Tao et al., 2026)	Recurring requests with varying preferences	Preferences drift and differ across users	Offline, against recorded user choices	Per recommendation, from the user’s response
Embodied control	Environmental lessons, plans, executable skills	Voyager (Wang et al., 2024a) CLIN (Majumder et al., 2024) ViReSkill (Kagaya et al., 2025) Pragmatist Robot (Qu et al., 2026)	Familiar instructions under different states	Objects, resources, and initial states	Simulator success; physical evidence is limited	After each attempt, from task outcome
Scientific research	Successful solutions, direction experience, experimental strategies	DS-Agent (Guo et al., 2024) EvoScientist (Lyu et al., 2026)	Modeling tasks recur; research directions vary	Task distributions and executor search procedures	Run status is checkable but budget-dependent; research value is not	Delayed and graded: run, baseline, evaluation
Financial trading and forecasting	Prior decisions, outcomes, strategy lessons	TradingGroup (Tian et al., 2025) FinCon (Yu et al., 2024) Live-Evo (Zhang et al., 2026m)	Similar market or question conditions	Market conditions shift	Backtests or resolved forecasts, not prospective returns	Final outcomes delayed; applicability changes can be checked earlier

gains from stored content and learned retrieval (Section 8.1). The practical question is which retained content and processing mechanism justify their cost in the intended application; a system-level gain alone does not

settle that choice.

8.9 Design Decisions across Applications

A practitioner adding experience to an agent faces the four decisions named at the start of this section: whether to invest in experience, what to retain, how strongly to use it, and when to revise it. The application evidence and the reuse conditions in Table 20 bear on each of them; Sections 5 and 6 describe the mechanisms involved, and Section 7 the comparisons needed to test their effects.

Whether to invest depends on expected reuse. Experience is worth building only when its expected reuse repays the cost of acquiring, checking, and maintaining it, so the first step is to estimate how many suitable future tasks there will be and which benefit matters: answer correctness, repair success, operation time, or items users accept. SkillWeaver’s website exploration and EAM’s graph construction illustrate acquisition costs, MemGovern’s additional tokens per issue illustrate the cost of use, and Memento’s case-count results show that more retrieval need not buy better answers (Zheng et al., 2025; Qin et al., 2026; Wang et al., 2026e; Zhou et al., 2025). The full cost accounting is specified in Section 7.4.2. Repeated operations on a stable website can justify tested functions, whereas a rarely recurring question type may justify only a lightweight precedent. When tasks repeat but interfaces change rapidly, adaptable workflows and checks at the changing interaction points matter more, and frequent reuse pays off only if its savings also cover repeated revalidation.

What to retain depends on which conditions can be recorded and checked with it, and these differ by application. Software diagnosis needs repair-stage and dependency context; workplace procedures need the intended artifact and role (Shen et al., 2026a; Belikova et al., 2026). Experience in recommender agents also needs the preference and feedback conditions under which earlier choices were evaluated, and whether it describes one user or the recommender’s behavior across users (Zhu et al., 2025; Tao et al., 2026). Negative scientific or robot experience should retain the failing stage, resources, and state (Lyu et al., 2026; Kagaya et al., 2025); an unfinished experiment under a limited budget supports at most a qualified warning about the direction. These conditions also inform the choice among the three forms (Sections 2.2.2 and 3.5); few studies compare forms built from the same trajectories, and no form wins across the tested benchmarks.

How strongly experience should influence the current task depends on how well its conditions can be checked. Experience helps most when it enters at the decision it can inform, which the selection mechanisms of Section 5.2 help identify: a repair agent may need a localization precedent, a question-answering agent a search alternative, and a recommender a hint about its recurring ranking errors (Shen et al., 2026a; Zhao et al., 2024a; Wang et al., 2025c). Uncertain experience is best introduced as a candidate explanation, planning option, or checklist. A tested procedure can be allowed to execute when its inputs, starting state, and required output can be checked, and operations spanning changing states need checks between the affected steps. Current observations still have to settle the present request, since an earlier answer, preference match, or accepted item cannot, and the agent should be able to reject retrieved experience whose conditions do not hold.

Finally, review should be triggered by observed changes, not by age: an interface update, new output requirements, a role switch, or worsening outcomes on comparable tasks. AFTER’s cross-role losses and MAGNET’s rising success after revising its memory in a new environment (Section 8.3) both follow such changes, which age alone would not detect (Belikova et al., 2026; Sun et al., 2026b). The maintenance operations of Section 6 then determine whether to narrow applicability, refine the guidance, or retire the experience from active use. For delayed outcomes, as in forecasting, revisions justified by the final result must wait until it resolves (Zhang et al., 2026m). Earlier revision can still follow independently verified changes, such as a failed assumption, a withdrawn source, or a changed tool; such evidence concerns current applicability, not the unresolved outcome.

Section 9 develops the remaining questions about exploration, transfer, composition, and the correction of errors across later learning.

9 Challenges and Future Directions

An experience-driven agent should learn from earlier tasks, use what it has learned in new situations, and correct mistaken experience. Doing this reliably over many tasks remains difficult: familiar strategies can limit exploration, and errors in stored experience can spread into later experience. Building on the mechanisms, evaluations, and applications reviewed above, this section examines eight open problems: how agents acquire, extend, and safely revise experience, and how to show that the resulting improvement accumulates. Each problem starts from a limitation that the reviewed studies report or leave untested.

We organize these problems into three groups. The first, acquiring and extending experience, asks how agents can learn from human records and their own exploration, transfer experience across domains, and combine known procedures for new tasks. The second, keeping experience useful, revisable, and safe, asks how experience can remain useful as executors learn or share it, how propagated errors can be corrected, and how malicious interactions can be prevented from corrupting it. The third, evaluating cumulative progress, asks how to test whether agents continue to improve as they gain experience. Table 21 summarizes, for each problem, why it remains open, the route developed here, and what would count as progress. The section closes by explaining how the problems depend on one another (Section 9.9).

9.1 Forming Experience from Human Expert Records

Human experts leave records of their task trajectories that agents could learn from, such as issue threads with merged patches, incident tickets, operation logs, and recorded demonstrations, and these records can hold strategies that an agent would need many attempts to find. Because the records were produced by a human expert, with tools and under conditions that the records rarely state, it is unclear which parts of a human trajectory remain valid guidance for an agent executor. Converting such records into experience is already feasible: offline AWM induces workflows from annotated demonstrations (Wang et al., 2025e), and MemGovern converts GitHub repair records into searchable experience cards (Wang et al., 2026e). The harder step is deciding what the conversion should keep, mark as uncertain, or discard.

Human records differ from an agent’s own trajectories in three ways. They usually keep the final path and omit rejected alternatives, failed attempts, and the reasons for a choice, which is the evidence that failure, comparative, and corrective experience need (Section 2.2.4). The expert acted with perception, permissions, tools, and domain knowledge that the agent may lack, so a step that worked for the expert may be unavailable or unsafe for the agent. Moreover, success is often implied by curation, such as a merged patch or a closed ticket, rather than established by a check the agent can rerun (Section 3.1).

A closed incident ticket that records restarting an unresponsive service shows what such records leave out. The ticket does not record that the operator first reproduced the outage and ruled out a full disk. Monitoring then confirmed the recovery, but under conditions the agent cannot inspect. A lesson stating *restart the service* keeps the action but drops the diagnosis that justified it and the system state it assumed. Where the disk is in fact full, following the lesson repairs nothing and can hide the real cause.

Experience formation can separate what a human record shows from what the agent infers. In the restart example, the unrecorded diagnosis that ruled out a full disk remains a hypothesis about the operator’s reasoning, and sufficient free disk space becomes an unverified applicability condition for reusing the fix. More generally, inferred reasons and rejected alternatives can be kept as hypotheses that explain the guidance, whereas the conditions on which reuse depends must be checked before the guidance is applied.

The agent can test those conditions by rerunning the recorded procedure in its own environment, starting with steps where the expert’s tools or observations differ from its own. ICAL, for example, executes rewritten demonstrations and collects human feedback on the actions that fail (Sarch et al., 2024). Where re-execution is impossible, asking the expert only about decision points on which inferred explanations disagree may cost less human attention than annotating every step.

The same separation applies when no trajectory was recorded at all. Expert knowledge collected apart from any recorded trajectory, such as the interviews that inform part of OpsMem’s initial memory (Sun et al., 2026c), counts as experience that experts formed from their own practice (Section 2.1). Its execution evidence, however, stays implicit. Its reliability therefore remains provisional until the claim at issue is checked against

Table 21 The eight open problems of this section, with the route each subsection develops and the evidence that would count as progress. Routes and evaluation designs are our proposals, derived from the mechanisms reviewed above; any resulting improvement claim also needs the controls in Table 18.

Open problem	Why it remains open	Route developed here	What would count as progress
Acquiring and extending experience			
Forming experience from human expert records (§9.1)	Records omit rejected alternatives, reasons, and environment assumptions, and reflect tools and permissions the agent may lack.	Record inferred assumptions as unverified conditions and inferred rationales as hypotheses, then check them by re-execution or targeted expert questions.	Human records reduce the interactions needed for a given success rate, under a budget that counts human effort, without importing assumptions the agent cannot meet.
Preserving informative exploration (§9.2)	Successes under a dominant strategy cannot show whether an alternative would also have worked, and an abandoned strategy may never be retried.	Keep competing explanations and run small test interactions on which they predict different outcomes, starting where a wrong lesson costs most.	The agent notices when a once-effective lesson has become too restrictive, at lower interaction cost than exploring only after failures or on a schedule.
Transferring principles across domains (§9.3)	The base model may already state the general principle; knowing when it applies and when it does not is the hard part.	Identify the condition that drove the source decision, look for matching evidence in the target domain, and withhold the principle when none is found.	On held-out domains, the agent decides when to apply or withhold the principle more accurately than with domain-specific lessons or the base model alone.
Composing experience for unseen tasks (§9.4)	Two procedures can each succeed alone and fail together, because a stored link cannot show that one produces what the next requires.	Record what each procedure requires and produces, and test first the connections that many workflows depend on.	The agent solves unseen combinations of known procedures and detects incompatible ones with limited checking.
Keeping experience useful, revisable, and safe			
Co-evolving experience representations and executors (§9.5)	As the executor learns, and across executors that share a repository, some guidance becomes redundant and some must change form; a gain in one round does not show sustained improvement.	Build several representations from the same source evidence; compare them with the executor held fixed, then update the executor and compare again.	The executor drops support it no longer needs while still handling exceptions and learning from new experience.
Tracing and correcting propagated errors (§9.6)	A flawed premise can pass into later experience without its wording or a recorded link, and privacy limits how much history can be kept.	Record which trajectories support each unit and which guidance was available or followed in each trajectory; test suspect units where the premise holds and where it fails.	The agent removes the influence of an inherited flawed premise despite incomplete records, without discarding behavior that has independent support.
Learning under adversarial interaction (§9.7)	An adversary can craft experience that passes checks on each unit, spread harm across units, or delay its effect.	Record who could influence each trajectory, isolate experience formed under untrusted influence, and check the guidance to be supplied as a whole.	Defenses limit adaptive attacks under stated access assumptions while losing little learning on benign tasks.
Evaluating cumulative progress			
Establishing cumulative capability gains (§9.8)	Rankings of learning strategies can reverse when later tasks reward different capabilities, so one task stream cannot settle them.	Run strategies on a shared early stream, then test their resulting states on several different future streams.	Gains hold across successive snapshots of the agent, reported with regressions and total cost and with final tests kept out of update decisions.

agent trajectories, inspectable records of other agents’ or humans’ work, or targeted expert clarification. Recording the source of each unit also lets maintenance compare human-derived and agent-derived experience when they conflict.

The quantity to measure is how many interactions human records save in reaching a given success rate on a fixed set of expert-resolved tasks. The comparison covers experience formed from raw records, from records with inferred rationale, and from records checked by agent re-execution, as well as experience the agent forms from its own attempts. All arms receive a matched budget that counts human effort, such as corrections and answered questions, together with agent interactions. In addition, test tasks should include look-alikes in

which an assumption of the expert’s environment fails, because a saving counts only if the agent withholds the guidance there instead of importing conditions it cannot meet or verify. With experience frozen during evaluation, this comparison tests fixed experience reuse.

9.2 Preserving Informative Exploration during Reuse

When a lesson keeps working, the agent has little reason to test where it stops working, yet it must decide when such a test is worth extra interaction. The decision matters because experience changes both how an agent solves a task and what it has an opportunity to learn: once a useful strategy dominates action selection, the agent may stop producing the trajectories that would reveal its limitations or support alternatives.

Two reviewed studies suggest this risk; however, neither directly measures how cross-episode reuse changes exploration. In machine-learning engineering, adding debugging memory to tree search made the search more stable but less diverse; because part of that reuse stays within one coding run and retrieval is indexed by task, the result says little about cross-task transfer (Section 8.6). In a controlled ALFWorld reuse study, raw-trajectory memory acquired on other tasks solved sharply fewer of the tasks that a no-memory agent failed than memory acquired on the target tasks; insight memory largely removed this effect (Section 7.5.2). The result concerns lost new learning, which is narrower than exploration, and comes from one transfer direction with two runs per condition.

Deliberate exploration could target uncertainty about retained experience even when reuse currently succeeds. Voyager (Wang et al., 2024a), SkillWeaver (Zheng et al., 2025), and the memory-aware test-time scaling of ReasoningBank (Ouyang et al., 2026) already illustrate exploration or evidence enrichment (Section 3.1). The further question is how to choose new interactions that reveal the limits of currently successful guidance.

A web-shopping agent shows how reuse can cut off the evidence that would test a working lesson. Its lesson *use the site search instead of the category menus* has worked on several tasks and now sends it to the search box whenever it looks for a product. Later successes show that searching can work in the conditions encountered, but provide little evidence that it was necessary. Conversely, if an earlier attempt to browse the category menus failed, the agent may stop trying them, and so never learns whether that attempt failed because of a fixable mistake, such as opening the wrong menu, or because the site offers no suitable category.

An agent could preserve competing explanations and choose *probes*, small test interactions for which the competing explanations predict different outcomes. Repeating familiar tasks adds little when both explanations predict success. More informative probes could vary a suspected prerequisite, omit a supposedly necessary step, or retry a previously failed strategy after fixing the implementation defect that may have caused its failure. A controller could prioritize these probes by the consequence of an incorrect lesson, disagreement among supported alternatives, and the expected relevance of the answer to future tasks. Such probes target uncertainty about the guidance used to complete a task, not uncertainty about completing the task itself.

Execution uncertainty alone cannot show whether a retained lesson is necessary. ExpSeek’s entropy is only a modest indicator of step correctness (Section 8.1); even a reliable signal about step correctness would leave the lesson’s necessity untested. Where the environment can be restored, rerunning the same state with and without the disputed step provides a direct comparison. In the shopping example, successfully completing the task through the category menus after restoring the starting page would show that searching was unnecessary in that case. Reflecting again on the unchanged trajectory cannot supply this observation of the alternative.

The test streams should be ones in which an initially effective lesson later becomes unnecessarily restrictive, since that is the change probing is meant to detect. Under the same interaction budget, controllers that select probes from uncertainty about retained experience are compared with controllers that explore only after task failure or on a fixed schedule. A copy whose experience is frozen before the lesson becomes restrictive provides the reference for gains from continued updates. Relevant outcomes include discovery of valid alternatives, preservation of task success, and the interactions spent before the question is resolved. The main outcome is whether a controller narrows the lesson’s applicability conditions once the lesson has become limiting, before repeated reuse cuts off the evidence that would reveal the limit.

9.3 Transferring Principles across Domains

Experience formed in one domain does not always carry into another. When the base model already knows the general principle involved, source trajectories may still help it recognize the principle’s applicability conditions, exceptions, and appropriate timing in the new domain. ExpeL (Zhao et al., 2024a) and Agent KB (Tang et al., 2025) motivate studying how trajectory-derived experience improves these decisions across tasks and domains. Reviewed transfer results include both gains and losses. When coding experience is formed on benchmarks other than the target, gains come mainly from high-level guidance such as validation routines, whereas domain-mismatched precedents and misapplied best practices can cause failures on tasks the agent solves without memory (Section 7.4.3). Across roles, a PDF-processing skill evolved for one professional role lowers accuracy when moved to another role that uses it for a different purpose (Section 8.3).

The principle *check whether evaluation prerequisites were satisfied before rejecting a solution* shows what can carry over from software debugging to an experimental workflow. In the source trajectory, a functional test reported failure only because its test database never started. In the target domain, the corresponding observation might be a positive control that also returns a negative result, showing that the assay, not the tested hypothesis, failed. The two observations share no tools or entities, yet both show that the evaluation itself did not run correctly, so its negative feedback cannot yet be trusted. Once independent checks confirm that the evaluation ran correctly, however, further questioning only delays accepting a genuine failure. What transfers is therefore the maxim together with its stopping rule: look for evidence that the evaluation failed, and accept the negative feedback once its prerequisites are confirmed. Such a principle remains semantic-level experience (Section 2.2.2), and its broader wording still has to earn broader applicability through checks in the new domain.

Transferring a principle involves three connected tasks: identify the condition that changed the source decision, locate corresponding evidence in the target domain, and decide whether that evidence supports invoking the principle. The second task is the hardest, because it requires establishing that two observations play the same decision-relevant role despite different tools and entities, as the failed functional test and the failed positive control do. EMG (Wang et al., 2026f) matches action decision graphs, which represent the action structure of successful trajectories, and so offers a mechanism for recovering shared decision patterns and corrective edits; matching across domains with different action vocabularies remains a further research question. Candidate conditions could be tested against contrasting trajectories in which a prerequisite holds, fails, or remains unknown, and the retained experience should connect each condition to concrete observations and counterexamples. If no target observation can distinguish those cases, the agent should seek an informative check or withhold the principle.

An evaluation of such transfer holds out entire domains and pairs structurally equivalent tasks with superficially similar tasks that violate a key premise. Principles whose applicability conditions were learned from source trajectories are compared with concrete source experience, domain-specific lessons, and principles that the same base model generates without source trajectories, under matched context and interaction budgets. The last comparison controls for what the base model already knows: conditions learned from source trajectories add value only if they improve the agent’s decisions to invoke or withhold a principle that the base model can already state correctly. Transfer with experience frozen should be reported separately from adaptation using limited target-domain feedback.

9.4 Composing Experience for Unseen Tasks

Several procedures, each reliable under its own conditions, can fail when chained. An experience collection supports composition when an agent can combine supported experience to solve a task whose complete procedure it has never observed. Whether two procedures combine depends on their *interfaces*: the starting conditions each requires and the effects it produces. Organization links related experience, and maintenance can revise linked experience together, as in Auto-Dreamer’s region-level consolidation (Ye et al., 2026b) and CoEvo-Mem’s graph updates (Ye et al., 2026a) (Sections 4 and 6); these operations change what is stored and how it is linked, and testing whether linked procedures fit together is a separate step. Contract checks come closer, testing recorded preconditions and whether one skill’s output matches the next skill’s input (Section 4.4), but they can test only the interface assumptions that have been recorded. The remaining challenge is to discover the assumptions that isolated successful trajectories never test, and to verify them in

new combinations under a limited budget.

For example, a formatting skill may reflow a source file to the project’s style and shift its line numbers, while a patching skill locates its edit by the line that a traceback recorded before formatting. Both can succeed in isolation and fail when combined, because the patch then changes the wrong line. Each skill’s successful trajectories record only the conditions they met, so neither reveals that the patch depends on line numbers from before formatting. The agent must discover this mismatch without such evidence and then determine whether a mapping step, such as rerunning the tests to refresh the traceback, can connect the procedures.

Each procedure could carry an explicit interface, namely its observed preconditions, effects, resource assumptions, and unresolved conditions. Contrasting runs and checks that vary one input or state property can help turn a plausible interface description into supported experience. Exhaustively testing every pair or subset is infeasible, so a controller should first verify connections that several candidate workflows depend on, transitions where the procedures disagree about a shared resource, and transitions whose failure would invalidate many later steps. For the line-number example, checking whether the recorded line still holds the failing statement after formatting may resolve compatibility more cheaply than running the full repair and test suite. Applicability conditions can play the same role for semantic-level experience, indicating which lessons can be applied together, although their compatibility often needs empirical checks. Composition evidence should also inform maintenance: a rarely used recovery procedure may be worth retaining because it makes several otherwise fragile workflows usable.

Testing composition means withholding complete workflows while their constituent procedures can still be acquired, then introducing controlled interface mismatches. With the experience collection frozen, the comparison covers independent retrieval, composition using inferred interfaces, and composition with targeted interface verification under equal interaction budgets, and it reports success on unseen combinations and detection of incompatible ones together with the checking effort each requires.

9.5 Co-evolving Experience Representations and Executors

Experience must stay useful while the executor that applies it is itself learning. Explicit experience can be used to train the executor, and a better-trained executor in turn needs different external help: part of the guidance becomes redundant or must be expressed differently, while part must stay accessible so that later errors can be corrected. By *representation* we mean the form and level of detail in which retained experience presents its guidance. It spans the three representational forms of Section 2.2.2, from a concrete precedent to a lesson or a procedure, as well as differences in detail within one form, such as a full diagnostic procedure versus a short exception check; it does not mean the format of the carrier that stores the experience. The Experience Compression Spectrum (Zhang et al., 2026l) motivates representation selection, while Sections 3.5 and 5.5 discuss coexisting representations and internalization. The difficulty is how to divide guidance between the executor’s parameters and explicit experience so that, after familiar behavior has entered the parameters, the executor can still handle rare exceptions and learn from new guidance. The division requires deciding which behavior to train into the executor, which conditions to retain explicitly, and when to return to concrete precedents.

A routine diagnostic procedure, for example, may become unnecessary after training, while an environment-specific exception remains essential. Removing the whole procedure would lose that exception, but retaining every step would keep paying access costs for behavior the executor has already learned. Choosing between them is difficult because, when a piece of guidance stops improving results, the cause is ambiguous: the executor may have learned the behavior, current tasks may rarely need it, or the guidance may no longer suit the updated executor. Those explanations imply different retention and reconstruction decisions.

Existing methods already couple experience updates with policy learning, leaving open whether the updated executor can still learn from new experience. EDGE (Xie et al., 2026a) combines experience-bank updates with internalization, and ReSkill (He et al., 2026c) compares conditional versions of a skill during policy optimization, so that the selected version keeps pace with the changing executor. Because ReSkill still triggers skills after training, its results support updating skills and policy together, not removing external guidance altogether. A three-round web-reasoning study separates performance after internalization from the additional benefit of supplying the updated experience pool (Chen et al., 2026c). Its comparisons of experience form,

injection position, and distillation setting show that these choices affect performance in later rounds, so claims of sustained improvement must be judged across rounds.

Other evidence bears on what should stay explicit once guidance has been distilled, and it suggests keeping concrete evidence alongside the selected abstractions. SkillEvolBench (Lei et al., 2026) reports settings in which raw trajectories outperform distilled skills under deployment shifts, and WikiSkill (Tang et al., 2026) separates trajectory, knowledge, and skill resources. These observations motivate retaining source evidence, though they do not show which evidence an updated executor will later need.

Executors that share a repository raise the same question across recipients: executors with different roles or capabilities may need different guidance from the same experience. Section 5.2.3 reviews two relevant results: reusing another backbone model’s memory sometimes lowered accuracy, and a more accurate teacher did not produce more helpful memory for its student (Chang et al., 2026; Kim et al., 2026b). They motivate comparing allocation policies across recipients, one of the comparisons that Section 5.6 lists as missing.

Several linked representations can be built from the same source evidence and compared in two alternating steps: compare how much each helps while the executor is held fixed, then update the executor and repeat the comparison. In the first step, each representation is tested on routine cases and relevant exceptions, and the results decide which guidance to use for training and which experience to retain explicitly. After the executor is updated, the same checks are repeated, and new failures are used to revise the external guidance. In the diagnostic example, a short exception check may replace the routine procedure, while the source trajectories remain available if a tool change demands more detail. Reconstructing a detailed precedent requires retained source evidence; otherwise, the missing evidence must be acquired again.

A two-by-two design separates the contributions of executor and representation and their interaction: old and new executors are crossed with old and new representations, built from common source evidence with compatible interfaces. Withholding external guidance tests remaining dependence on it; changing applicability conditions tests whether internalized behavior still responds to exceptions. Across successive rounds, the comparison is with a copy that keeps the starting executor and repository fixed. After each update, newly acquired experience is also supplied to test whether the executor can use it to correct previously successful but now invalid behavior. Training effects are attributed through the matched-budget comparisons in Section 7.4.4, counting training, representation conversion, and re-evaluation costs. With executors of different roles or capabilities in place of executor versions, the same crossing would compare how guidance from a shared repository is allocated across recipients.

9.6 Tracing and Correcting Propagated Errors

Keeping experience revisable becomes harder once its guidance has shaped trajectories from which further experience is formed. Versioning, revocation, and runtime recovery in Sections 6 and 6.6 address important parts of this problem. They leave open how to find which later experience inherited a flawed premise, especially when the record of how that experience was formed is incomplete. Such a premise may survive without its original wording or a direct derivation link. Effective correction must locate that influence while preserving experience that has independent support.

Suppose the lesson *restart the service* from Section 9.1 carries a flawed premise: that an unresponsive service signals a transient fault. In one trajectory the outage actually comes from a full disk, but the check used to judge the outcome only confirms that the service came back up, so the trajectory appears successful and a recovery skill (reproduce the outage, then restart the service) is formed from it. Retiring the lesson leaves the skill’s inherited premise available for reuse. Correction must therefore also find the premise’s *descendants*: the conclusions the agent later learned from trajectories that the premise influenced, such as this skill.

Dependency tracing can be extended from runtime recovery to experience formation by keeping two kinds of records. *Derivation links* identify the trajectories supporting an experience unit, and *exposure records* identify guidance available or followed during a trajectory; exposure nominates candidates for investigation, which further checks must confirm. Dependency-guided rollback (Yu et al., 2026a) already traces memory-to-action dependencies for runtime recovery (Section 6.6); once a premise is found to be flawed, the same tracing must follow derivation links and exposure records to the premise’s descendants. Where links are missing, similar decisions and shared assumptions can guide additional checks, but cannot substitute for independent support.

Privacy limits how complete these records can be, because retained history can itself leak what it contains (Section 6.6). The records should therefore keep only what selective repair needs, such as which units a trajectory drew on and which units it supported, and omit private trajectory content. It is not yet known how little such information still suffices for repair.

Locating a descendant does not settle whether it is wrong: a valid observation may support a mistaken explanation, and a descendant exposed to a flawed premise may still have independent support. OEP (Wang et al., 2026d) shows that locally plausible experience can induce harmful generalization, so plausibility alone cannot clear a descendant either. Targeted tests should run suspect experience on tasks where the premise holds and where it fails. For a procedure, the tests should record intermediate results as well as task success; when a step’s contribution remains unclear, omitting or replacing it in matched runs can help determine what to keep or change.

For the recovery skill, the two kinds of test task would have a genuine transient fault or a full disk. The tests should check whether reproduction exposes each fault, then check that the service stays up after the restart. If reproduction exposes both faults but restarting resolves only transient ones, those observations support retaining reproduction and making the restart conditional on a confirmed transient fault. This repairs the unsupported step without discarding the whole skill. With a limited repair budget, the agent could prioritize descendants whose support is most uncertain and whose guidance affects the most later units.

Update-robustness evaluation (Section 7.5.3) becomes a test of correction when an early premise is invalidated after several rounds of reuse and derivation. The retained history should vary in completeness, down to histories with missing links or with private details removed. Targeted repair is compared with retiring only the unit that stated the premise and with removing every descendant that exposure records nominate; the measures are how much the flawed premise still affects behavior after repair, what the repair costs, and how much valid capability is lost by removing too much.

9.7 Learning from Experience under Adversarial Interaction

Learning from interaction exposes an agent to whoever can influence that interaction. Section 6.6 reviews attacks on this channel, from poisoned stores to malicious writes induced through ordinary queries (Chen et al., 2024b; Dong et al., 2025). Correction in Section 9.6 treats errors as accidents that can be traced once found. An adversary instead designs experience to pass admission checks, spreads its effect across units that are each acceptable, and can wait until the guidance has been reused. The agent must keep learning from interaction without letting an adversary steer what it learns, and the two goals conflict: the openness that lets an agent learn quickly from new tasks, through broad admission and generalization, also enlarges what an adversary can shape.

Spreading an attack across several units can defeat checks that inspect units one at a time. EvoBreak (Yan et al., 2026a) shows that units judged harmless individually can elicit harmful behavior when activated together, even though its adversary never edits the experience store and only submits tasks and reads the resulting experience (Section 6.6). For a shopping agent, one lesson might skip a confirmation step when it times out, and another might prefer a payment link named in the task. If confirmation times out on a task that names an unvetted link, applying both lessons sends the payment through that link without any check.

Influence over a trajectory can be treated as part of its evidence. Recording who could shape a trajectory, such as the task submitter or a retrieved web page, lets reliability evaluation and applicability conditions reflect that exposure. Experience formed under untrusted influence could stay isolated until independent trajectories support it, and guidance that relaxes a check could require stronger support than guidance that adds one. When selecting experience for use, the agent could check the whole set it would supply as well as each unit, extending the compatibility judgment that selection already makes (Section 5.2.2). Existing defenses such as A-MemGuard (Wei et al., 2026) combine related checks; however, their protection is shown only for the attacks they evaluate (Section 6.6). The derivation and exposure records of Section 9.6 then support revoking what these checks miss.

Evaluations of such defenses should define each threat model by the attacker’s access, from weakest to strongest: submitting tasks only, submitting tasks and reading the experience they produce, or writing directly to the store. Attack success should be reported under adaptive attackers who know the defense, together with

the learning gain lost on benign task streams, since a defense that rejects all new experience stops attacks trivially. Repair can be judged as in MemSecBench (Chen et al., 2026h): targeted malicious content should no longer be presented as guidance, while benign content present before the attack remains.

9.8 Establishing Cumulative Capability Gains

Finite experiments can support feedback-driven sustained improvement (Section 2.5) only within specified tasks, durations, and budgets. Whether learning costs pay off also depends on the future workload, which may reward different capabilities, and on how long benefits are given to emerge. Existing controls for repository and mechanism changes (Sections 7.5.1 and 7.5.4) therefore need extensions that vary future workloads and evaluation horizons.

Consider two strategies for an agent that repeatedly works on the same website. One stores a cheap workaround, such as a fixed click sequence that reaches a settings page buried several menus deep; the other spends interactions learning how the site’s navigation is organized. The workaround may lead while the site stays stable, and the investment in navigation pays off only if the site changes in a way the workaround cannot absorb, such as a redesign that moves the page; if the site never changes, the additional investment may never be recovered. An evaluation that stops before the change ranks the two strategies differently from one that runs past it.

Competing strategies can share a common early task stream and then have their resulting states evaluated on several distinct future streams. These continuations should vary how often old tasks recur, when environmental conditions change, and whether new tasks require unfamiliar compositions. AgentStream’s comparison of isolated, sequential, and interleaved tasks under permitted feedback updates already shows that the configuration of the task stream affects results (Yan et al., 2026b). Combining these variations with the comparisons of frozen checkpoints (snapshots of the agent that receive no updates while they are tested) in Section 7.5.1 would show when newly acquired capabilities appear, which earlier capabilities survive, and how rankings change with the evaluation horizon.

Gains should be reported across successive checkpoints within the tested range of demands, alongside regressions and total resource use. Faster subsequent learning—solving unfamiliar tasks with fewer additional trials or less feedback—is one valuable outcome; broader capability, retained performance, and recovery after change are others. Moreover, independent evaluation consumes interactions, so the total budget should include the cost of checking progress, and the final test tasks should stay out of any decision about which updates to accept. A stronger claim that the learning mechanism itself improved requires the separate subsequent-learning comparisons in Section 7.5.4.

9.9 How the Open Problems Depend on One Another

The open problems are connected by the guidance that each learning step passes to the next. An unsupported assumption inferred from a human record or introduced during transfer or composition can shape later trajectories and become a premise in new experience. Correcting that error requires tracing both the trajectories supporting a unit and the guidance that influenced those trajectories. Tests in which the premise holds or fails then help identify what to revise while preserving behavior with independent support. Adversarial interaction makes the same task harder: an attacker may design errors to evade these tests or spread their effect across several units.

These dependencies also limit what separate evaluations can establish. A transfer method can improve immediate performance while leaving an invalid generalization in later experience, and a defense can suppress attacks by rejecting useful new experience. The evaluations proposed in the preceding subsections test individual mechanisms under selected conditions. To establish cumulative progress, those mechanisms must also be evaluated through successive updates on the varied future task streams of Section 9.8, tracking whether early gains survive, propagated errors are corrected, and learning remains worth its cost.

10 Conclusion

LLM agents can plan and act within a task, but learning from one task to improve the next remains a central challenge. Turning trajectories into reusable experience offers a path toward capabilities that grow with use. Realizing this potential requires a common understanding of what experience is, how it should be accumulated and revised, and what evidence would demonstrate improvement.

This survey develops that understanding through a unified framework for agent experience: action-guiding information grounded in task trajectories and retained for later episodes where it applies. The framework connects what agents retain with how they manage experience through four lifecycle functions, and distinguishes three improvement claims by the evidence they require. Together, these perspectives provide a basis for comparing methods across representations and storage formats, informing recommendations for seven application settings and an agenda of eight open problems. The analysis reveals a gap between successful experience reuse and sustained learning from experience. Evidence is strongest for fixed experience reuse, chiefly on tasks with tests or reference answers; support for feedback-driven sustained improvement and improvement of the learning mechanism remains partial. Research has also advanced further in building and supplying experience than in checking and revising it. This imbalance motivates our central design principle: keep guidance connected to its applicability conditions and supporting or challenging evidence, so that experience can be checked and corrected as new information arrives.

The next challenge is to turn successful reuse into reliable learning across changing tasks and agents. Explicit experience offers one route toward the Era of Experience anticipated by [Silver and Sutton \(2025\)](#), allowing what agents learn to remain open to inspection and revision. A complementary route is *experience internalization*: training transfers behavior supported by experience into model parameters, reducing dependence on external guidance at runtime. The two routes can work together, with stable, broadly useful behavior internalized and applicability conditions, exceptions, and supporting evidence kept explicit for later correction. Both can draw on human practice, other agents, and an agent’s own trajectories. The broader goal is to make improvement cumulative, transferable, and correctable as agents continue to learn from use.

These conclusions have limits. The survey centers on explicitly accessible experience and reviews a selective, mechanism-oriented set of studies (Section 2.6), many of them preprints in a fast-moving field. Matching studies to improvement claims involves our interpretation, the design principle rests mainly on mechanism analysis and results from individual systems, and the evaluations we propose remain untested. Like the experience it describes, this framework should be revised as new evidence arrives.

References

- Agnar Aamodt and Enric Plaza. Case-based reasoning: Foundational issues, methodological variations, and system approaches. *AI Communications*, 7(1):39–59, 1994. <https://www.cs.auckland.ac.nz/~ian/CBR/AICom.pdf>.
- Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Wang. Agent S: An open agentic framework that uses computers like a human. In *International Conference on Learning Representations*, volume 2025, pages 22924–22946, 2025. <https://arxiv.org/abs/2410.08164>.
- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Christian Alexander Calvo, Weiyuan Chen, and Tu Vu. EvoSkill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026. <https://arxiv.org/abs/2603.02766>. First Workshop on Agent Skills.
- Tong Bai, Zhenglin Wan, Pengfei Zhou, Xingrui Yu, Yang You, and Ivor W. Tsang. SkillDAG: Self-evolving typed skill graphs for LLM skill selection at scale. *arXiv preprint arXiv:2606.03056*, 2026.
- Julia Belikova, Rauf Parchiev, Evgeny Egorov, Grigori Davydenko, Gleb Gusev, Andrey Savchenko, and Maksim Makarenko. Managing procedural memory in LLM agents: Control, adaptation, and evaluation. *arXiv preprint arXiv:2606.23127*, 2026.
- Nishant Bhargava and Rodrigo Sobral Barrento. MEMAUDIT: An exact package-oracle evaluation protocol for budgeted long-term LLM memory writing. *arXiv preprint arXiv:2605.02199*, 2026.
- Igor Bogdanov, Chung-Horng Lung, Thomas Kunz, Jie Gao, Adrian Taylor, and Marzia Zaman. FORGE: Self-Evolving

- Agent Memory With No Weight Updates via Population Broadcast. In *Proceedings of the ACM Conference on AI and Agentic Systems (CAIS)*, pages 292–310, 2026. doi: 10.1145/3786335.3813155. <https://arxiv.org/abs/2605.16233>.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.
- Yuxuan Cai, Wei Li, Jie Zhou, Qin Chen, Xin Li, Bo Zhang, and Liang He. Ask only when needed: Proactive retrieval from memory and skills for experience-driven lifelong agents. *arXiv preprint arXiv:2604.20572*, 2026.
- Yuzheng Cai, Siqi Cai, Yuchen Shi, Zihan Xu, Lichao Chen, Yulei Qin, Xiaoyu Tan, Gang Li, Zongyi Li, Haojia Lin, Yong Mao, Ke Li, and Xing Sun. Training-free group relative policy optimization. *arXiv preprint arXiv:2510.08191*, 2025.
- Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. Remember me, refine me: A dynamic procedural memory framework for experience-driven agent evolution. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 16803–16822. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.829. <https://aclanthology.org/2026.findings-acl.829/>.
- Yurui Chang, Yiran Wu, Qingyun Wu, and Lin Lin. MemCollab: Cross-Model Memory Collaboration via Contrastive Trajectory Distillation. *arXiv preprint arXiv:2603.23234*, 2026.
- Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchu Li, Feiyu Xiong, and Zhiyu Li. HaluMem: Evaluating hallucinations in memory systems of agents. *arXiv preprint arXiv:2511.03506*, 2025a.
- Hanwu Chen, Hanyu Lin, Zhanjiang Yang, Linhao Zhang, Aoyan Li, Jinxi Li, Meng Li, Yin Chen, and Daoguang Zan. EnvPilot: Systematic Design and Evaluation of an Experience-Augmented Agent for Software Environment Setup. *ACM Transactions on Software Engineering and Methodology*, 2026a. To appear; preprint arXiv:2609.07357.
- Jialuo Chen, Lingqi Jiang, Xinhao Deng, Xiaohu Du, Jianan Ma, Yunhao Feng, Yuqi Qing, Zhihao Yuan, Linkang Du, and Jingyi Wang. When experience becomes instruction: Trajectory poisoning in self-evolving agent skill systems. *arXiv preprint arXiv:2608.05563*, 2026b.
- Jingwen Chen, Wenkai Yang, Shengda Fan, Wenbo Nie, Chenxing Sun, Shaodong Zheng, Yangen Hu, Lu Pan, Ke Zeng, and Yankai Lin. Rethinking continual experience internalization for self-evolving LLM agents. *arXiv preprint arXiv:2606.04703*, 2026c. doi: 10.48550/arXiv.2606.04703.
- Kai Chen, Yan Pang, and Tianhao Wang. MRMMIA: Membership inference attacks on memory in chat agents. *arXiv preprint arXiv:2605.27825*, 2026d.
- Kunfeng Chen, Qihuang Zhong, Juhua Liu, and Bo Du. SkillCAT: Contrastive, assessment-augmented and topology-aware skill self-evolution for LLM agents. *arXiv preprint arXiv:2606.13317*, 2026e. <https://arxiv.org/abs/2606.13317v2>. Version 2.
- Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. AutoManual: Constructing instruction manuals by LLM agents via interactive environmental learning. In *Advances in Neural Information Processing Systems*, volume 37, pages 589–631. Curran Associates, Inc., 2024a. doi: 10.52202/079017-0019. https://proceedings.neurips.cc/paper_files/paper/2024/hash/0142921fad7ef9192bd87229cdafa9d4-Abstract-Conference.html.
- Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution. *arXiv preprint arXiv:2507.23361*, 2025b. doi: 10.48550/arXiv.2507.23361. <https://arxiv.org/abs/2507.23361v1>.
- Tianyi Chen, Yinheng Li, Michael Solodko, Sen Wang, Nan Jiang, Tingyuan Cui, Junheng Hao, Jongwoo Ko, Sara Abdali, Leon Xu, Suzhen Zheng, Hao Fan, Pashmina Cameron, Justin Wagle, and Kazuhito Koishida. CUA-Skill: Develop skills for computer using agent. *arXiv preprint arXiv:2601.21123*, 2026f.
- Xuan Chen, Chengpeng Wang, Lu Yan, and Xiangyu Zhang. SLBench: Evaluating how LLM agents follow logical relations in skills. *arXiv preprint arXiv:2607.09016*, 2026g.
- Xuanze Chen, Xukang Xie, Wentao Fu, Jiajun Zhou, Shanqing Yu, and Qi Xuan. MemSecBench: Tracking agent memory poisoning from persistence to consequence and repair. *arXiv preprint arXiv:2607.27080*, 2026h.

- Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems*, 2024b.
- Yu Cheng, Yongkang Hu, Jiuan Zhou, Yushuo Zhang, Yihang Chen, Huichi Zhou, Mingang Chen, Zhizhong Zhang, Kun Shao, Yuan Xie, and Zhaoxia Yin. TAME: A trustworthy test-time evolution of agent memory with systematic benchmarking. *arXiv preprint arXiv:2602.03224*, 2026.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Yumin Choi, Sangwoo Park, Minki Kang, Jinheon Baek, and Sung Ju Hwang. PREPING: Building agent memory without tasks. *arXiv preprint arXiv:2605.13880*, 2026.
- Sijia Cui, Aiyao He, Shuai Xu, Hongming Zhang, Yanna Wang, Qingyang Zhang, Yajing Wang, and Bo Xu. Self-guided function calling in large language models via stepwise experience recall. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10842–10854, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-emnlp.574. <https://aclanthology.org/2025.findings-emnlp.574/>.
- Yuzhuo Cui, Zongye Zhang, and Qingjie Liu. LivingRAG: Augmenting graph RAG with experience. *arXiv preprint arXiv:2608.25960*, 2026.
- Zijie Dai, Shiyuan Deng, Sheng Guan, Yizhou Tian, Xin Yao, Xiao Yan, and James Cheng. RecMem: Recurrence-based memory consolidation for efficient and effective long-running LLM agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 32353–32376, San Diego, California, United States, July 2026a. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-acl.1619. <https://aclanthology.org/2026.findings-acl.1619/>.
- Zijie Dai, Siuhin He, Hui Li, Qihui Zhou, Jiajun Li, Mingcong Song, Guoping Long, Hongjie Si, Xin Yao, Lin Zhang, James Cheng, and Xiao Yan. Metis: Bridging text and code memory for self-evolving agents. *arXiv preprint arXiv:2606.24151*, 2026b.
- Zihao Deng, Yining Zhu, Leiming Wang, Junbo Wang, and Jingfei Lu. Tree-of-Experience: Hierarchical experience management for self-evolving agents. *arXiv preprint arXiv:2608.09044*, 2026. <https://arxiv.org/abs/2608.09044v2>. Version 2.
- Jie Ding, Rui Sun, Xinyuan Zhang, Zeyu Zhang, and Xin Liu. APEX: Distillation of agent procedural experience for adaptive deep research question answering. *arXiv preprint arXiv:2609.02253*, 2026.
- Prakhar Dixit, Sadia Kamal, and Tim Oates. Honest lying: Understanding memory confabulation in reflexive agents. *arXiv preprint arXiv:2605.29463*, 2026.
- Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. Memory injection attacks on LLM agents via query-only interaction. In *Advances in Neural Information Processing Systems*, 2025.
- Zihan Dong, Zhiyuan Ma, Zekun Wang, Yunqing Li, Zirou Liu, Ruixuan Deng, Qishi Zhan, and Rui Qian. How benchmarks mis-score computer-use agents. *arXiv preprint arXiv:2607.28367*, 2026.
- Salma El Yadouni and Guanyi Li. TraceCompiler: Skill-Guided Mining and Compilation of LLM Agent Traces into Mostly Deterministic Workflows. *arXiv preprint arXiv:2608.02680*, 2026.
- Jingxiang Fan, Junbao Zhuo, and Bochao Zou. RRM: Experience-driven reflective retrieval memory for long-horizon multimodal reasoning. *arXiv preprint arXiv:2607.28156*, 2026a.
- Shengda Fan, Xin Cong, Zhong Zhang, Yuepeng Fu, Yesai Wu, Hao Wang, Xinyu Zhang, Enrui Hu, and Yankai Lin. Generalizing experience for language agents with hierarchical MetaFlows. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Zhiyuan Fan, Wenwei Jin, Feng Zhang, Bin Li, Yihong Dong, Yao Hu, and Jiawei Li. Evolving-RL: End-to-end optimization of experience-driven self-evolving capability within agents. *arXiv preprint arXiv:2605.10663*, 2026b.
- Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems. *arXiv preprint arXiv:2508.07407*, 2025.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, HuaJun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. In *Findings of the Association for Computational*

- Linguistics: ACL 2026*, pages 17490–17502. Association for Computational Linguistics, 2026a. doi: 10.18653/v1/2026.findings-acl.866. <https://aclanthology.org/2026.findings-acl.866/>.
- Yue Fang, Zhibang Yang, Fangkai Yang, Xiaoting Qin, Liqun Li, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. ToolAtlas: Learning once, reusing everywhere with tool-side memory. *arXiv preprint arXiv:2607.11126*, 2026b.
- Tao Feng, Chongrui Ye, Tianyang Luo, Jingjun Xu, Xueqiang Xu, Haozhen Zhang, Zhigang Hua, Yan Xie, Shuang Yang, Ge Liu, and Jiaxuan You. ExpGraph: Model-agnostic experience learning with graph-structured memory for LLM agents. *arXiv preprint arXiv:2605.30712*, 2026a. <https://arxiv.org/abs/2605.30712v1>. Version 1.
- Yiyang Feng, Biddut Sarker Bijoy, Niranjana Balasubramanian, and Jiawei Zhou. Break it down, pass it on: Cross-task skill transfer in LLM agents. *arXiv preprint arXiv:2608.20274*, 2026b.
- Thomas Palmeira Ferraz, Romain Deffayet, Vassilina Nikoulina, Hervé Déjean, and Stéphane Clinchant. Retrieval-augmented LLM agents: Learning to learn from experience. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2026. <https://arxiv.org/abs/2603.18272>.
- Saman Forouzandeh, Wei Peng, Parham Moradi, Xinghuo Yu, and Mahdi Jalili. Learning hierarchical procedural memory for LLM agents through Bayesian selection and contrastive refinement. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2026. doi: 10.65109/fkyo8341.
- Dawei Fu, Cheng Jiang, Sitian Qian, Huainan Wang, and Zhongkai Hao. SE-GoS: Self-evolving graph-of-skills for skill library at scale. *arXiv preprint arXiv:2609.08228*, 2026.
- Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. AutoGuide: Automated generation and selection of context-aware guidelines for large language model agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Qihan Ren, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*, 2026a. <https://openreview.net/forum?id=CTr3bovS5F>.
- Jifeng Gao, Kang Xia, Yi Zhang, Xiaobin Hong, Mingkai Lin, Xingshen Wei, Wenzhong Li, and Sanglu Lu. MemPoison: Uncovering persistent memory threats and structural blind spots in LLM agents. *arXiv preprint arXiv:2607.14651*, 2026b.
- Xingze Gao, Chuanrui Hu, Hongda Chen, Pengfei Yao, Zhao Wang, Yi Bai, Zhengwei Wu, Yunyun Han, Xiaofeng Cong, Jie Gui, Yafeng Deng, and Teng Li. EvoAgentBench: Benchmarking agent self-evolution via ability transfer. *arXiv preprint arXiv:2607.05202*, 2026c.
- Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J. Szostkiewicz, Jon M. Laurent, Muhammed T. Razzak, Andrew D. White, Michaela M. Hinks, and Samuel G. Rodrigues. Robin: A multi-agent system for automating scientific discovery. *arXiv preprint arXiv:2505.13400*, 2025.
- Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. DS-agent: Automated data science by empowering large language models with case-based reasoning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 16813–16848. PMLR, 2024. <https://proceedings.mlr.press/v235/guo24b.html>.
- Yuchen Guo, Junli Gong, Weicheng Wang, Hongmin Cai, Yiu ming Cheung, and Weifeng Su. PEAM: Parametric embodied agent memory through contrastive internalization of experience in Minecraft. *arXiv preprint arXiv:2605.27762*, 2026a.
- Zhongxin Guo, Danrui Qi, Hanwen Gu, Peng Cheng, and Yongqiang Xiong. Skill-DisCo: Distilling and compiling agent traces into reusable procedural skills. *arXiv preprint arXiv:2606.26669*, 2026b.
- Priyanshu Gupta, Shashank Kirtania, Ananya Singha, Sumit Gulwani, Arjun Radhakrishna, Gustavo Soares, and Sherry Shi. MetaReflection: Learning instructions for language agents using past reflections. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8369–8385, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.477. <https://aclanthology.org/2024.emnlp-main.477/>.

- Sina Hajimiri, Masih Aminbeidokhti, Jose Dolz, Ismail Ben Ayed, Issam H. Laradji, Spandana Gella, and Nicolas Gontier. Are online skill and memory modules always worth their tokens? a budget-constrained study of web agents. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2026. <https://arxiv.org/abs/2606.15017>.
- Dongge Han, Camille Couturier, Daniel Madrigal Diaz, Xuchao Zhang, Victor Rühle, and Saravan Rajmohan. LEGOMem: Modular procedural memory for multi-agent LLM systems for workflow automation. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2026a. doi: 10.65109/VLUA1303. <https://arxiv.org/abs/2510.04851>. Extended abstract; full version available on arXiv.
- Jinyi Han, Yuanjian Xu, Ying Liao, Xinyi Wang, Zishang Jiang, Zixiang Di, Fanyang Lu, Zhichao Hu, and Yanghua Xiao. Skill-Use: Can LLMs actually use skills in agentic harnesses? *arXiv preprint arXiv:2608.04828*, 2026b.
- Yu Hao, Jinxuan Cai, Qi Zhang, Yawen Li, Zhiqiang Zhang, Chuan Shi, and Cheng Yang. HiSkill: Empowering LLM agents with hierarchical skill graphs. *arXiv preprint arXiv:2607.25853*, 2026.
- Keno Harada, Lui Yoshida, Takeshi Kojima, Yusuke Iwasawa, and Yutaka Matsuo. Automated refinement of essay scoring rubrics for language models via Reflect-and-Revise. In *Proceedings of the 30th Conference on Computational Natural Language Learning*, pages 771–789. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.conll-main.47. <https://aclanthology.org/2026.conll-main.47/>.
- Kostas Hatalis, Despina Christou, and Vyshnavi Kondapalli. Review of case-based reasoning for LLM agents: Theoretical foundations, architectural components, and cognitive integration. *arXiv preprint arXiv:2504.06943*, 2025.
- Kang He and Kaushik Roy. SWE-Adept: An LLM-based agentic framework for deep codebase analysis and structured issue resolution. In *Proceedings of the 14th International Joint Conference on NLP of the Asian Federation of NLP (AACL-IJCNLP)*, 2026. <https://arxiv.org/abs/2603.01327>.
- Shiqi He, Yue Cui, Feijie Wu, Xinyu Ma, Jiaheng Lu, Yaliang Li, Bolin Ding, and Mosharaf Chowdhury. Beyond domains: Reusing web skills via transferable interaction patterns. *arXiv preprint arXiv:2606.17645*, 2026a.
- Xingwei He, Qianru Zhang, Pengfei Chen, Guanhua Chen, Linlin Yu, Yuan Yuan, and Siu-Ming Yiu. ConInstruct: Evaluating large language models on conflict detection and resolution in instructions. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(37):30969–30977, 2026b. doi: 10.1609/aaai.v40i37.40356. <https://ojs.aaai.org/index.php/AAAI/article/view/40356>.
- Zelin He, Haotian Lin, Boran Han, Wei Zhu, Haoyang Fang, Bernie Wang, Xuan Zhu, Runze Li, and Matthew Reimherr. ReSkill: Reconciling skill creation with policy optimization in agentic RL. *arXiv preprint arXiv:2606.01619*, 2026c.
- Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. Benchmarking agent memory in interdependent multi-session agentic tasks. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026d. <https://openreview.net/forum?id=JHYmxqS9Jv>. Preprint title: MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks, arXiv:2602.16313.
- Hao-Lun Hsu, Nikki Lijing Kuang, Boyi Liu, Zhewei Yao, and Yuxiong He. Organize then retrieve: Hierarchical memory navigation for efficient agents. *arXiv preprint arXiv:2606.11680*, 2026.
- Michael Y. Hu, Benjamin Van Durme, Jacob Andreas, and Harsh Jhamtani. Sample-efficient online learning in LM agents via hindsight trajectory rewriting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025a.
- Qisheng Hu, Quanyu Long, and Wenya Wang. When continual learning moves to memory: A study of experience reuse in LLM agents. *arXiv preprint arXiv:2604.27003*, 2026a.
- Tu Hu, Ronghao Chen, Shuo Zhang, Jianghao Yin, Mou Xiao Feng, Jingping Liu, Shaolei Zhang, Wenqi Jiang, Yuqi Fang, Sen Hu, Huacan Wang, and Yi Xu. Controlled self-evolution for algorithmic code optimization. *arXiv preprint arXiv:2601.07348*, 2026b.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*, 2025b.

- Jiahao Huang, Fei Cheng, Junfeng Jiang, Zefan Yu, and Akiko Aizawa. BenchTrace: A benchmark for testing reflection ability and controlled evolution in LLM agents. *arXiv preprint arXiv:2605.29225*, 2026.
- Tenghao Huang, Kinjal Basu, Ibrahim Abdelaziz, Pavan Kapanipathi, Jonathan May, and Muhao Chen. R2D2: Remembering, replaying and dynamic decision making with a reflective agentic memory. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 30318–30330. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1464. <https://aclanthology.org/2025.acl-long.1464/>.
- Che Jiang, Jincheng Zhong, Yu Fu, Kai Tian, Junlin Yang, Kaikai Zhao, Yuchong Wang, Tianwei Luo, Weizhi Wang, Yuxin Zuo, Guoli Jia, Xingtai Lv, Dianqiao Lei, Sihang Zeng, Yuru Wang, Zhenzhao Yuan, Xinwei Long, Ermo Hua, Can Ren, Xin Jiang, Shulei Xie, Yuanchun Zheng, Youbang Sun, Biqing Qi, Ning Ding, Kaiyan Zhang, and Bowen Zhou. Self-improving agents in the era of experience: A survey of self- to meta-evolution. *OpenReview Archive*, 2026a. <https://openreview.net/pdf?id=IUltZSgLMm>.
- Eric Hanchen Jiang, Zhi Zhang, Yuchen Wu, Levina Li, Dong Liu, Xiao Liang, Rui Sun, Yubei Li, Edward Sun, Haozheng Luo, Zhaolu Kang, Aylin Caliskan, Kai-Wei Chang, and Ying Nian Wu. Memory as a controlled process: Learned adaptive memory management for LLM agents. *arXiv preprint arXiv:2607.13591*, 2026b.
- Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. XSkill: Continual learning from experience and skills in multimodal agents. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026c. <https://openreview.net/forum?id=AjP1yvCyoG>.
- Yuxin Jin, Siyuan Zhang, Hanchen Wang, Lu Qin, Ying Zhang, and Wenjie Zhang. EXG: Self-evolving agents with experience graphs. *arXiv preprint arXiv:2605.17721*, 2026.
- Ruofei Ju, Xinrui Wang, Xin Ding, Yifan Yang, Hao Wu, Shiqi Jiang, Qianxi Zhang, Hao Wen, Xiangyu Li, Weijun Wang, Kun Li, Yunxin Liu, Haipeng Dai, Wei Wang, and Ting Cao. EmbodiSkill: Skill-aware reflection for self-evolving embodied agents. *arXiv preprint arXiv:2605.10332*, 2026. <https://arxiv.org/abs/2605.10332v2>. Version 2.
- Tomoyuki Kagaya, Thong Jing Yuan, Yuxuan Lou, Jayashree Karlekar, Sugiri Pranata, Akira Kinose, Koki Oguri, Felix Wick, and Yang You. RAP: Retrieval-augmented planning with contextual memory for multimodal LLM agents. *arXiv preprint arXiv:2402.03610*, 2024.
- Tomoyuki Kagaya, Subramanian Lakshmi, Anbang Ye, Thong Jing Yuan, Jayashree Karlekar, Sugiri Pranata, Natsuki Murakami, Akira Kinose, and Yang You. ViReSkill: Vision-grounded replanning with skill memory for LLM-based planning in lifelong robot learning. *arXiv preprint arXiv:2509.24219*, 2025.
- Qingcan Kang, Mingyang Liu, Shixiong Kai, Kaichao Liang, Zhentao Tang, Yuqi Cui, Tao Zhong, and Mingxuan Yuan. Retain or consolidate? budget-dependent operator selection for language agent memory. *arXiv preprint arXiv:2607.17545*, July 2026a. <https://arxiv.org/abs/2607.17545v2>. Version 2.
- Ryangkyung Kang, Hongcheol Cho, and Youngeun Kim. SkillRet: A large-scale benchmark for skill retrieval in LLM agents. *arXiv preprint arXiv:2605.05726*, 2026b.
- Christopher Kevin, Narendran Raghavan, Jean-Francois Puget, Roshni Malani, Meghana Puvvadi, Moshe Abramovitch, Mohit Gupta, Rama Akkiraju, Subodh Prabhu, Yogesh Dangi, Wei Luo, and Seong Hee Lee. Evaluating skills, not just agents: Agentic continuous evaluation of skills. *arXiv preprint arXiv:2608.20614*, 2026.
- Kangsan Kim, Minki Kang, Taeil Kim, Yanlai Yang, Mengye Ren, and Sung Ju Hwang. Memory transfer learning: How memories are transferred across domains in coding agents. *arXiv preprint arXiv:2604.14004*, 2026a.
- Minsoo Kim, Victor Bursztyn, Eunye Koh, Shunan Guo, and Seung-won Hwang. RaDA: Retrieval-augmented web agent planning with LLMs. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 13511–13525, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.802. <https://aclanthology.org/2024.findings-acl.802/>.
- Taeil Kim, Kangsan Kim, and Sung Ju Hwang. Agent Memory Distillation: Empowering Small LLM Agents with Hierarchical Teacher Memory. *arXiv preprint arXiv:2608.07169*, 2026b.
- Shashank Kirtania, Param Biyani, Priyanshu Gupta, Yasharth Bajpai, Roshni Iyer, Sumit Gulwani, and Gustavo Soares. Improving language agents through BREW: Bootstrapping experientially-learned environmental knowledge. In *Conference on Language Modeling (COLM)*, 2026. <https://openreview.net/forum?id=y3Pu0eJoo6>.
- David A. Kolb. *Experiential Learning: Experience as the Source of Learning and Development*. Prentice-Hall, Englewood Cliffs, NJ, 1984. ISBN 0-13-295261-0.

- Sunjae Lee, Junyoung Choi, Jungjae Lee, Munim Hasan Wasi, Hojun Choi, Steve Ko, Sangeun Oh, and Insik Shin. MobileGPT: Augmenting LLM with human-like app memory for mobile task automation. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, pages 1119–1133. ACM, 2024. doi: 10.1145/3636534.3690682. <https://doi.org/10.1145/3636534.3690682>.
- Yingtie Lei, Zhongwei Wan, Jiankun Zhang, Samiul Alam, Zixuan Zhong, Peizhou Huang, Xin Wang, Jingxuan Zhang, Donghao Zhou, Yunta Hsieh, Zhihao Dou, Hui Shen, Yan Xu, Dimitrios Dimitriadis, Tuo Zhang, and Mi Zhang. SkillEvolBench: Benchmarking the evolution from episodic experience to procedural skills. *arXiv preprint arXiv:2605.24117*, 2026.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale. *arXiv preprint arXiv:2603.02176*, 2026a.
- Qinfeng Li, Dalin He, Yuntai Bao, Ying Yang, Ruoxi Chen, Xinyan Yu, Lizhou Liang, Ge Su, Wenqi Zhang, and Xuhong Zhang. SkillAligner: Treating retrieved skills as adaptable drafts at execution time. *arXiv preprint arXiv:2608.06880*, 2026b.
- Sijia Li, Yuchen Huang, Zifan Liu, Zijian Li, Jingjing Fu, Lei Song, Jiang Bian, Jun Zhang, and Rui Wang. Experience-evolving multi-turn tool-use agent with hybrid episodic-procedural memory. In *International Conference on Machine Learning (ICML)*, 2026c.
- Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han-chung Lee, and Dawn Song. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026d.
- Xuancheng Li, Haitao Li, Yujia Zhou, Yiqun Liu, and Qingyao Ai. Beyond experience retrieval: Learning to generate utility-optimized structured experience for frozen LLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 39467–39482. Association for Computational Linguistics, 2026e. doi: 10.18653/v1/2026.acl-long.1831. <https://aclanthology.org/2026.acl-long.1831/>.
- Yanzhou Li, Yiran Zhang, Xiaoyu Zhang, Xiaoxia Liu, and Yang Liu. CODESKILL: Learning self-evolving skills for coding agents. *arXiv preprint arXiv:2605.25430*, 2026f.
- Yifei Li, Heng Wang, Lingling Zhang, Muye Huang, Xinyu Zhang, Jiashuai Liu, Hang Yan, and Rongman Xu. Beyond retrieval: Query-conditioned reuse of long-horizon agent trajectories. *arXiv preprint arXiv:2608.12847*, 2026g.
- Zhiyuan Li, Linyuan Gao, Xuechun Ding, Hongwei Chen, Yuan Wu, and Yi Chang. CaSKG: Counterfactual-causal skill graphs for scalable agent skill retrieval. *arXiv preprint arXiv:2608.25500*, 2026h.
- Zijian Li, Xin Guan, Bo Zhang, Shen Huang, Houquan Zhou, Shaopeng Lai, Ming Yan, Yong Jiang, Pengjun Xie, Fei Huang, Jun Zhang, and Jingren Zhou. WebWeaver: Structuring web-scale evidence with dynamic outlines for open-ended deep research. In *International Conference on Learning Representations (ICLR)*, 2026i. <https://openreview.net/forum?id=MtNCJlrKt>.
- Junbo Jacob Lian, Huiling Chen, Hanzhang Qin, and Chung-Piaw Teo. Admission without answers: Label-free certification and experience learning for LLM-based optimization modeling. *arXiv preprint arXiv:2608.15565*, 2026.
- Sirui Liang, Pengfei Cao, Jian Zhao, Wenhao Teng, Xiangwen Liao, Jun Zhao, and Kang Liu. Learning how to remember: A meta-cognitive management method for structured and transferable agent memory. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 30733–30753. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.1535. <https://aclanthology.org/2026.findings-acl.1535/>.

- Yusheng Liao, Shuyang Jiang, Yanfeng Wang, and Yu Wang. ReflecTool: Towards reflection-aware tool-augmented clinical agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 13507–13531, 2025. doi: 10.18653/v1/2025.acl-long.663.
- Huawei Lin, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. MUSE-Autoskill: Self-evolving agents via skill creation, memory, management, and evaluation. *arXiv preprint arXiv:2605.27366*, 2026a.
- Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3-4):293–321, 1992. doi: 10.1007/BF00992699.
- Zibin Lin, Shengli Zhang, Taotao Wang, Yihan Xia, Deen Ma, and Guofu Liao. Workflow-Localized Mechanism Learning: Attribution-Guided Repair and Knowledge Reuse for Structured Agent Skills. *arXiv preprint arXiv:2607.20999*, 2026b.
- Dawei Liu, Zongxia Li, Hongyang Du, Xiyang Wu, Shihang Gui, Yongbei Kuang, and Lichao Sun. Graph-of-Skills: Dependency-aware structural retrieval for massive agent skills. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2026a. <https://arxiv.org/abs/2604.05333>.
- Genglin Liu, Shijie Geng, Sha Li, Hejie Cui, Sarah Zhang, Xin Liu, and Tianyi Liu. WebCoach: Self-evolving web agents with cross-session memory guidance. *arXiv preprint arXiv:2511.12997*, 2025.
- Hongyi Liu, Haoyan Yang, Tao Jiang, Bo Tang, Feiyu Xiong, Yuyu Luo, and Zhiyu Li. SkillsVote: Lifecycle governance of agent skills from collection, recommendation to evolution. *arXiv preprint arXiv:2605.18401*, 2026b.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, volume 36, pages 21558–21572, 2023. doi: 10.52202/075280-0943. https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract.html.
- Shuaiqi Liu, Zhengkai Lin, Yuxiang Zhang, Yuanyi Ren, Yue Wu, Yongbin Li, Zheng Wang, Zhihang Fu, and Jieping Ye. The path to recursive self-improving agents: Foundation, framework, and future directions. Preprints.org, 2026c. <https://www.preprints.org/manuscript/202608.0051>. Preprint, version 1.
- Yuxuan Liu, Tianchi Yang, Shaohan Huang, Zihan Zhang, Haizhen Huang, Furu Wei, Weiwei Deng, Feng Sun, and Qi Zhang. Calibrating LLM-based evaluator. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 2638–2656. ELRA and ICCL, 2024. <https://aclanthology.org/2024.lrec-main.237/>.
- Yuxuan Liu, Zhaochen Su, Lingyun Xie, Yuhao Zhang, Qing Zong, Jiahe Guo, Zhongwei Xie, Yiyan Ji, Yauwai Yim, Hongyu Luo, Xiyu Ren, Chenyu Ruan, Haoran Li, and Yangqiu Song. SkillRevise: Improving LLM-authored agent skills via trace-conditioned skill revision. In *Findings of the Association for Computational Linguistics: EMNLP 2026*. Association for Computational Linguistics, 2026d. <https://arxiv.org/abs/2606.01139>.
- Zeyuan Liu, Jeonghye Kim, Xufang Luo, Dongsheng Li, and Yuqing Yang. Exploratory memory-augmented LLM agent via hybrid on- and off-policy optimization. In *International Conference on Learning Representations (ICLR)*, 2026e. <https://openreview.net/forum?id=UOzxviKVFO>.
- Zhou Liu, Ligang Huang, Zeli Su, Zewei Pan, Zhaoyang Han, Xing Chen, Yuanfeng Song, and Wentao Zhang. SkillLens: Visual skill cards for retrieval-augmented GUI action prediction and on-policy distillation. *arXiv preprint arXiv:2608.10775*, 2026f.
- You Lu, Xinyu Huang, Bihuan Chen, and Xin Peng. SkillSentry: Reliable skill execution for LLM agents via runtime assurance. *arXiv preprint arXiv:2608.09253*, 2026a. doi: 10.48550/arXiv.2608.09253.
- Yuxing Lu, Yicheng Chen, Shanchan Wu, and Sercan Ö. Arık. Procedural graphs: Self-evolving execution structures for LLM agents. *arXiv preprint arXiv:2609.09153*, 2026b.
- Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. SKILL0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*, 2026c. doi: 10.48550/arXiv.2604.02268.
- Jinghao Luo, Yuchen Tian, Chuxue Cao, Ziyang Luo, Hongzhan Lin, Kaixin Li, Chuyi Kong, Ruichao Yang, and Jing Ma. From storage to experience: A survey on the evolution of LLM agent memory mechanisms. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 41622–41652. Association for Computational Linguistics, 2026. <https://aclanthology.org/2026.findings-acl.2069/>.

- Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent Lightning: Train ANY AI agents with reinforcement learning. *arXiv preprint arXiv:2508.03680*, 2025.
- Youngang Lyu, Xi Zhang, Yuyue Zhao, Konstantinos Papakostas, Xinhao Yi, Shuyu Guo, Wenxiang Hu, Jan Piotrowski, Jakub Kaliski, Jacopo Urbani, Zaiqiao Meng, Lun Zhou, Zhaochun Ren, and Xiaohui Yan. EvoScientist: Towards multi-agent evolving AI scientists for end-to-end scientific discovery. *arXiv preprint arXiv:2603.08127*, 2026.
- Weitao Ma, Xiaocheng Feng, Lei Huang, Xiachong Feng, Zhanyu Ma, Jun Xu, Jiuchong Gao, Jinghua Hao, Renqing He, and Bing Qin. Fine-mem: Fine-grained feedback alignment for long-horizon memory management. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 19666–19684. Association for Computational Linguistics, 2026a. doi: 10.18653/v1/2026.acl-long.900. <https://aclanthology.org/2026.acl-long.900/>.
- Yijun Ma, Zehong Wang, Yiyang Li, Ziming Li, Xiaoguang Guo, Weixiang Sun, Chuxu Zhang, and Yanfang Ye. ProPlay: Procedural world models for self-evolving LLM agents. *arXiv preprint arXiv:2606.12780*, 2026b.
- Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Peter Jansen, Oyvind Tafjord, Niket Tandon, Li Zhang, Chris Callison-Burch, and Peter Clark. CLIN: A continually learning language agent for rapid task adaptation and generalization. In *Conference on Language Modeling (COLM)*, 2024. <https://openreview.net/forum?id=xS6zx1aBI9>.
- Wenyu Mao, Haoyang Liu, Haosong Tan, Yaorui Shi, Jiancan Wu, An Zhang, and Xiang Wang. Joint optimization of multi-agent memory system. *arXiv preprint arXiv:2603.12631*, 2026. <https://arxiv.org/abs/2603.12631v2>. Version 2.
- Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-Pro: Learning reusable skills from experience via non-parametric PPO for LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. <https://openreview.net/forum?id=9kJQjx2B80>.
- Yongliang Miao, Ziyang Yu, Liang Zhao, Bowen Zhu, and Hasibul Haque. SkillLens: Adaptive multi-granularity skill reuse for cost-efficient LLM agents. *arXiv preprint arXiv:2605.08386*, 2026.
- Shikhar Murty, Christopher Manning, Peter Shaw, Mandar Joshi, and Kenton Lee. BAGEL: Bootstrapping agents by guiding exploration with language. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 2024.
- Andy Nguyen, Danh Doan, Hoang Pham, Bao Ha, Dat Pham, Linh Nguyen, Hieu Nguyen, Thien Nguyen, Cuong Do, Phat Nguyen, and Toan Nguyen. ByteRover: Agent-native memory through LLM-curated hierarchical context. *arXiv preprint arXiv:2604.01599*, 2026.
- Dang Nguyen, Viet Dac Lai, Seunghyun Yoon, Ryan A. Rossi, Handong Zhao, Ruiyi Zhang, Puneet Mathur, Nedim Lipka, Yu Wang, Trung Bui, Franck Dernoncourt, and Tianyi Zhou. DynaSaur: Large language agents beyond predefined actions. In *Conference on Language Modeling*, 2025. <https://arxiv.org/abs/2411.01747>.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2Skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026. <https://arxiv.org/abs/2603.25158v3>. Version 3.
- Kolby Nottingham, Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Sameer Singh, Peter Clark, and Roy Fox. Skill set optimization: Reinforcing language model behavior via transferable skills. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 2024.
- Nous Research. Hermes Agent: Curator. Official software documentation, 2026. <https://hermes-agent.nousresearch.com/docs/user-guide/features/curator>. Accessed September 15, 2026.
- OpenAI. Introducing ChatGPT. OpenAI, November 2022. <https://openai.com/index/chatgpt/>.
- Ciyan Ouyang and Rui Hou. MemLineage: Lineage-guided enforcement for LLM agent memory. *arXiv preprint arXiv:2605.14421*, 2026.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations (ICLR)*, 2026.
- Hongji Pu, Xinyuan Song, and Liang Zhao. SkillOps: Managing LLM agent skill libraries as self-maintaining software ecosystems. *arXiv preprint arXiv:2605.13716*, 2026.

- Zerui Qin, Sheng Yue, Xingyuan Hua, Yongjian Fu, and Ju Ren. Executable agentic memory for GUI agent. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. <https://openreview.net/forum?id=8metOwZHjD>.
- Kaixian Qu, Guowei Lan, René Zurbrügg, Changan Chen, Christopher E. Mower, Haitham Bou-Ammar, and Marco Hutter. A pragmatist robot: Learning to plan tasks by experiencing the real world. *IEEE Robotics and Automation Letters*, 11(5):5606–5613, 2026. doi: 10.1109/LRA.2026.3669804.
- Alireza Rezazadeh, Zichao Li, Wei Wei, and Yujia Bao. From isolated conversations to hierarchical schemas: Dynamic tree memory representation for LLMs. In *International Conference on Learning Representations (ICLR)*, 2025. <https://openreview.net/forum?id=moXtEmCleY>.
- Gabriel Sarch, Lawrence Jang, Michael J. Tarr, William W. Cohen, Kenneth Marino, and Katerina Fragkiadaki. VLM agents generate their own memories: Distilling experience into embodied programs of thought. In *Advances in Neural Information Processing Systems*, volume 37, pages 75942–75985. Curran Associates, Inc., 2024. doi: 10.52202/079017-2418. <https://arxiv.org/abs/2406.14596>.
- Vishnu Sarukkai, Zhiqiang Xie, and Kayvon Fatahalian. Self-generated in-context examples improve LLM agents for sequential decision-making tasks. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, pages 64392–64425. Curran Associates, Inc., 2025. https://proceedings.neurips.cc/paper_files/paper/2025/file/5d1f02132ef51602adf0700ca5b6138-Paper-Conference.pdf.
- Yuval Shalev, Zifeng Ding, and Mateja Jamnik. Training language agents to learn from experience. *arXiv preprint arXiv:2605.20477*, 2026.
- Jiaqi Shao, Yiyi Lu, Yunzhen Zhang, and Bing Luo. When stored evidence stops being usable: Scale-conditioned evaluation of agent memory. *arXiv preprint arXiv:2605.07313*, 2026a.
- Jie-Jing Shao, Haiyan Yin, Yueming Lyu, Xingrui Yu, Lan-Zhe Guo, Ivor Tsang, James Kwok, and Yu-Feng Li. Lifting traces to logic: Programmatic skill induction with neuro-symbolic learning for long-horizon agentic tasks. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026b. <https://openreview.net/forum?id=D0Dg8ISjq0>.
- Kangning Shen, Jingyuan Zhang, Chenxi Sun, Wencong Zeng, and Yang Yue. Structurally aligned subtask-level memory for software engineering agents. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026a. <https://openreview.net/forum?id=2CoRS45Ucj>.
- Yiting Shen, Kun Li, Wei Zhou, and Songlin Hu. Mem2ActBench: A benchmark for evaluating long-term memory utilization in task-oriented autonomous agents. *arXiv preprint arXiv:2601.19935*, 2026b.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.
- Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, and Yu Su. AgentCL: Toward rigorous evaluation of continual learning in language agents. *arXiv preprint arXiv:2606.02461*, 2026.
- David Silver and Richard S. Sutton. Welcome to the Era of Experience. Google DeepMind, 2025. <https://storage.googleapis.com/deepmind-media/Era-of-Experience%20/The%20Era%20of%20Experience%20Paper.pdf>. Preprint of a chapter for *Designing an Intelligence*, MIT Press.
- Saksham Sahai Srivastava and Haoyu He. MemoryGraft: Persistent compromise of LLM agents via poisoned experience retrieval. *arXiv preprint arXiv:2512.16962*, 2025.
- Adam Stein, Matthew Trager, Benjamin Bowman, Michael Kleinman, Aditya Chattopadhyay, Wei Xia, and Stefano Soatto. Experience-guided adaptation of inference-time reasoning strategies. *arXiv preprint arXiv:2511.11519*, 2025.
- Hongjin Su, Ruoxi Sun, Jinsung Yoon, Pengcheng Yin, Tao Yu, and Serkan Ö. Arik. Learn-by-interact: A data-centric framework for self-adaptive agents in realistic environments. In *International Conference on Learning Representations (ICLR)*, 2025.
- Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. <https://openreview.net/forum?id=li6ZCvfiQJ>.
- Haoran Sun, Shaoning Zeng, and Bob Zhang. H-MEM: Hierarchical memory for high-efficiency long-term reasoning in LLM agents. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational*

- Linguistics (Volume 1: Long Papers)*, pages 341–350. Association for Computational Linguistics, 2026a. doi: 10.18653/v1/2026.eacl-long.15. <https://aclanthology.org/2026.eacl-long.15/>.
- Haotian Sun, Yuchen Zhuang, Ling kai Kong, Bo Dai, and Chao Zhang. AdaPlanner: Adaptive planning from feedback with language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 58202–58245. Curran Associates, Inc., 2023. doi: 10.52202/075280-2537. https://proceedings.neurips.cc/paper_files/paper/2023/hash/b5c8c1c117618267944b2617add0a766-Abstract-Conference.html.
- Libo Sun, Jiwen Zhang, Siyuan Wang, and Zhongyu Wei. MAGNET: Towards adaptive GUI agents with memory-driven knowledge evolution. *arXiv preprint arXiv:2601.19199*, 2026b.
- Yongqian Sun, Rongchen Gao, Yu Luo, Wenwei Gu, Shenglin Zhang, Qingyi Guo, Qiuai Fu, Yaoliang Wu, and Dan Pei. OpsMem: Dual-memory reasoning with cross-memory resonance for failure diagnosis. *arXiv preprint arXiv:2607.11357*, 2026c.
- Zhongxiang Sun, Qipeng Wang, Weijie Yu, Jingxuan Yang, Haolang Lu, and Jun Xu. Deep search with hierarchical meta-cognitive monitoring inspired by cognitive neuroscience. *arXiv preprint arXiv:2601.23188*, 2026d.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106. Association for Computational Linguistics, 2026. <https://aclanthology.org/2026.eacl-long.333/>.
- Haoran Tan, Zeyu Zhang, Zhicheng Cao, Rui Li, and Xu Chen. DELTAMEM: Incremental experience memory for LLM agents via residual trees. *arXiv preprint arXiv:2606.03083*, 2026a.
- Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. SkillZip: Contract-preserving graph compression for scalable agent skill libraries. *arXiv preprint arXiv:2608.05604*, 2026b.
- Yunteng Tan, Zhi Gao, and Xinxiao Wu. Enhancing web agents with a hierarchical memory tree. *arXiv preprint arXiv:2603.07024*, 2026c.
- Liyan Tang, Cyrus Rashtchian, Chun-Sung Ferng, Andrew Tomkins, Da-Cheng Juan, and Tu Vu. WikiSkill: Compiling agent experience into persistent knowledge for skill evolution. *arXiv preprint arXiv:2608.27454*, 2026. doi: 10.48550/arXiv.2608.27454.
- Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, Ge Zhang, Jiaheng Liu, Xingyao Wang, Sirui Hong, Chenglin Wu, Hao Cheng, Chi Wang, and Wangchunshu Zhou. Agent KB: Leveraging cross-domain experience for agentic problem solving. *arXiv preprint arXiv:2507.06229*, 2025.
- Zhen Tao, Riwei Lai, Chenyun Yu, Weixin Chen, Li Chen, Beibei Kong, Lei Cheng, Chengxiang Zhuo, Zang Li, and Qingqiang Sun. SAGER: Self-evolving user policy skills for recommendation agent. *arXiv preprint arXiv:2604.14972*, 2026.
- Feng Tian, Flora D. Salim, and Hao Xue. TradingGroup: A multi-agent trading system with self-reflection and data-synthesis. *arXiv preprint arXiv:2508.17565*, 2025.
- Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In *Advances in Neural Information Processing Systems*, volume 36, pages 38975–38987, 2023. doi: 10.52202/075280-1693. https://proceedings.neurips.cc/paper_files/paper/2023/hash/7a92bcdede88c7afd108072faf5485c8-Abstract.html.
- Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, and Shumin Deng. SkillX: Automatically constructing skill knowledge bases for agents. *arXiv preprint arXiv:2604.04804*, 2026a.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024a. ISSN 2835-8856. <https://openreview.net/forum?id=ehfRiFOR3a>.
- Haoyu Wang, Guangyuan Dong, He Liang, Zijing Zhang, Jiachen Luo, Chuang Liu, Chao Xue, and Hao Tang. MemGuard: Persisting verifier signals for LLM-agent memory governance. *arXiv preprint arXiv:2608.21867*, 2026b.
- Juntong Wang, Haoyue Zhao, Guanghui Pan, Xiyuan Wang, Yanbo Wang, Qiyang Deng, and Muhan Zhang. SAGE: A self-evolving agentic graph-memory engine for structure-aware associative memory. *arXiv preprint arXiv:2605.12061*, 2026c.

- Kaixiang Wang, Jiong Lou, Zhaojiacheng Zhou, and Jie Li. OEP: Poisoning self-evolving LLM agents via locally correct but non-transferable experiences. *arXiv preprint arXiv:2605.18930*, 2026d.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024b.
- Qihao Wang, Ziming Cheng, Shuo Zhang, Fan Liu, Rui Xu, Heng Lian, Kunyi Wang, Xiaoming Yu, Jianghao Yin, Sen Hu, Yue Hu, Shaolei Zhang, Yanbing Liu, Ronghao Chen, and Huacan Wang. MemGovern: Enhancing code agents through learning from governed human experiences. *arXiv preprint arXiv:2601.06789*, 2026e.
- Wenjun Wang, Yuchen Fang, Fengrui Liu, Zibo Liang, and Kai Zheng. Experience memory graph: One-shot error correction for agents. *arXiv preprint arXiv:2607.13884*, 2026f.
- Yibo Wang, Nikki Lijing Kuang, Philip S. Yu, Zhewei Yao, and Yuxiong He. Learning to retrieve: Dual-level long-term memory for text-to-SQL agents. *arXiv preprint arXiv:2606.00547*, 2026g.
- Yuyao Wang, Zhongjian Zhang, Mo Chi, Kaichi Yu, Yuhan Li, Miao Peng, Bing Tong, Chen Zhang, Yan Zhou, and Jia Li. EvoMemBench: Benchmarking agent memory from a self-evolving perspective. *arXiv preprint arXiv:2605.18421*, 2026h.
- Zhenhailong Wang, Haiyang Xu, Junyang Wang, Xi Zhang, Ming Yan, Ji Zhang, Fei Huang, and Heng Ji. Mobile-Agent-E: Self-evolving mobile assistant for complex tasks. *arXiv preprint arXiv:2501.11733*, 2025a.
- Zhiruo Wang, Graham Neubig, and Daniel Fried. TroVE: Inducing verifiable and efficient toolboxes for solving programmatic tasks. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 51177–51191. PMLR, 2024c. <https://proceedings.mlr.press/v235/wang24az.html>.
- Zihao Wang, Shaofei Cai, Anji Liu, Yonggang Jin, Jinbing Hou, Bowei Zhang, Haowei Lin, Zhaofeng He, Zilong Zheng, Yaodong Yang, Xiaojian Ma, and Yitao Liang. JARVIS-1: Open-world multi-task agents with memory-augmented multimodal language models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(3):1894–1907, 2025b. doi: 10.1109/TPAMI.2024.3511593. <https://doi.org/10.1109/TPAMI.2024.3511593>.
- Ziyan Wang, Yingpeng Du, Zhu Sun, Haoyan Chua, Kaidong Feng, Wenya Wang, and Jie Zhang. Re2LLM: Reflective reinforcement large language model for session-based recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(12):12827–12835, 2025c. doi: 10.1609/aaai.v39i12.33399.
- Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. Inducing programmatic skills for agentic tasks. In *The Second Conference on Language Modeling*, 2025d. <https://arxiv.org/abs/2504.06821>.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025e. <https://openreview.net/forum?id=NTAhi2JEEE>.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.
- Qianshan Wei, Tengchao Yang, Yaochen Wang, Xinfeng Li, Lijun Li, Zhenfei Yin, Yi Zhan, Thorsten Holz, Zhiqiang Lin, and XiaoFeng Wang. A-MemGuard: A proactive defense framework for LLM-based agent memory. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. <https://openreview.net/forum?id=udqe7UZUZ6>.
- Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. Evo-memory: Benchmarking LLM agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*, 2025.
- Lukas Weidener, Marko Brkić, Mihailo Jovanović, Ritvik Singh, Chiara Baccin, Emre Ulgac, Alex Dobrin, and Aakaash Meduri. Rethinking the AI scientist: Interactive multi-agent workflows for scientific discovery. *arXiv preprint arXiv:2601.12542*, 2026.
- Cheng-Kuang Wu, Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, and Hung-yi Lee. StreamBench: Towards benchmarking continuous improvement of language agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 37, 2024a. <https://openreview.net/forum?id=8hUUy3hoS8>.
- Di Wu, Zixiang Ji, Asmi Kawatkar, Bryan Kwan, Jia-Chen Gu, Nanyun Peng, and Kai-Wei Chang. LongMemEval-V2: Evaluating long-term agent memory toward experienced colleagues. *arXiv preprint arXiv:2605.12493*, 2026a.

- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, and Botian Shi. From interactions to principles: Experience-driven self-distillation for evolving LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026b. <https://openreview.net/forum?id=BamDfCP5R3>. Preprint title: EvolveR: Self-Evolving LLM Agents through an Experience-Driven Lifecycle, arXiv:2510.16079.
- Xiaodong Wu, Yu Shi, Qi Li, Zhimin Zhao, Xiangman Li, Bram Adams, Ahmed E. Hassan, and Jianbing Ni. EVOMAL: Self-poisoning in self-evolving coding agents. *arXiv preprint arXiv:2608.25776*, 2026c.
- Yaxiong Wu and Yongyue Zhang. Agent skills from the perspective of procedural memory: A survey. *TechRxiv*, 2026. doi: 10.36227/techrxiv.176857932.25697838/v1. <https://yashonwu.github.io/agent-skills-survey/>.
- Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. OS-Copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*, 2024b.
- Yunjia Xi, Weiwen Liu, Jianghao Lin, Bo Chen, Ruiming Tang, Weinan Zhang, and Yong Yu. MemoCRS: Memory-enhanced sequential conversational recommender systems with large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 2585–2595. Association for Computing Machinery, 2024. doi: 10.1145/3627673.3679599. <https://doi.org/10.1145/3627673.3679599>.
- Siyu Xia, Zekun Xu, Jiajun Chai, Wentian Fan, Yan Song, Xiaohan Wang, Guojun Yin, Wei Lin, Haifeng Zhang, and Jun Wang. From experience to strategy: Empowering LLM agents with trainable graph memory. *arXiv preprint arXiv:2511.07800*, 2025.
- Han Xiao, Hongjun Xu, Xin Zhang, Yidong Chen, and Xiaodong Shi. TARL: Transaction-aware reliable ledgers for executable memory management in long-term agents. *arXiv preprint arXiv:2608.03699*, 2026. <https://arxiv.org/abs/2608.03699v2>. Version 2.
- Can Xie, Yuyi Zhou, Wen Yang, Ziyi Zhang, Siyao Song, Yingzhuo Deng, Shuo Ren, and Jiajun Zhang. EDGE: Experience-distillation for guided exploration in agentic reinforcement learning. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2026a. <https://arxiv.org/abs/2608.21946>.
- Weiwei Xie, Shaoxiong Guo, Fan Zhang, Tian Xia, Xue Yang, Lizhuang Ma, Junchi Yan, and Qibing Ren. MemEvoBench: Benchmarking safety risks from memory misevolution in LLM agents. *arXiv preprint arXiv:2604.15774*, 2026b.
- Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 623–645, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.27. <https://aclanthology.org/2026.acl-long.27/>.
- Haoran Xu, Jiacong Hu, Ke Zhang, Lei Yu, Yuxin Tang, Xinyuan Song, Yiqun Duan, Lynn Ai, and Bill Shi. SEDM: Scalable self-evolving distributed memory for agents. *arXiv preprint arXiv:2509.09498*, 2025a. <https://arxiv.org/abs/2509.09498v1>. Version 1.
- Jinghan Xu, Yiyong Xiao, Wanru Shao, Hankai Liu, and Xinjin Li. Memory provenance laundering in LLM agents: A non-amplification firewall for persistent memory. *arXiv preprint arXiv:2607.29167*, 2026a.
- Ruiyao Xu, Tiankai Yang, and Wei-Chieh Huang. HyperSkill: Self-evolving LLM agents via hypergraph-structured skill memory. *arXiv preprint arXiv:2608.16114*, 2026b.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025b.
- Yisen Xu, Jiayuan Zhou, Ruiqi Pan, and Tse-Hsun Chen. Reusing past repairs through hierarchical trajectory abstraction for coding agents. *arXiv preprint arXiv:2607.29658*, 2026c.
- Bingyu Yan, Xiaoming Zhang, Chaozhuo Li, Ziyi Zhou, Yirui Qi, and Litian Zhang. Benign Alone, Harmful Together: Exploiting Experience Composition in Self-Evolving LLM Agents. *arXiv preprint arXiv:2608.01759*, 2026a.
- Dong Yan, Jian Liang, Dapeng Hu, Ran He, Nicholas Jing Yuan, Qi Zhang, and Tieniu Tan. AgentStream: How Well Do Self-Evolving LLM Agents Perform Under Streaming Tasks? *arXiv preprint arXiv:2608.00155*, 2026b.

- Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schuetze, Volker Tresp, and Yunpu Ma. Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12805–12825. Association for Computational Linguistics, 2026c. doi: 10.18653/v1/2026.acl-long.583. <https://aclanthology.org/2026.acl-long.583/>.
- Cheng Yang, Xuemeng Yang, Licheng Wen, Daocheng Fu, Jianbiao Mei, Rong Wu, Pinlong Cai, Yufan Shen, Nianchen Deng, Botian Shi, Yu Qiao, and Haifeng Li. Learning on the job: An experience-driven self-evolving agent for long-horizon tasks. *arXiv preprint arXiv:2510.08002*, 2025a.
- Ke Yang, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and ChengXiang Zhai. PlugMem: A task-agnostic plugin memory module for LLM agents. In *International Conference on Machine Learning (ICML)*, 2026a.
- Ling Yang, Zhaochen Yu, Tianjun Zhang, Shiyi Cao, Minkai Xu, Wentao Zhang, Joseph E. Gonzalez, and Bin Cui. Buffer of Thoughts: Thought-augmented reasoning with large language models. In *Advances in Neural Information Processing Systems*, volume 37, pages 113519–113544. Curran Associates, Inc., 2024. doi: 10.52202/079017-3607. <https://arxiv.org/abs/2406.04271>.
- Lu Yang, Shusheng Xu, Zhuoran Li, Tongkai Yang, and Longbo Huang. When personal memory has no single answer: Evaluating LLM agents under irreducible conflict. *arXiv preprint arXiv:2608.13921*, 2026b.
- Wei Yang, Jinwei Xiao, Hongming Zhang, Qingyang Zhang, Yanna Wang, and Bo Xu. Coarse-to-fine grounded memory for LLM agent planning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 13029–13056, Suzhou, China, 2025b. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.659. <https://aclanthology.org/2025.emnlp-main.659/>.
- Xidong Yang, Xingyi Zhang, Wenhao Li, Wenyan Liu, Junjie Sheng, Yun Hua, Wei Yin, Tao Fang, Chuyun Shen, and Xiangfeng Wang. PATH-Bench: Path-dependent evaluation of lifelong agents. *arXiv preprint arXiv:2608.01149*, 2026c.
- Yi Yang, Zhennan Chen, Yihong Zhuang, Tiehan Fan, Yinan Chen, Jian Li, Jian Yang, and Ying Tai. RoMeRL: Balancing feedback coverage and the memory-reward trap in self-evolving agent memory via reduced-order utility states. *arXiv preprint arXiv:2608.02508*, 2026d. <https://arxiv.org/abs/2608.02508v3>. Version 3.
- Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive strategy for self-evolving agent skills. *arXiv preprint arXiv:2605.23904*, 2026e.
- Yutao Yang, Junsong Li, Qianjun Pan, Jie Zhou, Kai Chen, Qin Chen, Jingyuan Zhao, Ningning Zhou, Xin Li, and Liang He. PsychAgent: An experience-driven lifelong learning agent for self-evolving psychological counselor. *arXiv preprint arXiv:2604.00931*, 2026f.
- Chao Yao, Yangbo Wei, Zhen Huang, Junhong Qian, Chenle Chen, Shaoqiang Lu, Chen Wu, and Lei He. Forgetting without restarting: Execution-state unlearning for stateful LLM agents. *arXiv preprint arXiv:2609.04875*, 2026.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. https://openreview.net/forum?id=WE_vluYUL-X.
- Weiran Yao, Shelby Heinecke, Juan Carlos Niebles, Zhiwei Liu, Yihao Feng, Le Xue, Rithesh Murthy, Zeyuan Chen, Jianguo Zhang, Devansh Arpit, Ran Xu, Phil Mui, Huan Wang, Caiming Xiong, and Silvio Savarese. Retroformer: Retrospective large language agents with policy gradient optimization. In *International Conference on Learning Representations (ICLR)*, 2024.
- Bowen Ye, Yongchao Xu, Zhijian Li, Xiang Yin, Junkai Ma, and Wenzhao Li. CoEvo-Mem: Co-evolving retrieval policy and memory bank for LLM agents. *arXiv preprint arXiv:2608.01739*, 2026a.
- Chongrui Ye, Yuxiang Liu, Yu Wang, Haofei Yu, Yining Zhao, Ge Liu, Julian McAuley, and Jiaxuan You. Auto-dreamer: Learning offline memory consolidation for language agents. *arXiv preprint arXiv:2605.20616*, 2026b.
- Shicheng Ye, Chao Yu, Kaiqiang Ke, Chengdong Xu, and Yinqi Wei. H²R: Hierarchical hindsight reflection for multi-task LLM agents. *arXiv preprint arXiv:2509.12810*, 2025.

- Yongshi Ye, Hui Jiang, Feihu Jiang, Tian Lan, Yichao Du, Biao Fu, Xiaodong Shi, Qianghuai Jia, Longyue Wang, and Weihua Luo. UMEM: Unified memory extraction and management framework for generalizable memory. In *International Conference on Machine Learning (ICML)*, 2026c.
- Zonghao Ying, Xiangfan Wu, Huiyu Wu, Xing Zheng, Huangsheng Cheng, Xiaorong Shi, and Jing Guo. SkillJack: Persistent skill backdoors in self-evolving agents. *arXiv preprint arXiv:2608.03509*, 2026.
- Caili Yu, Yiqi Wang, Jiaqi Zhang, Yiqun Duan, Mingkai Zheng, Zhangkai Wu, Kaize Shi, and Taotao Cai. From faulty memories to corrected actions: Dependency-guided rollback repair for memory-augmented agents. *arXiv preprint arXiv:2608.10502*, 2026a.
- Simon Yu, Gang Li, Weiyan Shi, and Peng Qi. PolySkill: Learning generalizable skills through polymorphic abstraction for continual learning. In *International Conference on Learning Representations*, 2026b. <https://openreview.net/forum?id=KdEsujyiSV>.
- Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yupeng Cao, Zhi Chen, Jordan W. Suchow, Rong Liu, Zhenyu Cui, Zhaozhuo Xu, Denghui Zhang, Koduvayur Subbalakshmi, Guojun Xiong, Yueru He, Jimin Huang, Dong Li, and Qianqian Xie. FinCon: A synthesized LLM multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- Ye Yu, Xiaopeng Yuan, Haibo Jin, Heming Liu, Yaoning Yu, and Haohan Wang. Do self-evolving agents forget? capability degradation and preservation in lifelong LLM agent adaptation. *arXiv preprint arXiv:2605.09315*, 2026c.
- Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21457–21483, San Diego, California, United States, July 2026d. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.981. <https://aclanthology.org/2026.acl-long.981/>.
- Zhuoyun Yu, Xin Xie, Wuguannan Yao, Chenxi Wang, Lei Liang, Xiang Qi, and Shumin Deng. SkillAdaptor: Self-adapting skills for LLM agents from trajectories. *arXiv preprint arXiv:2606.01311*, 2026e.
- Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi R. Fung, Hao Peng, and Heng Ji. CRAFT: Customizing LLMs by creating and retrieving from specialized toolsets. In *The Twelfth International Conference on Learning Representations*, 2024. <https://openreview.net/forum?id=G0vdDSt9XM>.
- Yanwei Yue, Guibin Zhang, Boci Peng, Xuanbo Fan, Jiabin Guo, Qiankun Li, and Yan Zhang. Mem-T: Densifying rewards for long-horizon memory agents. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. <https://openreview.net/forum?id=8ppVmLtA2V>.
- Xiangjun Zai, Xingyu Tan, Chen Chen, Xiaoyang Wang, and Wenjie Zhang. Trace only what you need: Structure-aware on-demand hypergraph memory for long-document question answering. *arXiv preprint arXiv:2606.10921*, 2026.
- Yaopei Zeng, Congchao Wang, Jianhang Chen, Nan Wang, Yurui Chang, and Lu Lin. Critic Experience Bank: Self-evolving step-level confidence estimation for LLM agents. *arXiv preprint arXiv:2607.12397*, 2026. doi: 10.48550/arXiv.2607.12397.
- Dylan Zhang, Yanshan Lin, Zhengkun Wu, Yihang Sun, Bingxuan Li, Dianqi Li, and Hao Peng. Useful memories become faulty when continuously updated by LLMs. *arXiv preprint arXiv:2605.12978*, 2026a.
- Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu, Kun Wang, and Shuicheng Yan. G-memory: Tracing hierarchical memory for multi-agent systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Guibin Zhang, Muxin Fu, and Shuicheng Yan. MemGen: Weaving generative latent memory for self-evolving agents. In *International Conference on Learning Representations (ICLR)*, 2026b. <https://openreview.net/forum?id=vI56m4Iu4e>.
- Guibin Zhang, Haotian Ren, Chong Zhan, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. MemEvolve: Meta-evolution of agent memory systems. In *International Conference on Machine Learning (ICML)*, 2026c.
- Guilin Zhang, Wei Jiang, Xiejiashan Wang, Aisha Behr, Kai Zhao, Jeffrey Friedman, Xu Chu, and Amine Anoun. Adaptive memory admission control for LLM agents. *arXiv preprint arXiv:2603.04549*, 2026d.
- Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu. CoEvoSkills: Self-evolving agent skills via co-evolutionary verification. In *Conference on Language Modeling (COLM)*, 2026e. <https://openreview.net/forum?id=gQjmJlichQ>.

- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. MemSkill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026f.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *International Conference on Learning Representations (ICLR)*, 2026g. <https://openreview.net/forum?id=pUpzQZTvGY>.
- Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *arXiv preprint arXiv:2603.19461*, 2026h.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026i. <https://arxiv.org/abs/2510.04618>.
- Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. MemRL: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026j.
- Wenqi Zhang, Ke Tang, Hai Wu, Mengna Wang, Yongliang Shen, Guiyang Hou, Zeqi Tan, Peng Li, Yueting Zhuang, and Weiming Lu. Agent-pro: Learning to evolve via policy-level reflection and optimization. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5348–5375, Bangkok, Thailand, August 2024a. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.292. <https://aclanthology.org/2024.acl-long.292/>.
- Wenyuan Zhang, Xinghua Zhang, Haiyang Yu, Shuaiyi Nie, Bingli Wu, Juwei Yue, Tingwen Liu, and Yongbin Li. ExpSeek: Self-triggered experience seeking for web agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 29255–29283. Association for Computational Linguistics, 2026k. doi: 10.18653/v1/2026.findings-acl.1462. <https://aclanthology.org/2026.findings-acl.1462/>.
- Xing Zhang, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He. Experience compression spectrum: Unifying memory, skills, and rules in LLM agents. *arXiv preprint arXiv:2604.15877*, 2026l.
- Xuchao Zhang, Tanish Mittal, Chetan Bansal, Rujia Wang, Minghua Ma, Zhixin Ren, Hao Huang, and Saravan Rajmohan. FLASH: A workflow automation agent for diagnosing recurring incidents. Microsoft Research, October 2024b. https://www.microsoft.com/en-us/research/wp-content/uploads/2024/10/FLASH_Paper.pdf.
- Yan Zhang and Shibo Li. ConsistencyGate: Preventing memory contamination in LLM agents via self-consistency admission control. *arXiv preprint arXiv:2607.22962*, 2026.
- Yaolun Zhang, Yiran Wu, Yijiong Yu, Qingyun Wu, and Huazheng Wang. Live-evo: Online evolution of agentic memory from continuous feedback. *arXiv preprint arXiv:2602.02369*, 2026m.
- Yunxiang Zhang, Yiheng Li, Ali Payani, and Lu Wang. AdaMEM: Test-time adaptive memory for language agents. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026n. <https://openreview.net/forum?id=RzlmkviaNy>.
- Ziao Zhang, Kou Shi, Shiting Huang, Avery Nie, Yu Zeng, Yiming Zhao, Zhen Fang, Qishen Su, Haibo Qiu, Wei Yang, Qingnan Ren, Shun Zou, Wenxuan Huang, Lin Chen, Zehui Chen, and Feng Zhao. SkillFlow: Benchmarking lifelong skill discovery and evolution for autonomous agents. *arXiv preprint arXiv:2604.17308*, 2026o.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024a.
- Haiteng Zhao, Chang Ma, Guoyin Wang, Jing Su, Lingpeng Kong, Jingjing Xu, Zhi-Hong Deng, and Hongxia Yang. Empowering large language model agents through action learning. In *Conference on Language Modeling*, 2024b. <https://openreview.net/forum?id=KqK5XcgEhR>.
- Qiyang Zhao, Xiaofeng Zhang, Bo Liu, Minda Chen, Wei Xiong, Jingyang Chen, Guanting Ye, Wenhao Yu, Xiaosong Yuan, Shijie Han, Da-Han Wang, Jianmin Ji, Fei Huang, and Xu-Yao Zhang. Coalition-Aware Skill Reliability for Self-Evolving Agents. *arXiv preprint arXiv:2608.22610*, 2026a.
- Weixiang Zhao, Yingshuo Wang, Yichen Zhang, Yang Deng, Yanyan Zhao, Wanxiang Che, Bing Qin, and Ting Liu. Large language model agents are not always faithful self-evolvers. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026b. <https://openreview.net/forum?id=kTjSSqgqGf>.

- Weixiang Zhao, Yingshuo Wang, Yichen Zhang, Yanyan Zhao, Yu Zhang, Yang Wu, Dandan Tu, Bing Qin, and Ting Liu. Rethinking experience utilization in self-evolving language model agents. *arXiv preprint arXiv:2605.07164*, 2026c.
- Xinyu Zhao, Junpeng Wang, Yuzhong Chen, Menghai Pan, Chin-Chia Michael Yeh, Jiarui Sun, Yan Zheng, Mahashweta Das, and Tianlong Chen. Demystify the role of memory in machine learning engineering agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 10811–10826. Association for Computational Linguistics, 2026d. doi: 10.18653/v1/2026.findings-acl.525. <https://aclanthology.org/2026.findings-acl.525/>.
- Yang Zhao, Chengxiao Dai, Mengying Kou, and Yue Xiu. MemoRepair: Barrier-first cascade repair in agentic memory. *arXiv preprint arXiv:2605.07242*, 2026e.
- Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.
- Congjie Zheng, Chuanyi Xue, Bin Liang, Jun Yang, and Changshui Zhang. SEAGym: An evaluation environment for self-evolving LLM agents. *arXiv preprint arXiv:2606.17546*, 2026.
- Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. Synapse: Trajectory-as-exemplar prompting with memory for computer control. In *International Conference on Learning Representations (ICLR)*, 2024.
- Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. SkillLearnBench: Benchmarking continual learning methods for agent skill generation on real-world tasks. In *Conference on Language Modeling (COLM)*, 2026. <https://openreview.net/forum?id=ulrZEJrNXA>.
- Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2024.
- Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, and Jun Wang. Memento: Fine-tuning LLM agents without fine-tuning LLMs. *arXiv preprint arXiv:2508.16153*, 2025.
- Ruiwen Zhou, Yingxuan Yang, Muning Wen, Ying Wen, Wenhao Wang, Chunling Xi, Guoqiang Xu, Yong Yu, and Weinan Zhang. TRAD: Enhancing LLM agents with step-wise thought retrieval and aligned decision. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3–13. ACM, 2024. doi: 10.1145/3626772.3657788.
- Sizhe Zhou, Sheldon Yu, Hui Wei, Junda Wu, Siru Ouyang, Yizhu Jiao, Shijia Pan, Julian McAuley, Yu Zhang, Tong Yu, and Jiawei Han. Filesystem-based memory for LLM agents: Organization, evolution, and sustainability. *arXiv preprint arXiv:2607.26637*, 2026a.
- Wei Zhou, Xuanhe Zhou, Shaokun Han, Hongming Xu, Guoliang Li, Zhiyu Li, Feiyu Xiong, and Fan Wu. Are we ready for an agent-native memory system? *arXiv preprint arXiv:2606.24775*, 2026b.
- Yingli Zhou, Wang Shu, Yaodong Su, Wenchuan Du, Yixiang Fang, and Xuemin Lin. A comprehensive survey on agent skills: Taxonomy, techniques, and applications. *arXiv preprint arXiv:2605.07358*, 2026c.
- Yujun Zhou, Kehan Guo, Haomin Zhuang, Xiangqi Wang, Yue Huang, Zhenwen Liang, Pin-Yu Chen, Tian Gao, Nuno Moniz, Nitesh V. Chawla, and Xiangliang Zhang. Getting better at working with you: Compiling user corrections into runtime enforcement for coding agents. *arXiv preprint arXiv:2606.13174*, 2026d. doi: 10.48550/arXiv.2606.13174.
- Jiapeng Zhu, Jianxiang Yu, Yibo Zhao, Chengcheng Han, Qi Gu, Xunliang Cai, Xiang Li, and Weining Qian. Skill0.5: Joint skill internalization and utilization for out-of-distribution generalization in agentic reinforcement learning. *arXiv preprint arXiv:2605.28424*, 2026a.
- Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. SWE context bench: A benchmark for context learning in coding. *arXiv preprint arXiv:2602.08316*, 2026b.
- Shiding Zhu, Yudi Qi, Yajie Wang, Jiaze Li, Chao Song, Yaorui Shi, Yibo Miao, Hanqi Gao, and Kai Zhang. Escaping the self-confirmation trap: An execute-distill-verify paradigm for agentic experience learning. *arXiv preprint arXiv:2606.24428*, 2026c.
- Yaochen Zhu, Harald Steck, Dawen Liang, Yinhan He, Nathan Kallus, and Jundong Li. LLM-based conversational recommendation agents with collaborative verbalized experience. In *Findings of the Association for Computational*

Linguistics: EMNLP 2025, pages 2207–2220, Suzhou, China, November 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-emnlp.119. <https://aclanthology.org/2025.findings-emnlp.119/>.

A Experience-Formation Method Index

This index preserves the system-level details accompanying the mechanism comparisons in Section 3. Table 22 connects each system’s input evidence, formation mechanism, and candidate experience; Table 10 in the main text compares the mechanism choices.

Table 22 Representative experience-formation systems, grouped by the form of experience they construct. Read each row from input evidence through formation mechanism to candidate experience. Inputs include trajectories, relevant feedback, and existing experience where used; mechanisms summarize the construction operations discussed in Section 3.

System	Evidence used	Formation mechanism	Constructed candidate
Trajectory-level experience			
RAP (Kagaya et al., 2024)	Successful trajectories	Select successful trajectories with their task and interaction context	Contextualized trajectory precedent
Self-Generated ICE (Sarukkai et al., 2025)	Agent trajectories generated using a small human-written seed set	Select successful trajectories as further in-context exemplars	Bootstrapped collection of trajectory exemplars
SEER (Cui et al., 2025)	Successful tool-use trajectories	Annotate user intent and the sequence of tools used	Trajectory precedent with intent and toolchain annotations
JARVIS-1 (Wang et al., 2025b)	Successful trajectories with their plans and observed states	Retain a successful plan with its task and starting situation	Plan precedent paired with task and state context
TRAD (Zhou et al., 2024)	Expert demonstration trajectories	Generate a thought for each step, conditioned on the recorded action	Demonstration steps annotated with action-consistent thoughts
Synapse (Zheng et al., 2024)	Interface-interaction trajectories	Remove task-irrelevant interface details through state abstraction	Trajectory of abstracted states and actions
ECHO (Hu et al., 2025a)	Failed trajectories	Infer alternative goals and rewrite behavior in hindsight	Synthetic positive trajectory or workflow for an alternative goal
BAGEL (Murty et al., 2024)	Exploratory trajectories	Label trajectories with synthetic instructions and re-execute them until a filter accepts the pair	Synthetic demonstrations retrieved as in-context precedents
Learn-by-Interact (Su et al., 2025)	Attempts at tasks generated from environment documentation	Write a new instruction for each sub-trajectory; filter pairs with an LLM committee	Instruction–trajectory pairs retrieved as in-context precedents
BREW (Kirtania et al., 2026)	Failed trajectories and their outcome feedback	Propose alternative tasks and rerun the agent on them	Newly observed successful trajectories on alternative tasks, later distilled into recipes (§3.4)
Semantic-level experience			
Reflexion (Shinn et al., 2023)	A trajectory, evaluator feedback, and prior reflections	Reflect on the attempt and describe how the next one should change	Reflection text guiding a later attempt at the same task
Retroformer (Yao et al., 2024)	A failed trajectory, its return, and prior reflections	Reflect with a model fine-tuned by policy gradient on the change in return over the next trial	Reflection text guiding a later attempt at the same task
CLIN (Majumder et al., 2024)	Trajectories from repeated trials in one environment	Interpret action effects as proposed causal relations	Textual causal abstractions of action effects
AutoGuide (Fu et al., 2024)	Offline trajectories of the same task with different returns	Locate divergent actions and extract guidance for the shared context	State-aware guideline with an applicable context
ExpeL (Zhao et al., 2024a)	Success–failure trajectory pairs and groups of successful trajectories	Compare outcomes and extract recurring patterns across successes	Insights retained alongside source trajectories
ReasoningBank (Ouyang et al., 2026)	Trajectories with self-evaluated outcomes; repeated or refined attempts	Reflect on outcomes and, with parallel scaling, synthesize recurring patterns	Successful strategies and failure lessons
Training-Free GRPO (Cai et al., 2025)	Groups of attempts at each training problem, with reference answers	Contrast the attempts and edit a natural-language experience library	Experience library held fixed at test time
MCMA (Liang et al., 2026)	Trajectories of a frozen task model	Train a memory copilot on downstream-ranked preferences to structure and abstract them	Structured memories at several abstraction levels

Continued on next page

Table 22 continued

System	Evidence used	Formation mechanism	Constructed candidate
AutoManual (Chen et al., 2024a)	Trajectories with mixed outcomes and previously derived rules	Derive rules from trajectories and generalize existing rules	Rule-based manual recording rule dependencies
EMG (Wang et al., 2026f)	Successful expert trajectories across similar tasks	Match decision graphs and verbalize their shared subgraphs	Cross-task insights drawn from shared action structure
EDV (Zhu et al., 2026c)	Trajectories of heterogeneous agents on the same task	Use a separate distillation agent to integrate the trajectories	Candidate lessons combining evidence across agents
Procedural-level experience			
AWM (Wang et al., 2025e)	One or more demonstration or agent trajectories	Extract common subtasks and replace concrete values with variables	Workflow with a goal and observation-action steps
SSO (Nottingham et al., 2024)	High-reward subtrajectories from different trajectories	Extract similar subtrajectories and generate a subgoal with instructions	Subgoal-and-instruction skill
TraceCompiler (El Yadouni and Li, 2026)	Tool-call trajectories for a recurring procedure	Infer data dependencies and classify argument sources	Mostly deterministic workflow with explicit argument bindings
Voyager (Wang et al., 2024a)	Exploration trajectories, feedback, and retrieved code skills	Compose skill code and refine it using environment feedback and critiques	Executable skill program
SkillWeaver (Zheng et al., 2025)	Trajectories from repeated website practice	Distill practice into APIs and refine them through further practice	Reusable website API
ASI (Wang et al., 2025d)	Source trajectories	Induce code skills and replay trajectory prefixes rewritten to call them	Reusable code skills exercised in source-task replay
SkillCAT (Chen et al., 2026e)	Successful and failed trajectories from repeated runs of one task	Diagnose behavioral differences and propose skill patches for replay	Declarative patch to an existing skill
SkillOpt (Yang et al., 2026e)	Scored trajectories from a frozen executor	Propose bounded edits to skills from trajectory feedback	Candidate skill edits
Metis (Dai et al., 2026b)	Repeatedly selected plans from prior trajectories and related queries	Convert selected plans into code with reusable parameters	Parameterized code helpers

B Experience-Organization Method Index

This index collects the system-level tables that accompany the comparison of organization patterns in Section 4. Tables 23, 24, and 25 describe tree-structured, graph-structured, and hierarchical-graph designs by what their relations mean and what access they support; Table 11 in the main text summarizes when each pattern is worth its upkeep.

Table 23 Tree-structured experience organization. Value denotes supported access or interpretation. Following Section 2.6, *Adjacent* marks memory systems outside the experience scope, with their memory type, and *Comp* marks skill libraries of unknown authorship and practice origin, used here and in Tables 24 and 25 as complementary evidence about organization.

Paper / system	Nodes and parent–child relations	Value for reuse
Experience index trees: organize access to records		
H-MEM (Sun et al., 2026a)	Domains, categories, memory records, and detailed episodes, grouped broad to specific through predefined levels.	Adjacent (dialogue memory): narrow a query from topic descriptions to detailed memories.
HMT (Tan et al., 2026c)	Intents, stages, and action patterns from web trajectories, in an intent–stage–action hierarchy with stage preconditions and postconditions.	Locate stage-relevant actions using the task goal and current page state.
Filesystem study (Zhou et al., 2026a)	Folders, described files, and nested sections containing memories and skills, linked by directory containment and nested headings.	Read selected files or lines and search a directory without descending level by level.
HORMA (Hsu et al., 2026)	Working-memory content in a filesystem-like hierarchy, with containment exposed through Bash tools.	Adjacent (working memory): let a learned retrieval agent navigate to compact, sufficient context.
MemTree (Reza-zadeh et al., 2025)	Memory nodes at multiple levels, grouped hierarchically.	Adjacent (conversation and document memory): preserve multi-level organization while matching across all levels to avoid premature branch exclusion.
AgentSkillOS (Li et al., 2026a)	Publicly shared skills under a capability tree, with less-used skills in a separate searchable index.	Comp: narrow a large skill pool to candidates for a task.
Experience content trees: organize relations within guidance		
MetaFlowLLM (Fan et al., 2025)	Workflows at different levels of detail, in a hierarchical organization of reusable workflow structure.	Reuse workflow structure while leaving some subtasks for the agent to complete.
Tree-of-Experience (Deng et al., 2026)	Reasoning perspectives, refined from general dimensions to specific directions.	Select analytical paths and use their perspectives and reliability to guide reasoning.
PsychAgent (Yang et al., 2026f)	Theoretical paradigms, intervention stages, meta-skills, and atomic skills, linked from broad counseling guidance to concrete skills.	Locate atomic skills suited to the current dialogue state.
AWM (Wang et al., 2025e)	Induced workflows and more complex workflows composed from them.	Reuse workflow components when constructing more complex procedures.
MACLA (Forouzan-deh et al., 2026)	Procedures and meta-procedures grouping recurring combinations, composed with rules to continue, skip, repeat, or stop.	Retain coordination instructions, not just isolated procedural steps.
DeltaMem (Tan et al., 2026a)	Shared procedure bases and variants; each child stores only its changes to its ancestors.	Reduce repeated content and reconstruct a complete variant from its ancestor path.
STAIR (Xu et al., 2026c)	Repair trajectories abstracted into levels from diagnostic actions to repair strategies.	Adapt nodes from several levels into a plan for a new issue.
SkillX (Wang et al., 2026a)	Planning, functional, and atomic skills built from trajectories.	Supply the levels that suit the executor.
H ² R (Ye et al., 2025)	High-level planning lessons and low-level execution lessons.	Give the planner and the executor the lessons for their own decisions.

Table 24 Graph-structured experience organization and supported access. Adjacent and Comp are used as in Table 23.

Paper / system	Nodes	Edges / relations	Value for reuse
Record graphs: connect precedents and lessons			
A-MEM (Xu et al., 2025b)	General-memory notes.	Associative links among related notes.	Adjacent (general memory): return linked notes along with the matched note.
ExpGraph (Feng et al., 2026a)	Distilled skills and failure lessons.	Semantic-neighbor links.	Expand matched seeds to related guidance through diffusion and utility-aware ranking.
CoEvo-Mem (Ye et al., 2026a)	Experience records with utility estimates.	Semantic, lexical, and temporal links.	Recover temporal neighbors at retrieval; propagate feedback across typed relations.
EXG (Jin et al., 2026)	Successful and failed attempts.	Directed <code>fixed_by</code> links from failures to observed corrections.	Reach a recorded repair from a matching failure.
EMG (Wang et al., 2026f)	Task records with within-task correction or workflow insights.	Cross-task insights carried on edges.	Return both local lessons and guidance derived from comparing tasks.
Transition graphs: connect operations into possible continuations			
H-EPM (Li et al., 2026c)	Tools used in successful trajectories.	Weighted tool transitions, with state summaries where available.	Recall candidate next tools using current state or transition weights.
Procedural Graphs (Lu et al., 2026b)	Tools, skills, reasoning steps, or task states.	Transitions annotated with conditions, guidance, and pitfalls.	Return the guidance and conditions of use attached to a transition from the current operation.
ProPlay (Ma et al., 2026b)	Reusable procedures from successful trajectories.	Observed transitions with outcome-informed reliability records.	Assemble a task-specific procedural path as soft guidance.
Skill graphs: connect capabilities and preserve their joint-use context			
SkillDAG (Bai et al., 2026)	Reusable skills.	Dependency, composition, similarity, specialization, and conflict relations, revised through edits backed by trajectories.	Add related skills to a match and report conflicting skills separately.
Graph-of-Skills (Liu et al., 2026a)	Skills in an existing library.	Skill-dependency links.	Comp (for its larger libraries): recover supporting skills and upstream prerequisites under a content budget.
CaSKG (Li et al., 2026h)	Skills in an existing library.	Candidate skill relations calibrated by LLM-simulated counterfactual probes.	Comp: filter or downweight unsupported links before structural retrieval.
SkillCAT (Chen et al., 2026e)	Capability components within an evolved skill.	Dependency metadata, with shared core content retained alongside components.	Within one skill: load selected capabilities with their common instructions, not the entire skill.
HyperSkill (Xu et al., 2026b)	Subtasks and skills.	Trajectory hyperedges (group links) joining co-used units, with task context and lessons.	Retrieve skills with joint-execution guidance; co-use does not imply mutual necessity.
Combinations of trees and graphs			
SkillLens (Miao et al., 2026)	Policies, strategies, procedures, and primitives.	Links from broader capabilities to lower-level ones, plus lateral links among siblings within a level.	Return a subskill when only part of a broader strategy applies.
HiSkill (Hao et al., 2026)	Trajectory-derived skills and parameterized atomic operations (AtomicOps).	Decomposition edges, plus transition, compatibility, support, and recovery relations.	Return concrete action candidates together with the skill that contains them.

Table 25 Hierarchical graph experience organization and supported access. Comp is used as in Table 23.

Paper / system	Nodes / layers	Cross-layer or internal relations	Value for reuse
Layered experience graphs: connect corresponding units across roles			
G-Memory (Zhang et al., 2025)	Insights, historical task queries, and interaction records in three graphs.	Query nodes connect reusable guidance to the collaboration context behind it.	Return insights together with the queries and interactions behind them.
Trainable graph memory (Xia et al., 2025)	Task queries, canonical decision patterns, and high-level strategies.	Intermediate patterns and learned edge weights between queries and strategies.	Route a task match to strategic guidance. The method also trains the agent, so full-system gains combine the graph with that training.
PlugMem (Yang et al., 2026a)	High-level intents, concrete prescriptions (the guidance returned), and source trajectories (its episodic memory).	Intent links across tasks, with further links to sources.	Return other prescriptions that share an intent, and open the source trajectory when needed.
Metis (Dai et al., 2026b)	Textual plans, code tools, and custom helper functions.	References from plans to implementations, with tool dependencies.	Return the code tool a plan refers to, plus the helper functions it needs.
Nested experience graphs: open up the internal graph of a reusable unit			
SkillOps (Pu et al., 2026)	Skills in an outer graph; inside each, a contract graph over preconditions, operations, artifacts, validators, and failure modes.	Outer dependency, compatibility, redundancy, and alternative relations; each skill maps to its internal contract graph.	Check a whole skill’s preconditions, interfaces, and validators before using or chaining it.
SkillZip (Tan et al., 2026b)	Macro nodes replacing recurring section-level subgraphs.	Input–output boundaries, dependencies, validation links, and mappings back to source content.	Comp: present a macro as a name, contract, outline, or full source, expanding it when information is missing.
DocTrace (Zai et al., 2026)	Abstract question patterns and retained reasoning DAGs.	Links from a pattern to its DAG and to the patterns of its child subquestions.	Incorporate a previously retained subplan where it fits within a broader plan.

C Experience-Maintenance Method Index

Table 26 indexes the systems discussed in Section 6 beyond admission. The admission checks and maintenance decisions remain in Tables 15 and 16 in the main text.

Table 26 Representative systems reviewed in Section 6 beyond admission, grouped by the maintenance operation they mainly perform; admission checks and their example systems appear in Table 15. Adjacent marks conversational-memory systems outside the experience scope.

System	Input examined	Update mechanism	Evaluation setting
Consolidation: integrating repeated and related experience			
MUSE (Yang et al., 2025a)	Procedures from verified subtasks; experience from completed tasks	Inserts a procedure after verified subtask success; deduplicates and generalizes after the task	TheAgentCompany
EvolveR (Wu et al., 2026b)	Newly distilled principles after an offline update phase	Checks semantic equivalence against retained principles; a match attaches the new source instead of adding an entry	Search-based QA (NQ, HotpotQA, five others)
ExpeL (Zhao et al., 2024a)	The current insight list in each extraction round	Adds, edits, or votes on particular insights, keeping unit boundaries intact	HotpotQA, ALFWorld, WebShop; FEVER transfer
DeltaMem (Tan et al., 2026a)	A root and its later variations after repeated successful use	Merges the chain into one self-contained root; specialized variants stay separate	ALFWorld, ScienceWorld, WebShop
Trace2Skill (Ni et al., 2026)	Successful and failed trajectories against one initial skill	Merges proposed patches hierarchically; programmatic checks reject invalid file references, withhold overlapping edits, and validate the format	Spreadsheet, visual QA, and math tasks
SkillZip (Tan et al., 2026b)	Recurring sections within skill workflows; execution feedback	Replaces them with reusable macros that keep interfaces, dependencies, and verification steps; source pointers allow expansion, and feedback can trigger expansion, splitting, or retirement	SkillsBench, ALFWorld
Auto-Dreamer (Ye et al., 2026b)	A selected region of experience and its linked trajectories	A consolidator, run separately from per-session writing, generates replacement experience for the region	ScienceWorld; held-out ALFWorld, WebArena
Refinement: correcting retained guidance			
EmbodiSkill (Ju et al., 2026)	A failed trajectory compared with the current skill	Content defect: edits the skill body; valid but unfollowed instructions: appends an emphasis; abstains when evidence is insufficient	ALFWorld, EmbodiedBench
SkillAdaptor (Yu et al., 2026e)	The earliest actionable fault; the contribution of retrieved skills	Revises the implicated skill or proposes a new one; commits a change only if the aggregate outcome under the candidate collection is nonnegative	WebShop, PinchBench, Claw-Eval
Workflow-Localized Mechanism Learning (Lin et al., 2026b)	Trajectory-derived diagnoses bounded by registered workflow nodes and file sections	A faulty rule: edits the corresponding resource; an incorrect relation between components: edits the composition instructions; abstains on conflicting or insufficient evidence; keeps an edit only if it stays within its permitted address and improves on the original skill on the same batch	Spreadsheet tasks; WikiTableQuestions transfer in spreadsheet form
SkillRevise (Liu et al., 2026d)	Recorded suspected defects, verification requirements, and behavior to preserve	Re-executes candidates within a bounded episode; returns the best observed version under measured utility	SkillsBench and two other skill benchmarks; ALFWorld, BFCL-v4
SkillOpt (Yang et al., 2026e)	Scored trajectories from a frozen executor	An optimizer proposes revisions; replacement requires an improved held-out selection score	Six benchmarks, incl. SpreadsheetBench, OfficeQA, ALFWorld

Continued on next page

Table 26 continued

System	Input examined	Update mechanism	Evaluation setting
WikiSkill (Tang et al., 2026)	Proposed skill sets	Evaluates them on a validation split; rolls back unsuccessful proposals and keeps the accumulated knowledge	Math, web search, spreadsheet, document QA, ALFWorld
SkillOps (Pu et al., 2026)	Contract defects; consistently failing or obsolete skills	Rule-based repair of missing checks and type mismatches against explicit skill contracts; removes a retired skill’s links in the skill graph	ALFWorld
MACLA (Forouzandeh et al., 2026)	Outcomes of repeated uses, with the conditions of each use retained	Updates procedure success estimates and uncertainty; prunes by reliability, usage, and age	ALFWorld, WebShop, TravelPlanner, InterCodeSQL
Retirement: reducing availability			
Live-Evo (Zhang et al., 2026m)	A batch of poorly solved tasks	Reruns source tasks and keeps improvements; updates persistent experience weights from the memory-guided versus memory-free performance gap	Event forecasting (Prophet Arena); Xbench-DeepResearch
EDGE (Xie et al., 2026a)	Running average of the reward gain of rollouts with versus without the experience	Deletes experience below a utility threshold during agentic reinforcement learning	ALFWorld, WebShop; seven search-based QA sets
ReMe (Cao et al., 2026)	Retrieval counts of each experience	Prunes experience with insufficient observed utility after a minimum number of retrievals	BFCL-V3, AppWorld
Hermes Curator (Nous Research, 2026)	Inactive agent-created skills	Archives them and can restore them to the active skill directory	Engineering documentation; no controlled study
Coordination: aligning operations with formation and retrieval			
UMEM (Ye et al., 2026c)	Queries semantically near the source query	Optimizes extraction and management jointly by marginal utility over this neighborhood	AIME, GPQA-Diamond, HLE, HotpotQA, ALFWorld
Memory-R1 (Yan et al., 2026c)	Memory-bank states and incoming information	Learns ADD, UPDATE, DELETE, or leave-unchanged decisions from downstream answer correctness	Adjacent (conversational memory): LoCoMo, MSC, LongMemEval
CODESKILL (Li et al., 2026f)	An add, merge, or drop decision for one candidate skill at a time	Trained on skill-quality feedback, verifier gains over a no-skill baseline, and an alignment component	EnvBench, SWE-Bench Verified, Terminal-Bench 2
MemSkill (Zhang et al., 2026f)	Difficult training cases	A controller selects routines from a shared bank of memory-processing skills; a designer revises or adds routines	ALFWorld, AppWorld; routine-evolution ablation on LoCoMo (conversational)
CoEvo-Mem (Ye et al., 2026a)	Graph entries, relations, and utility estimates; the router’s retrieval mixtures	Trains a retrieval router against a fixed graph, then updates the graph under a fixed router	Operating-system interaction, function calling; conversational multiple-choice
Evolution safeguards: keeping updates correctable			
ByteRover (Nguyen et al., 2026)	Curation operations	Records the reason for each operation; writes files atomically	Adjacent (conversational memory): LoCoMo, LongMemEval
A-MemGuard (Wei et al., 2026)	Candidate memories used in reasoning	Validates agreement across reasoning paths from related memories; keeps a separate store of failure lessons consulted before later actions	Memory-poisoning attacks on QA, healthcare, and multi-agent tasks
MemoRepair (Zhao et al., 2026e)	A withdrawn premise and its affected descendants	Reconstructs successors from retained support and republishes them once validated and their predecessors are repaired	ToolBench, MemoryArena

Continued on next page

Table 26 continued

System	Input examined	Update mechanism	Evaluation setting
Dependency rollback (Yu et al., 2026a)	A faulty memory that has already been identified	Uses recorded dependencies to keep independently supported state and replay the affected computation	Constructed tool-use cases; stress test from LongMemEval-V2
Execution-state unlearning (Yao et al., 2026)	Target information that has already been identified	Uses provenance to replay the affected computation and to remove residues left in compressed context or the key-value cache	LongMemEval, ToolSandbox, AgentDojo

D Evaluation Protocol Index

This index collects the representative evaluation protocols discussed in Section 7. Table 28 groups them by the evaluation question they mainly address and records each study’s evidence role; Table 27 gives the source basis for the authored-skill studies marked Direct[†]. The evidence requirements for each improvement claim and the measurements this survey proposes remain in the main text (Tables 18 and 19).

For the authored-skill studies marked Direct[†] in Table 28, Table 27 separates reported sources from our interpretation of their practice origin. This designation concerns the utilization and downstream questions tested. SkillRet remains complementary because its automatically crawled corpus does not establish authorship or practice origin.

Table 27 Source basis for Direct[†] authored-skill studies. Reported authorship and source descriptions are distinct from our interpretation of practitioner experience; use tests do not certify practice provenance.

Study	Source explicitly reported	Basis and limit of the practice interpretation
SkillsBench (Li et al., 2026d)	Contributors independently author skills using public repositories or prior domain experience.	Prior domain experience is an explicit practice-origin statement for that branch. The repository branch does not establish each skill’s underlying practice.
Skill-Use (Han et al., 2026b)	Public GitHub skills and material from SkillsBench and SkillLearnBench, selected for task-specific procedures and observable use.	Practitioner experience is our reading of these curated sources. Exclusive constraints and masked-skill checks establish use relevance, not practice origin.
SLBench (Chen et al., 2026g)	Human-written skill files, from which workflows and relations are extracted and traced to the text.	Human authorship is reported; task practice is our interpretation. Text traceability does not recover source trajectories.
SkillAligner (Li et al., 2026b)	ComposioHQ/awesome-claude-skills, official anthropics/skills, and material adapted from SkillsBench.	We interpret the curated material as practitioner experience. Repository identity and interface normalization do not establish each skill’s practice origin.

The four studies test triggering, following, relations, adaptation, or downstream outcomes of supplied skills.

Table 28 Representative evaluation protocols, grouped by question. Groups follow the order of Section 7. B = benchmark or protocol; M = method experiments; E = empirical study; B/M combines contributions. I = intrinsic quality; P = experience utilization process; D = downstream effects. The evidence-role column distinguishes direct experience from adjacent, complementary, and system-level boundary evidence. †: practice origin read from the reported sources (Table 27). Not every task in a suite involves trajectory-derived experience. In the sustained-improvement group, the first four rows (Voyager, AWM, ReasoningBank, SkillFlow) report cumulative or streamed scores without a frozen-copy comparison; the closing note of each row states what its design supports.

Work / type	Evidence role	Evaluated object	Target	Comparison and scoring basis
Intrinsic experience quality				
MUSE-Autoskill (Lin et al., 2026a) <i>M</i>	Direct	Generated skill packages	I	Compare skill outputs with tests generated alongside each skill.
OEP (Wang et al., 2026d) <i>E</i>	Direct	Over-generalized experience rules	I, D	An evaluator model checks retained content for an induced overgeneralization; later attack effects are measured separately.
Honest Lying (Dixit et al., 2026) <i>E</i>	Direct	Reflections retained across attempts	I	Whether each retained reflection names the task’s target object, extracted from the task specification.

Continued on next page

Table 28 continued

Work / type	Evidence role	Evaluated object	Target	Comparison and scoring basis
SkillLearnBench (Zhong et al., 2026) <i>B</i>	Direct	Skills generated from agent trajectories	I, P, D	Model-judged skill quality and trajectory alignment against references; verifier-scored outcomes on the same tasks.
MEMAUDIT (Bhargava and Barrento, 2026) <i>B</i>	Adjacent	Budgeted memory writing	I	Preserved evidence relative to the attainable optimum under a storage budget.
Experience utilization process				
ProactAgent (Cai et al., 2026) <i>M</i>	Direct	Retrieval policy and experience repository	P, D	Retrieval strategies compared on success and efficiency; paired query/no-query continuations.
QCR (Li et al., 2026g) <i>B/M</i>	Direct	Fixed-source trajectory reuse	P, D	Fixed candidates and selected history; delivered forms compared on success, online cost, and rebinding.
Faithful Self-Evolvers (Zhao et al., 2026b) <i>E</i>	Direct	Raw and condensed experience	P, D	Experience removed or perturbed under frozen backbones; shows dependence, not benefit.
Coalition-Aware Skill Reliability (Zhao et al., 2026a) <i>B/M</i>	Adjacent	Memory-writing skill subsets	D	Full banks versus single-skill knockouts on identical examples, mainly for MemSkill routines on conversational memory and QA.
SkillRet (Kang et al., 2026b) <i>B</i>	Comp.	Retrieval over a crawled public skill corpus	P	Recall@ <i>k</i> , NDCG@ <i>k</i> , and latency for labeled queries.
Skill-Use (Han et al., 2026b) <i>B</i>	Direct [†]	Triggering and following curated public skills	P, D	Triggering, compliance, prohibited-action avoidance, and out-of-scope non-triggering; triggered runs versus no library.
SLBench (Chen et al., 2026g) <i>B</i>	Direct [†]	Relations within human-written skills	P	Executable tests of prerequisites and exceptions; runs labeled safe, unsafe, or inconclusive.
SkillAligner (Li et al., 2026b) <i>M</i>	Direct [†]	Adaptation of retrieved skills	P, D	Raw skills versus adapted guides and their components under fixed candidates; evaluator-labeled regressions.
Mem2ActBench (Shen et al., 2026b) <i>B</i>	Comp.	Memory-based tool calls	P	Memory-based tool arguments scored with the reference tool supplied.
LongMemEval-V2 (Wu et al., 2026a) <i>B</i>	Adjacent	Environment-history evidence gathering	P, D	Questions over web-agent histories; accuracy, latency, and retrieval or reading errors.
Scale-conditioned memory (Shao et al., 2026a) <i>B</i>	Adjacent	Retrieval under history growth	P, D	Fixed query evidence amid growing irrelevant sessions; correctness within a memory-call budget.
Experience-Following (Xiong et al., 2026) <i>E</i>	Direct	Stored trajectories and agent behavior	P, D	Fixed starting memory versus addition policies on the same streams; similarity, imitation, error propagation, and downstream performance.
MemoryArena (He et al., 2026d) <i>B</i>	Comp.	Memory and interdependent action	D	Subtask completion and task success across interdependent sessions.

Continued on next page

Table 28 continued

Work / type	Evidence role	Evaluated object	Target	Comparison and scoring basis
Downstream effects: task benefits and transfer				
ExpeL (Zhao et al., 2024a) M	Direct	Trajectory pool and insights	D	Frozen trajectories and insights versus retrieval and insight ablations on held-out tasks; fixed experience reuse, not sustained improvement.
SkillsBench (Li et al., 2026d) B	Direct [†]	Curated skill packages	D	Matched runs with and without curated skills; deterministic verifiers.
BenchTrace (Huang et al., 2026) B	Direct	Failure understanding and reuse	D	Later failure avoidance and task progress; diagnosis scored separately.
Memory Transfer Learning (Kim et al., 2026a) E	Direct	Cross-domain coding experience	D	Target benchmark excluded from memory sources; representations and producer models versus no memory.
Budget-constrained study (Hajimiri et al., 2026) E	Direct	Online skill and memory augmentation	D	Augmented agents versus a base agent at comparable total token cost.
EvoAgentBench (Gao et al., 2026c) B	Direct	Trajectory-derived Ability transfer	D	Every test task backed by training-side Ability units; automatic methods against a curated reference across backbones and scaffolds.
EDGE (Xie et al., 2026a) M	Direct	Experience-trained policy	D	Trained policy with and without retrieved experience; component ablations on ALFWorld.
Machine learning engineering memory study (Zhao et al., 2026d) E	Adjacent	Debugging memory in coding agents	D	Debugging memory in sequential refinement versus tree search.
Downstream effects: feedback-driven sustained improvement				
Voyager (Wang et al., 2024a) M	Direct	Persistent executable skills	D	Skills added after feedback and self-verification; exploration progress and novel-task success. Checkpoint retention not isolated.
AWM (Wang et al., 2025e) M	Direct	Offline and on-line workflows	D	Frozen offline workflows tested across tasks, websites, and domains; online streamed success. Offline arm: fixed experience reuse; online arm: stream averages only.
ReasoningBank (Ouyang et al., 2026) M	Direct	Accumulated reasoning strategies	D	Strategies added from self-evaluated outcomes over streamed tasks, starting from empty memory under fixed rules. Stream averages only.
SkillFlow (Zhang et al., 2026o) B	Direct	Evolving skill library	P, D	Skills updated between tasks from an empty library; completion, skill use, library size, and cost. Aggregate completion against a no-skill agent only.
AgentStream (Yan et al., 2026b) B	Direct, Boundary	Persistent state across task streams	D	Isolated, sequential, and interleaved streams without test labels; no independent frozen audit.
SEAGym (Zheng et al., 2026) B	Direct	Updating harness snapshots	D	Saved snapshots scored on held-out ID and OOD views and replayed training tasks, with cost records.

Continued on next page

Table 28 continued

Work / type	Evidence role	Evaluated object	Target	Comparison and scoring basis
PATH-Bench (Yang et al., 2026c) <i>B</i>	Direct	Path-dependent memory and skill reuse	D	Controlled helpful or interfering histories around recurring probes; forward and backward transfer and forgetting.
Evo-Memory (Wei et al., 2025) <i>B</i>	Direct	Streaming memory updates	D	Easy-to-hard versus hard-to-easy order; successful-only versus all stored experience.
Capability erosion study (Yu et al., 2026c) <i>E</i>	Direct	Retention under memory evolution	D	Held-out old-task probes after updates on a new task stream, with and without evidence-gated preservation.
AgentCL (Shu et al., 2026) <i>B</i>	Direct	Cross-task memory reuse	D	Updating first pass, frozen second pass, and held-out tasks (plasticity, stability, and generalization gains).
SkillEvolBench (Lei et al., 2026) <i>B</i>	Direct	Acquired procedural skills	D	Updates during acquisition, then frozen replay and shifted deployment; library reset between environments.
Trust-Memevo / TAME (Cheng et al., 2026) <i>B/M</i>	Direct	Evolved memory and trustworthy reuse	D	Memory evolved on benign tasks, then frozen for model-scored trustworthiness tests.
RoMeRL (Yang et al., 2026d) <i>M</i>	Direct	Persistence of noisy experience	D	Ten rounds after corrupting 10% of early entries; credit and remaining share of noisy entries.
Continual reuse study (Hu et al., 2026a) <i>E</i>	Direct	Retention and new-task learning	D	Retention and new learning split by no-memory success; cross-task versus target-only acquisition.
StreamBench (Wu et al., 2024a) <i>B</i>	Adjacent	Feedback-adapting agent	D	Accuracy or Pass@1 over an input–feedback stream.
MemEvoBench (Xie et al., 2026b) <i>B</i>	Adjacent	Misleading memory and defenses	P, D	Attack success and misleading-memory detection over multi-round tasks.
EvoMemBench (Wang et al., 2026h) <i>B</i>	Adjacent	Knowledge, task state, and procedures	D	Memory forms compared on within-episode and cross-episode subsets.
Downstream effects: improvement of the learning mechanism				
MemSkill (Zhang et al., 2026f) <i>M</i>	Direct, Adjacent	Memory routines and selector	D	Evolved versus fixed routines (conversational memory); held-out scores and routine transfer.
CoEvo-Mem (Ye et al., 2026a) <i>M</i>	Direct, Adjacent	Retrieval router and memory graph	D	Router and graph updated alternately in training, then frozen; tests training co-adaptation only.
MemEvolve (Zhang et al., 2026c) <i>M</i>	Direct	Experience-processing architecture	D	Architectures each started from empty memory, then fixed for transfer.
DGM (Zhang et al., 2026g) <i>M</i>	Boundary	Agent code and variant archive	D	Coding progress versus a fixed modifier and without the variant archive; coding only.

Continued on next page

Table 28 continued

Work / type	Evidence role	Evaluated object	Target	Comparison and scoring basis
Hyperagents (Zhang et al., 2026h) <i>M</i>	Boundary	Task-agent and meta-agent programs	D	Transferred meta-agent frozen in a new domain; system transfer, not a mechanism swap.